

Iterative Algorithmen

- Schleifenkonstrukte wie `for`- und `while`-Schleifen führen zu **iterativen Algorithmen**
- beschreiben Berechnungen über einer Menge von Variablen, die in jeder Iteration aktualisiert werden
- `for`-Schleifen alleine reichen im Allgemeinen nicht aus, um alles Berechenbare zu abzudecken (`while`-Schleifen notwendig).

Mehr hierzu in Berechenbarkeitstheorie!

- **Rekursion** (nächstes Thema) erlaubt uns einige Probleme viel einfacher und eleganter zu lösen.

Rekursion im Alltag

Ein Mensch ist **natürlich geborene(r) US-Staatsangehörige(r)**, falls sie/er

- in den USA geboren ist, oder **(Basisfall)**
- Kind zweier Eltern ist, die jeweils entweder eingebürgerte oder **natürlich geborene US-Staatsangehörige** sind und mindestens ein Elternteil bereits in den USA gelebt hat.

(Rekursiver Fall)

Rekursionsprinzip

(Induktionsprinzip)

Basisfall

- Definiere/Löse triviale Instanzen direkt.

(Induktionsanfang)

Rekursiver Fall

Definiere/Löse **nicht-triviale Instanzen I**, indem

- **einfachere Instanz(en) J** definiert/gelöst wurde(n),

(Induktionsvoraussetzung)

- **I** mittels Definition(en)/Lösung(en) von **J** und einfachen Operationen definiert/gelöst wird

(Induktionsschritt)

Rekursion in der Informatik

- I) Eine Art und Weise **Objekte/Probleme** zu **beschreiben**

Beispiel wohlgeklammerte Ausdrücke.

Wohlgeklammerte Ausdrücke	Klammerpaare
()	1
()() (())	2
()()() ()()() (()) (()) ((()))	3

Rekursive Definition?

Wohlgeklammerte Ausdrücke rekursiv definiert

Basisfall. Es gibt nur einen wohlgeklammerten Ausdruck mit $n=0$ **Klammerpaaren**, nämlich **das leere Wort**.

Rekursiver Fall.

Sei $n \geq 1$. Ein **wohlgeklammerter Ausdruck** e mit n **Klammerpaaren** ist ein Ausdruck der Art

$$e = (f) g$$

wobei f ein wohlgekl. Ausdruck mit i **Klammerpaaren**
und g ein wohlgekl. Ausdruck mit j **Klammerpaaren**
ist, sodass $i + j = n-1$ gilt.

Rekursion in der Informatik

II) Eine Art und Weise **Lösungen** zu **entwerfen**

Für $n > 0$ ist die **Fakultät von n** folgendermassen definiert:

$$n! = \prod_{i=1}^n i = 1 \cdot 2 \cdot 3 \cdots n$$

Rekursive Berechnung von $n!$

Basisfall. Für $n=1$ gilt $n!=1!=1$

Induktiver Fall.

Berechnung von $I=n!$ für $n > 1$:

- Berechne $J=(n-1)!$ rekursiv

- Berechne $I=n \cdot J$

In Python typischerweise durch
rekursiven Methodenaufruf

Ist korrekt, denn $I=n \cdot J=n \cdot (n-1)!=n!$

Rekursion

- algorithmisch: Eine Art und Weise Lösungen zu beschreiben mittels
 - Teile und Herrsche (**Divide & Conquer**) oder
 - Verkleinere und Herrsche (**Decrease & Conquer**)
 - **Idee**: Reduziere ein Problem in Unterprobleme (desselben grundsätzlichen Problems) und führe deren Lösungen zu einer Lösung des Gesamtproblems
- Semantisch: Eine Programmiertechnik bei der sich **Funktionen selbst aufrufen**
 - Beim Programmieren sollte keine Endlosrekursion entstehen
 - Basisfall/Basisfälle müssen direkt gelöst werden
 - Induktiver Fall muss auf einfacheren Fall überführbar sein

Multiplikation iterativ

ohne Beschränkung der Allgemeinheit

- Die Multiplikation $a * b$ ist äquivalent dazu a mit sich selbst b mal zu addieren (wir nehmen hier oBdA an, dass a und b nicht negativ sind)
- Der aktuelle **Zustand** der Multiplikation wird beschrieben durch
 - Eine **Iterationszahl** i , die bei b beginnt
 $i \leftarrow i - 1$ und stoppe sobald i gleich 0
 - einen aktuellen Berechnungswert (Zwischensumme) `ergebnis`
 $\text{ergebnis} \leftarrow \text{ergebnis} + a$

```
def mult_iterativ(a,b):  
    ergebnis=0  
    while b>0:  
        ergebnis+=a  
        b-=1  
    return ergebnis
```

Iteration
aktueller Berechnungswert, Zwischensumme
aktueller Wert der Iterationsvariablen

Multiplikation rekursiv

- **Rekursiver Fall**

- Man überlegt sich wie man das Problem auf eine **kleinere/einfachere Version** des Problems reduzieren kann

...vereinfachte Problem wiederholt...

- **Basisfall**

- ...bis einfacher Fall entsteht, der **direkt gelöst** werden kann

- **Beispiel:** Falls $b = 1$ dann gilt

$$a \cdot b = a$$

$$a \cdot b = \underbrace{a + a + a + \dots + a}_{b \text{ mal}}$$

b mal

$$= a + \underbrace{a + a + \dots + a}_{b-1 \text{ mal}}$$

b-1 mal

$$= a + \boxed{a \cdot (b - 1)}$$

*Rekursive
Reduktion*

```
def mult(a,b):  
    if b==1:  
        return a  
    else:  
        return a+mult(a,b-1)
```

Basisfall

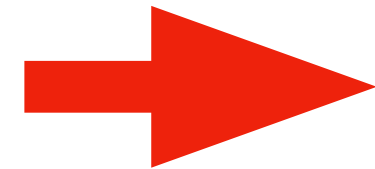
Rekursiver Fall

Fakultätsberechnung

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot (n - 3) \cdot \dots \cdot 1$$

- Für des welche n kennen wir die Fakultät?

$n = 1$



```
if n==1:
```

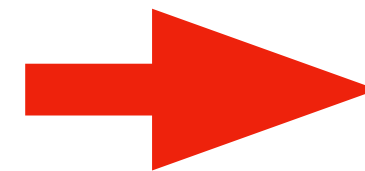
```
return 1
```

Basisfall

- Wie kann das Fakultätsberechnungsproblem reduziert werden?

Falls $n > 1$ so gilt

$$n! = n \cdot (n - 1)!$$



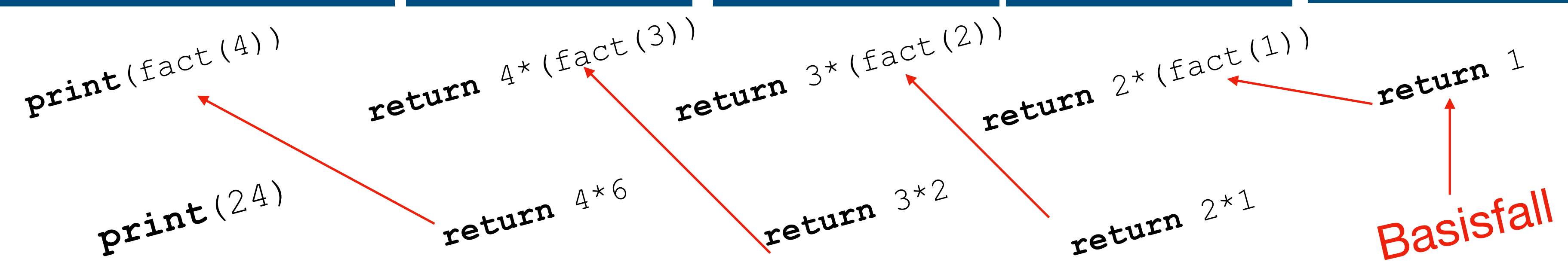
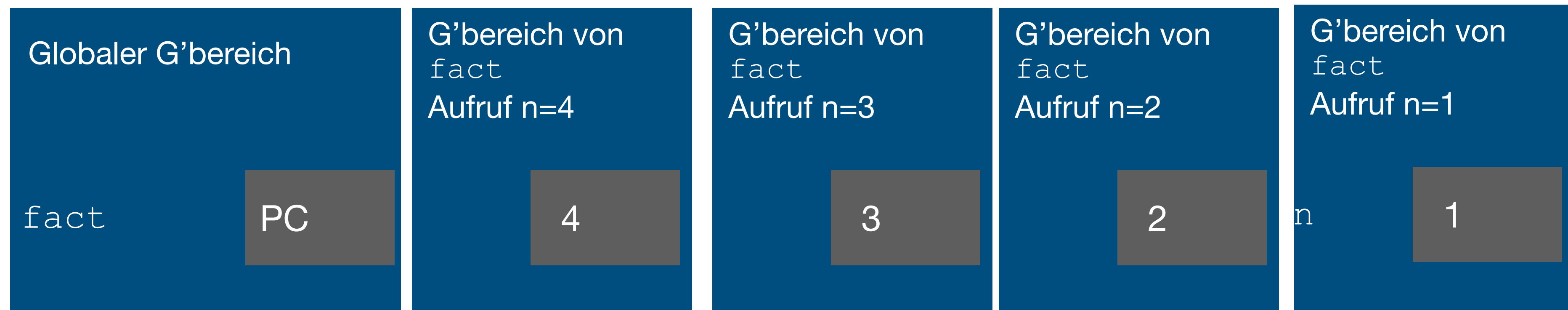
```
else:
```

```
return n*fakultaet(n-1)
```

Rekursiver Fall

Geltungsbereich rekursiver Funktionen

```
def fact(n):  
    if n==1:  
        return 1  
    else:  
        return n*fact(n-1)  
print(fact(4))
```



Ein paar Beobachtungen

- Jeder rekursive Aufruf einer Funktion erzeugt einen **eigenen Geltungsbereich**
- **Variablenbelegungen** im Geltungsbereich der rekursiven aufgerufenen Funktion hat keine Auswirkung auf den Aufrufer
- Der **Kontrollfluß kehrt zum Aufrufer** zurück sobald der rekursiv berechnete Wert zugewiesen wird

obwohl u.U. derselbe Variablenname verwendet wird, handelt es sich um unterschiedliche Objekte in verschiedenen Geltungsbereichen

Iteration vs. Rekursion

```
def fakultaet_iterativ(n):  
    produkt=1  
    for i in range(1,n+1):  
        produkt*=i  
    return produkt
```

```
def fakultaet(n):  
    if n==1:  
        return 1  
    else:  
        return n*fakultaet(n-1)  
print(fakultaet(4))
```

- Rekursion kann u.U. einfacher bzw. intuitiver sein
- Iteration kann u.U. einfacher bzw. intuitiver sein
- Rekursion kann effizienter aus der Sicht der Programmiers sein
- Rekursion kann u.U. weniger effizient sein aus der Sicht des Computers

Induktives Schlussfolgern

- Woher wissen wir dass unser rekursiver Code auch wirklich korrekt ist?

```
def mult_iterativ(a,b):  
    ergebnis=0  
    while b>0:  
        ergebnis+=a  
        b-=1  
    return ergebnis
```

- `mult_iterativ` terminiert, da `b` zu Beginn positiv ist und in jedem Schleifendurchlauf um 1 dekrementiert wird, muss also irgendwann einen Wert annehmen, der nicht größer 1 ist

```
def mult(a,b):  
    if b==1:  
        return a  
    else:  
        return a+mult(a,b-1)
```

- `mult` hat bei `b=1` keinen rekursiven Aufruf und hält an
- `mult` macht bei `b>1` einen rekursiven Aufruf mit einer kleineren Version von `b`, muss also irgendwann den Aufruf mit `b=1` erreichen

Mathematische Induktion

Menge der natürlichen Zahlen in der Informatik: $\mathbb{N} = \{0,1,2,\dots\}$

- Um zu zeigen, dass eine Aussage über allen natürlichen Zahlen n gilt, zeigt man zwei Dinge:

1. Man zeigt, dass die Aussage für das kleinste n gilt (also für $n=0$)

Induktionsanfang

2. Dann zeigt man, dass für alle natürlichen Zahlen n folgende Folgerung gilt:

Induktionsvoraussetzung

Wenn die Aussage für n gilt, **dann** gilt sie auch für $n+1$

Induktionsschritt

Um 2. zu beweisen, dürfen (und müssen) sogar annehmen, dass die Aussage für n **tatsächlich gilt**, um die Aussage für $n+1$ zu beweisen. Wir reduzieren den Fall für $n+1$ also auf den Fall n .

Induktion Beispiel

Aussage: Für alle $n \in \mathbb{N}$ gilt folgende Gleichheit

$$1 + 2 + 3 + \dots + n = (n \cdot (n + 1))/2$$

Beweis durch Induktion:

- Falls $n = 0$ so ist die linke Seite der Gleichung gleich 0 und die rechte Seite $(0 \cdot (0 + 1))/2 = 0$

Induktionsanfang

- Sei $n \geq 0$ eine beliebige aber feste natürliche Zahl größer 0

und nehmen wir an die obige Aussage gelte bereits für n ,

also $1 + 2 + 3 + \dots + n = (n \cdot (n + 1))/2$ (★)

Induktionsvoraussetzung

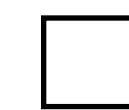
Unter der Annahme der Gleichung (★) müssen wir nun

$1 + 2 + 3 + \dots + n + (n + 1) = ((n + 1) \cdot (n + 2))/2$ zeigen:

Induktionsschritt

Es gilt: $1 + 2 + 3 + \dots + n + (n + 1) \stackrel{(\star)}{=} (n \cdot (n + 1))/2 + (n + 1)$

$$= (n \cdot (n + 1))/2 + 2 \cdot (n + 1)/2 = ((n + 2)(n + 1))/2$$



Relevanz bei Korrektheit von Programmen

- Wir wenden ebenfalls das Induktionsprinzip an.
- Zur Erinnerung: Wir haben angenommen, dass $a > 0$ und $b > 0$ gilt.

```
def mult(a,b):  
    if b==1:  
        return a  
    else:  
        return a+mult(a,b-1)
```

Keine Zuweisung sondern
mathematische Gleichheit gemeint!

- **Aussage:** Für alle $a > 0$ und alle $b > 0$ liefert `mult(a,b)` das Produkt $a * b$ zurück.

Beweis durch Induktion nach b

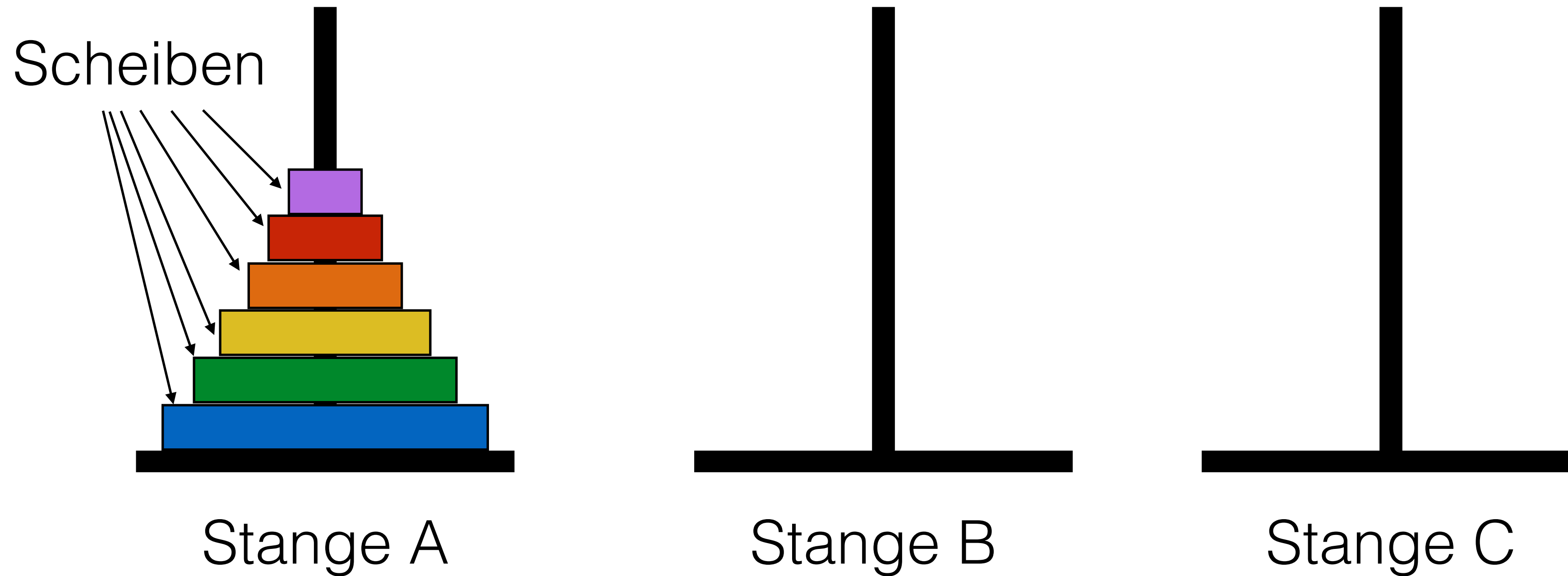
- **Induktionsanfang:** Für b gleich 1 gibt `mult` für alle a den Wert a zurück, korrekt da, wenn b gleich 1, für alle a gilt: $a * b = a$
- **Induktionsschritt:** Für alle a gibt `mult(a, b+1)` den Wert $a + \text{mult}(a, b+1-1)$ zurück

Es gilt $a + \text{mult}(a, b+1-1) = a + \text{mult}(a, b) = a + a * b = a * (b+1)$

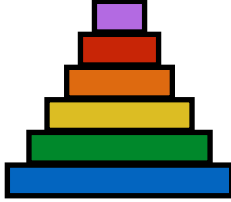
Induktionsvoraussetzung

Induktionsschritt

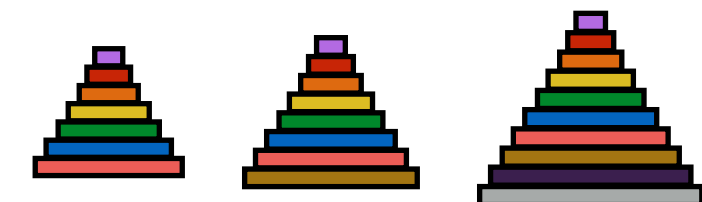
Die Türme von Hanoi



Die Regeln:

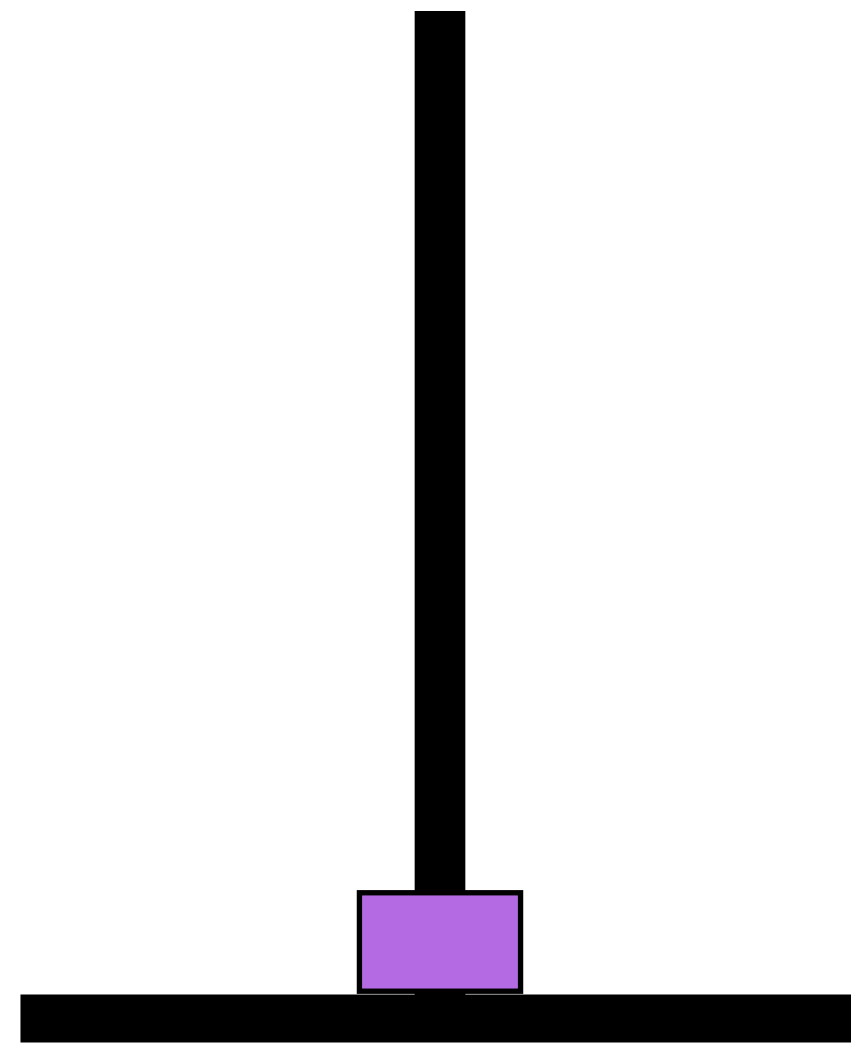
- Bewege einzelne Scheiben zwischen den Stangen, sodass sich  schliesslich auf Stange C befindet.
- **Bedingung:** Eine gerade gelegte Scheibe muss immer die kleinste der Stange sein, auf die sie gelegt wird!

Die Frage: Ist dies für jede Turmhöhe **n** möglich?

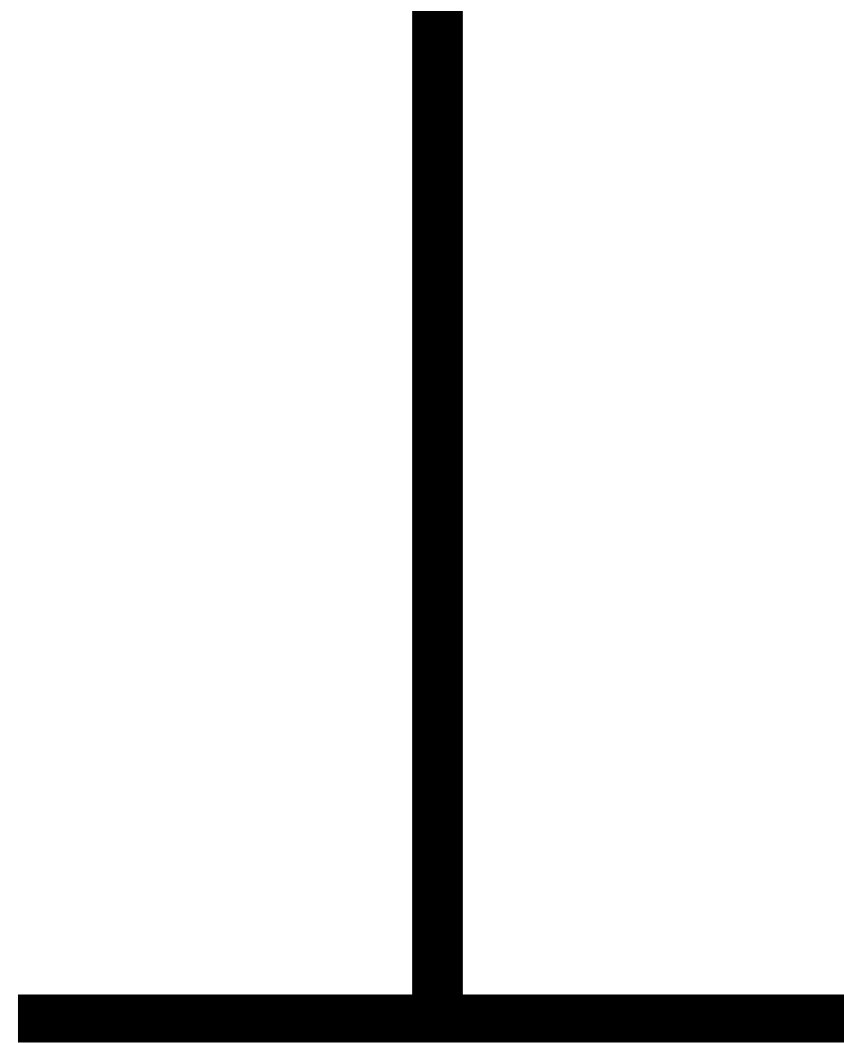


Kleine Türme von Hanoi

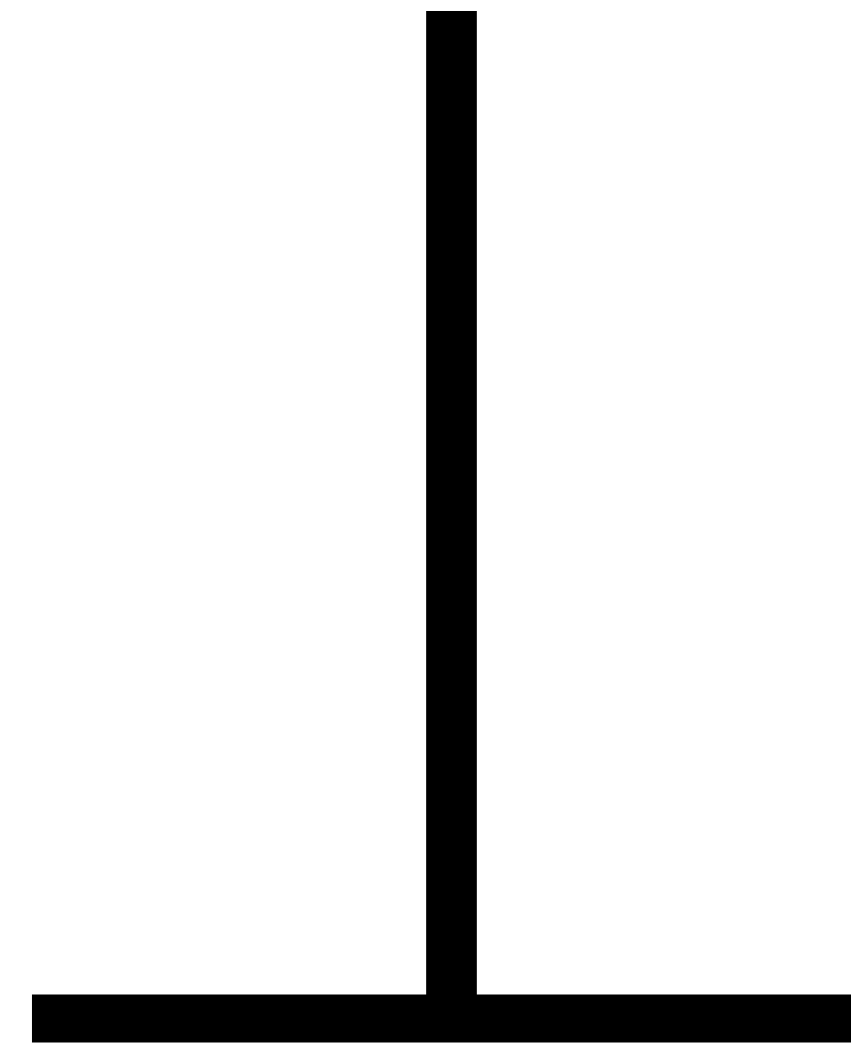
n=1



Stange A



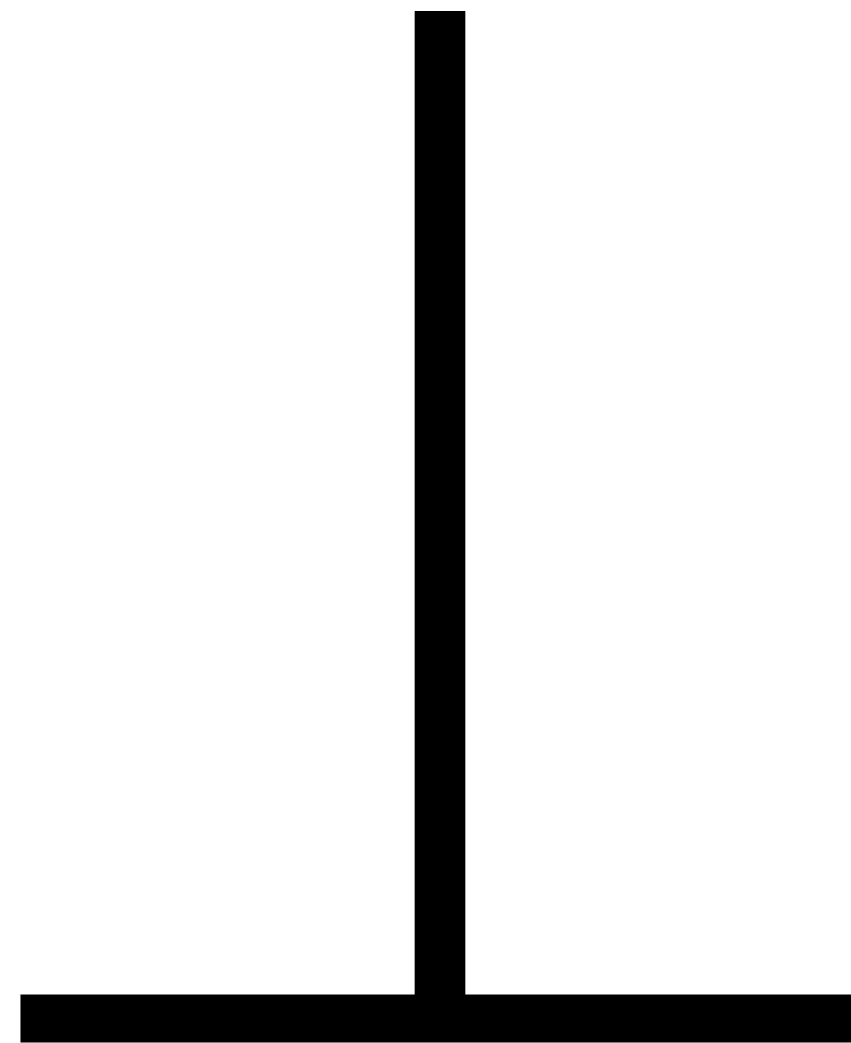
Stange B



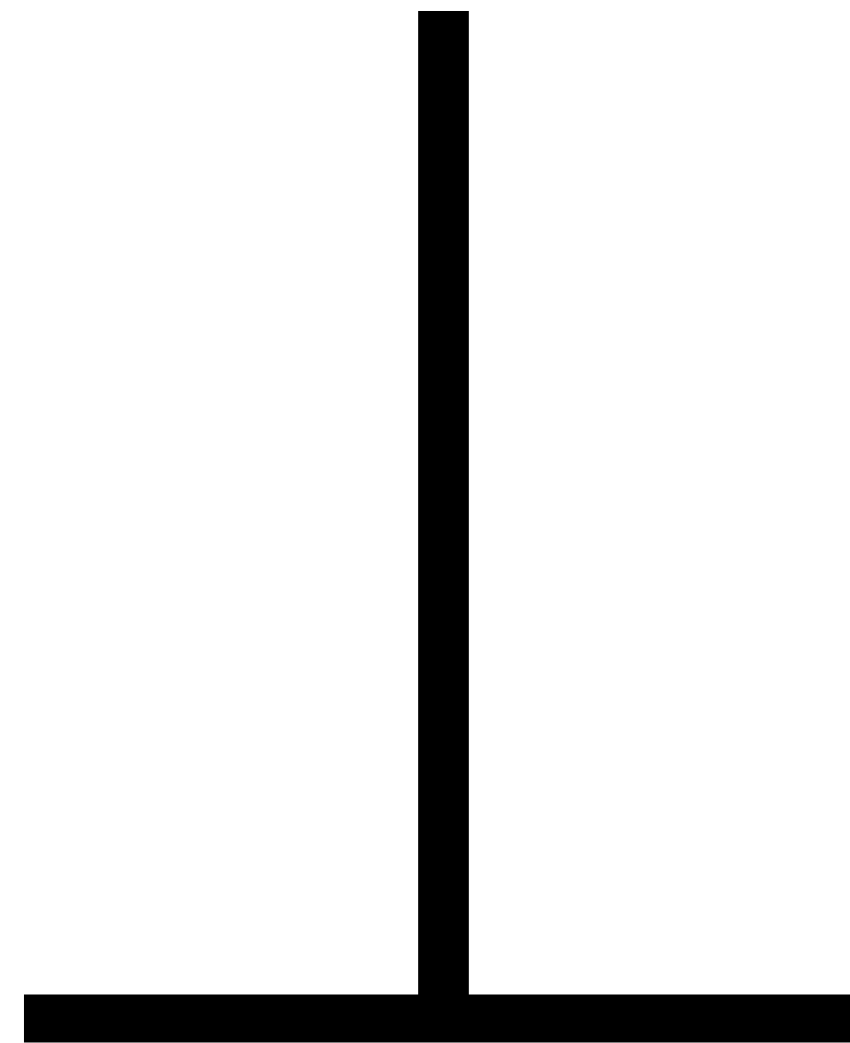
Stange C

Kleine Türme von Hanoi

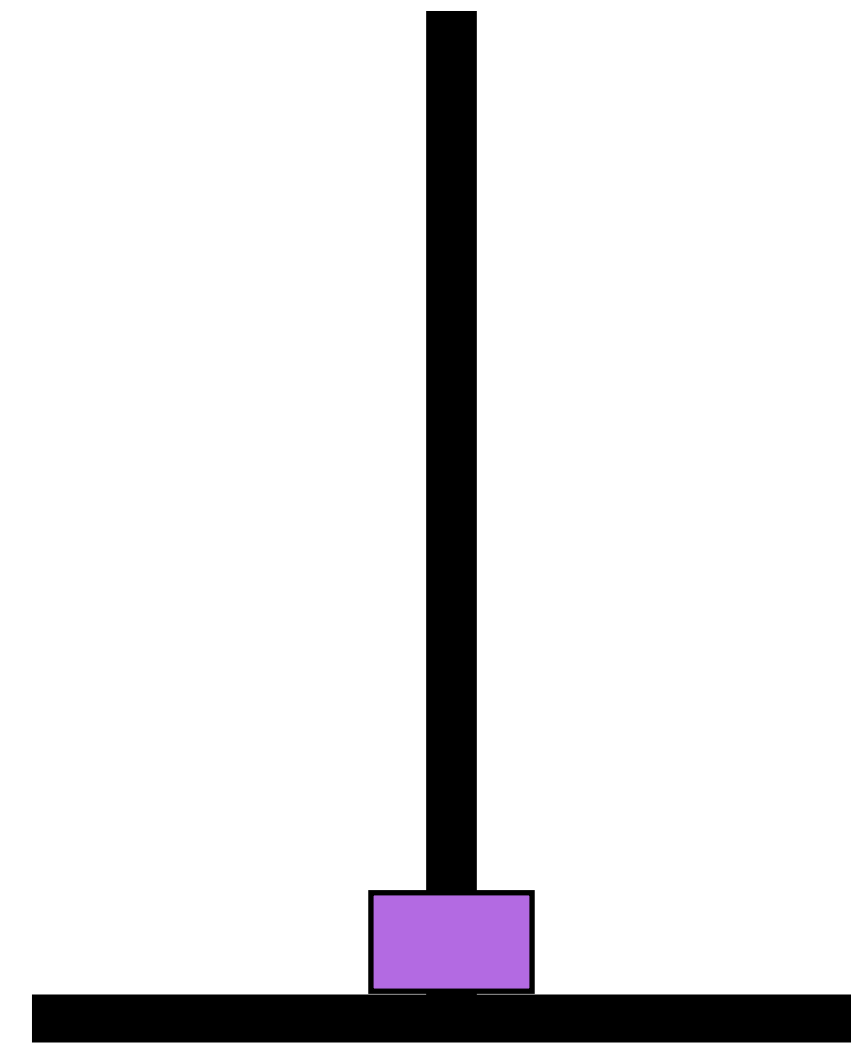
n=1



Stange A



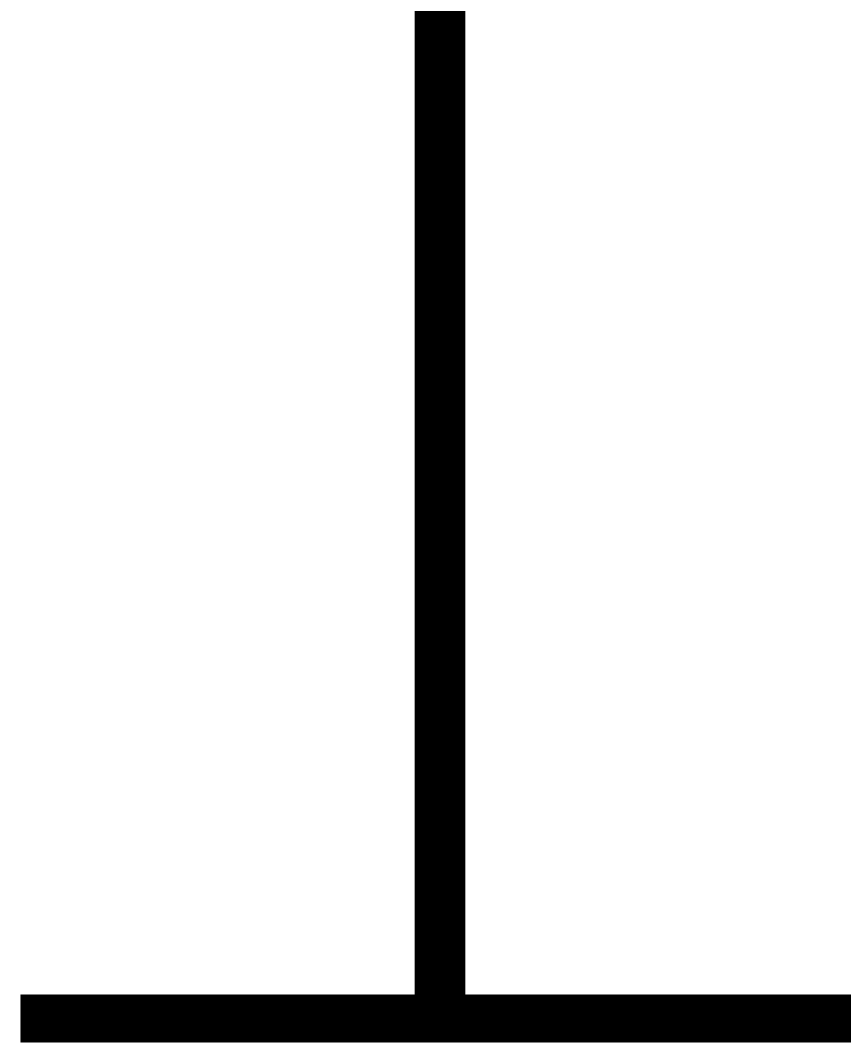
Stange B



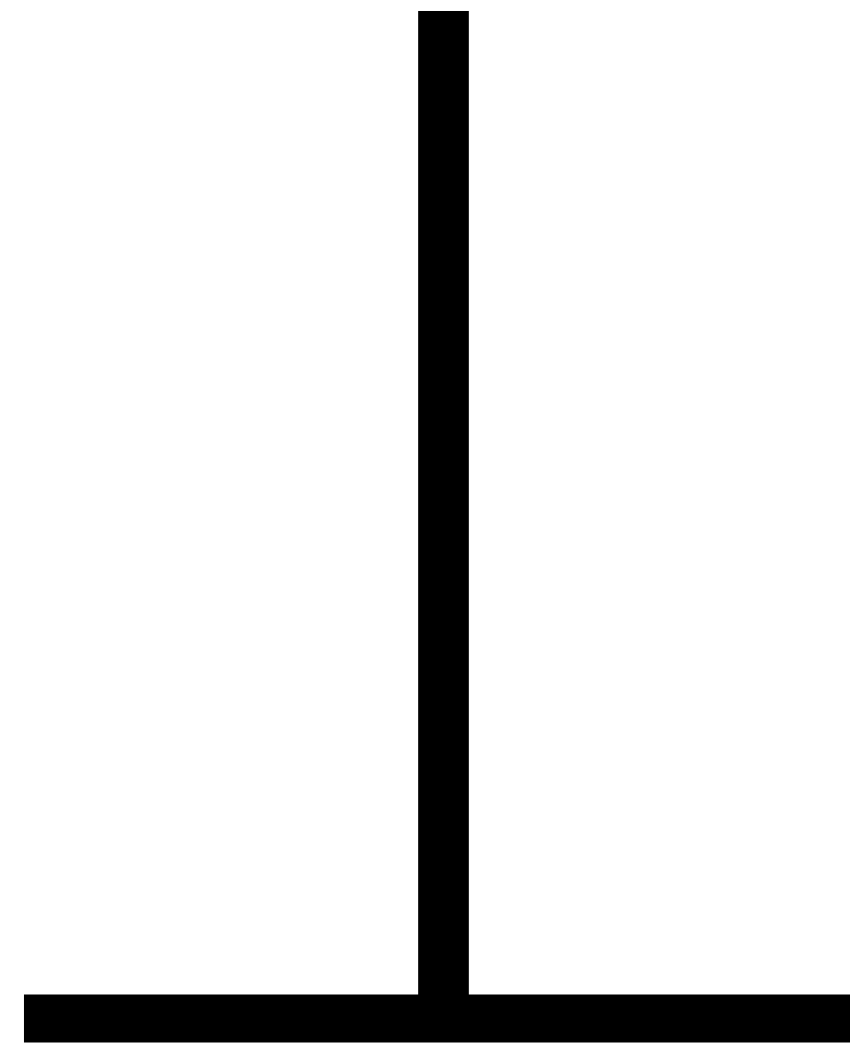
Stange C

Kleine Türme von Hanoi

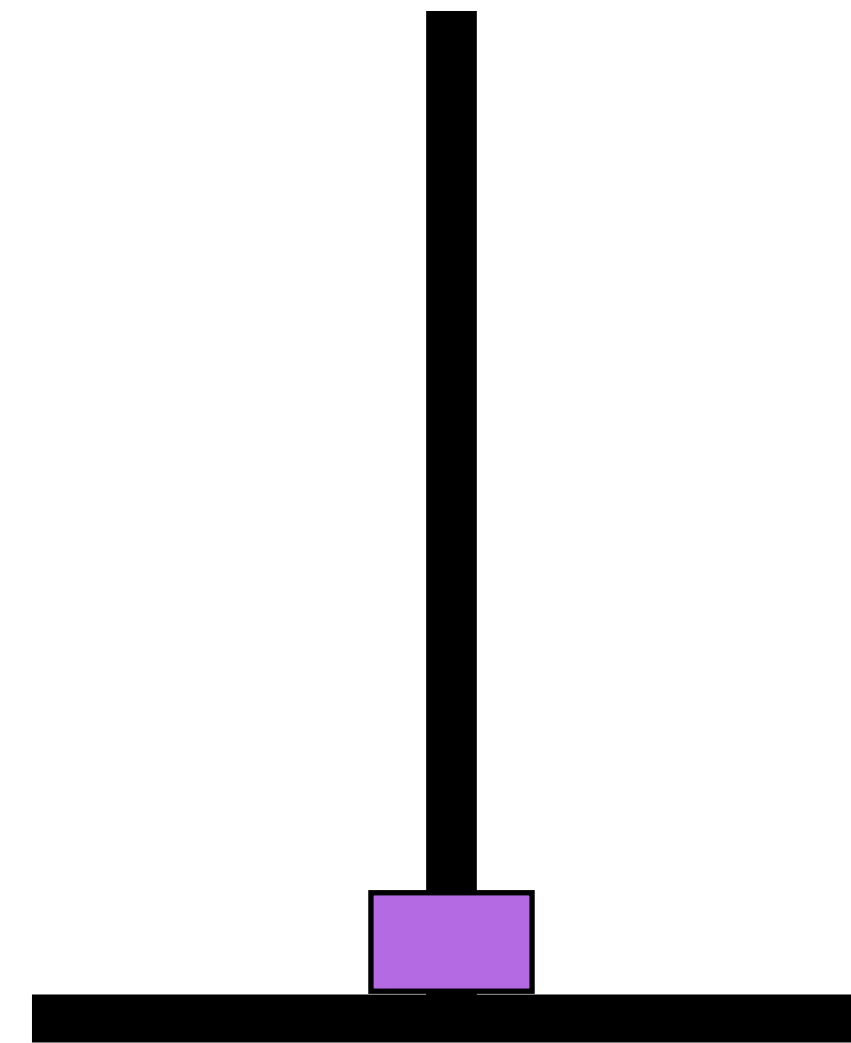
n=1



Stange A



Stange B

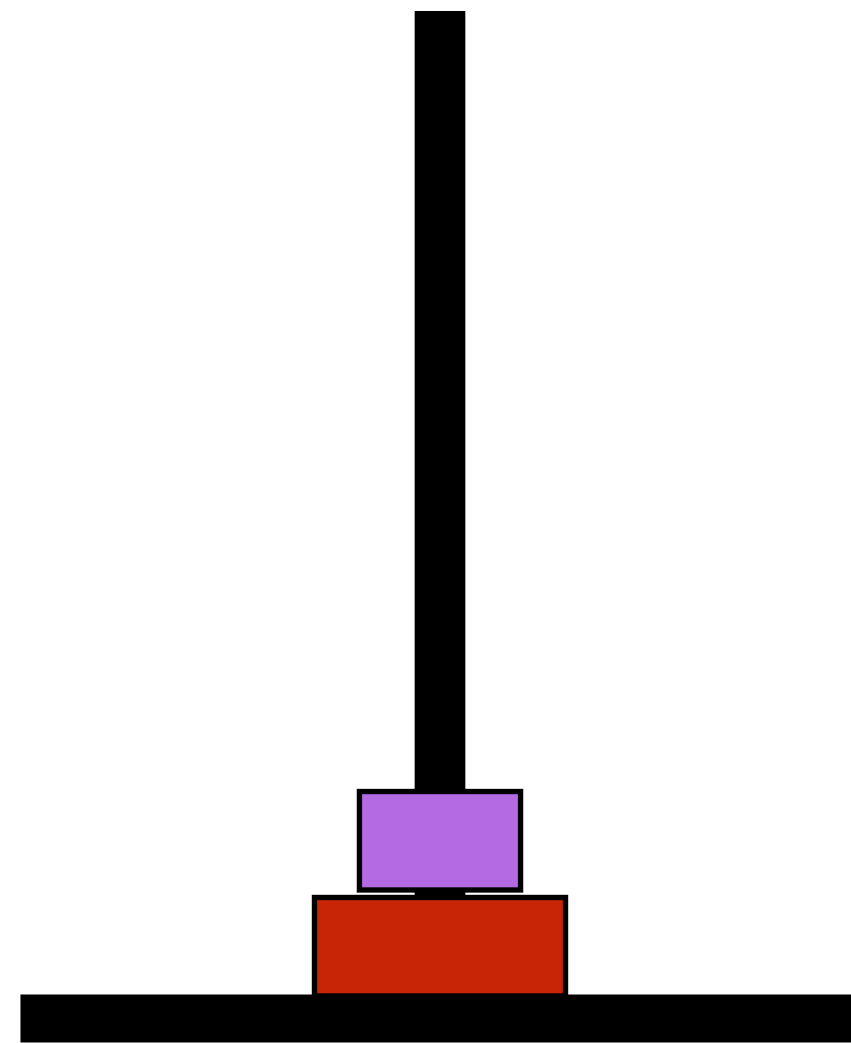


Stange C

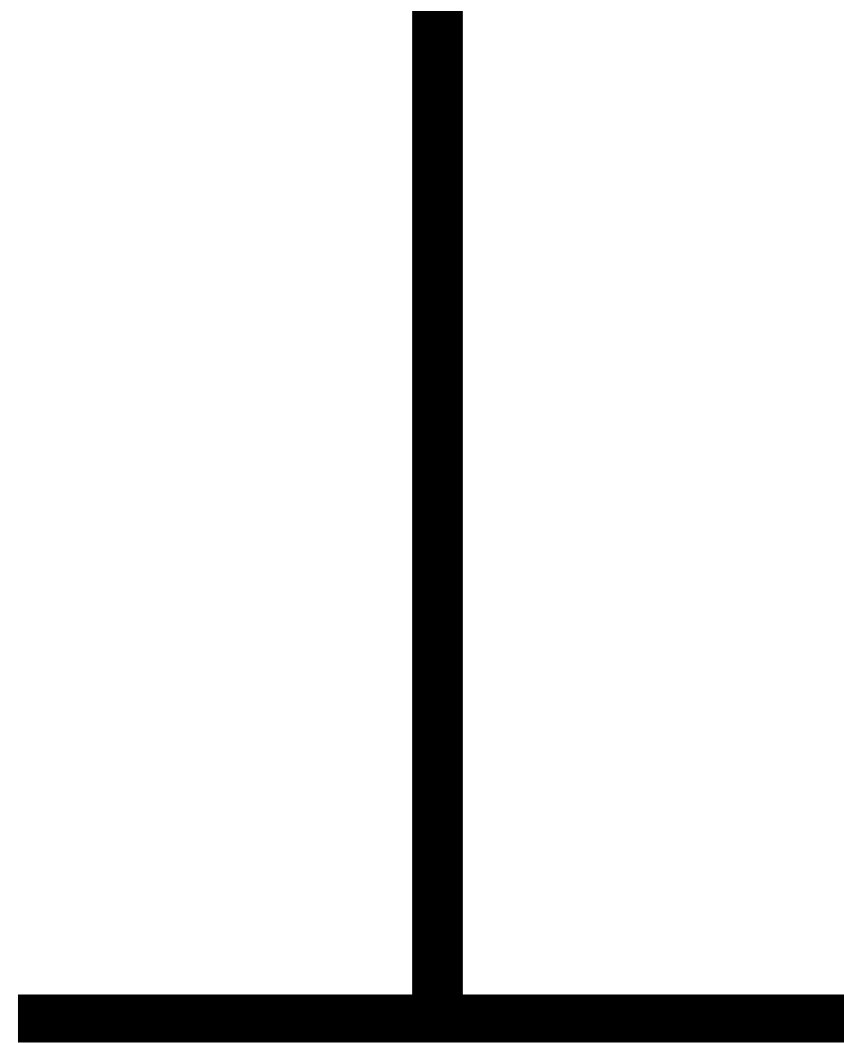


Kleine Türme von Hanoi

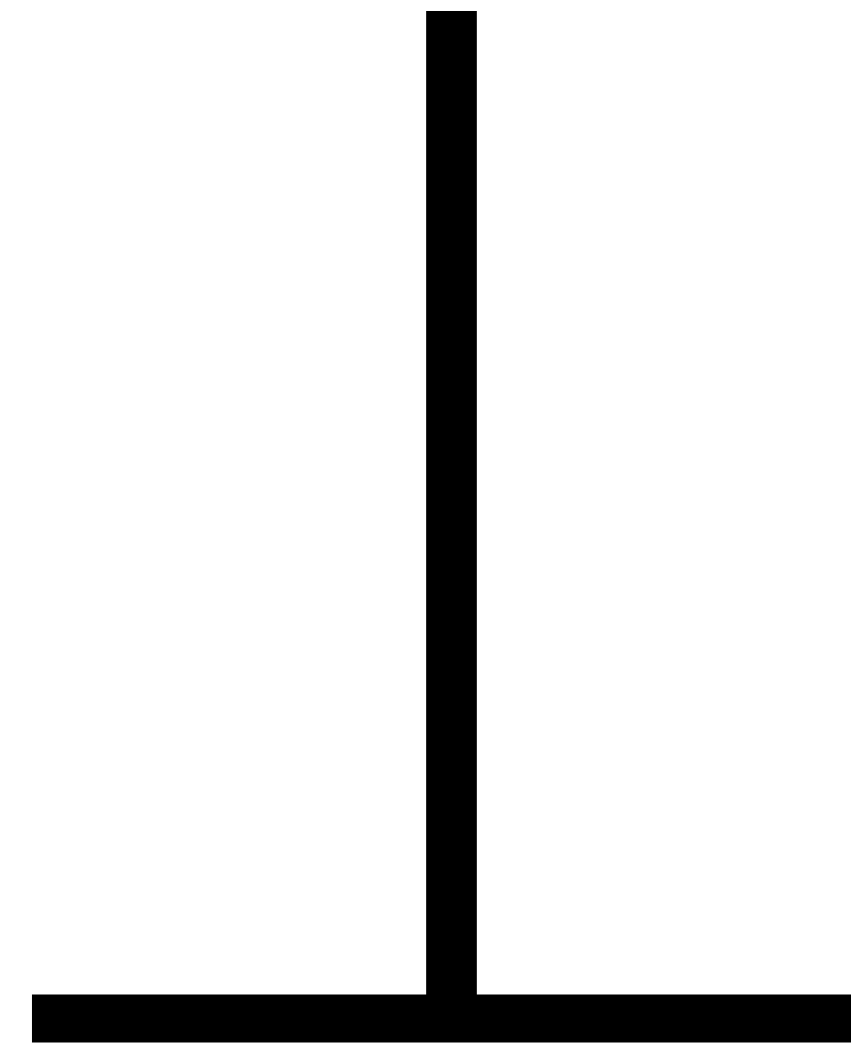
n=2



Stange A



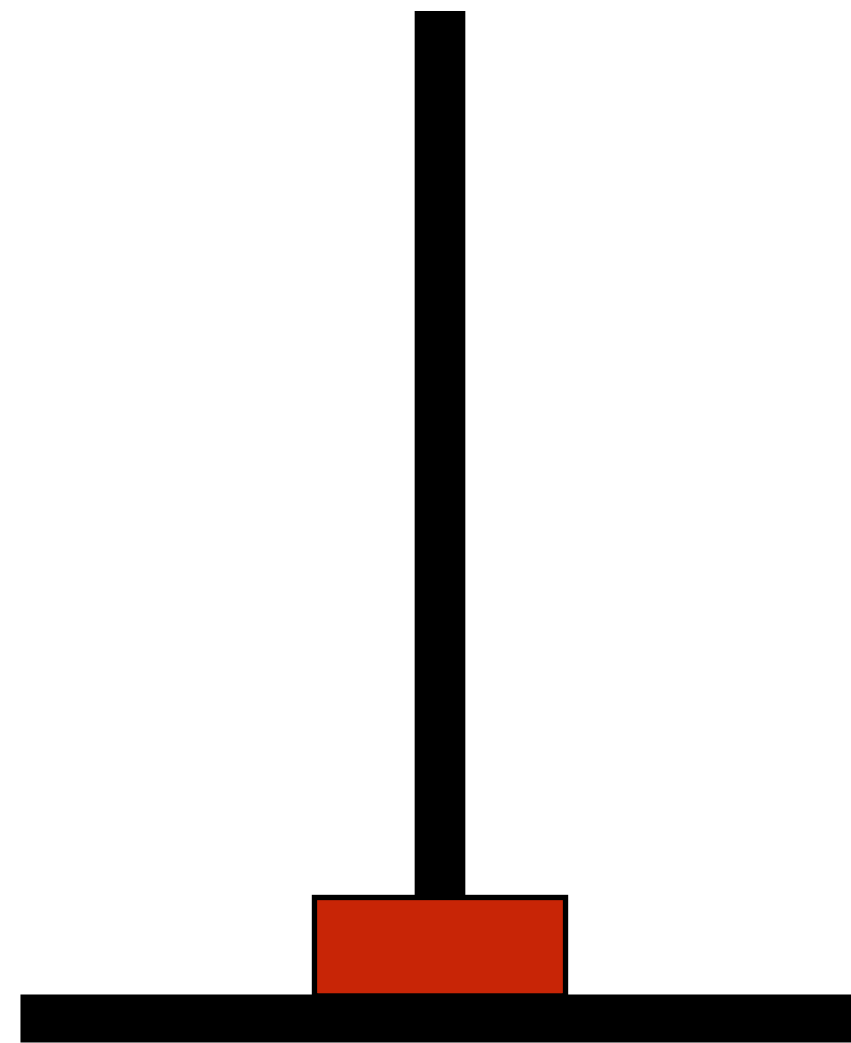
Stange B



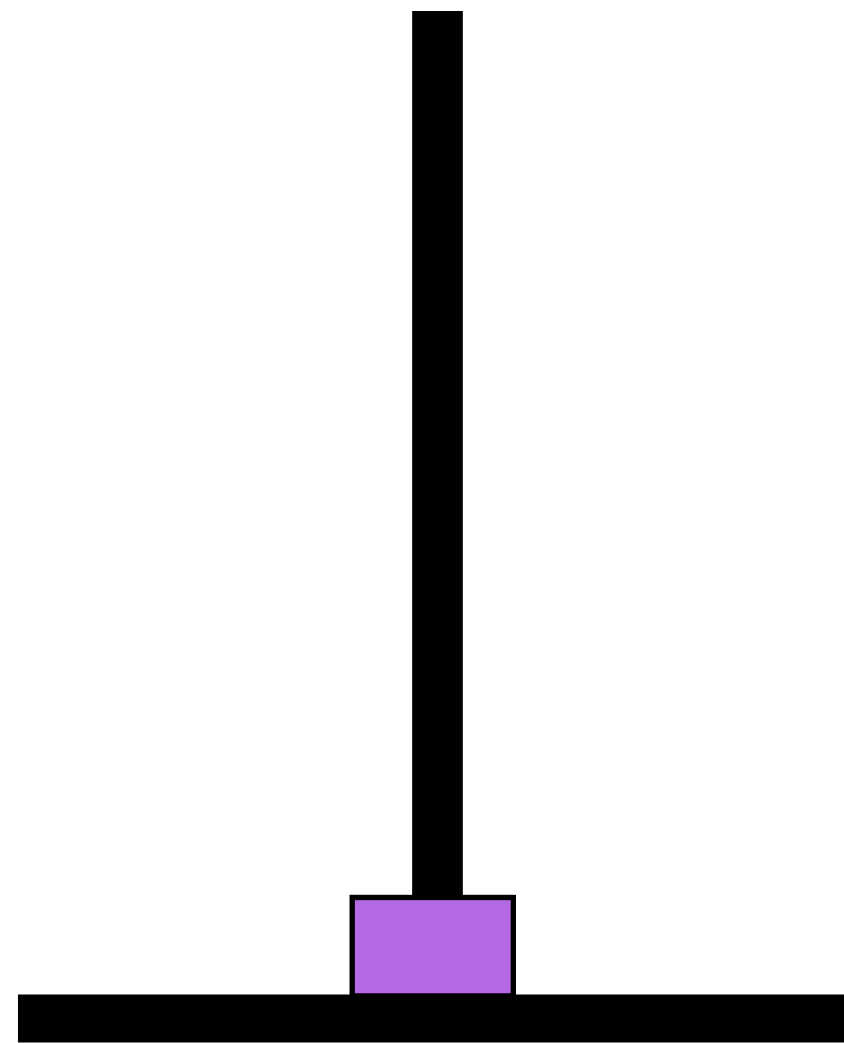
Stange C

Kleine Türme von Hanoi

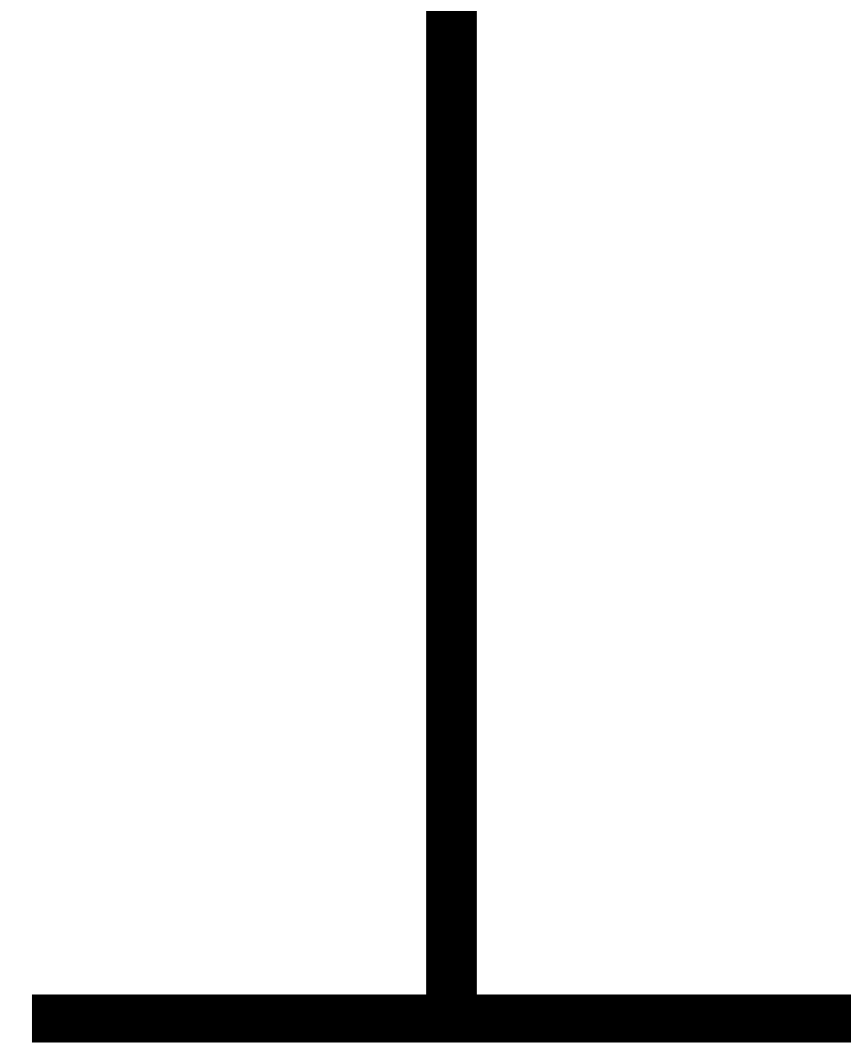
n=2



Stange A



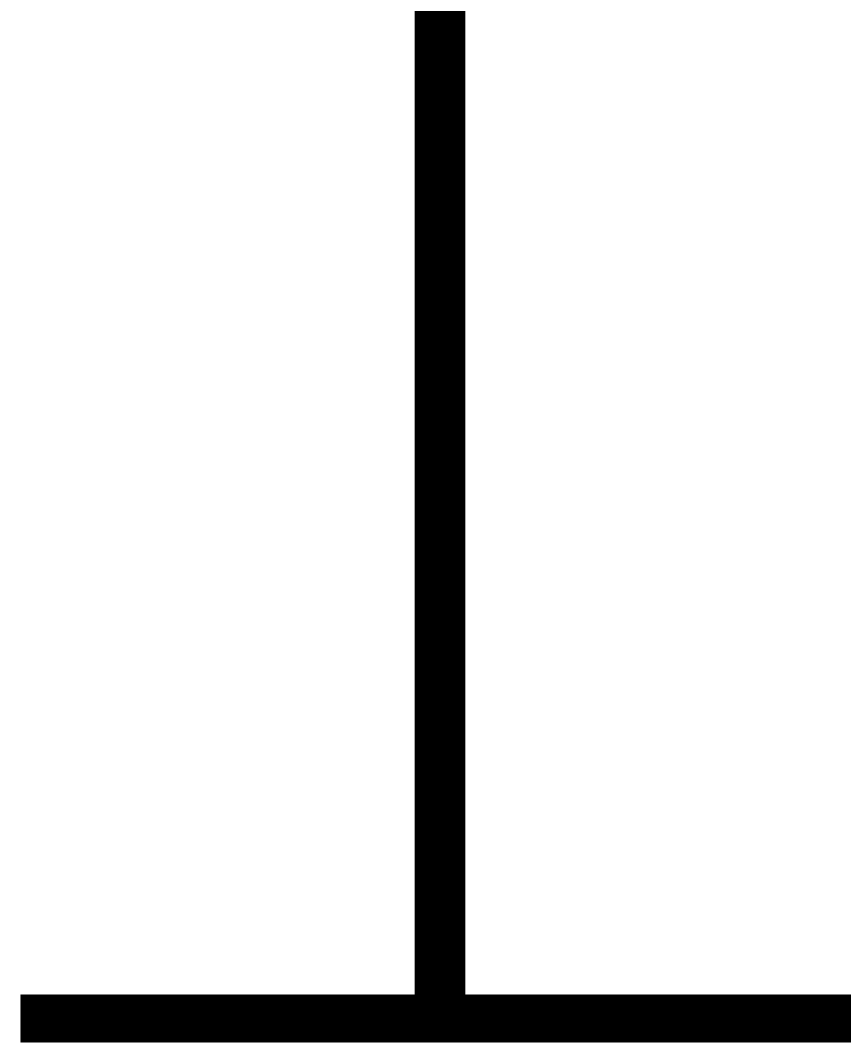
Stange B



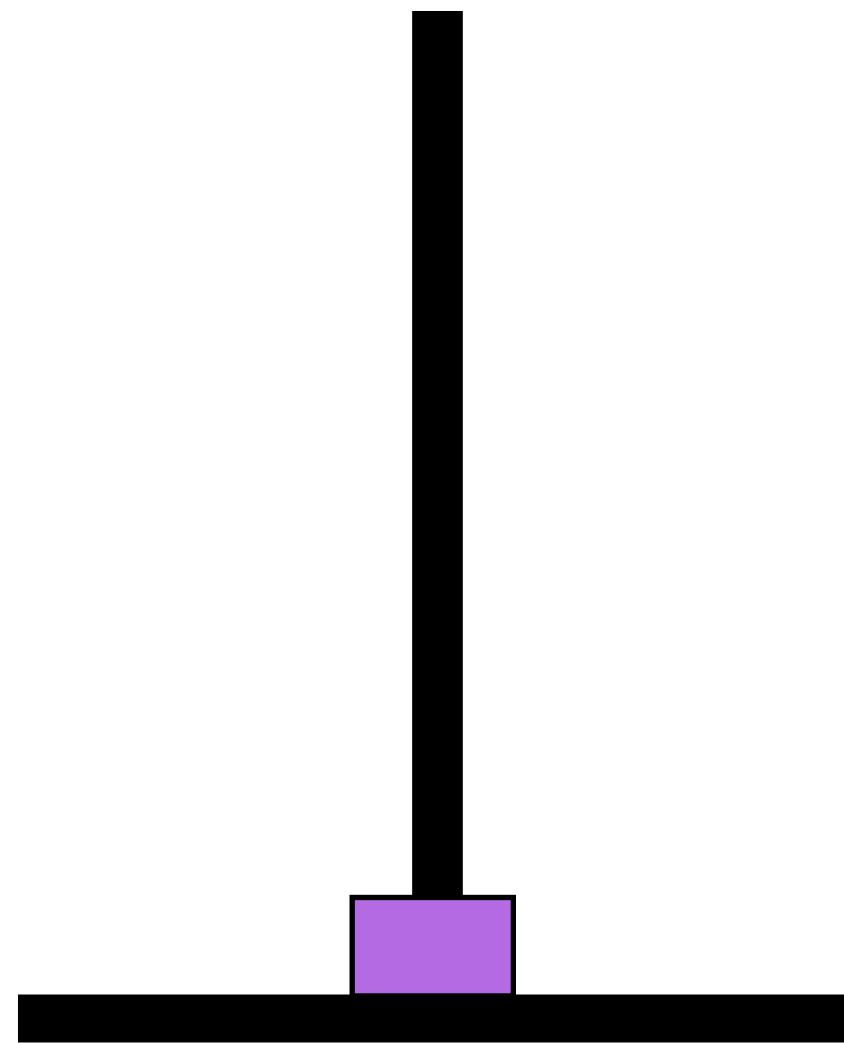
Stange C

Kleine Türme von Hanoi

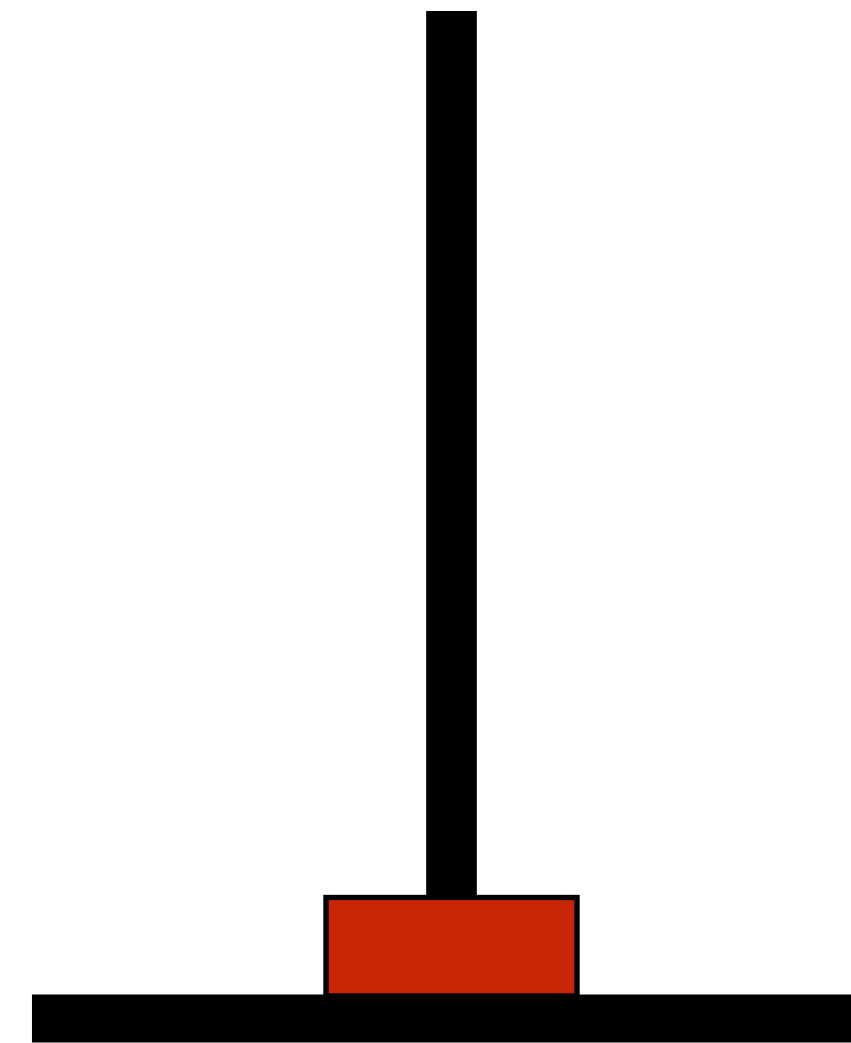
n=2



Stange A



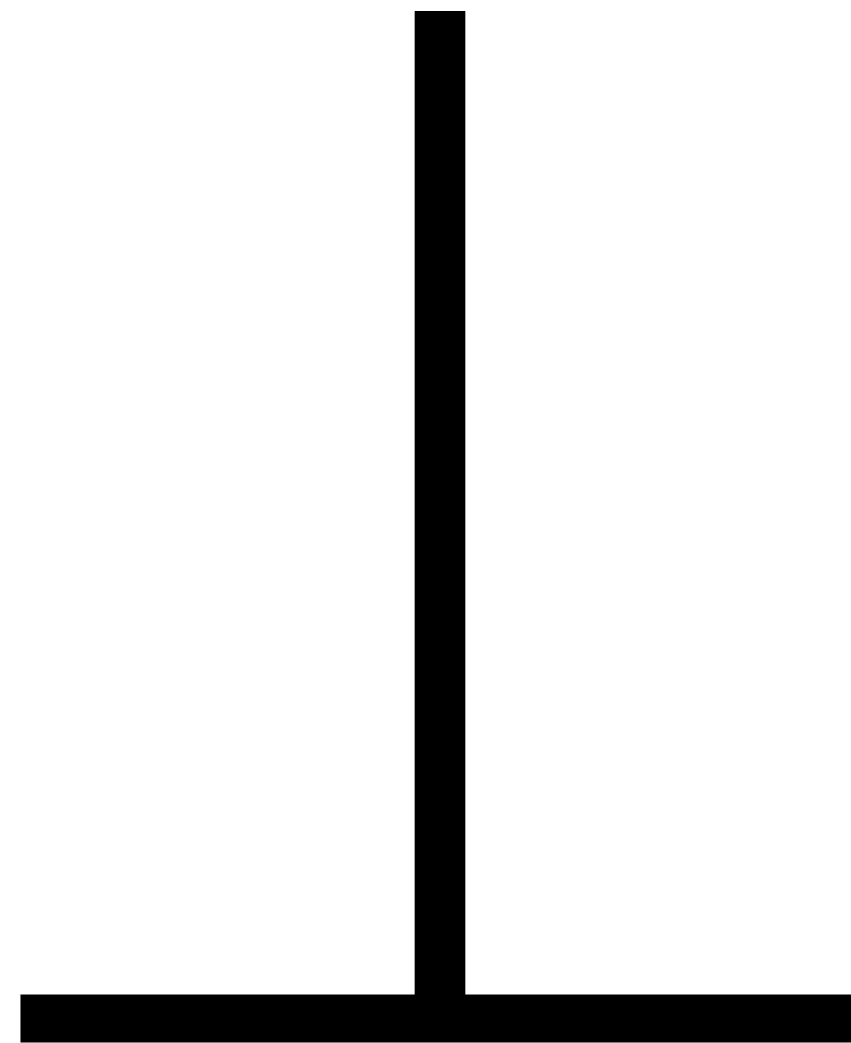
Stange B



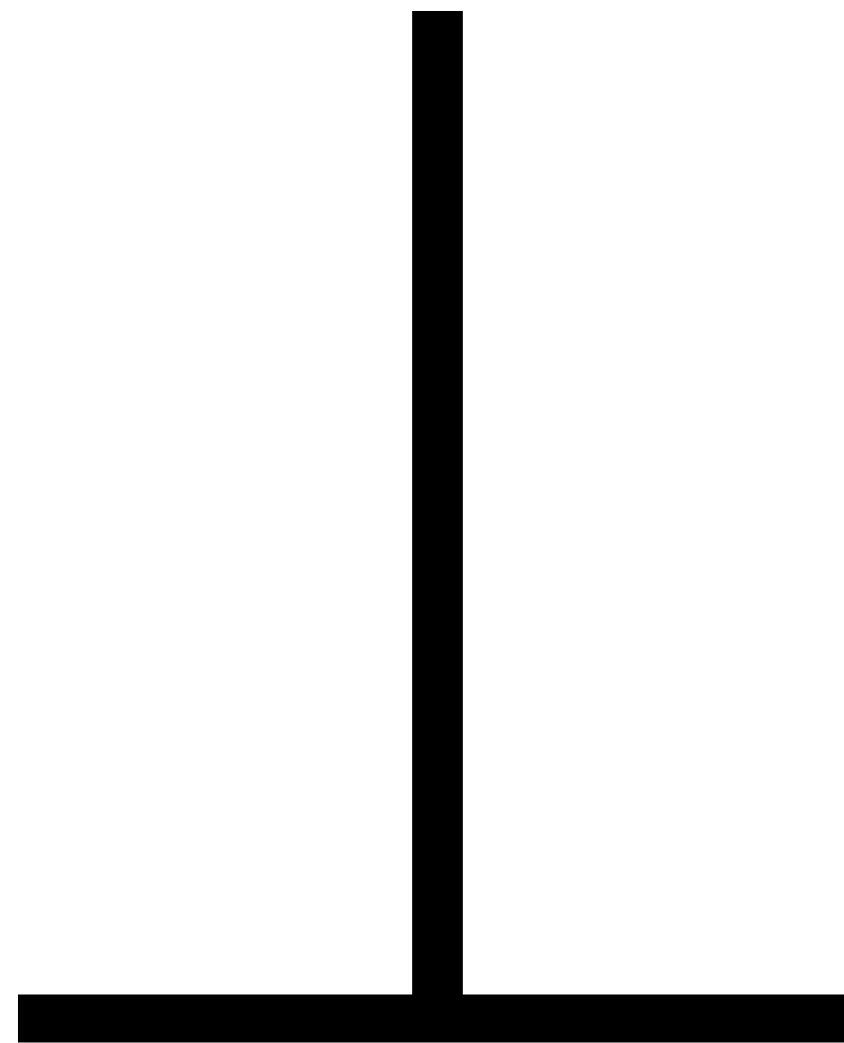
Stange C

Kleine Türme von Hanoi

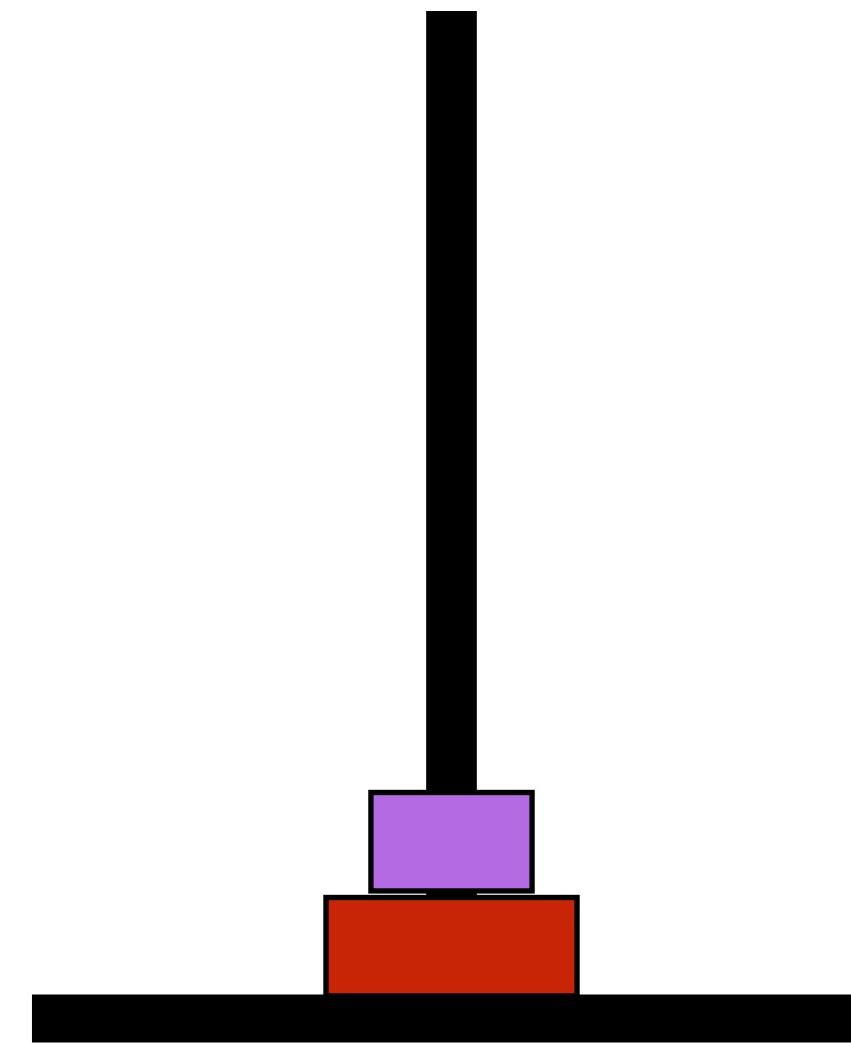
n=2



Stange A



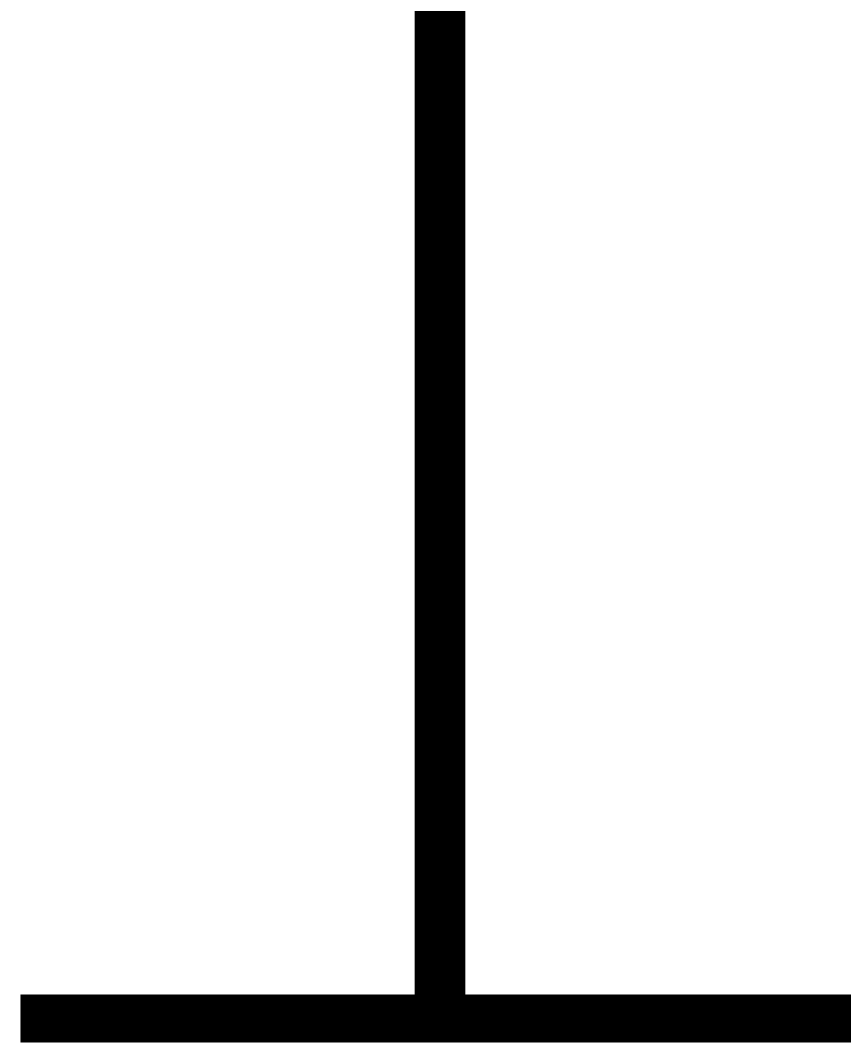
Stange B



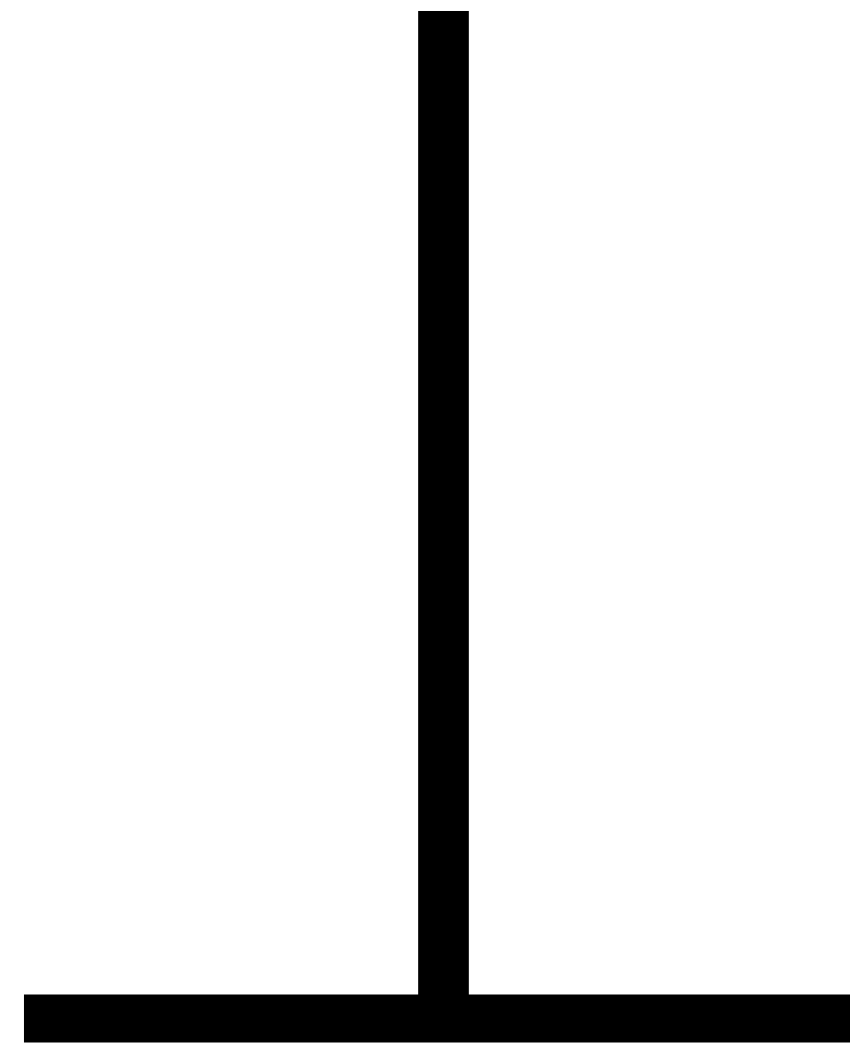
Stange C

Kleine Türme von Hanoi

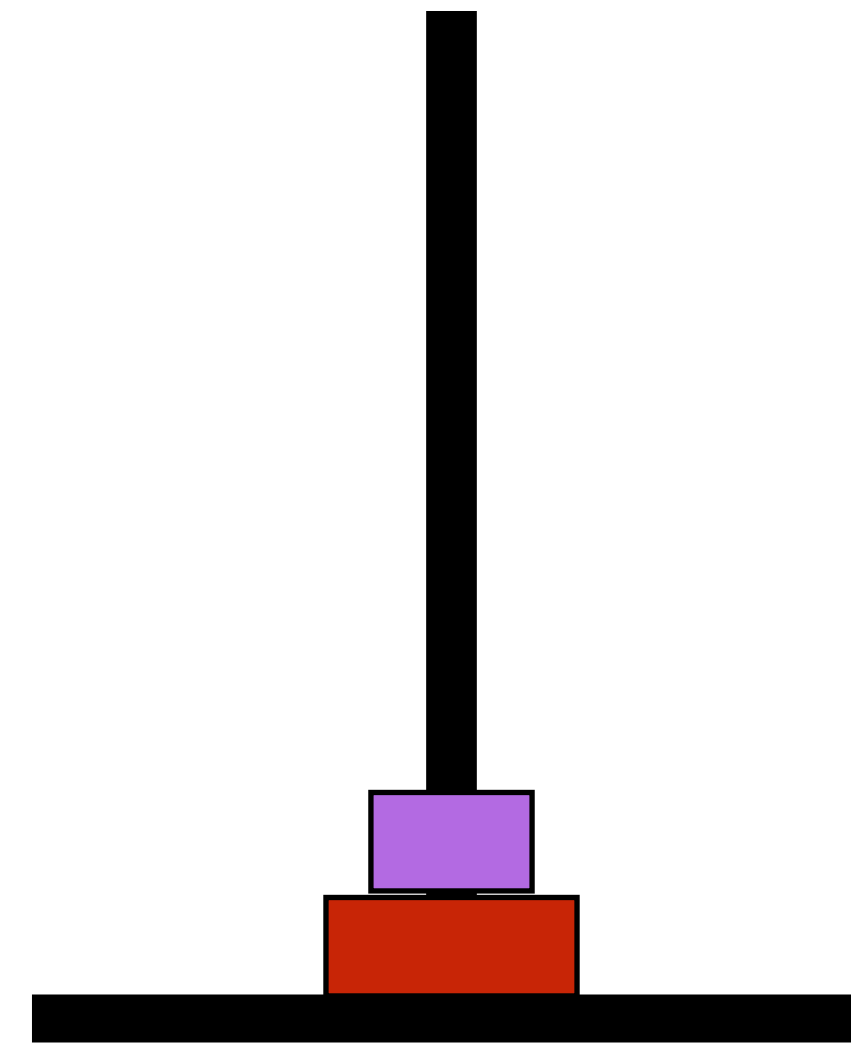
n=2



Stange A



Stange B

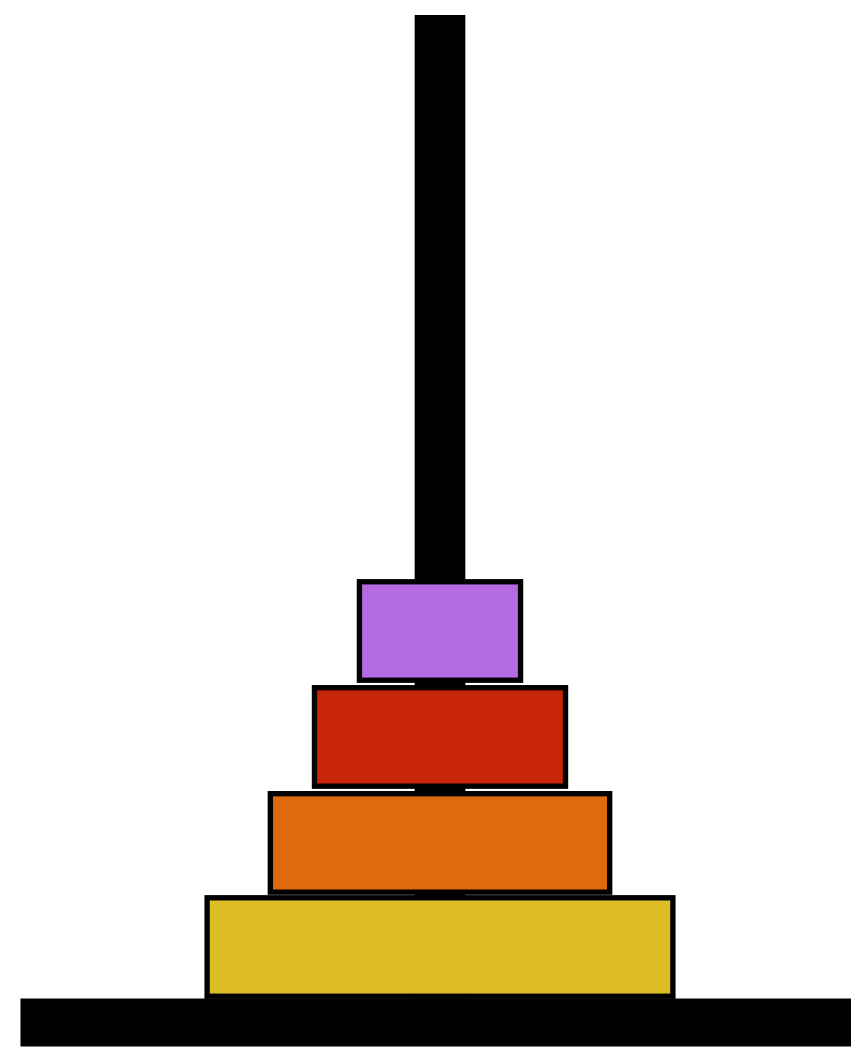


Stange C

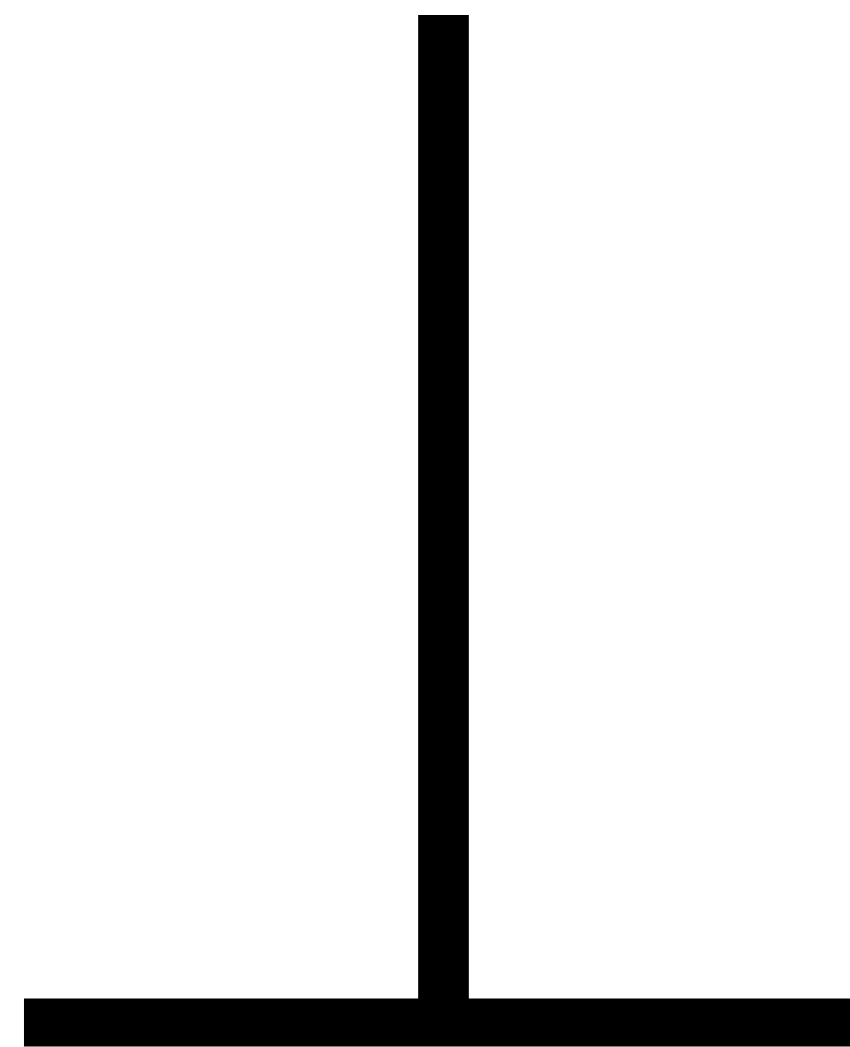


Kleine Türme von Hanoi

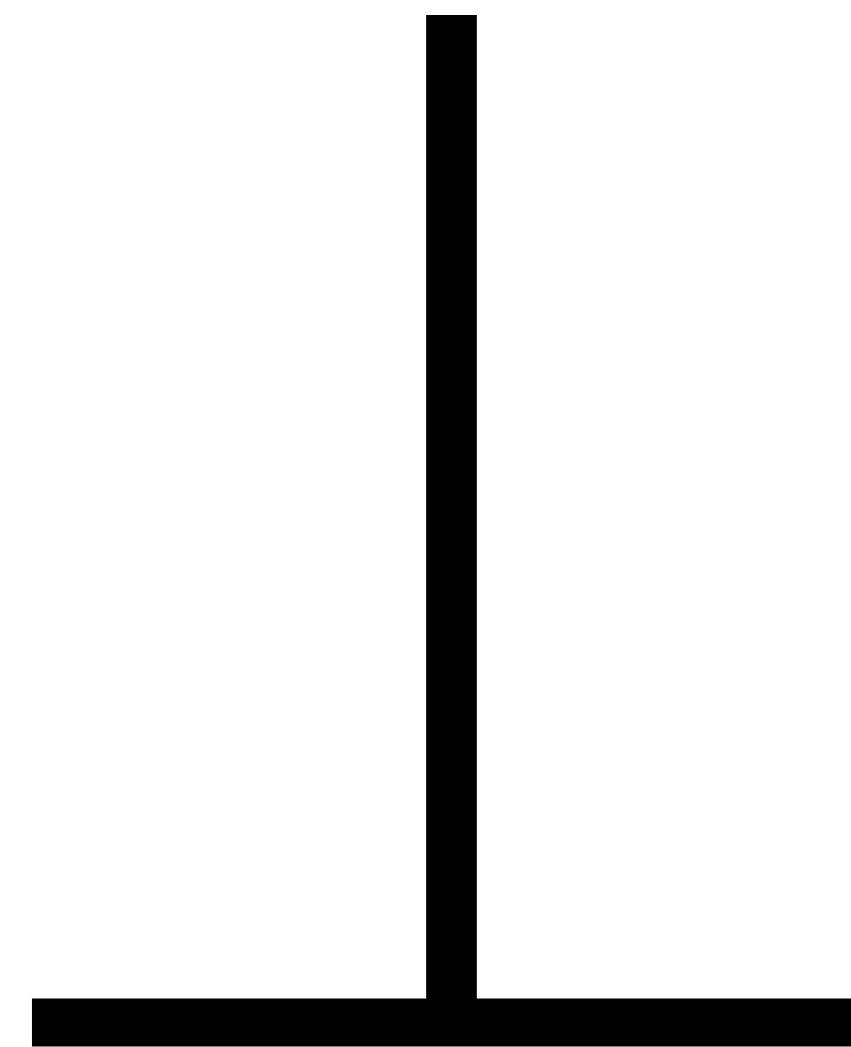
n=4



Stange A



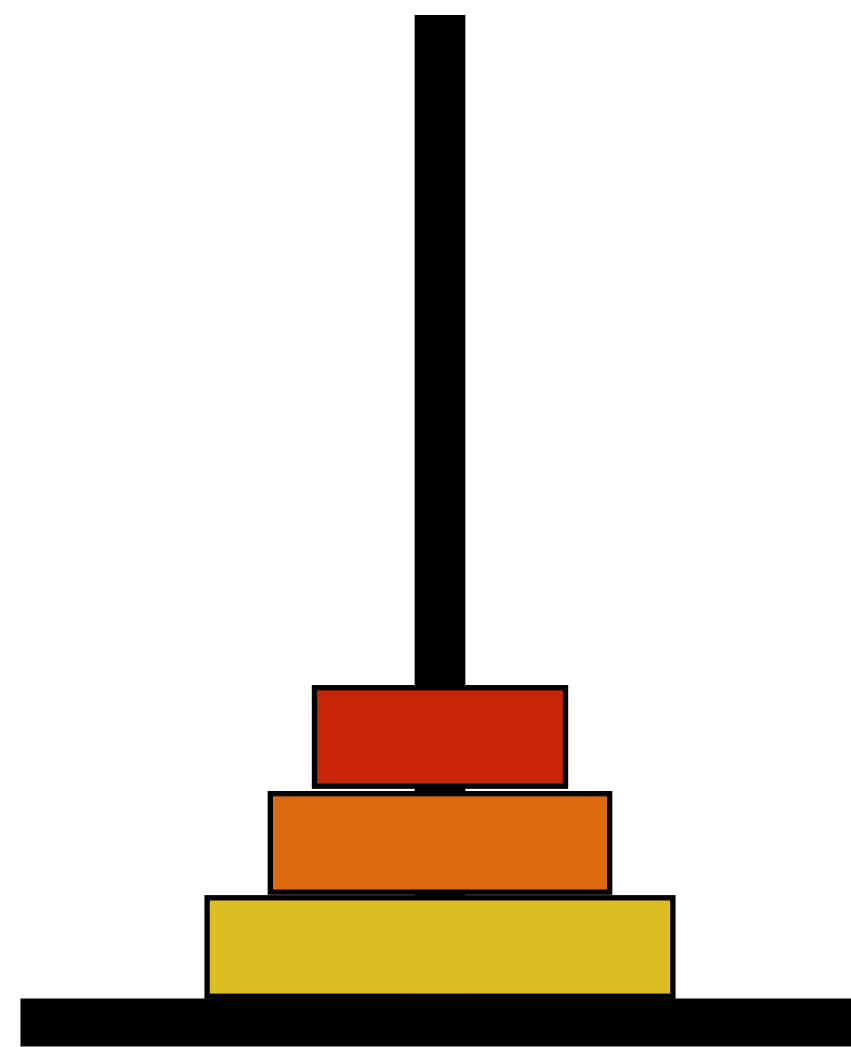
Stange B



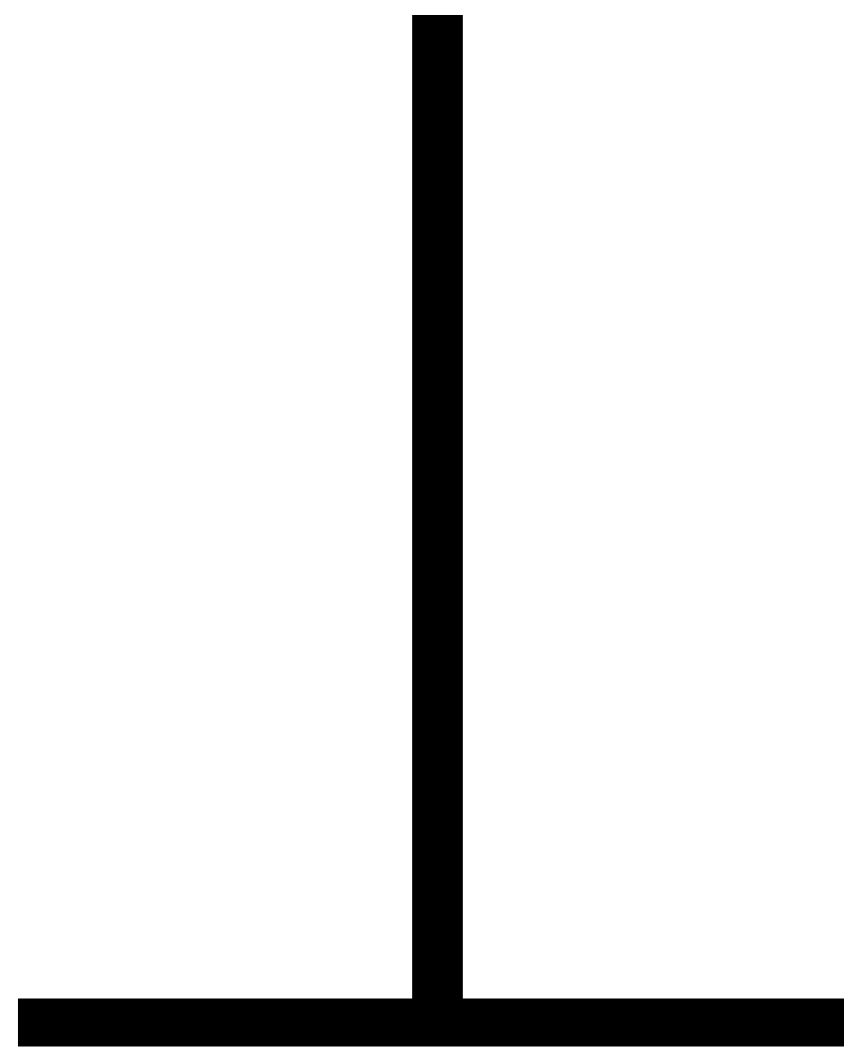
Stange C

Kleine Türme von Hanoi

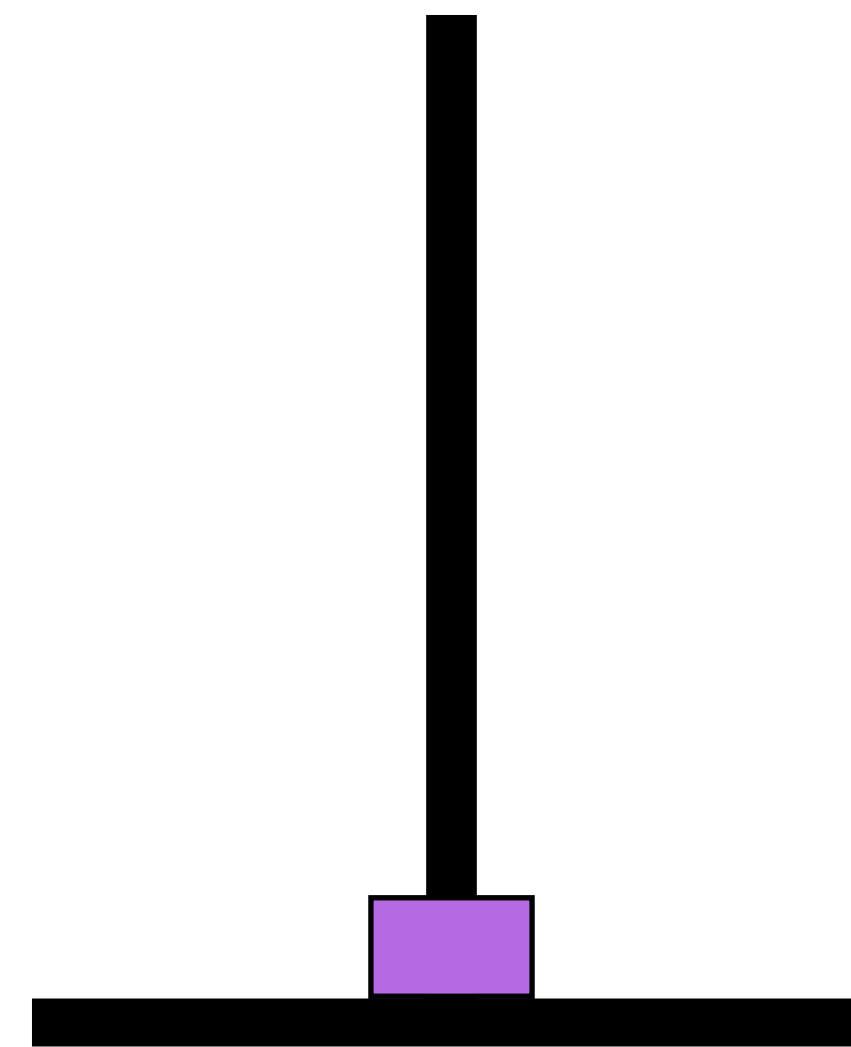
n=4



Stange A



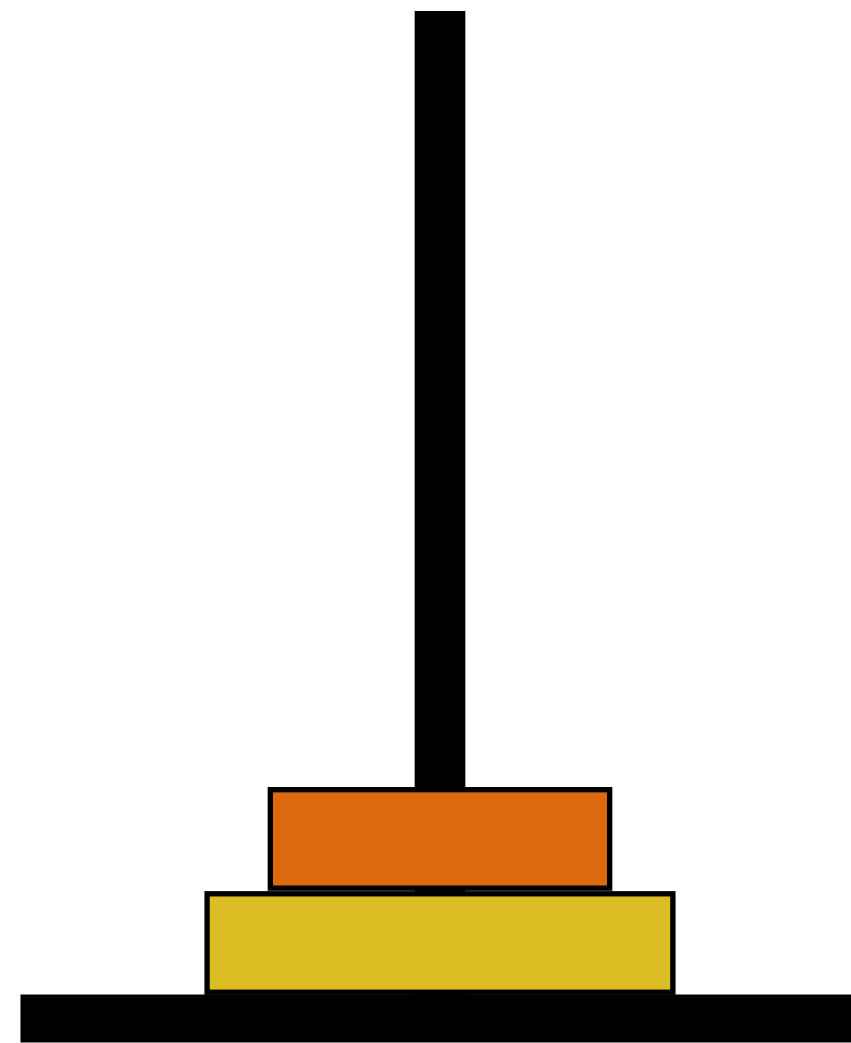
Stange B



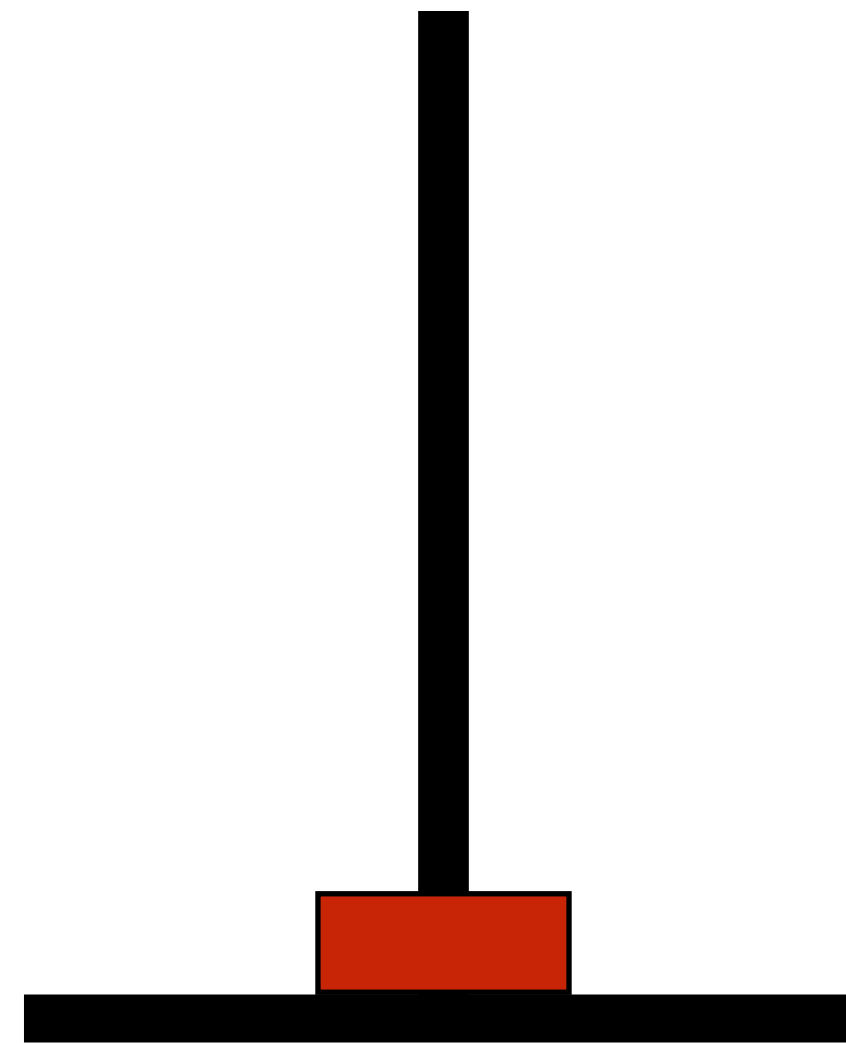
Stange C

Kleine Türme von Hanoi

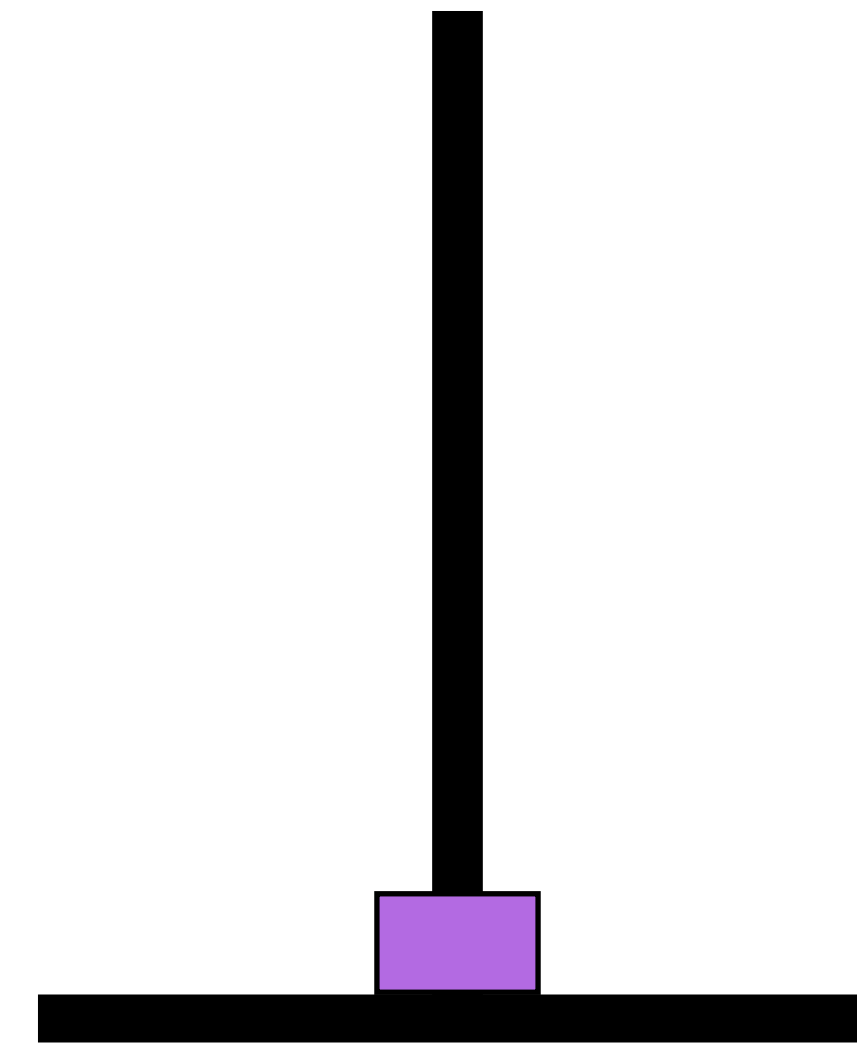
n=4



Stange A



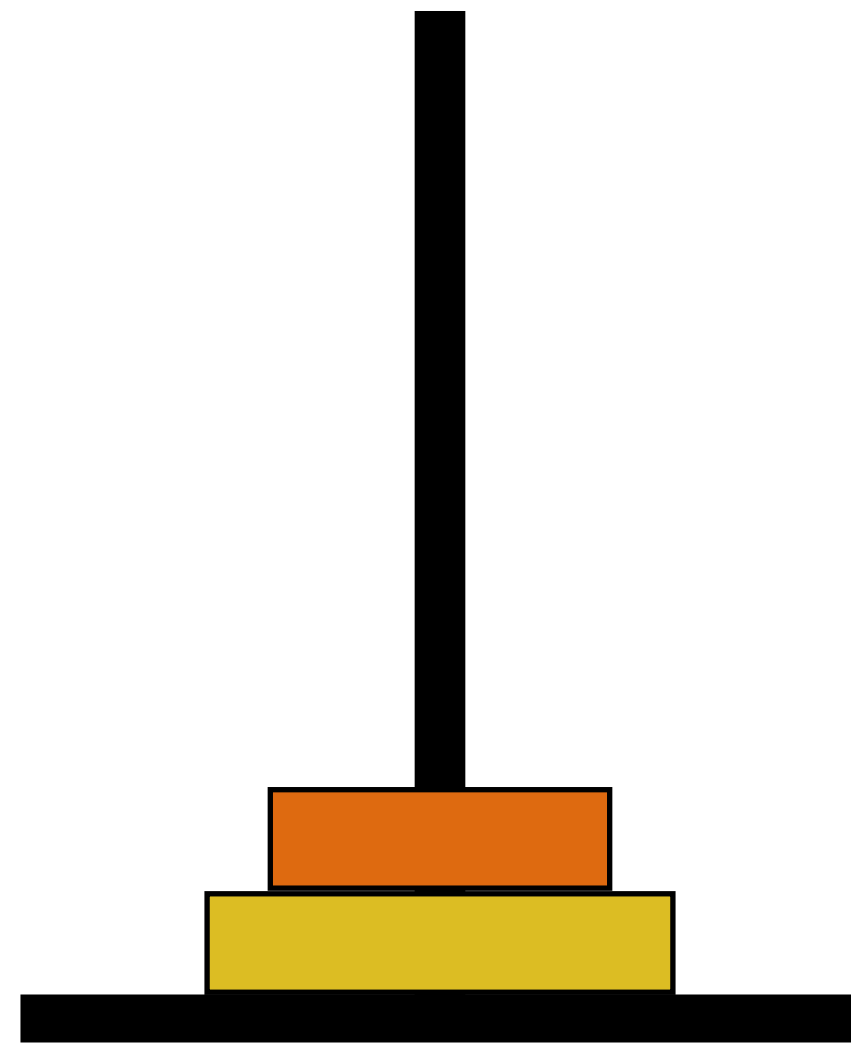
Stange B



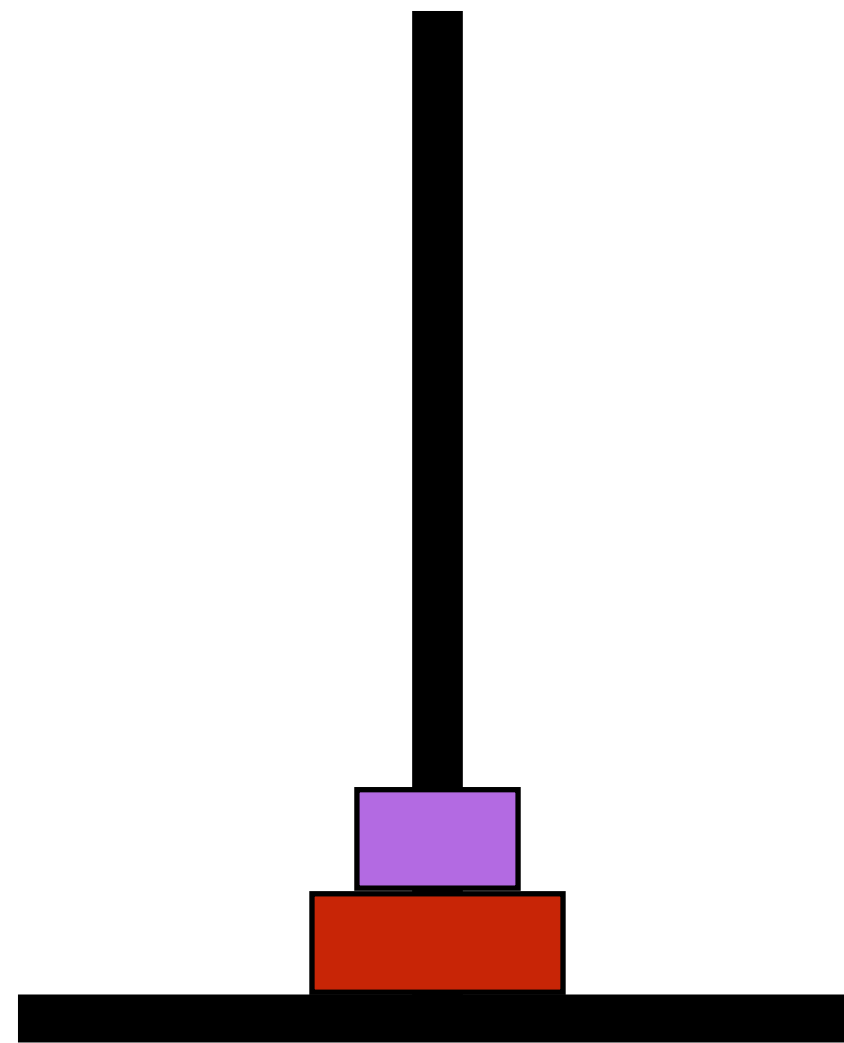
Stange C

Kleine Türme von Hanoi

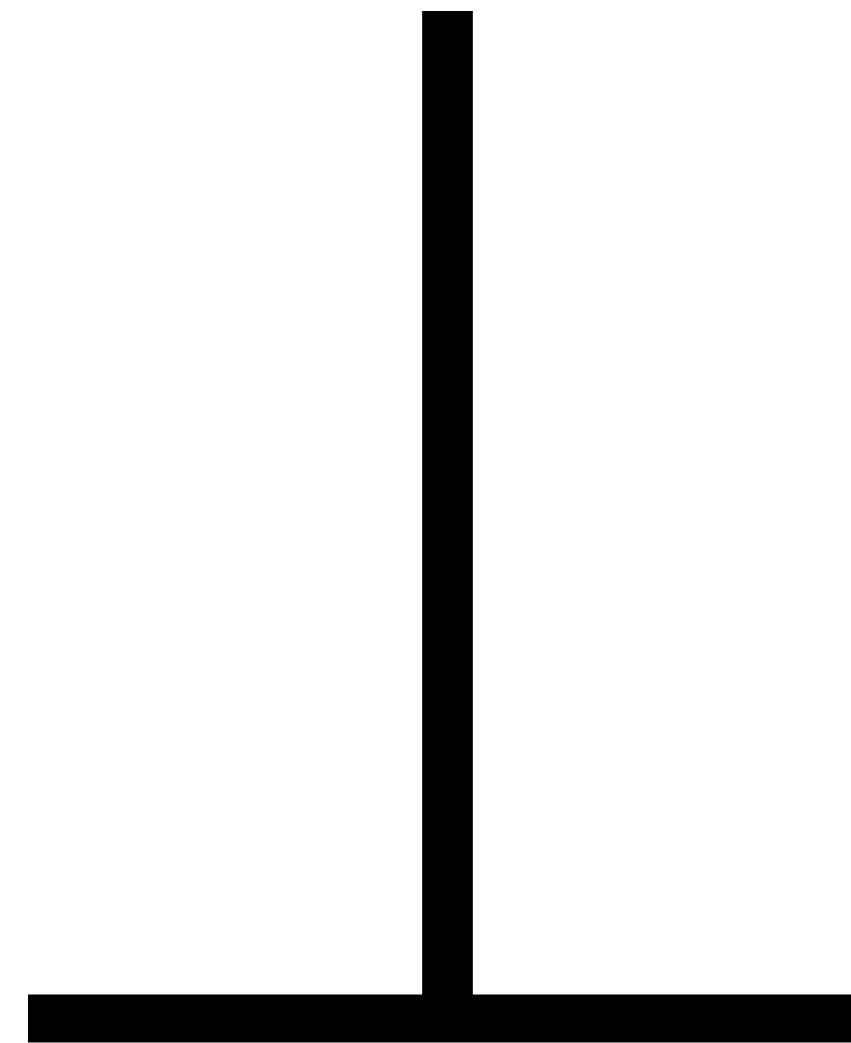
n=4



Stange A



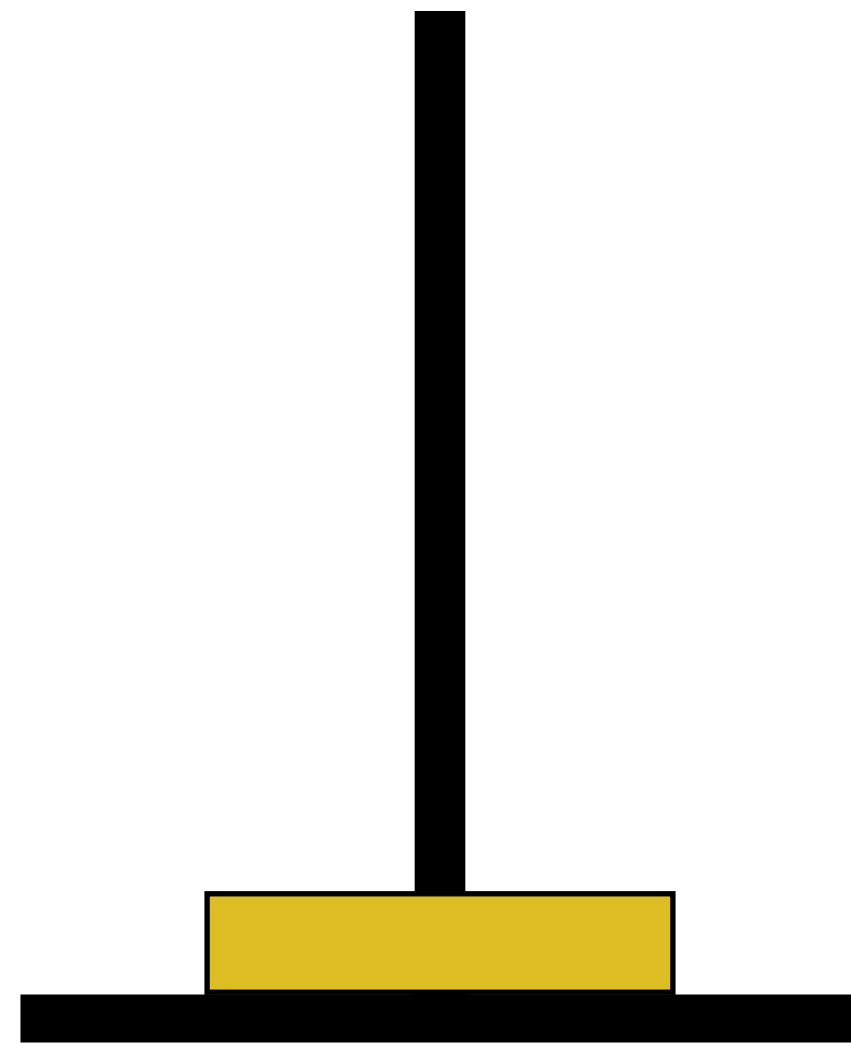
Stange B



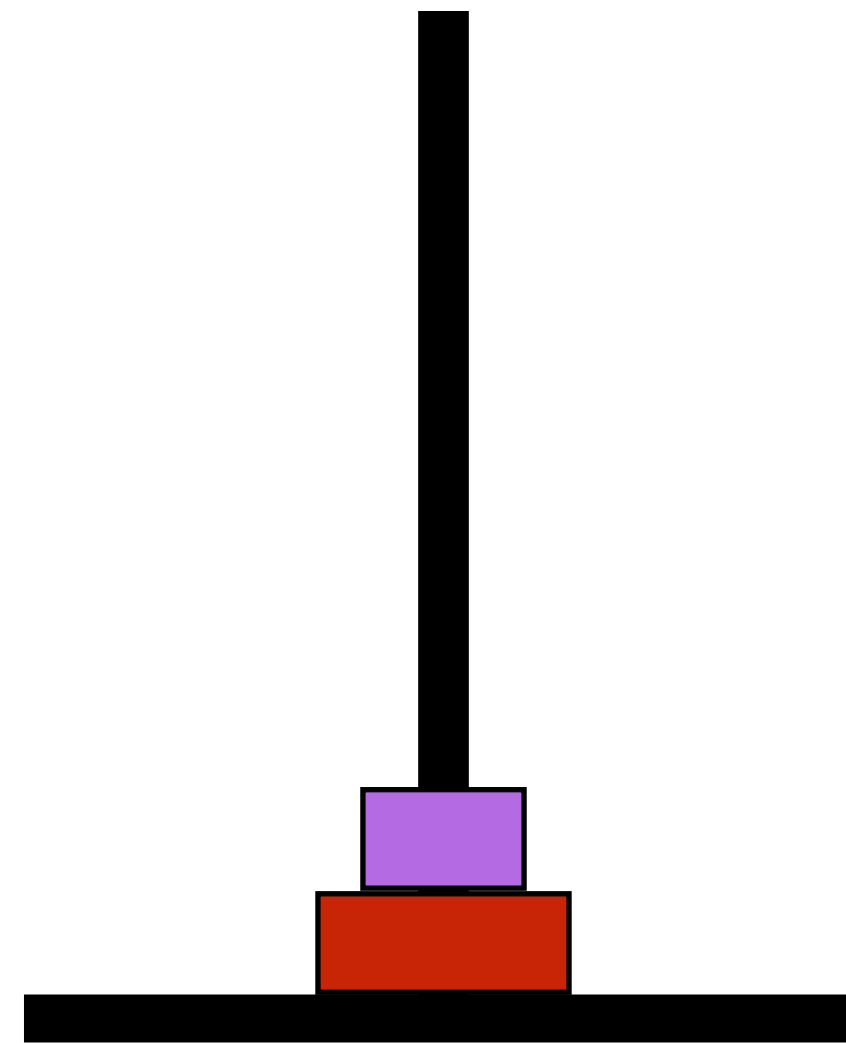
Stange C

Kleine Türme von Hanoi

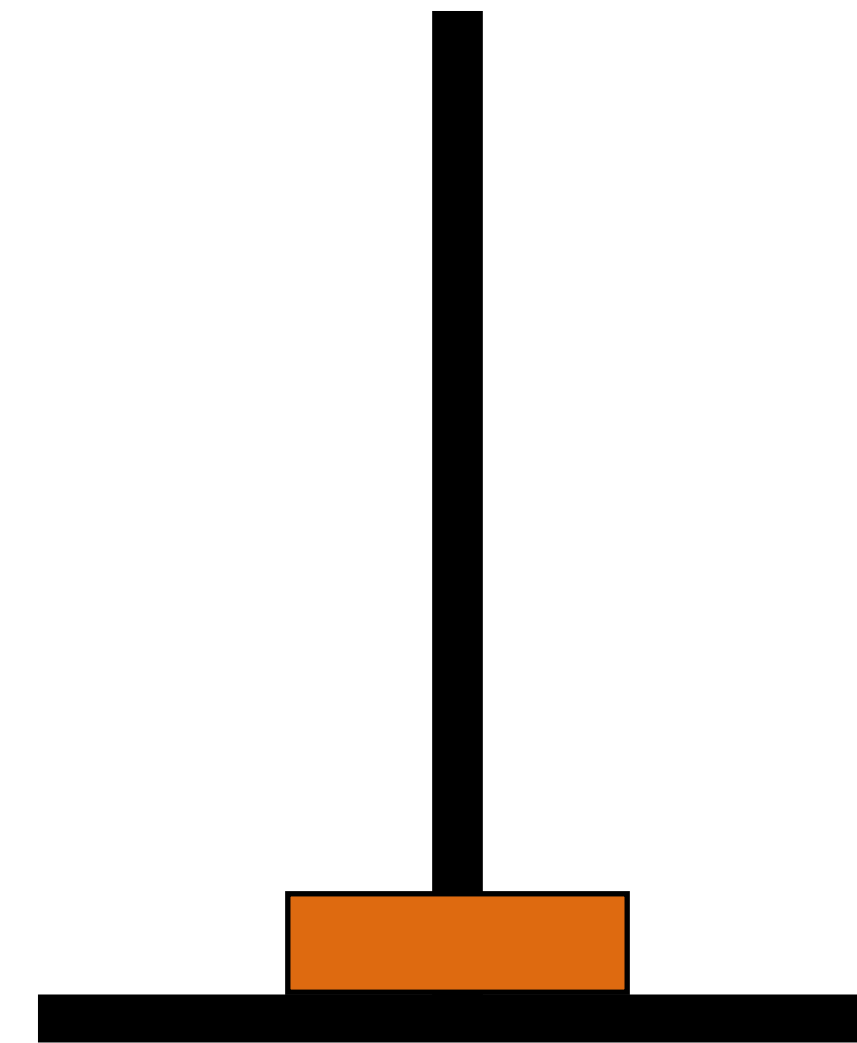
n=4



Stange A



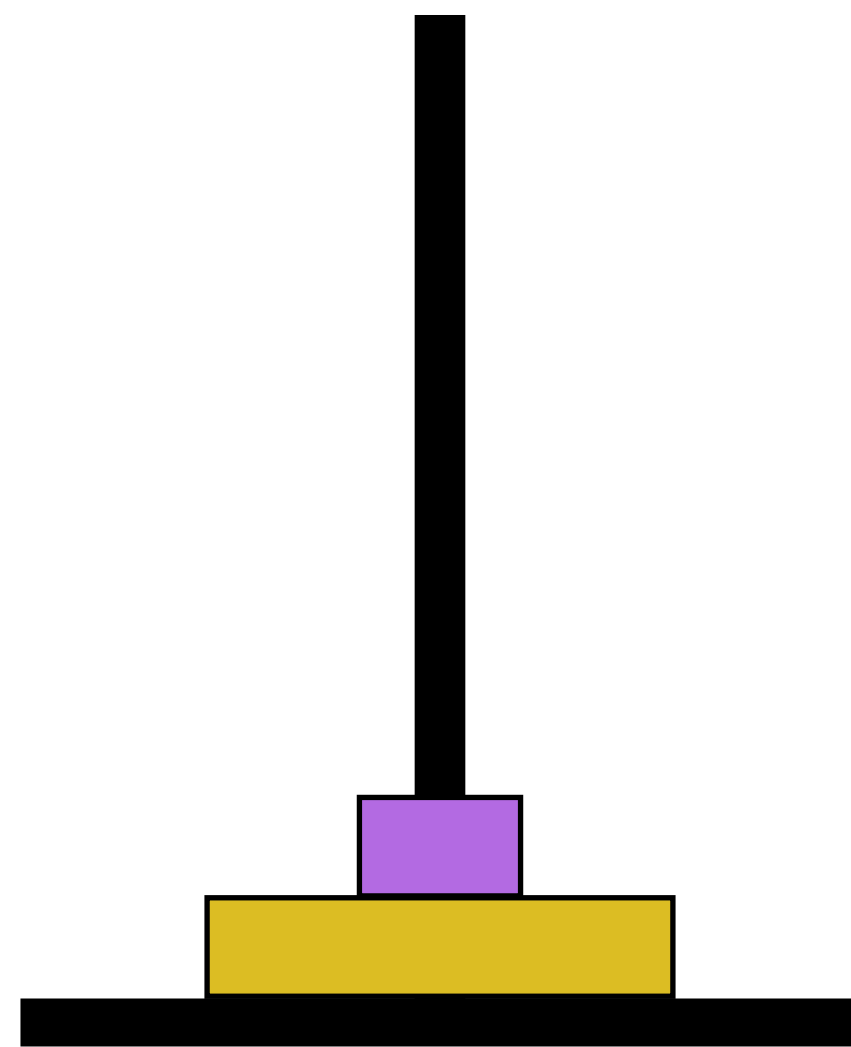
Stange B



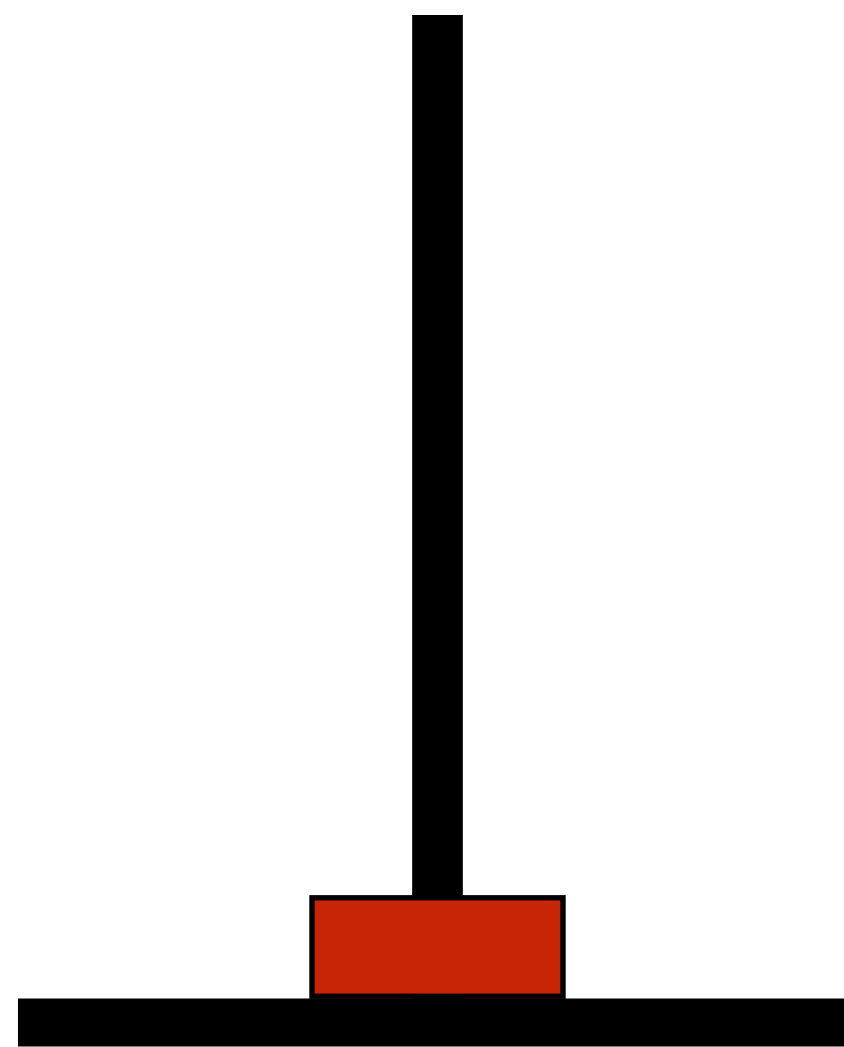
Stange C

Kleine Türme von Hanoi

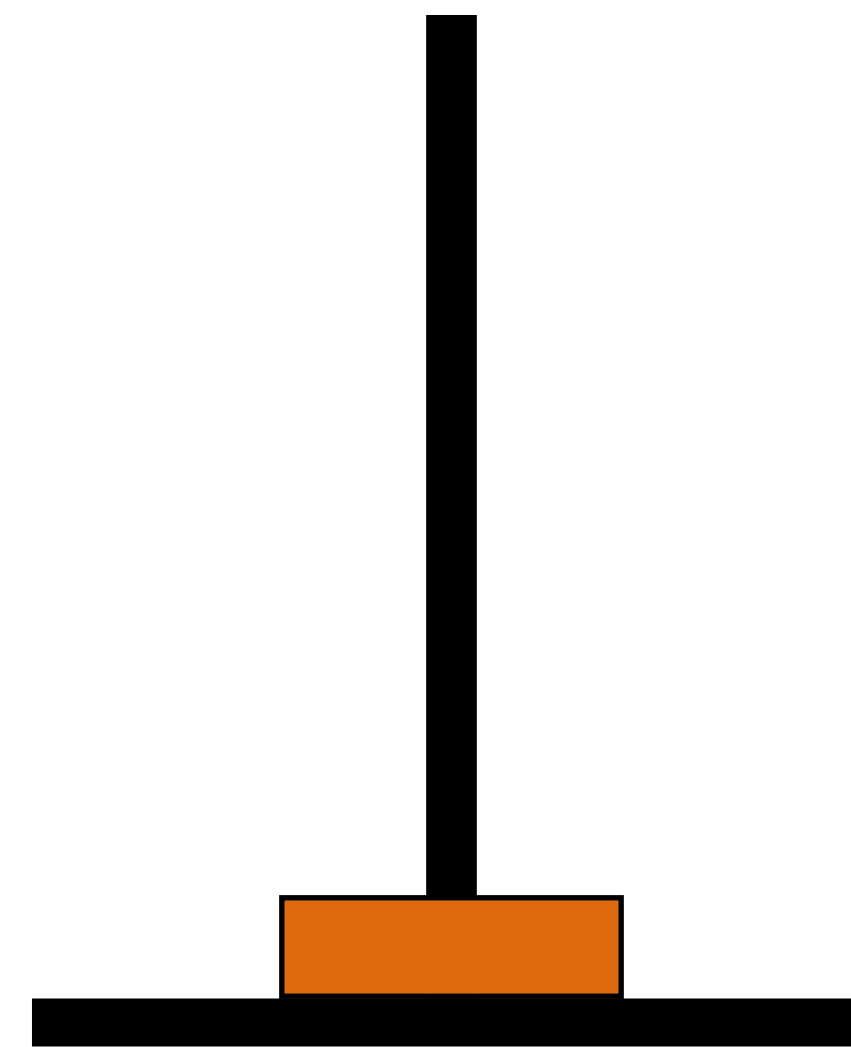
n=4



Stange A



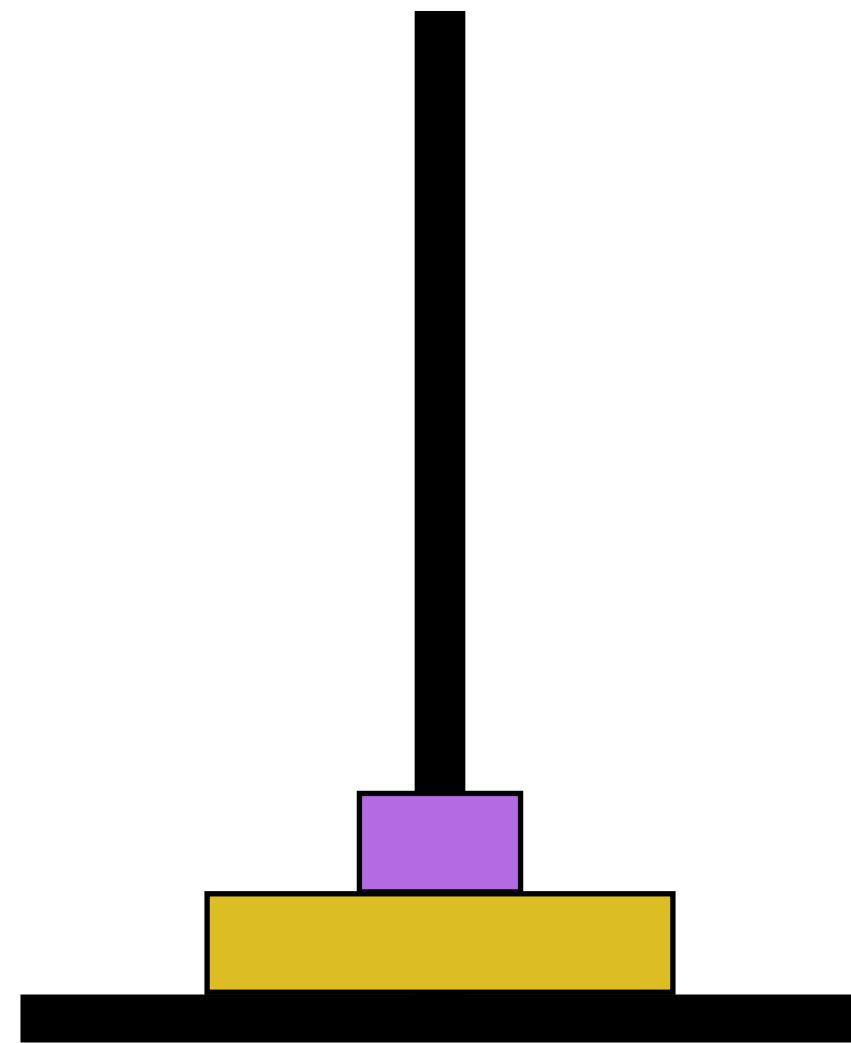
Stange B



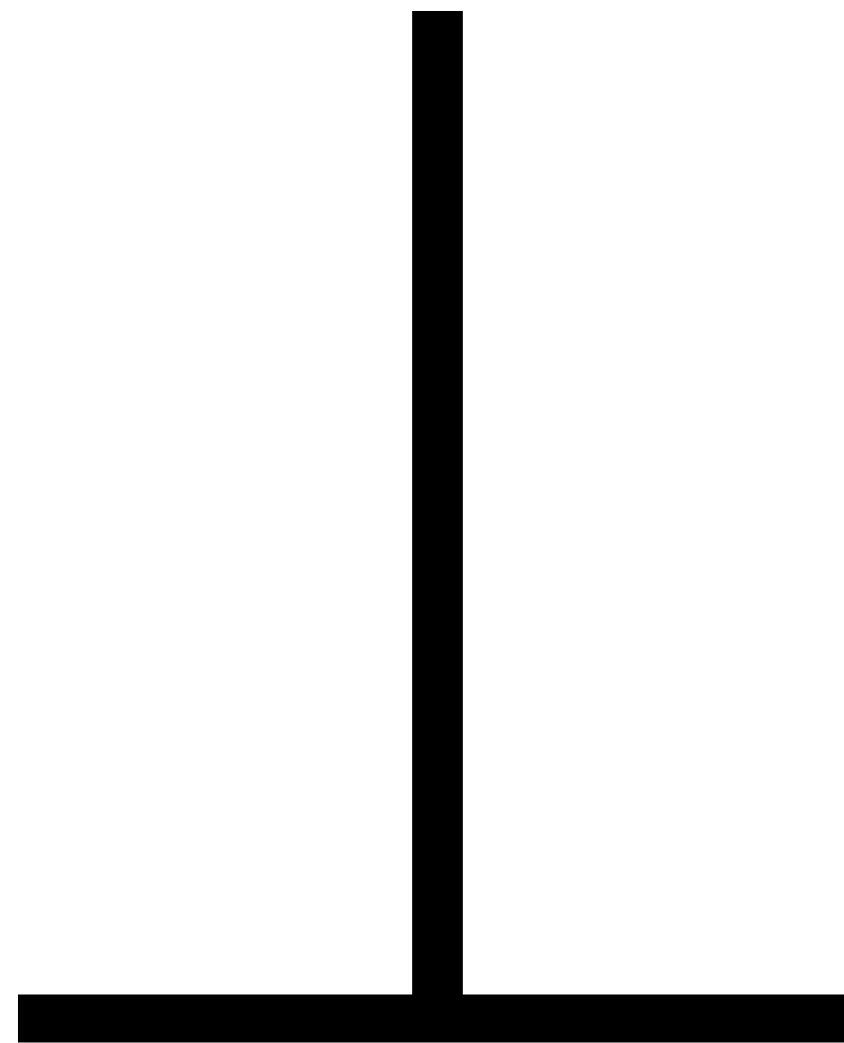
Stange C

Kleine Türme von Hanoi

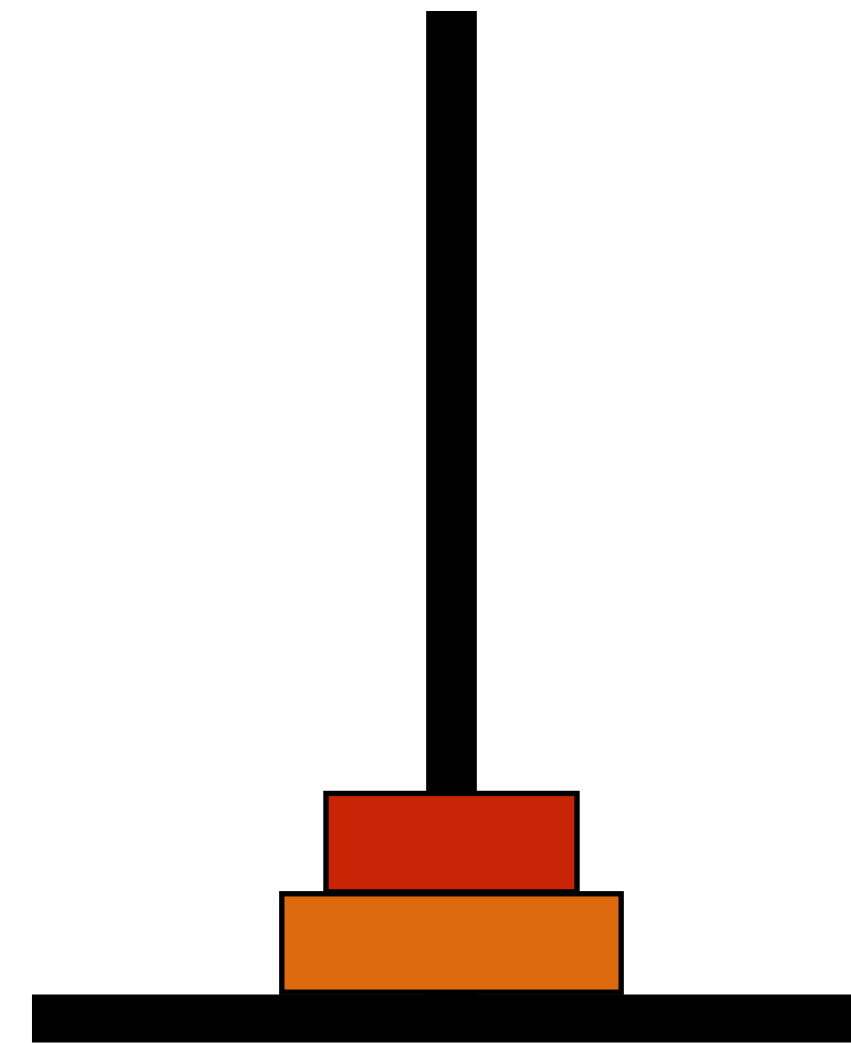
n=4



Stange A



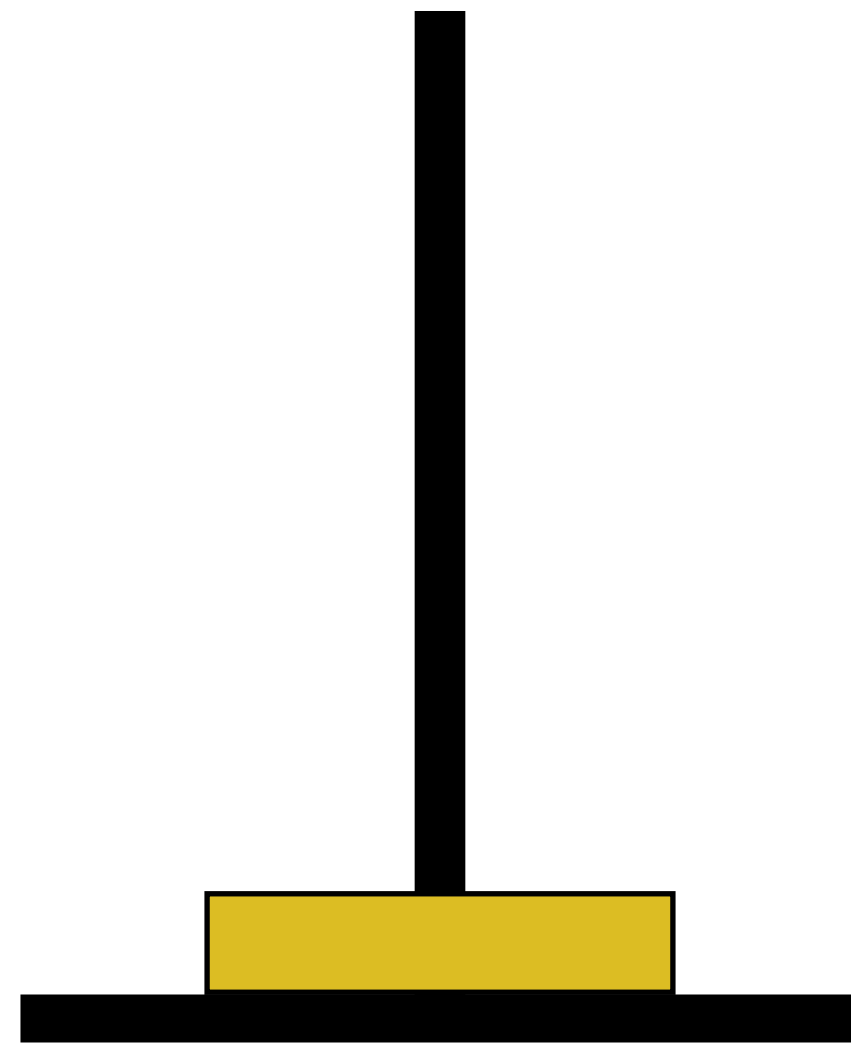
Stange B



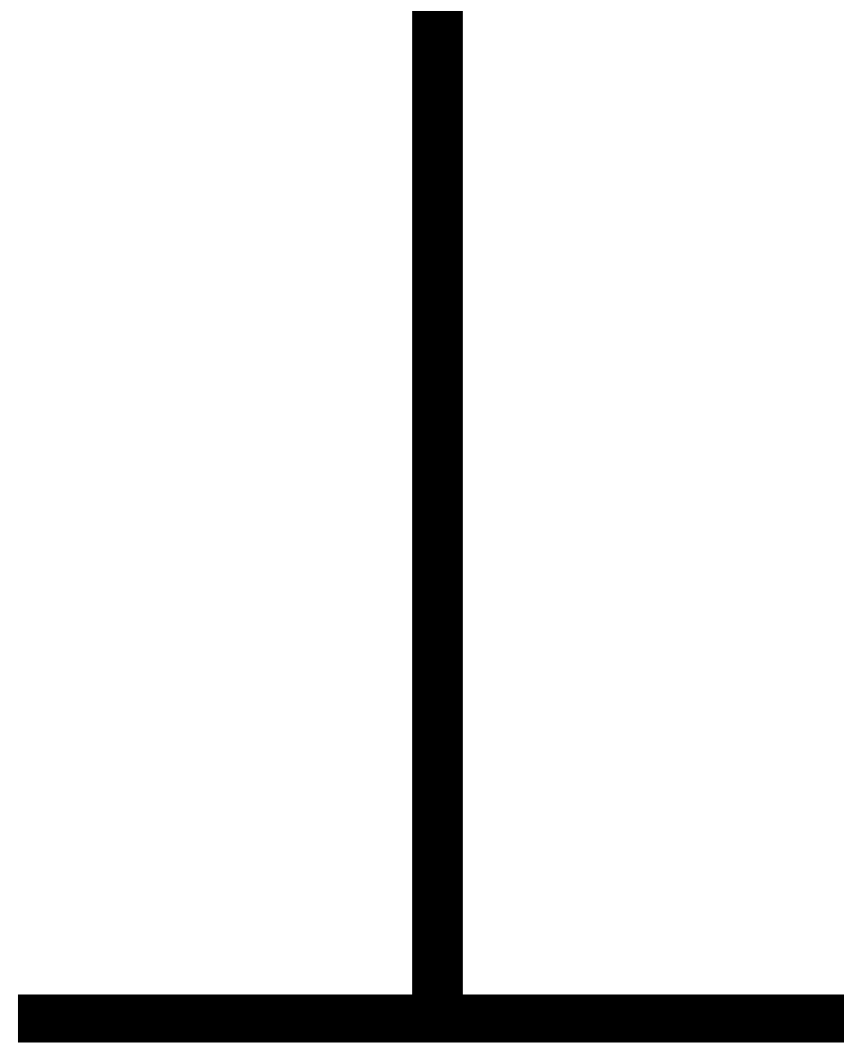
Stange C

Kleine Türme von Hanoi

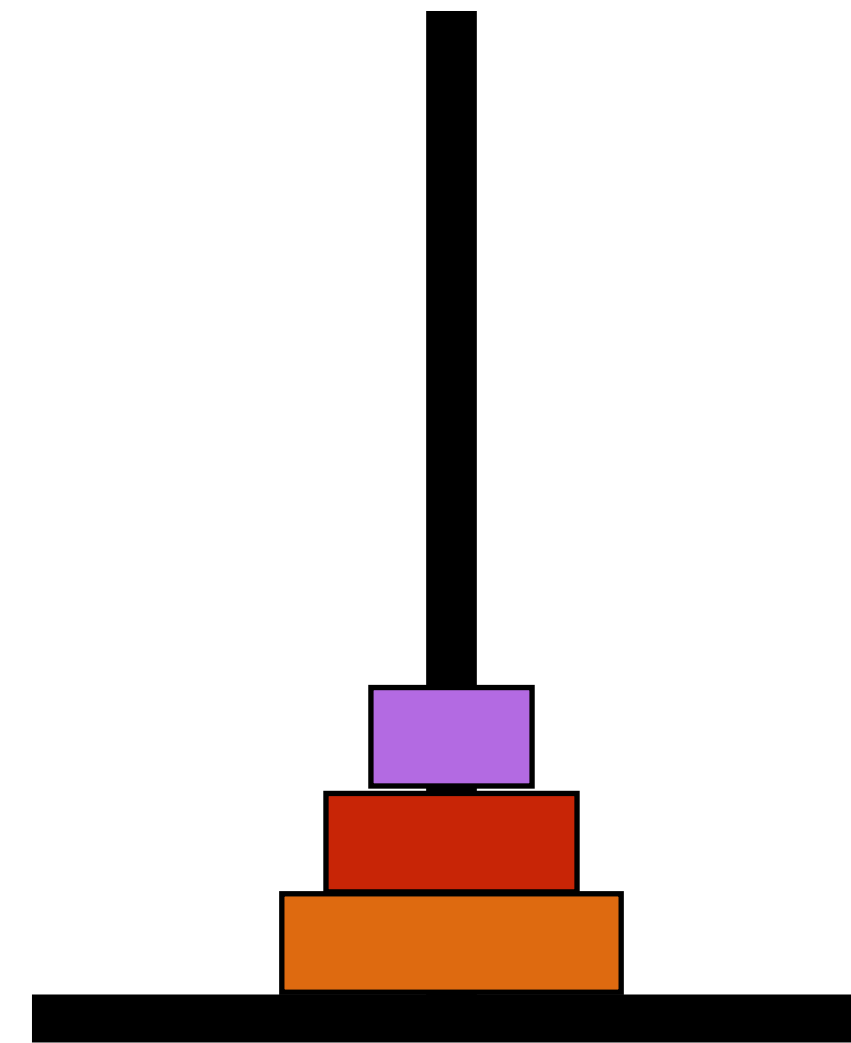
n=4



Stange A



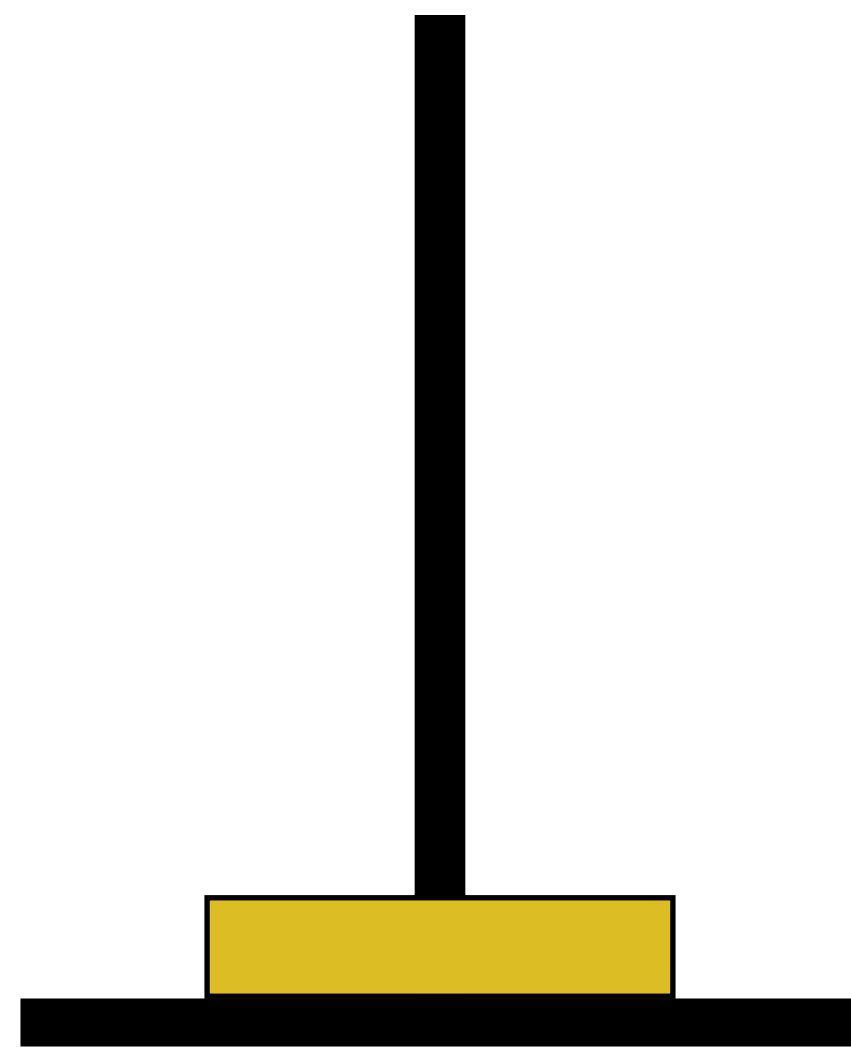
Stange B



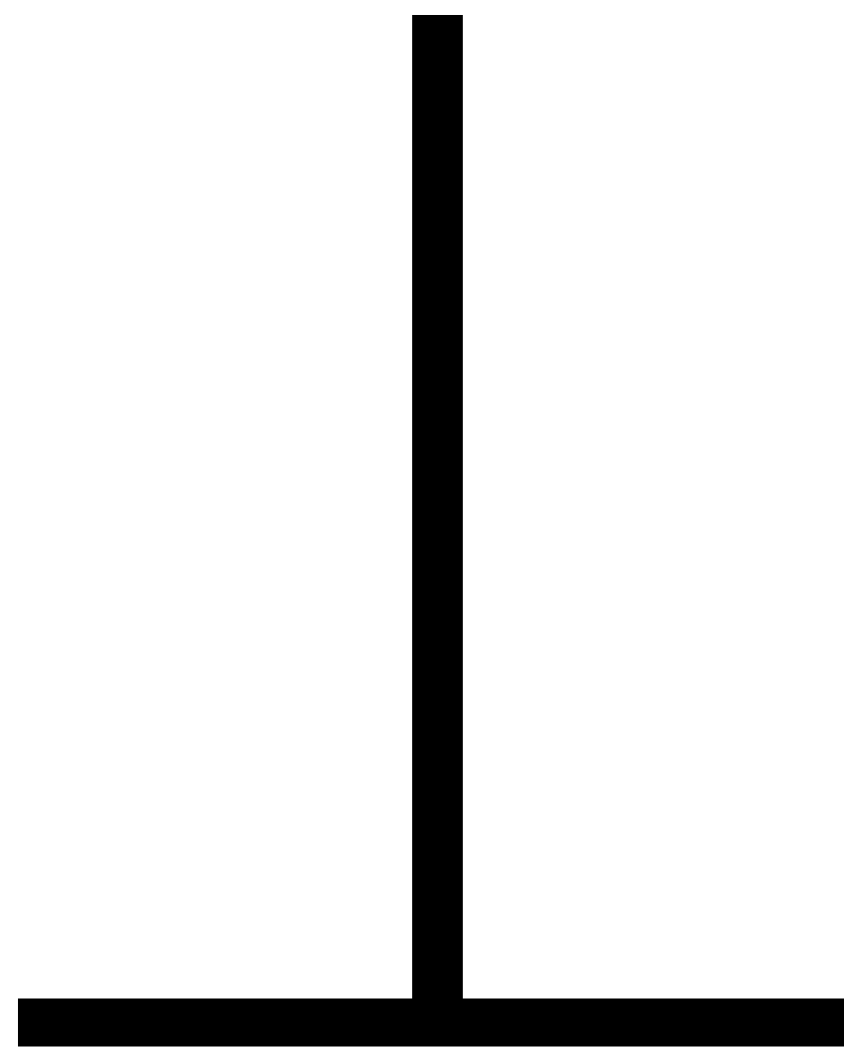
Stange C

Kleine Türme von Hanoi

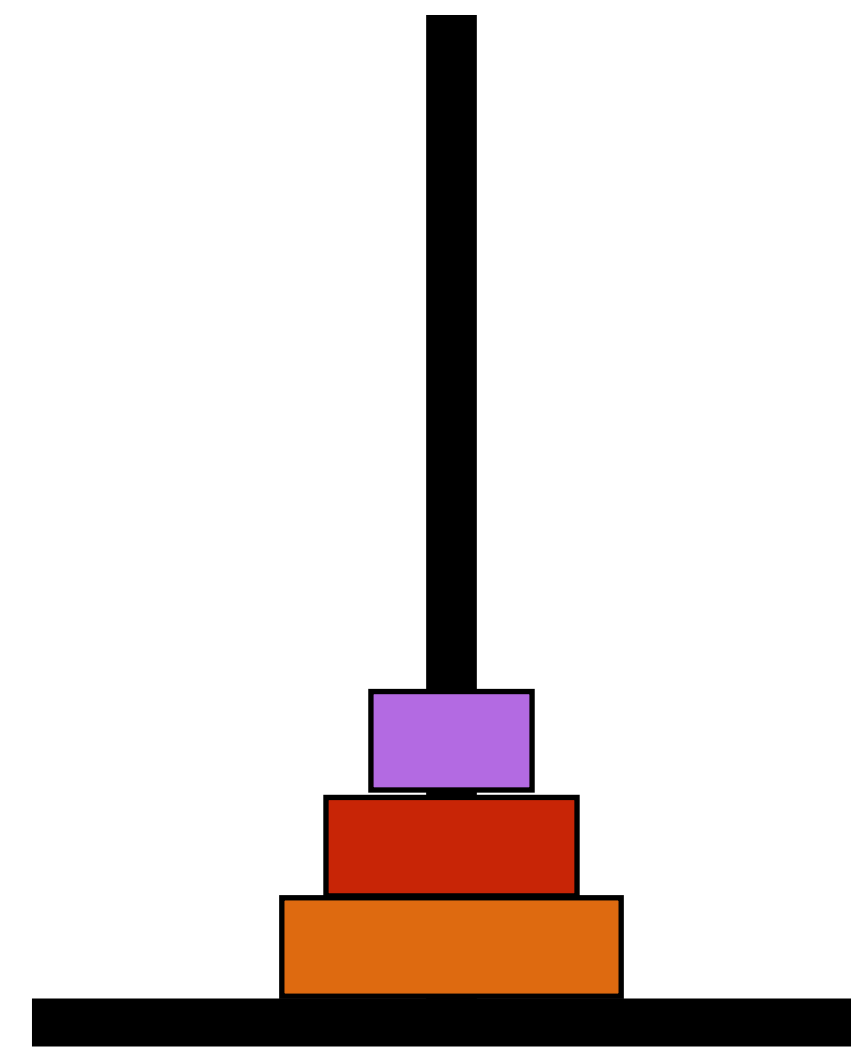
n=4



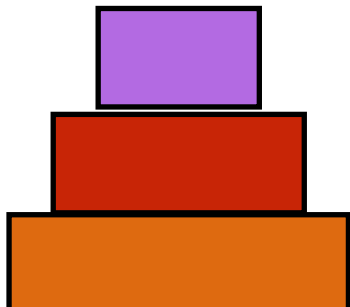
Stange A



Stange B

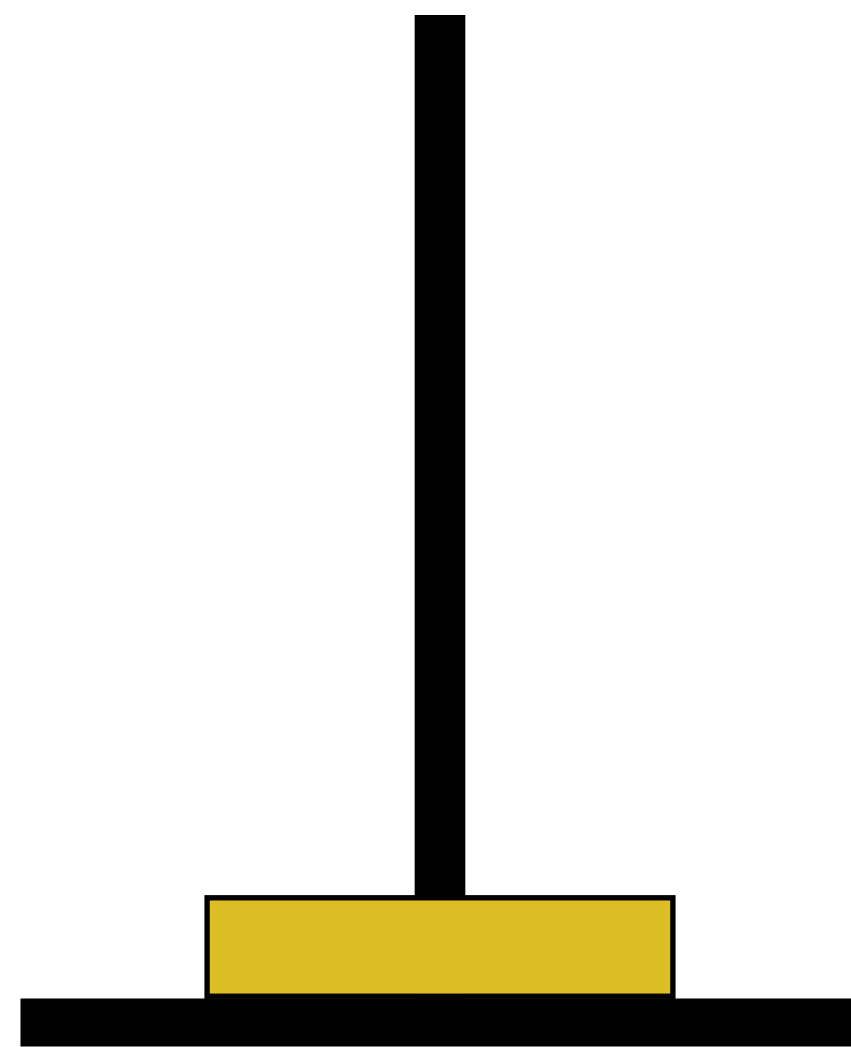


Stange C

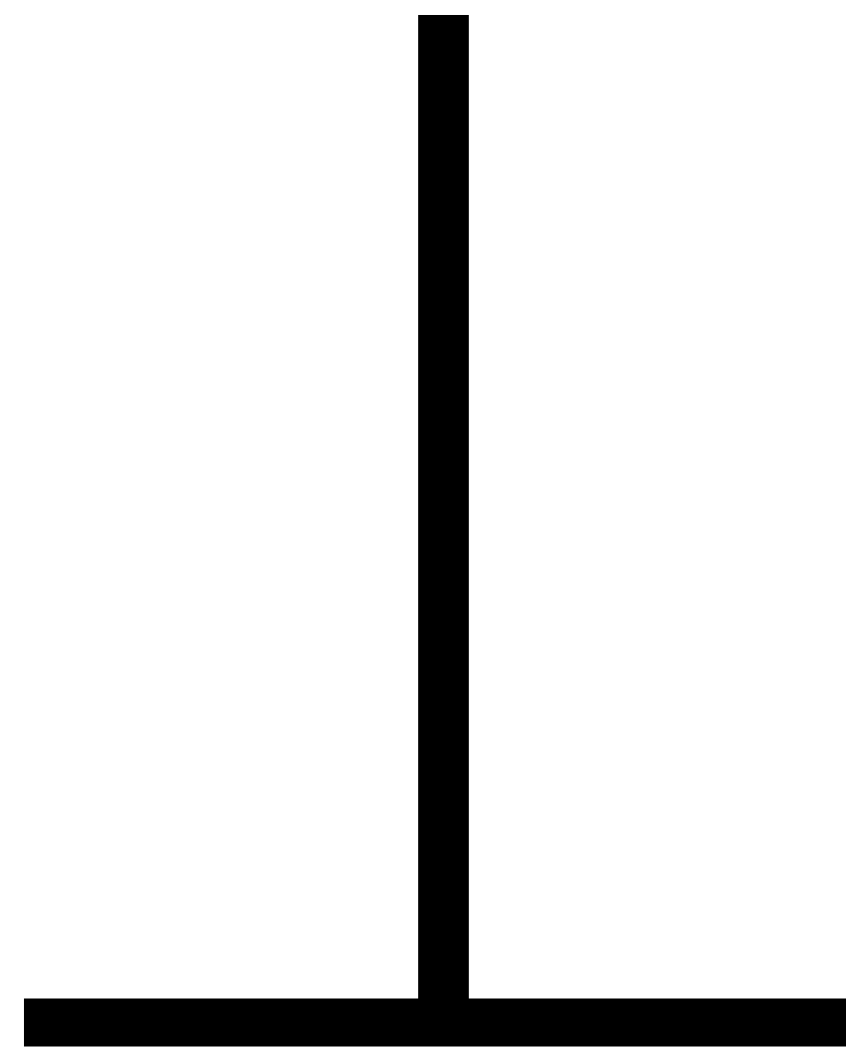
Eigentlich wollten wir  auf Stange C bewegen
direkt nachdem  auf Stange B bewegt wurde.

Kleine Türme von Hanoi

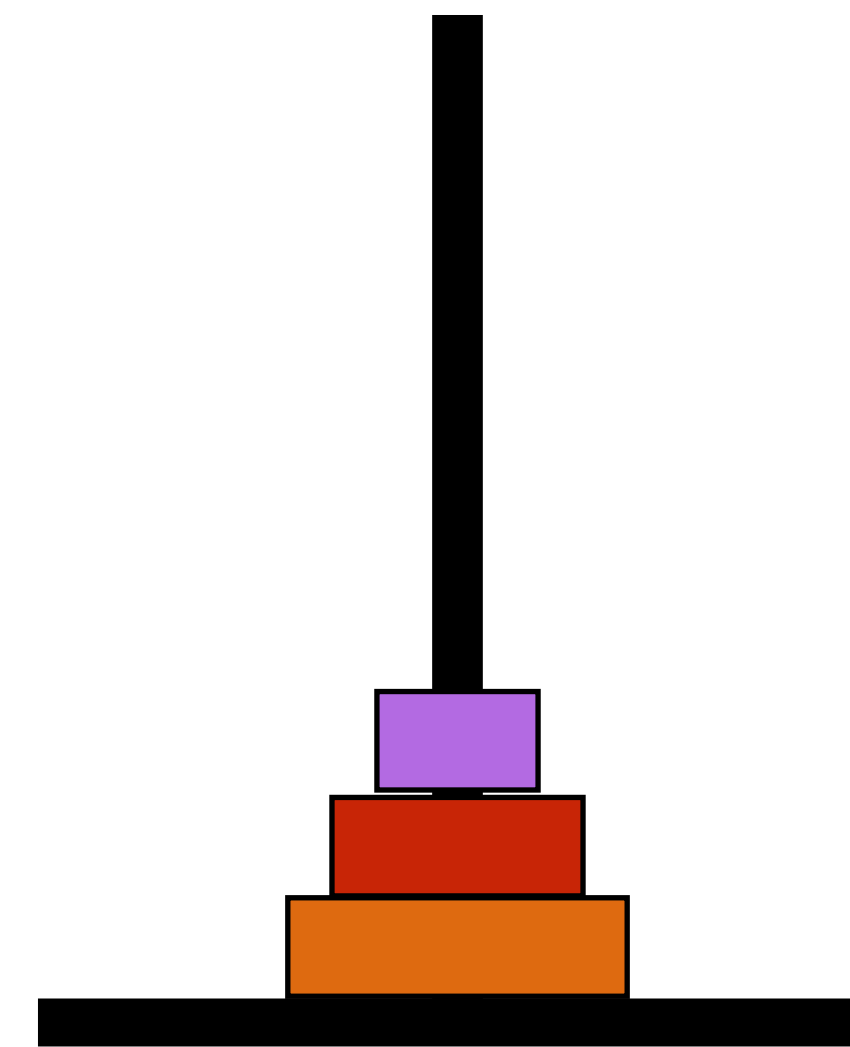
n=4



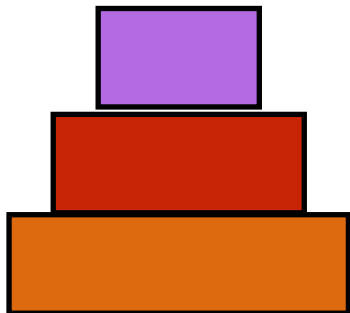
Stange A



Stange B



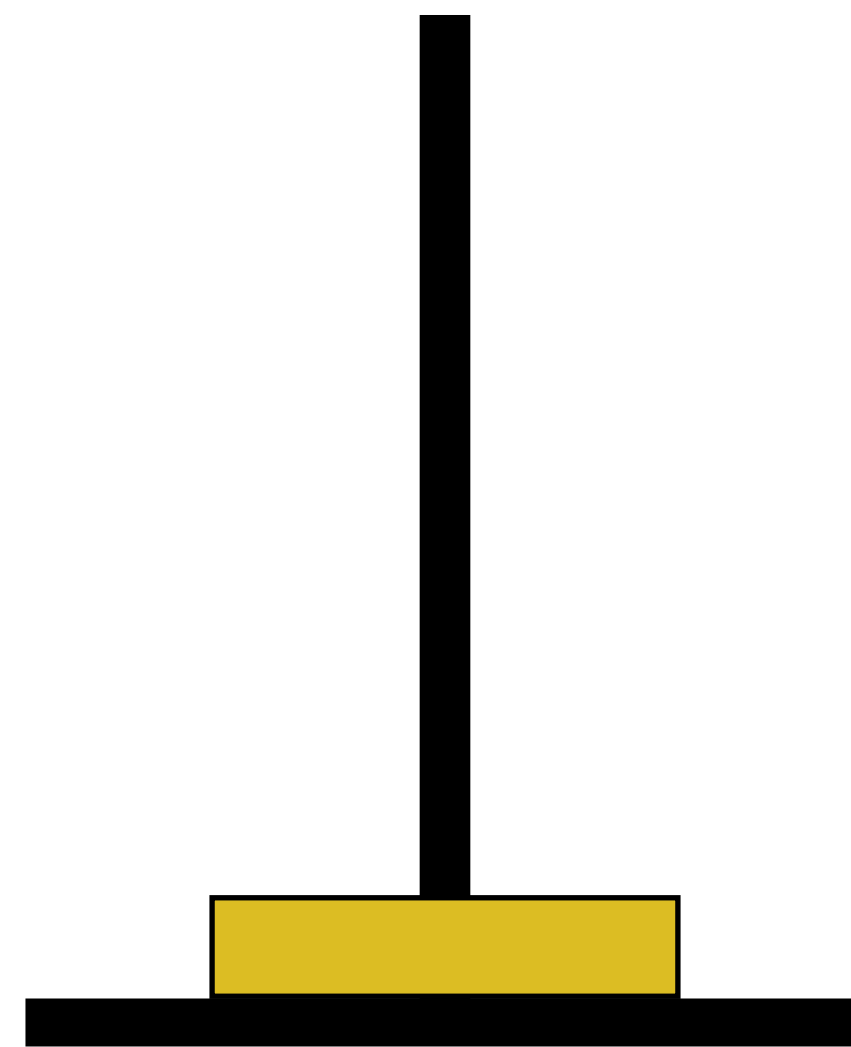
Stange C

Eigentlich wollten wir  auf Stange C bewegen
direkt nachdem  auf Stange B bewegt wurde.

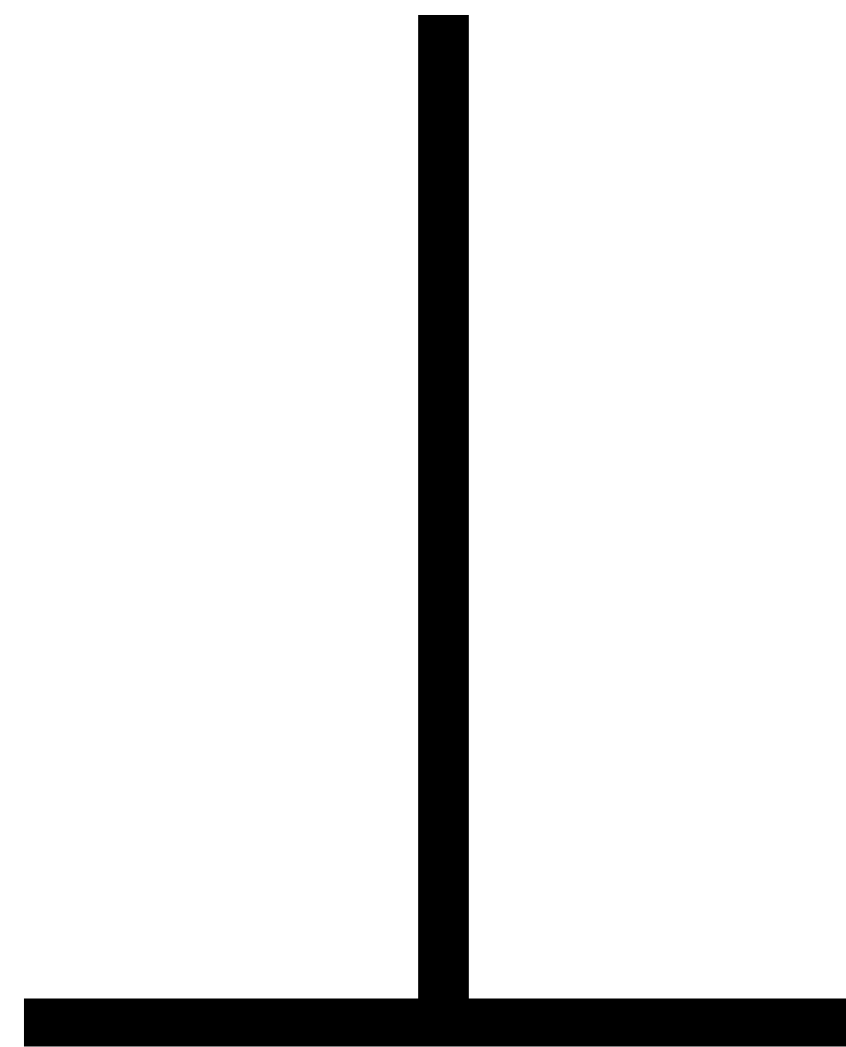


Kleine Türme von Hanoi

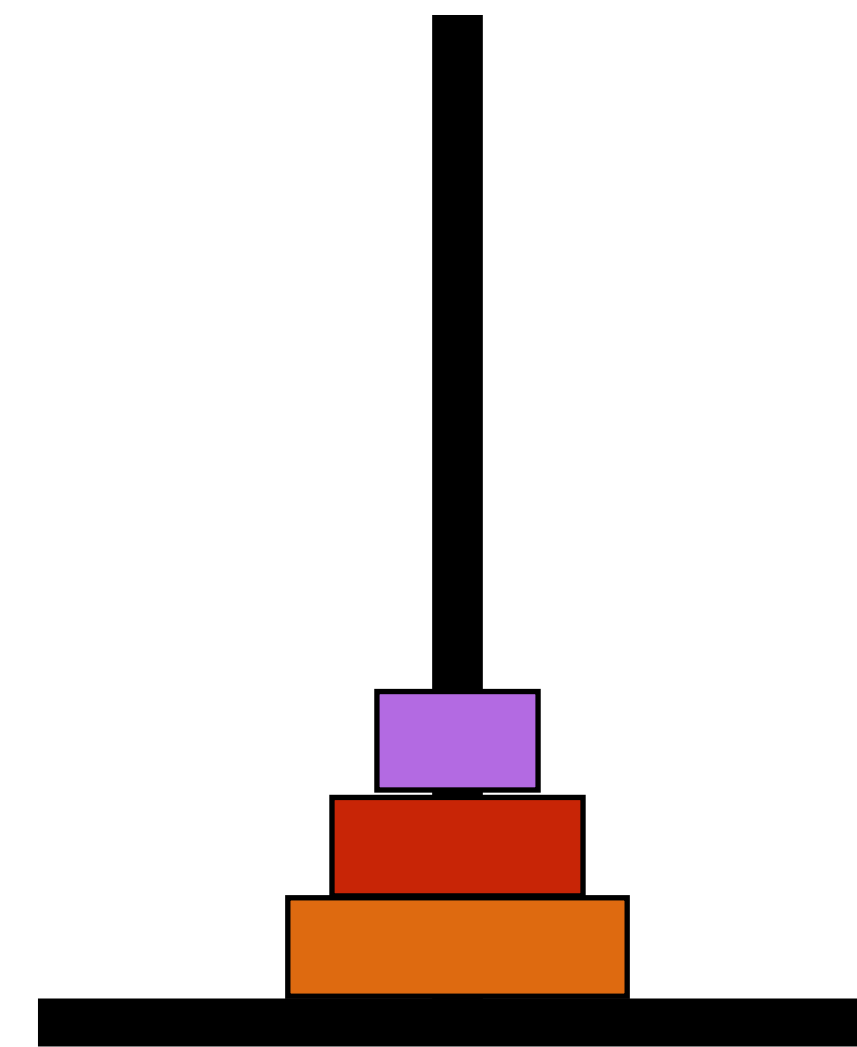
n=4



Stange A



Stange B

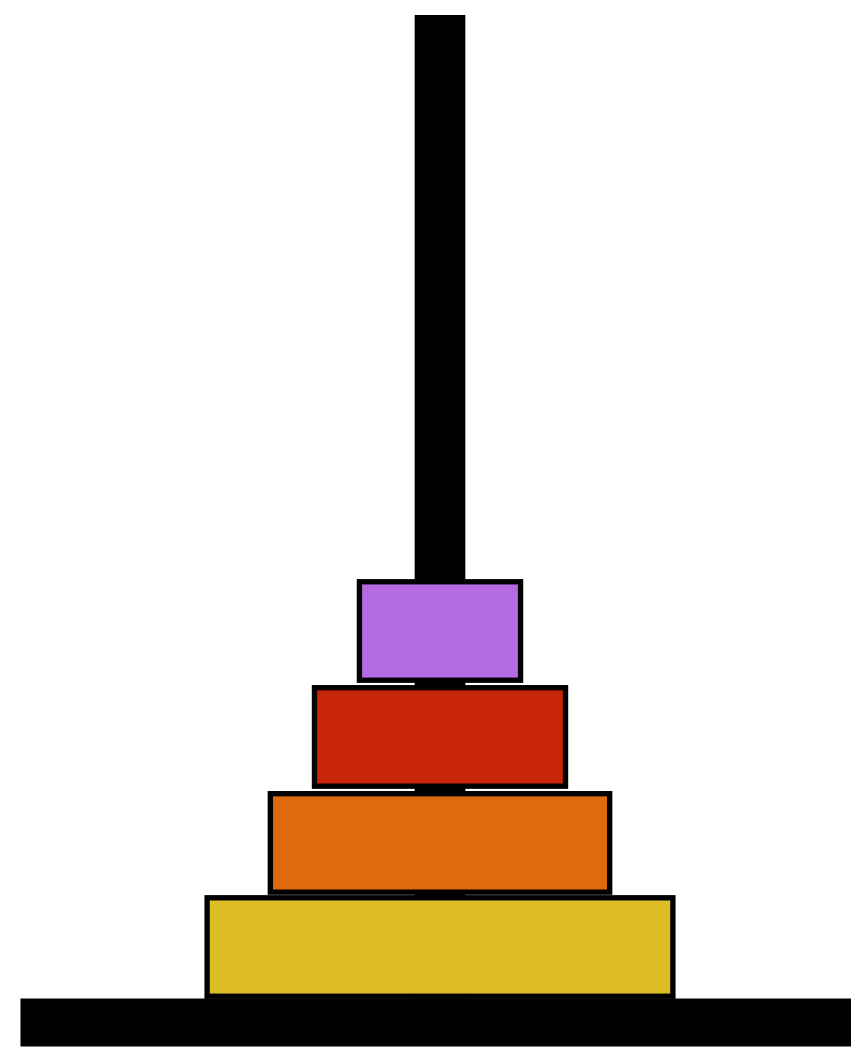


Stange C

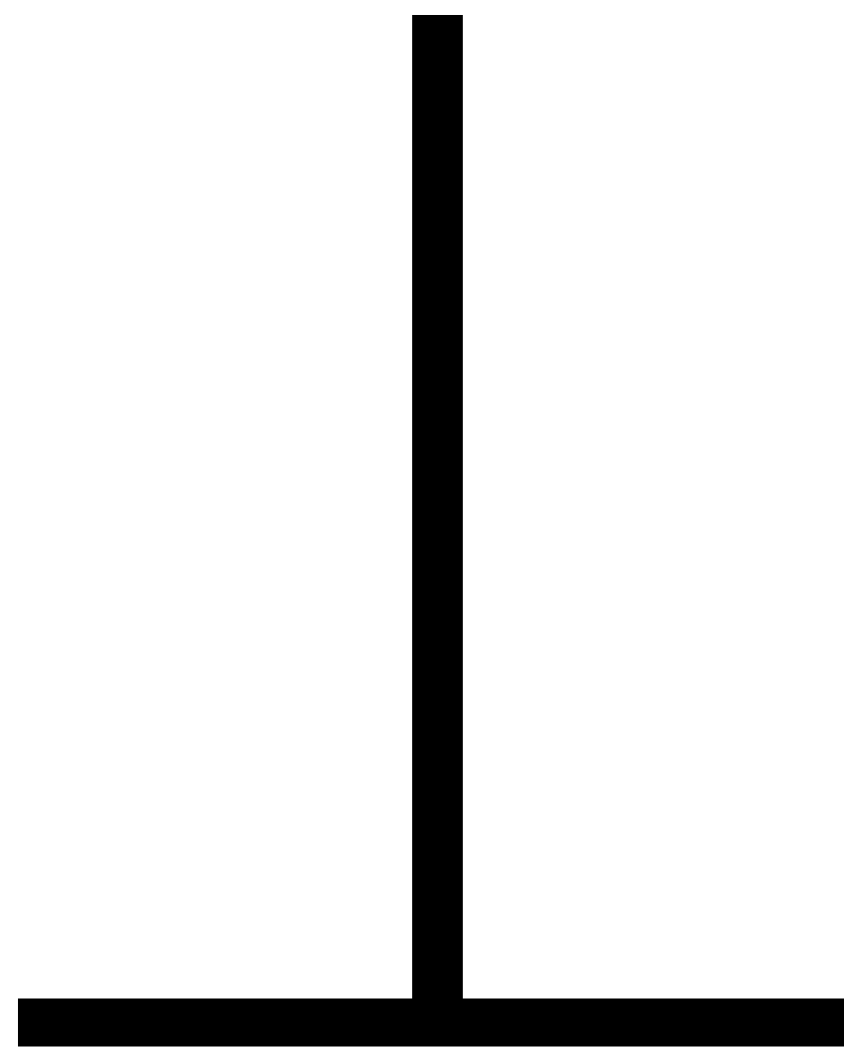
Versuchen wir es also erneut.

Kleine Türme von Hanoi

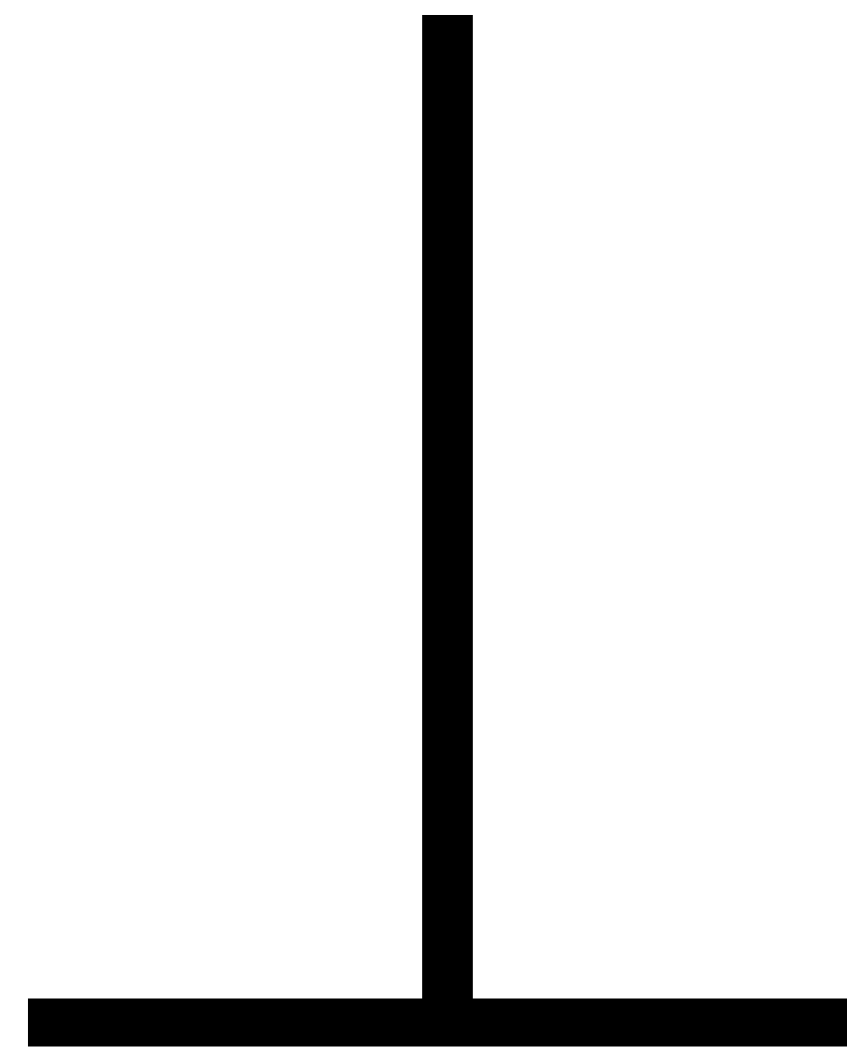
n=4



Stange A



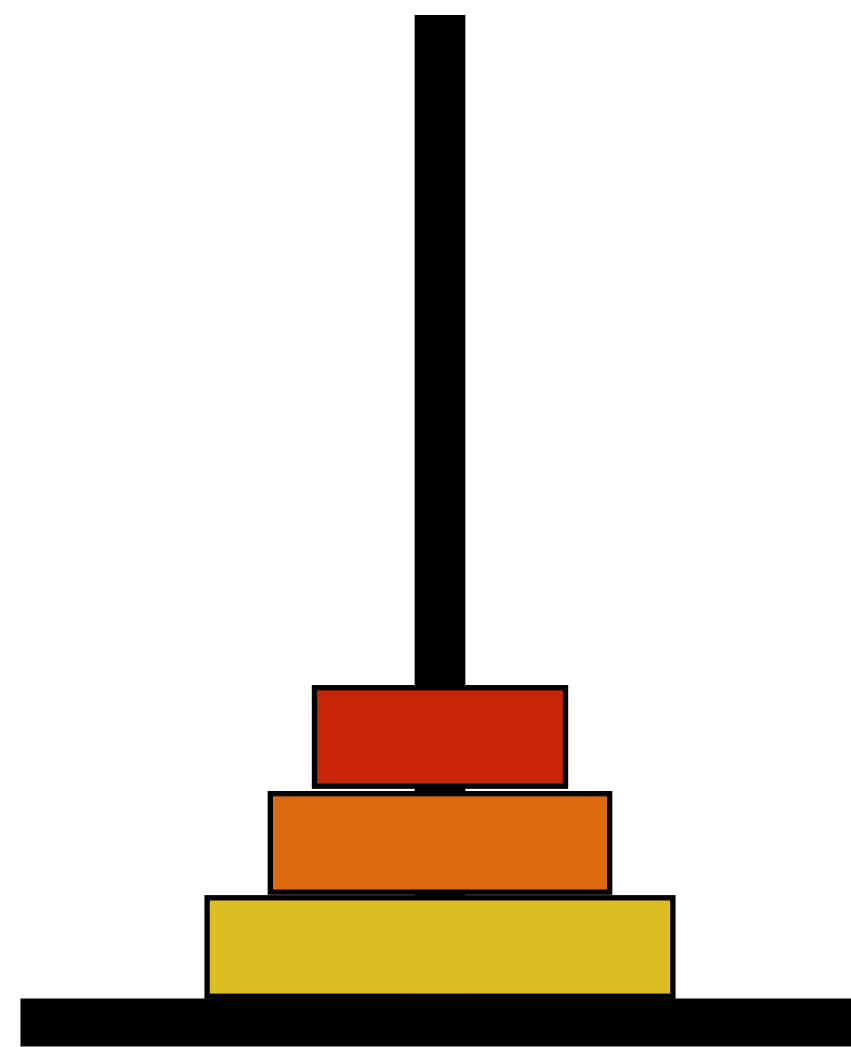
Stange B



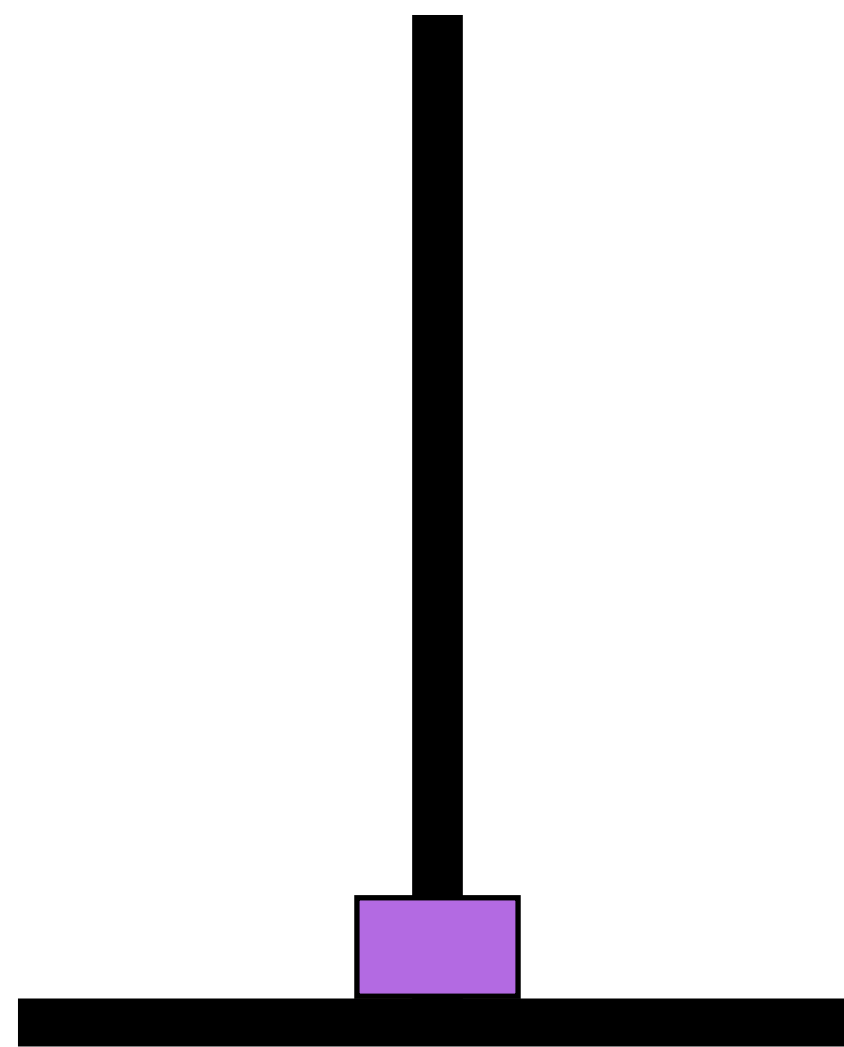
Stange C

Kleine Türme von Hanoi

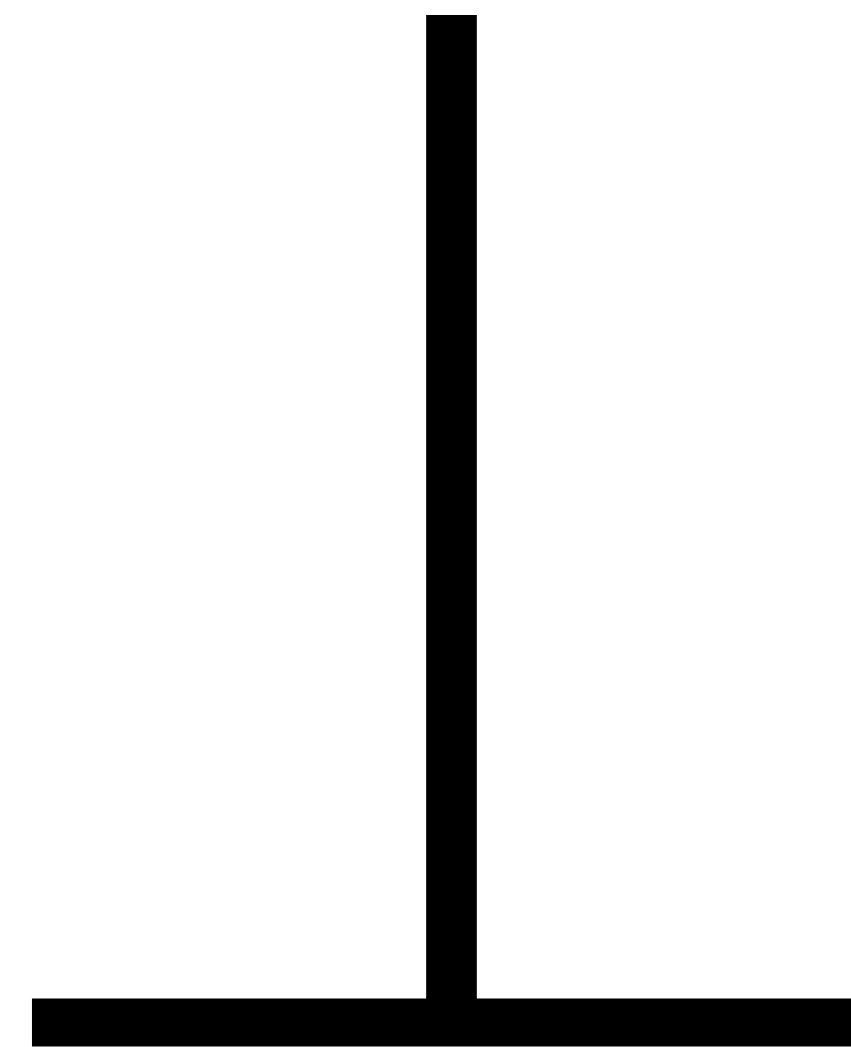
n=4



Stange A



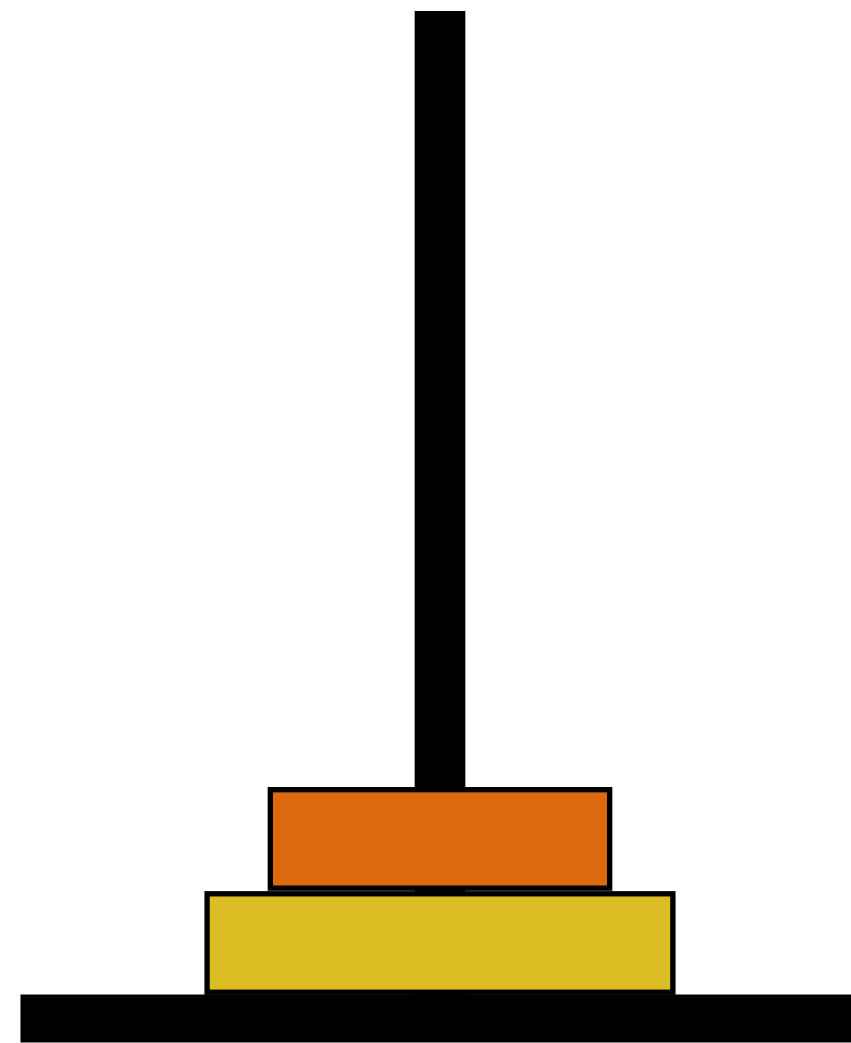
Stange B



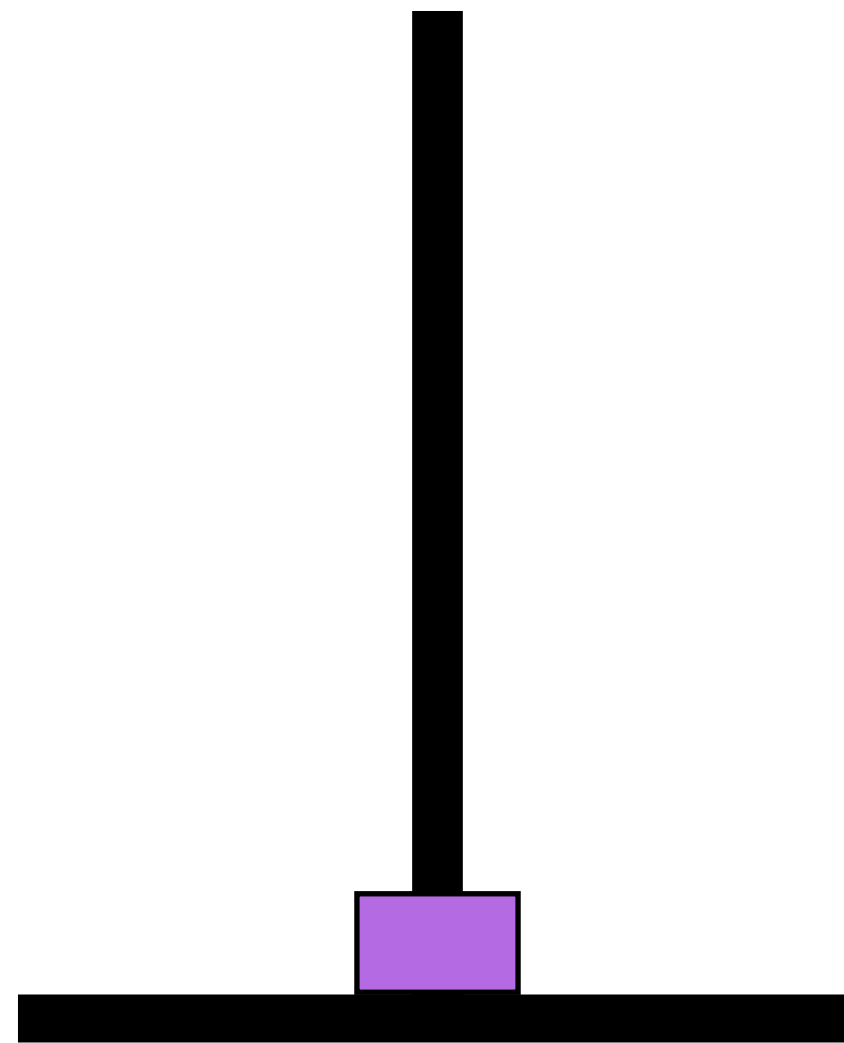
Stange C

Kleine Türme von Hanoi

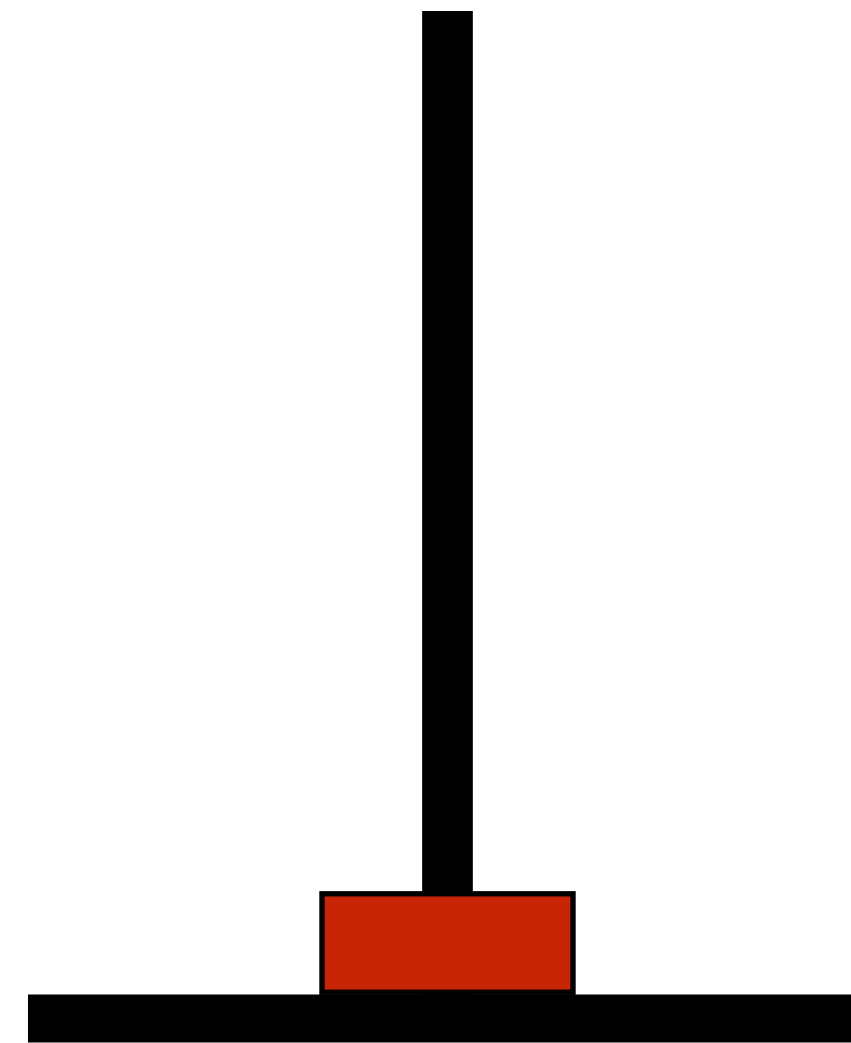
n=4



Stange A



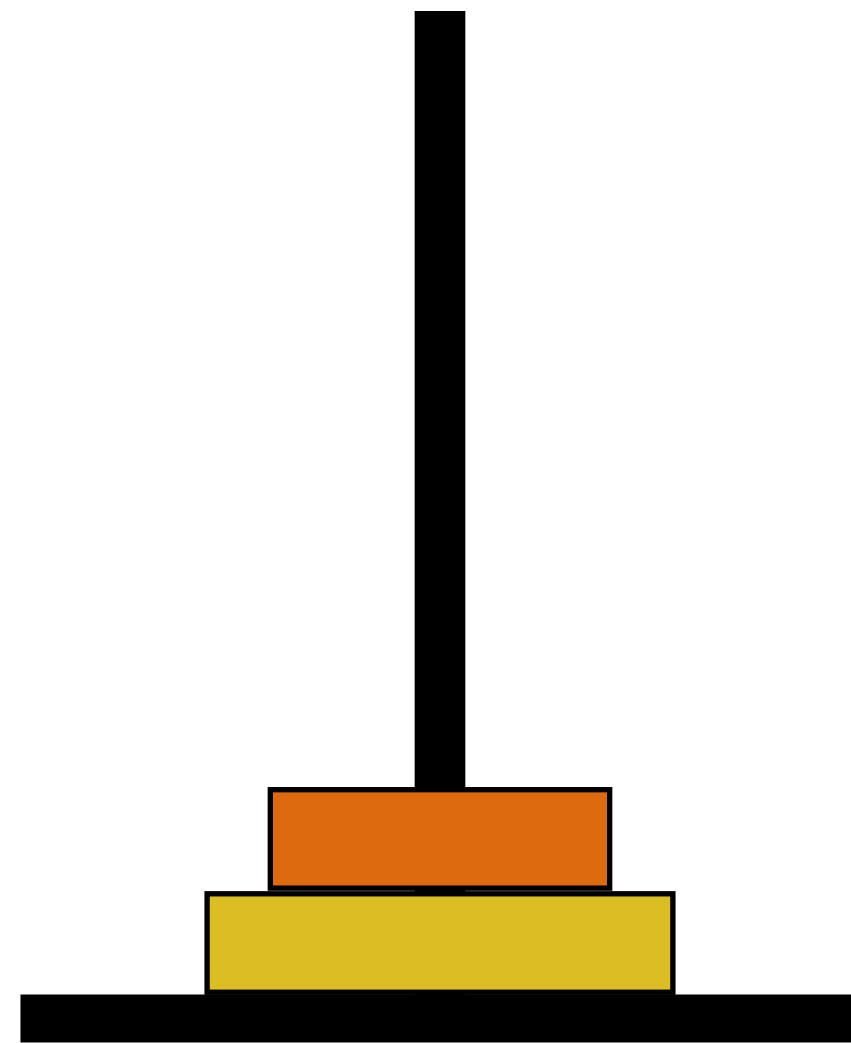
Stange B



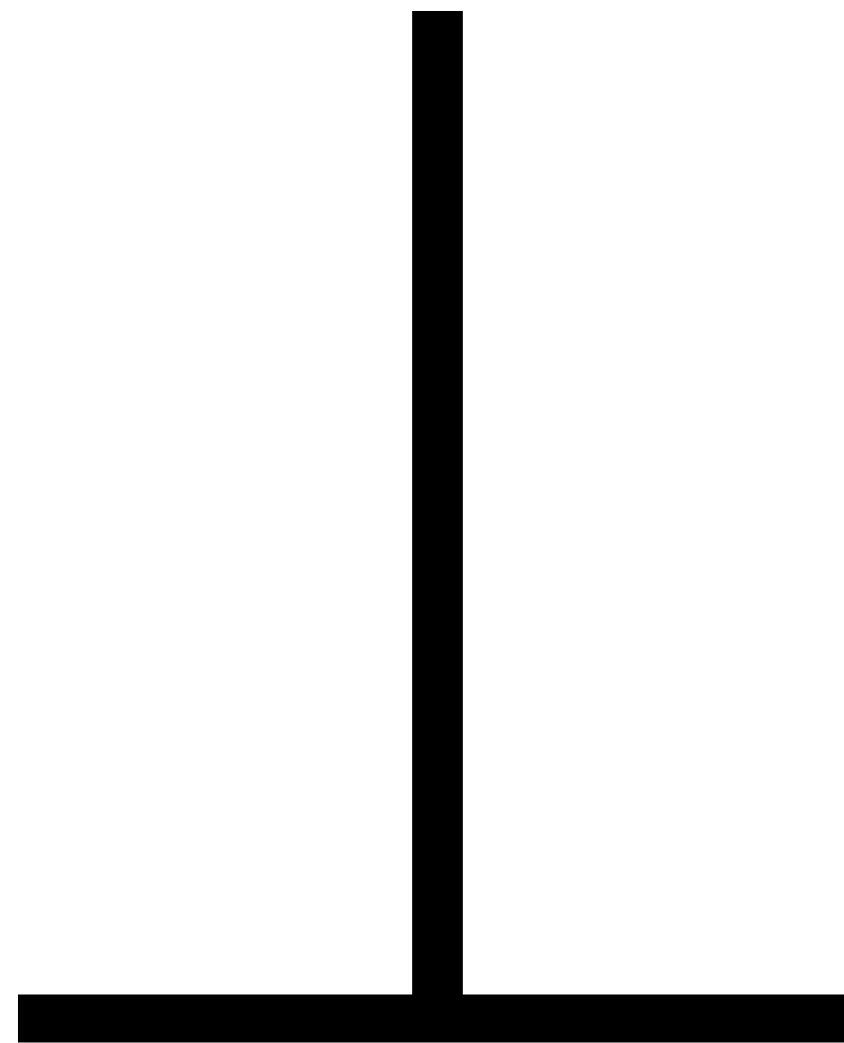
Stange C

Kleine Türme von Hanoi

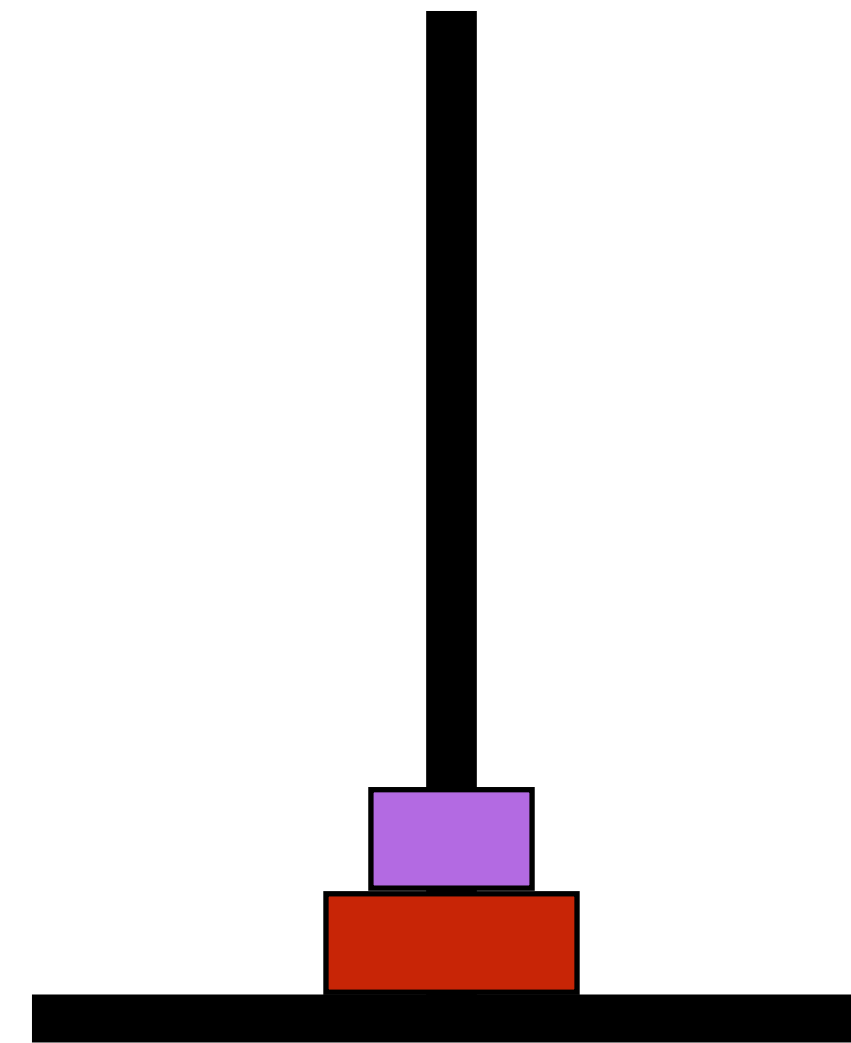
n=4



Stange A



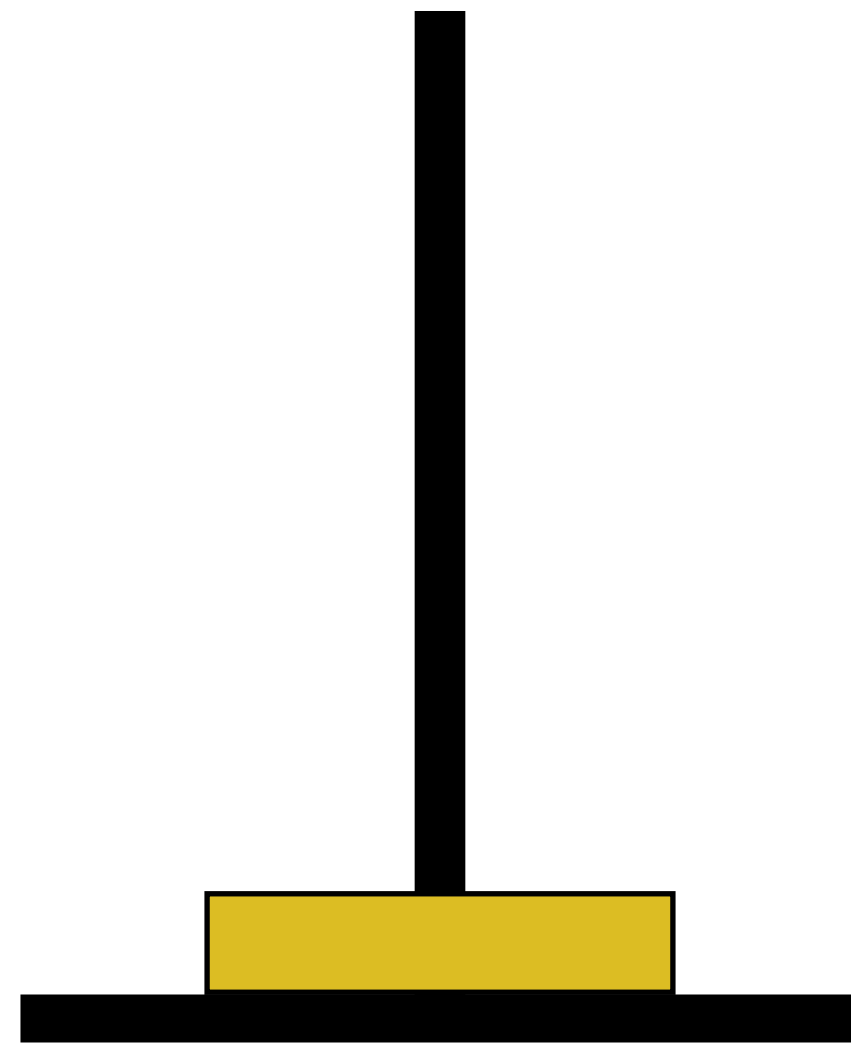
Stange B



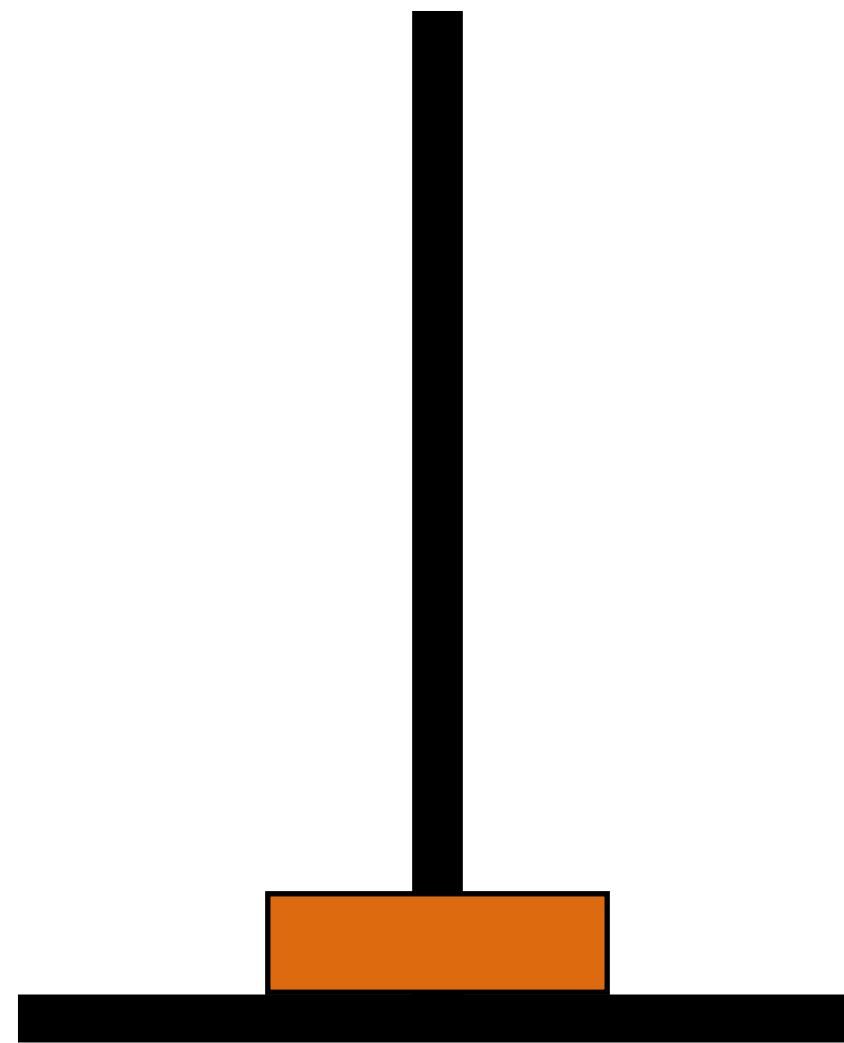
Stange C

Kleine Türme von Hanoi

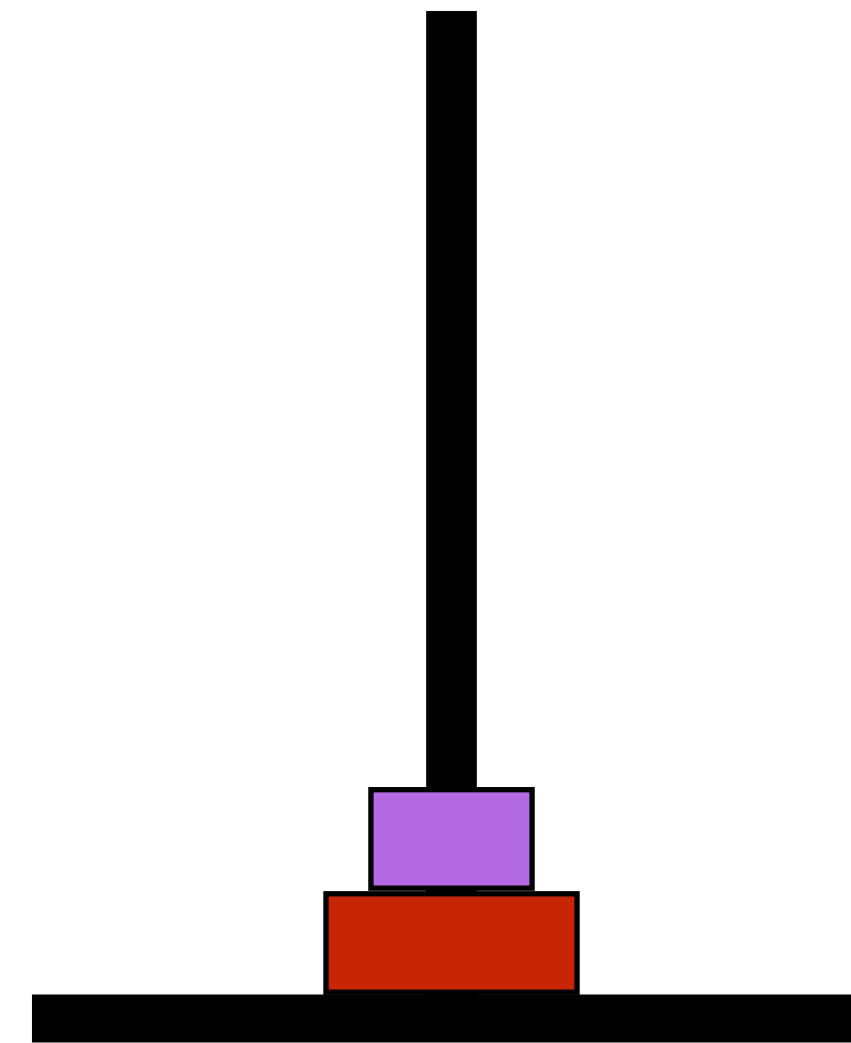
n=4



Stange A



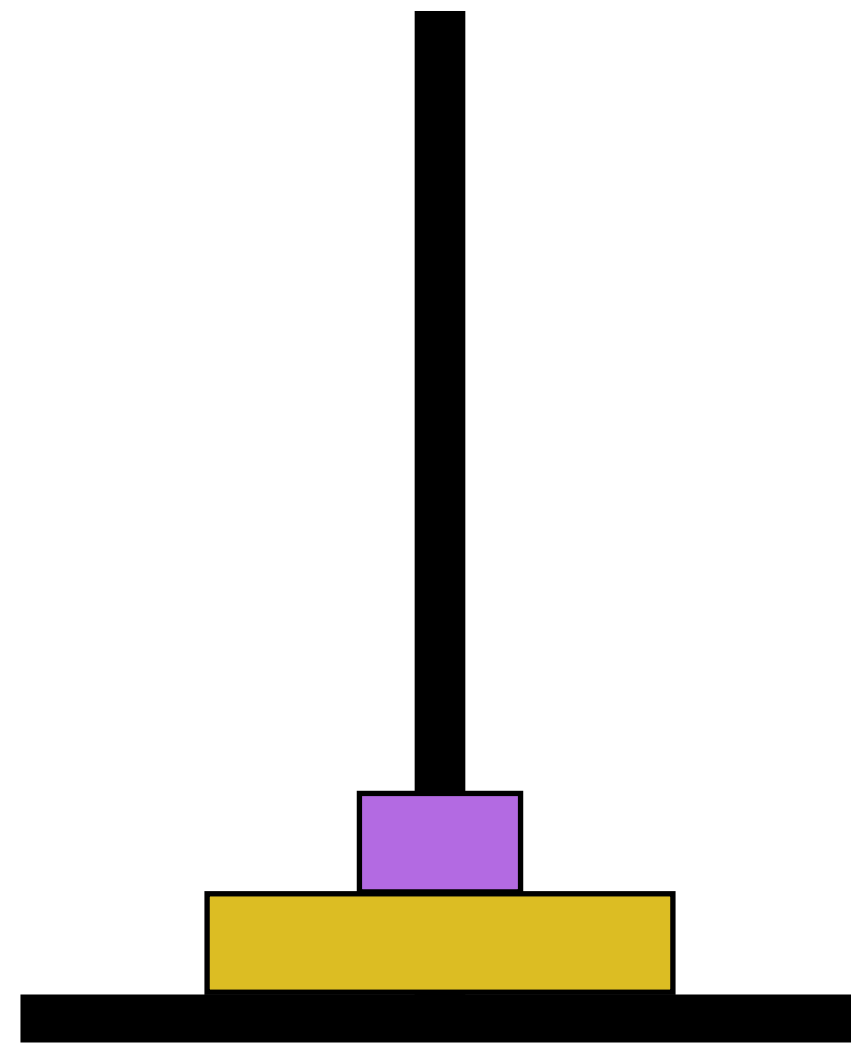
Stange B



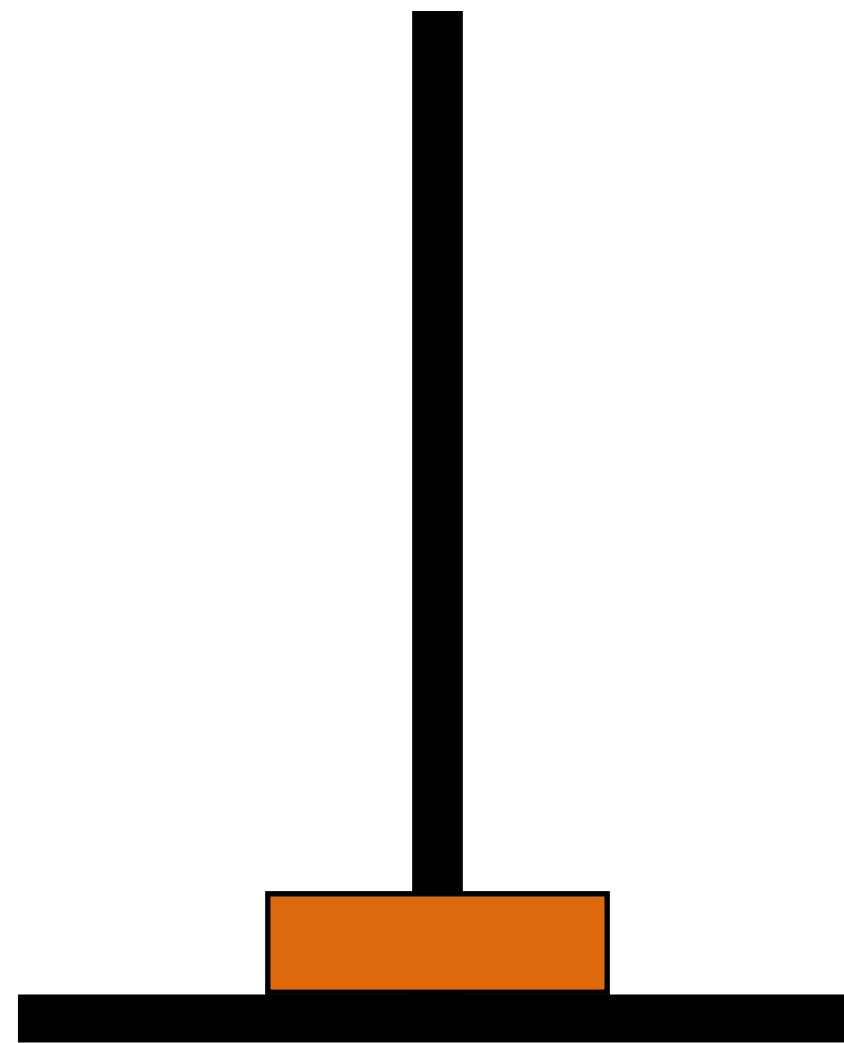
Stange C

Kleine Türme von Hanoi

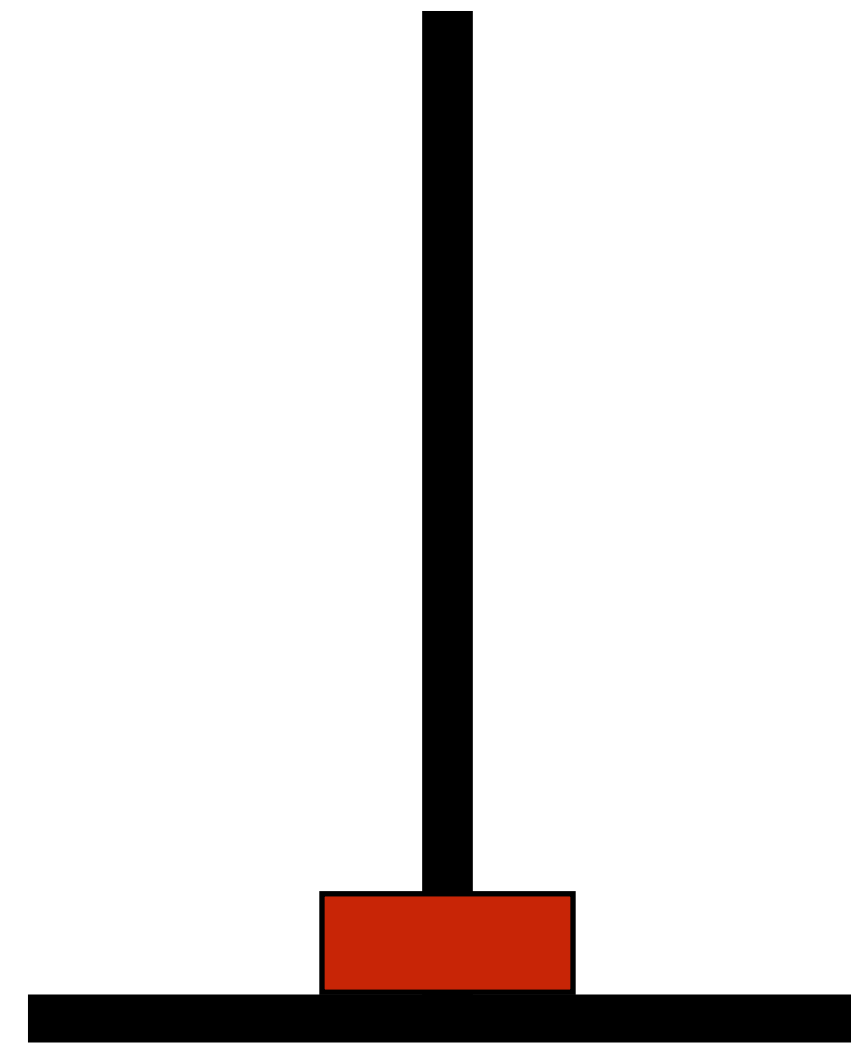
n=4



Stange A



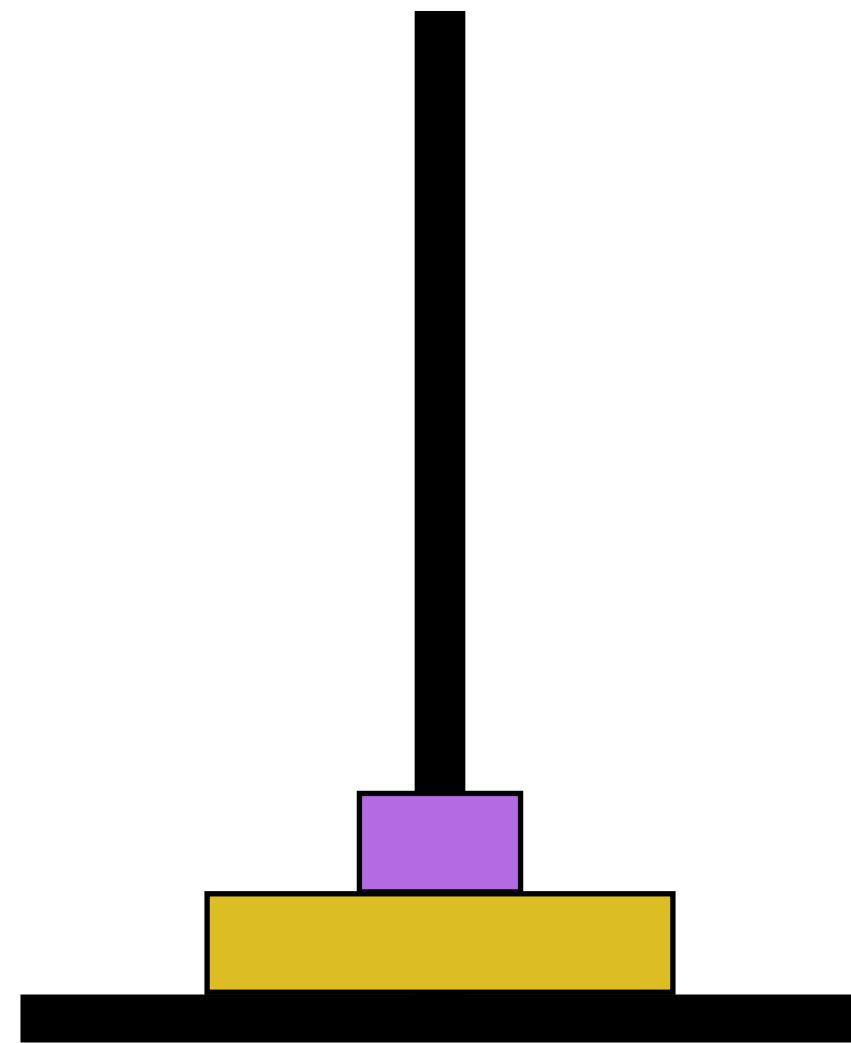
Stange B



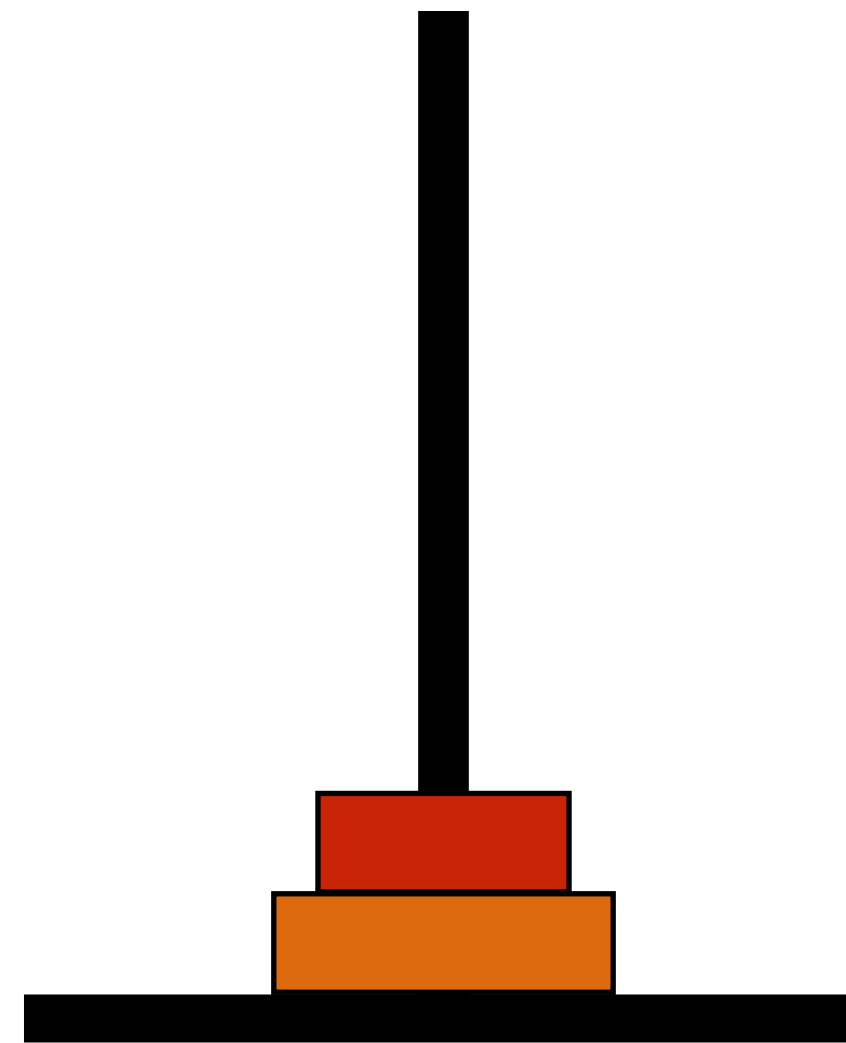
Stange C

Kleine Türme von Hanoi

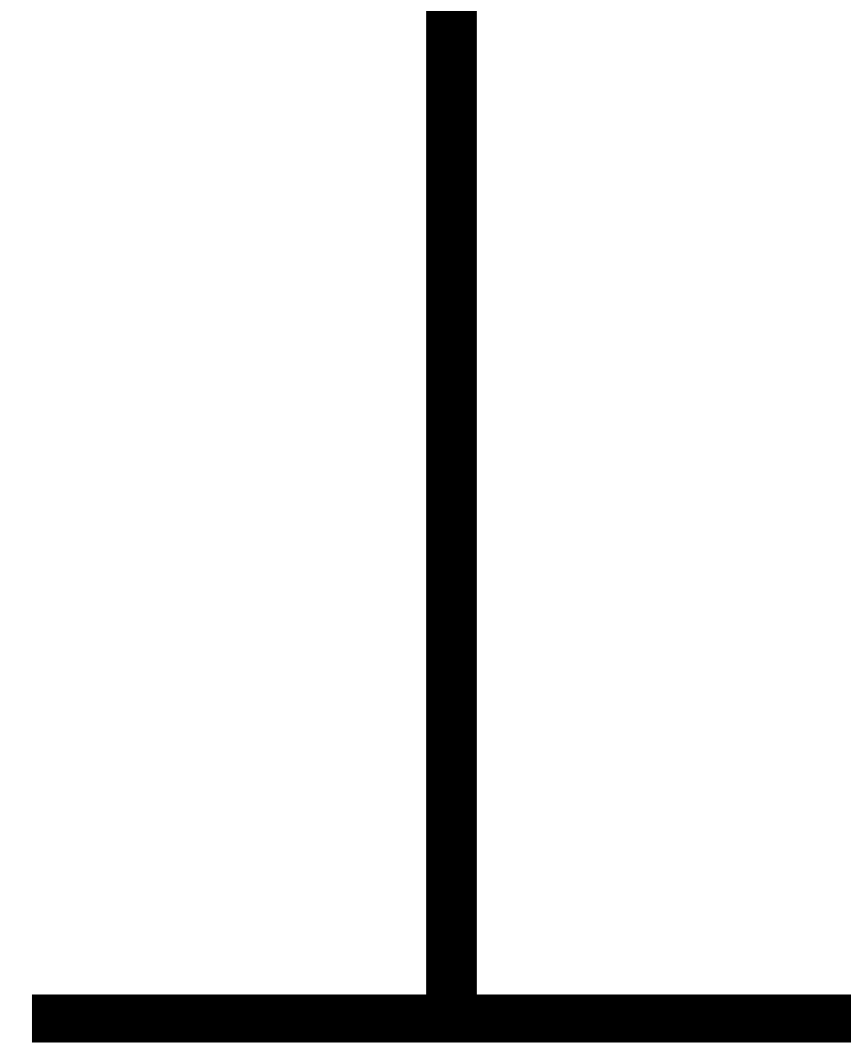
n=4



Stange A



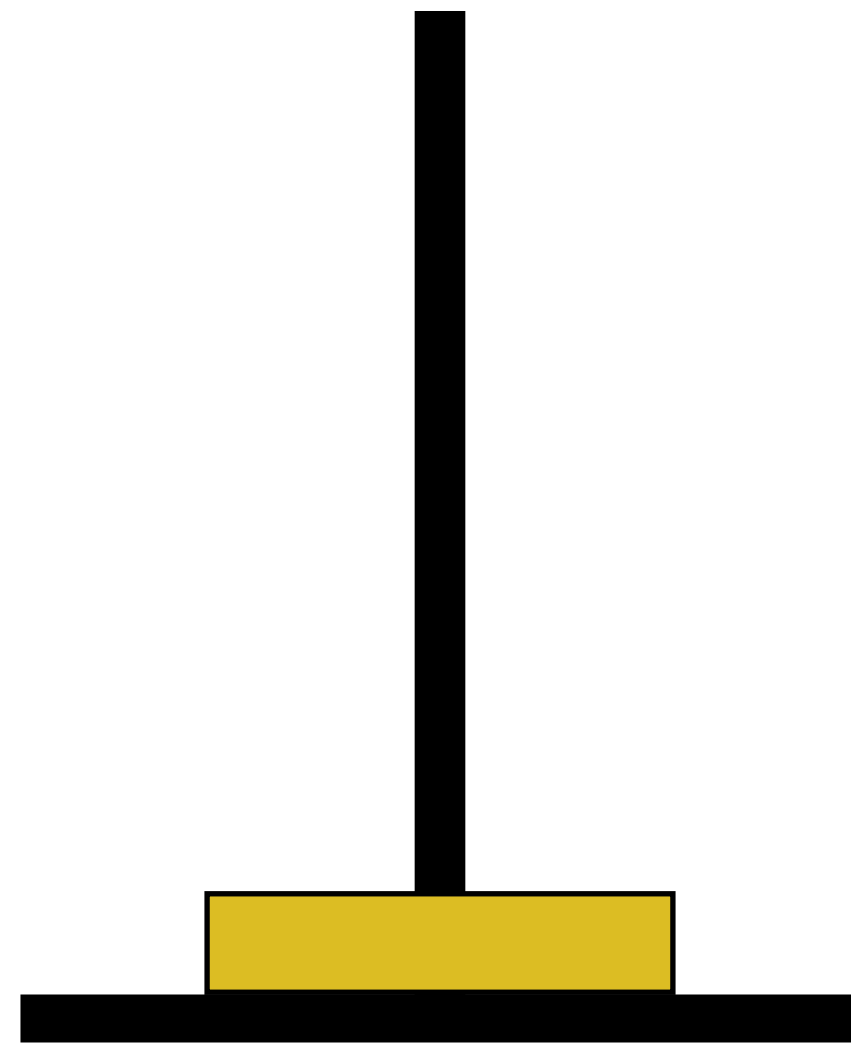
Stange B



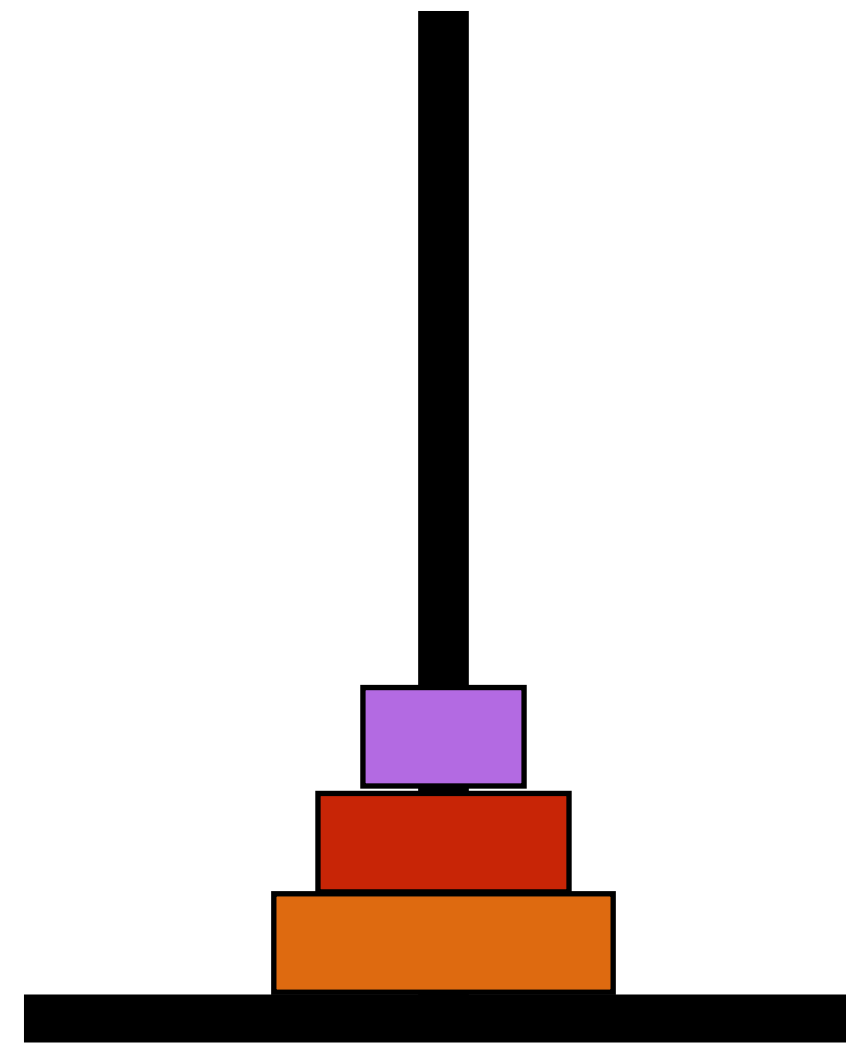
Stange C

Kleine Türme von Hanoi

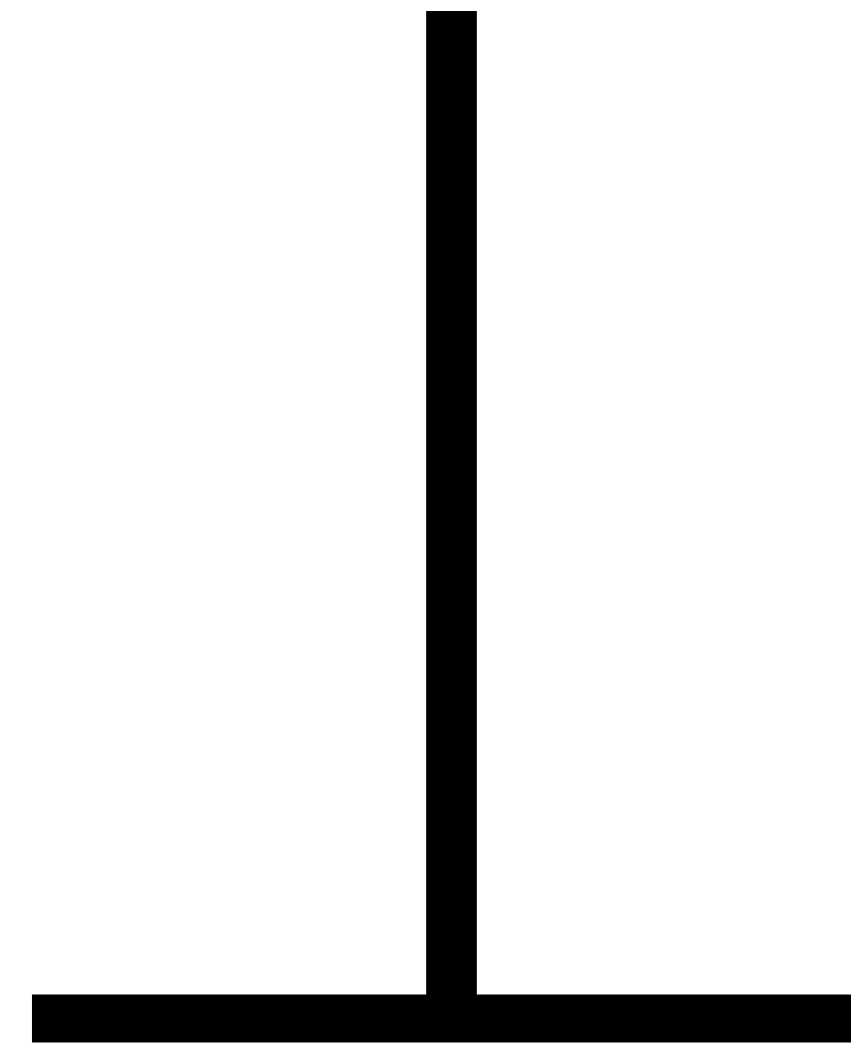
n=4



Stange A



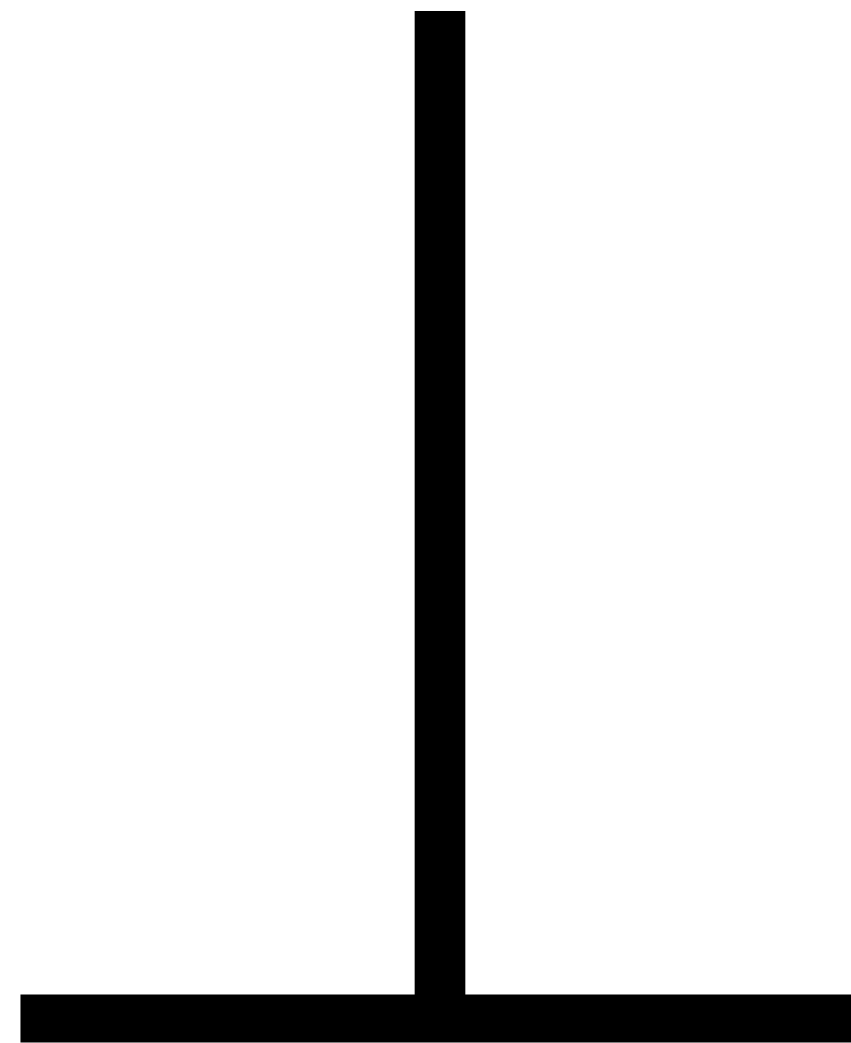
Stange B



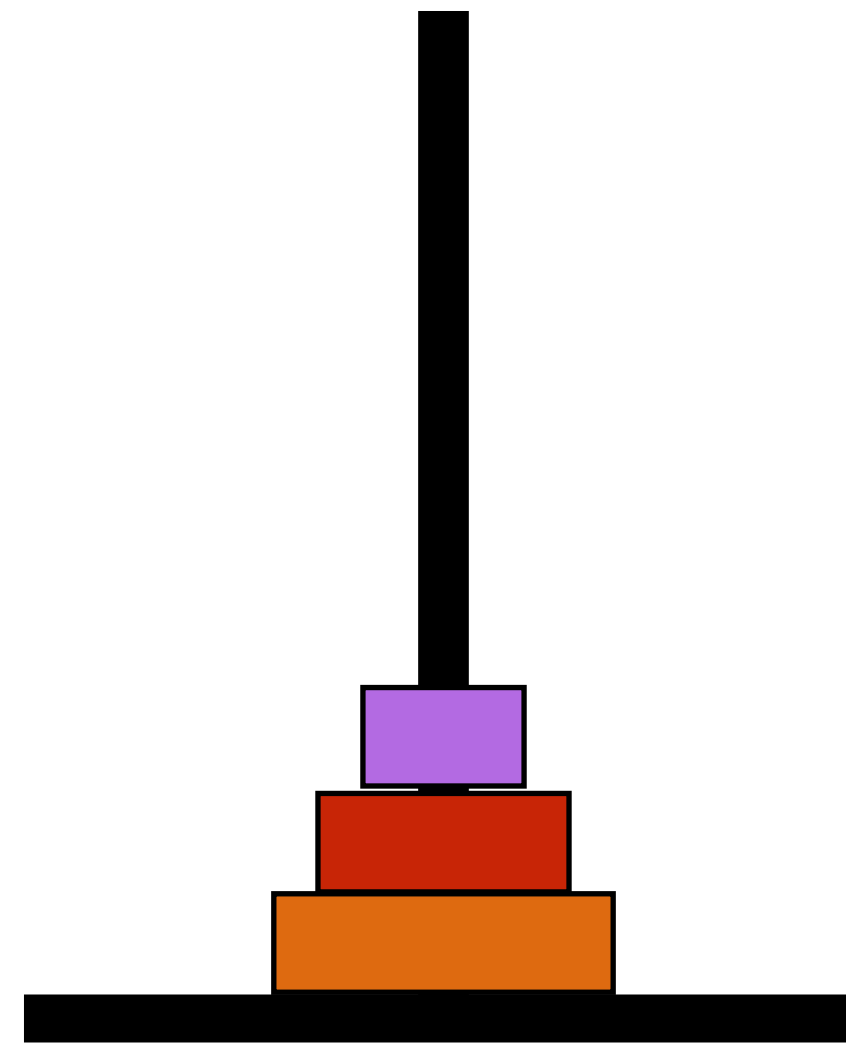
Stange C

Kleine Türme von Hanoi

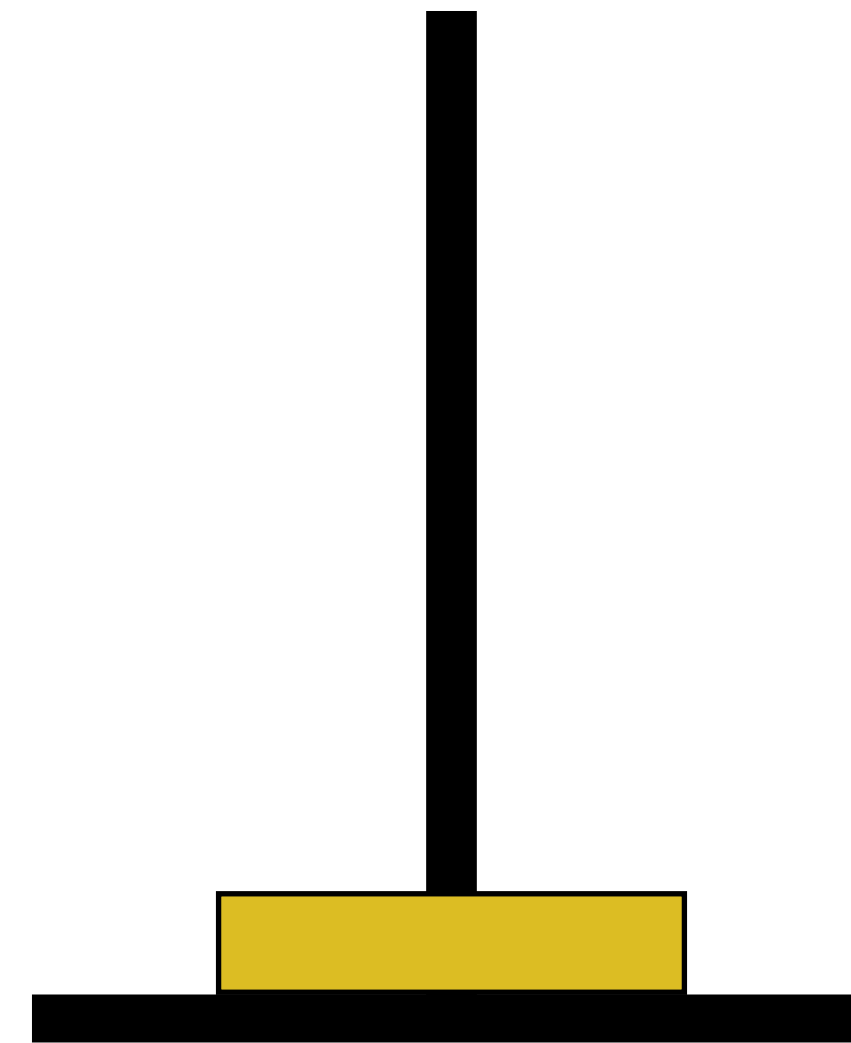
n=4



Stange A



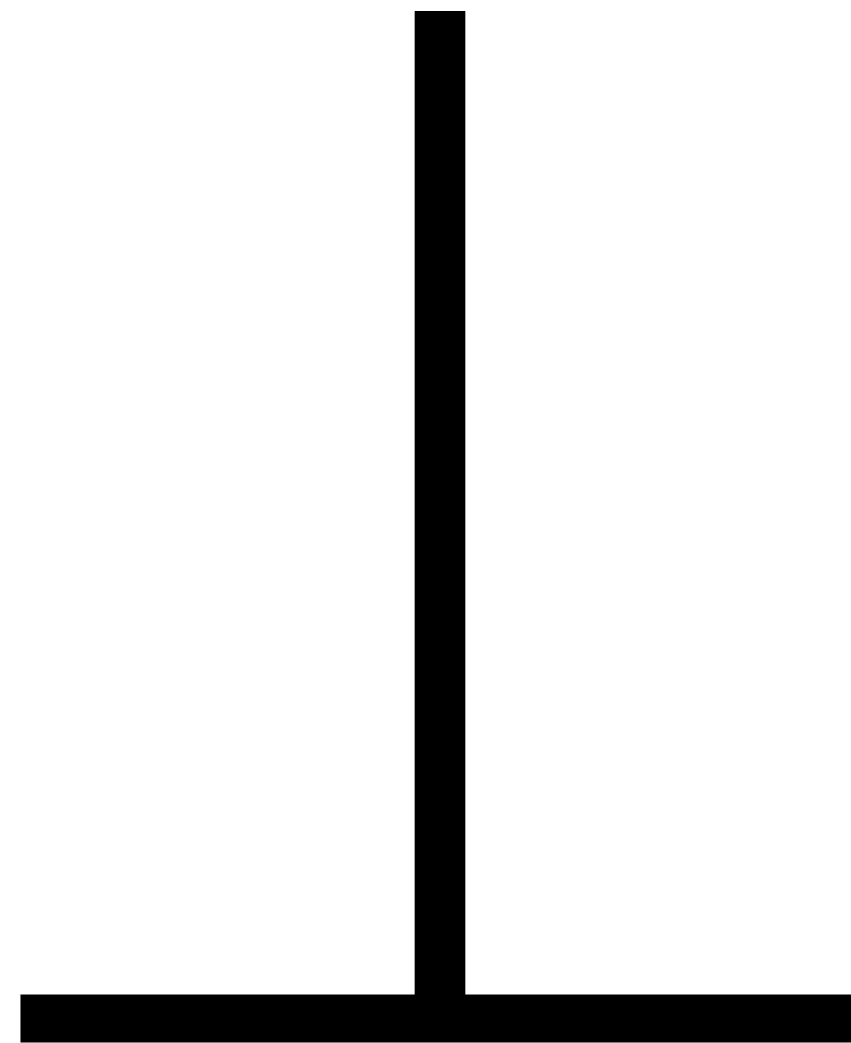
Stange B



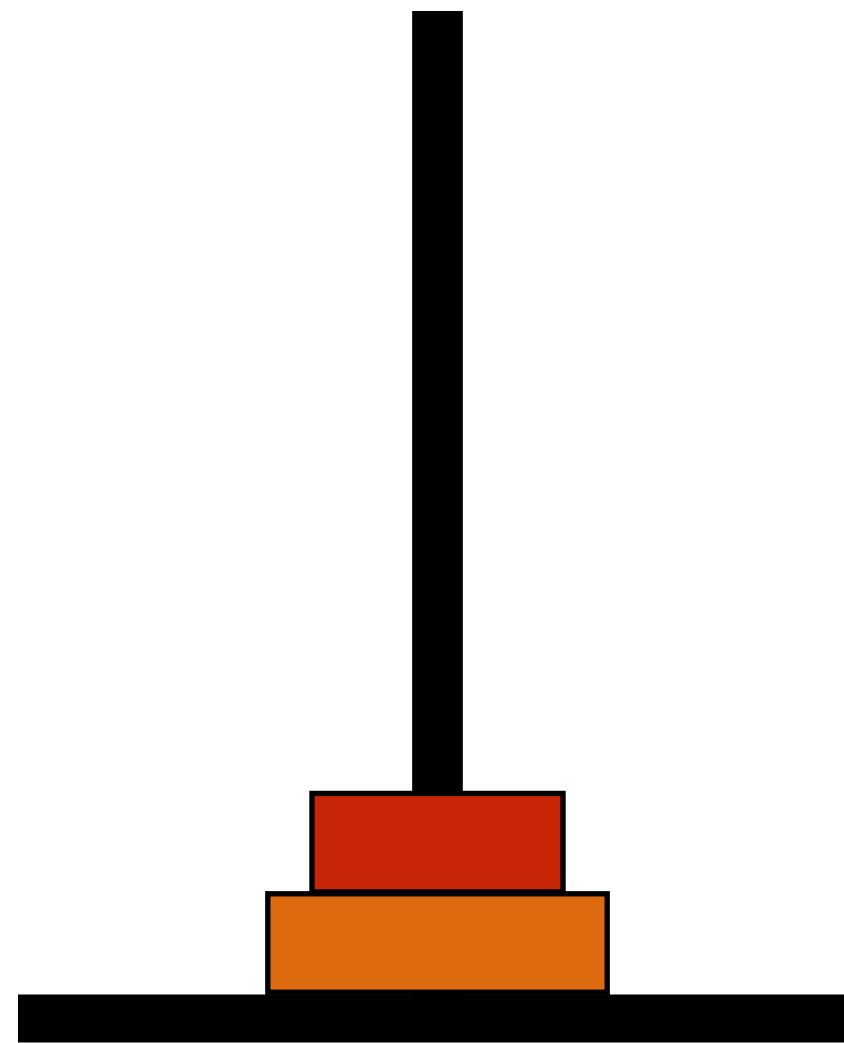
Stange C

Kleine Türme von Hanoi

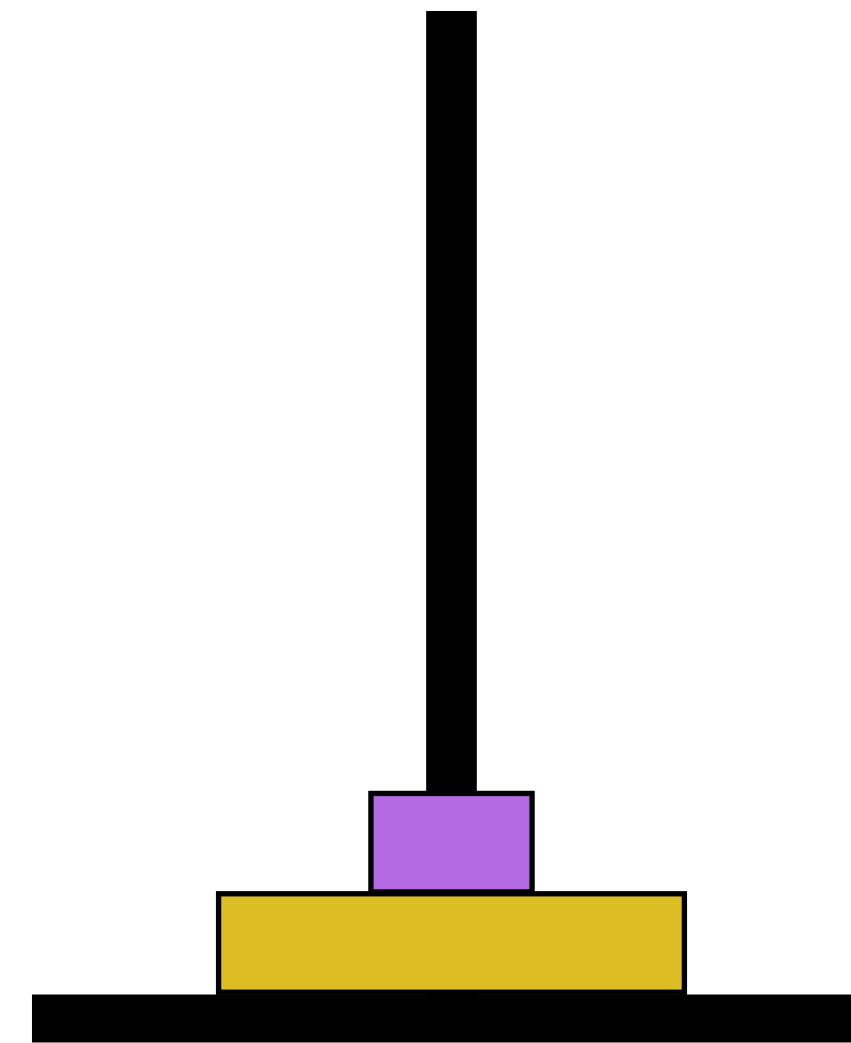
n=4



Stange A



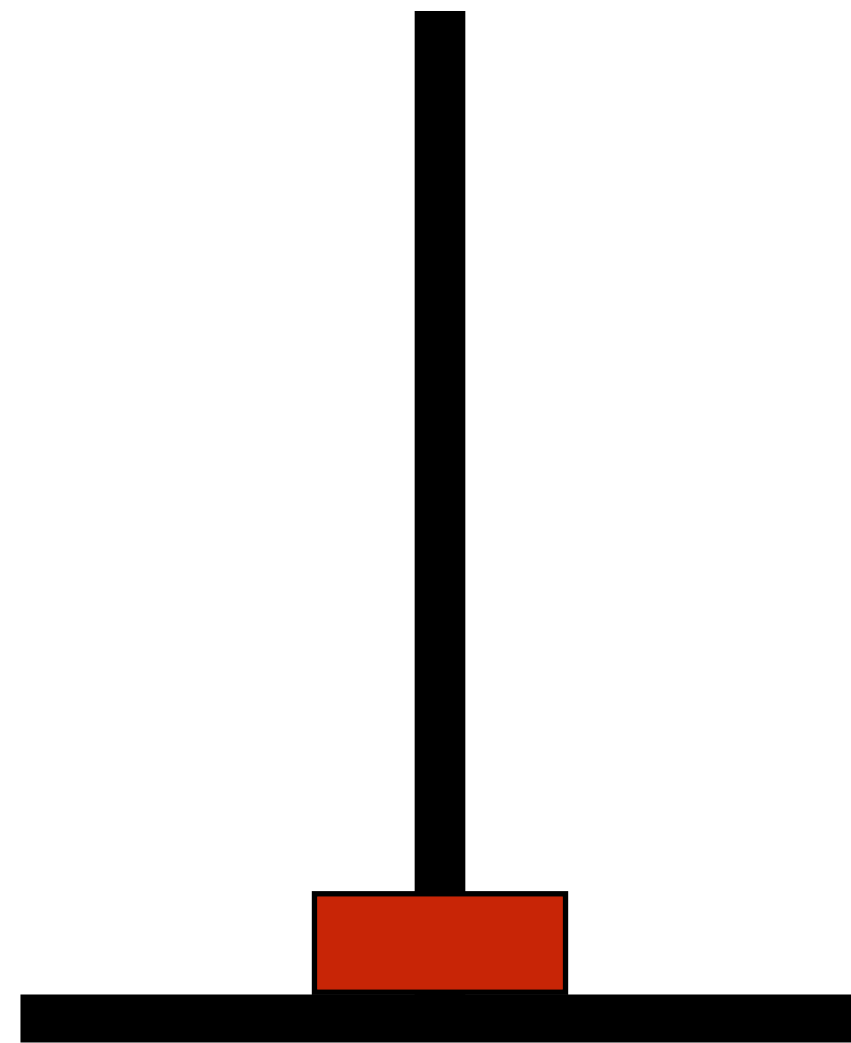
Stange B



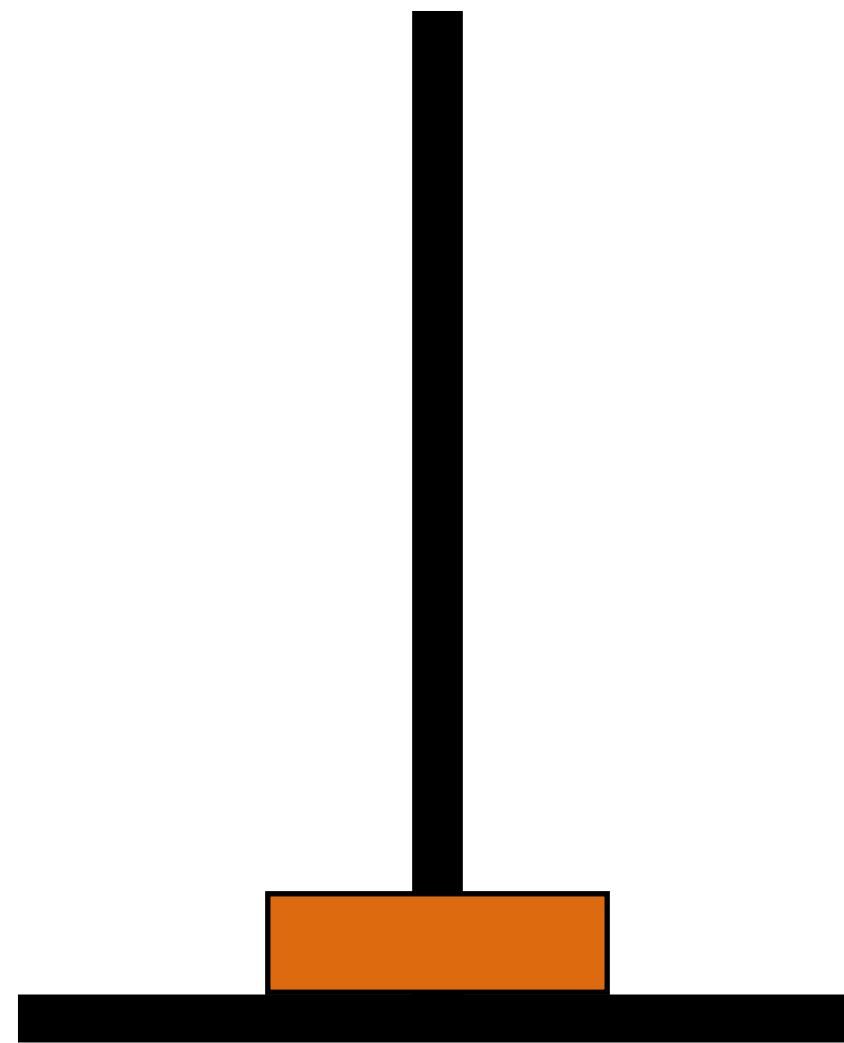
Stange C

Kleine Türme von Hanoi

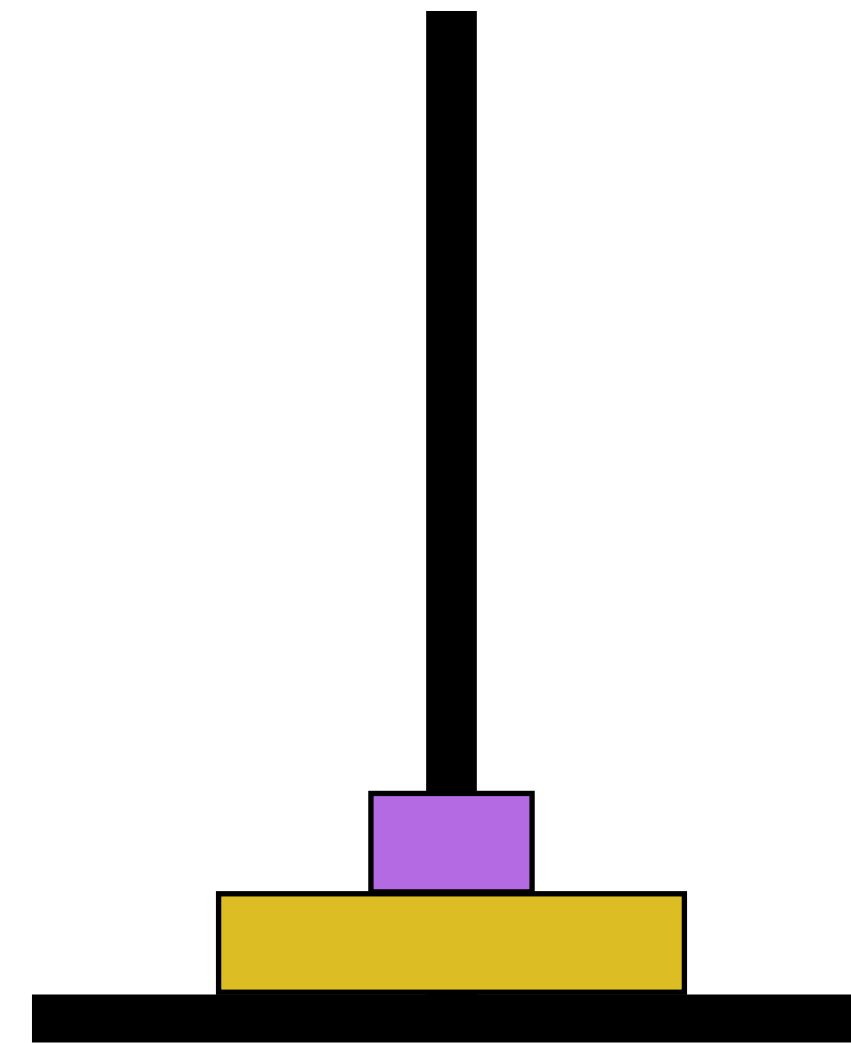
n=4



Stange A



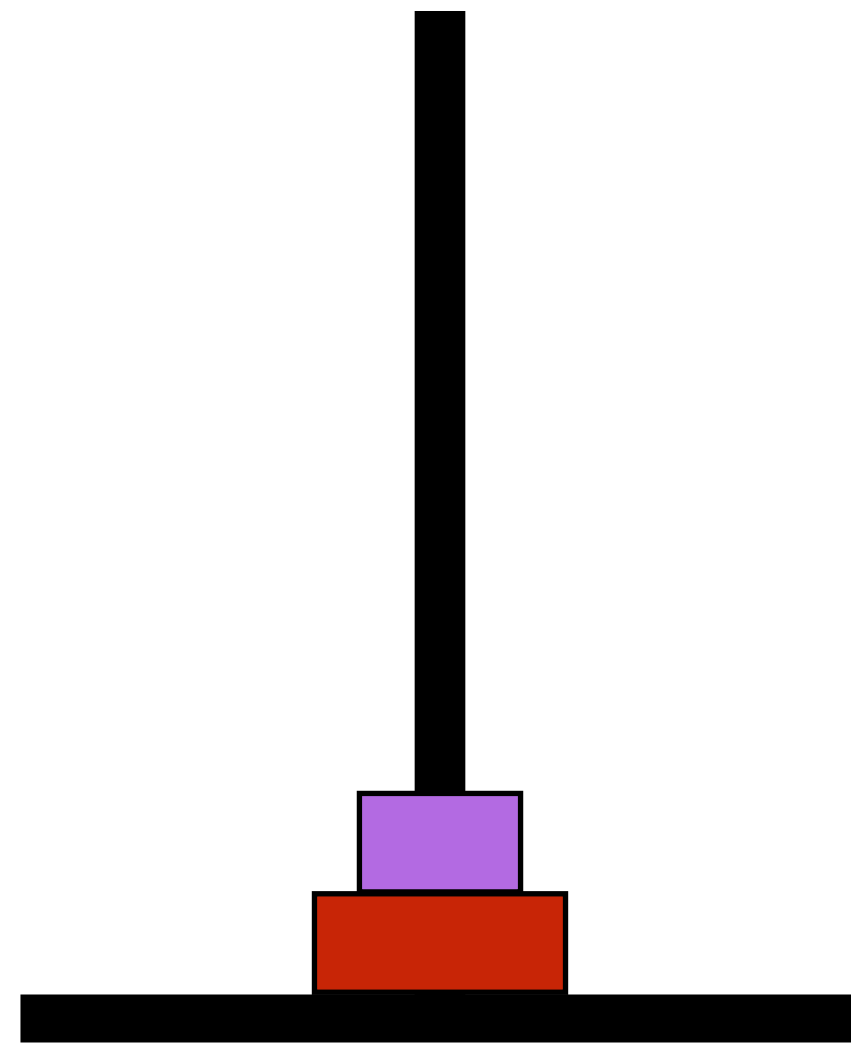
Stange B



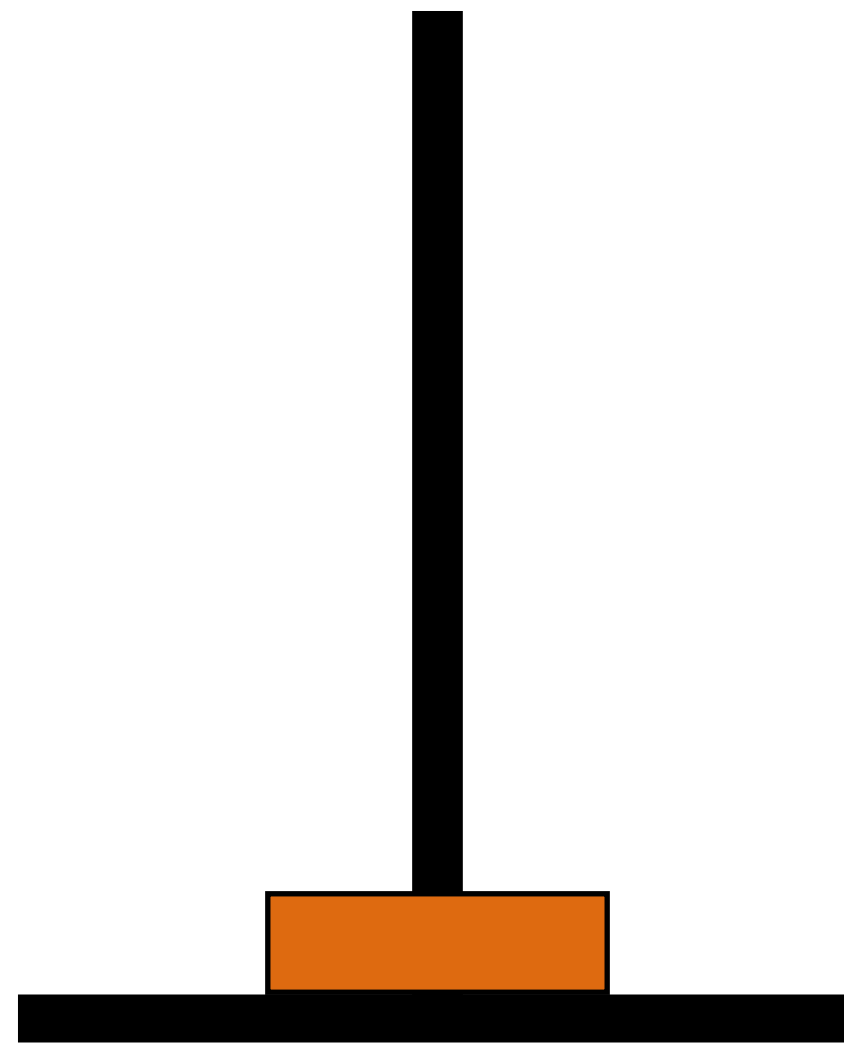
Stange C

Kleine Türme von Hanoi

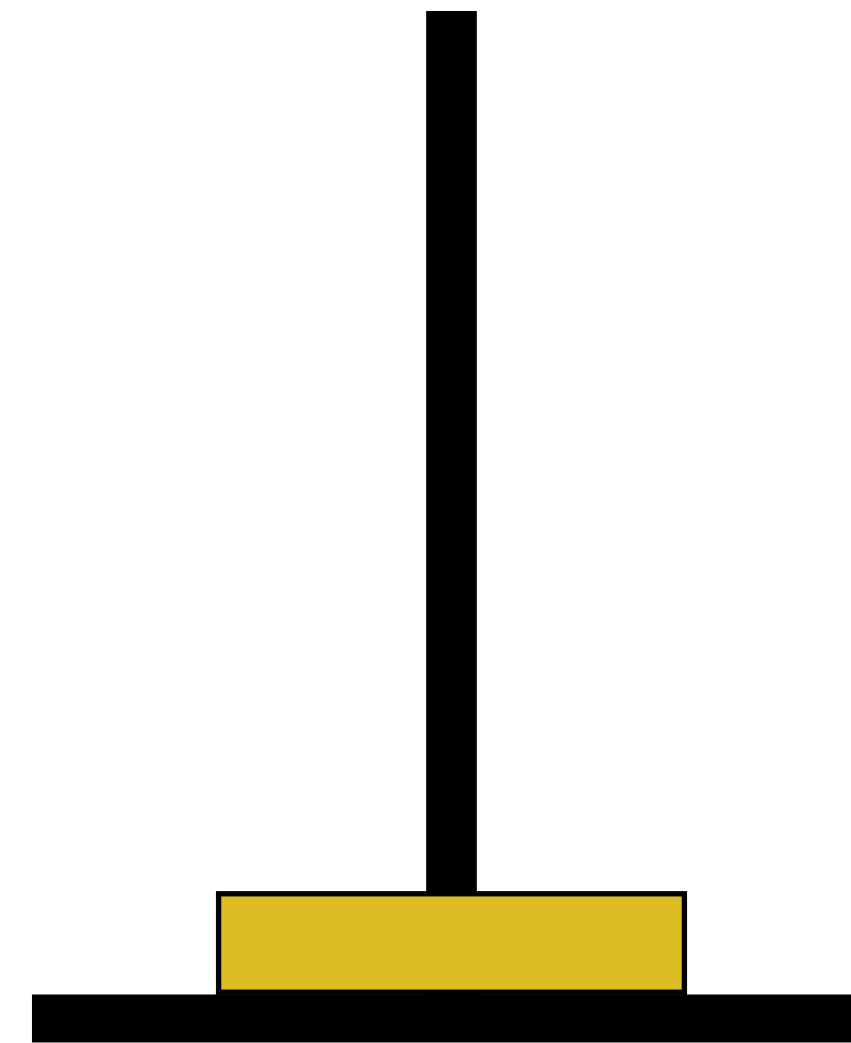
n=4



Stange A



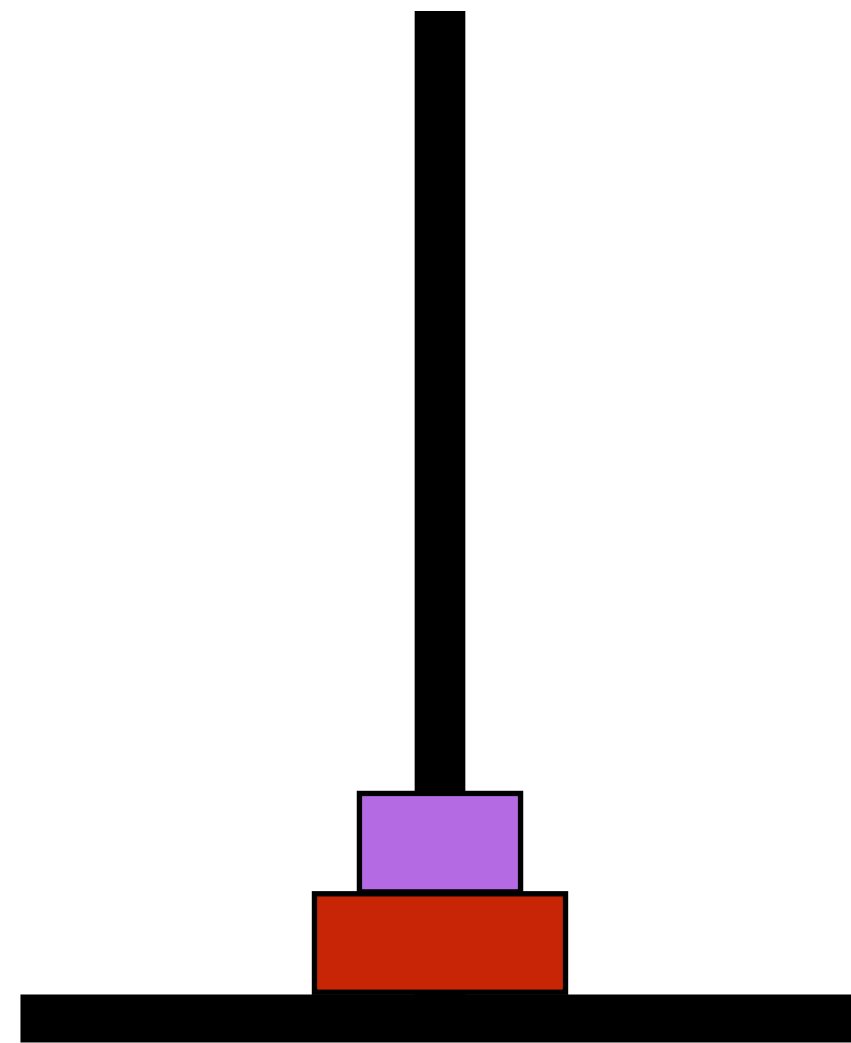
Stange B



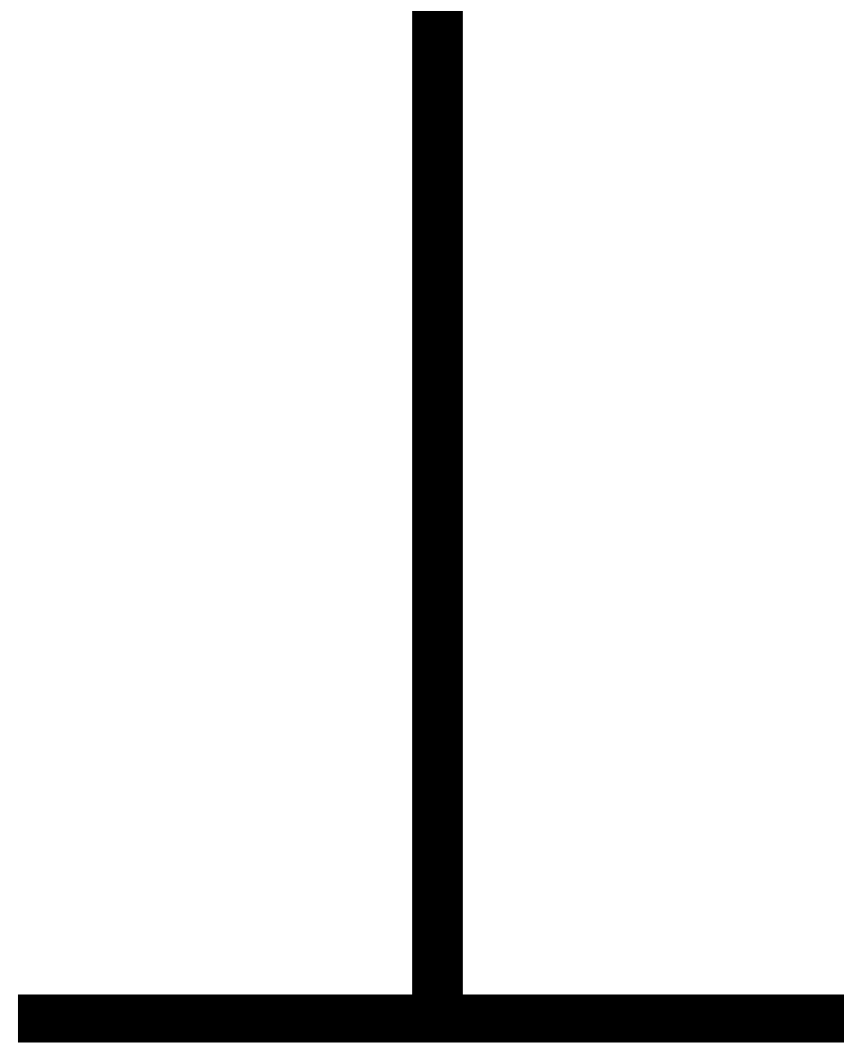
Stange C

Kleine Türme von Hanoi

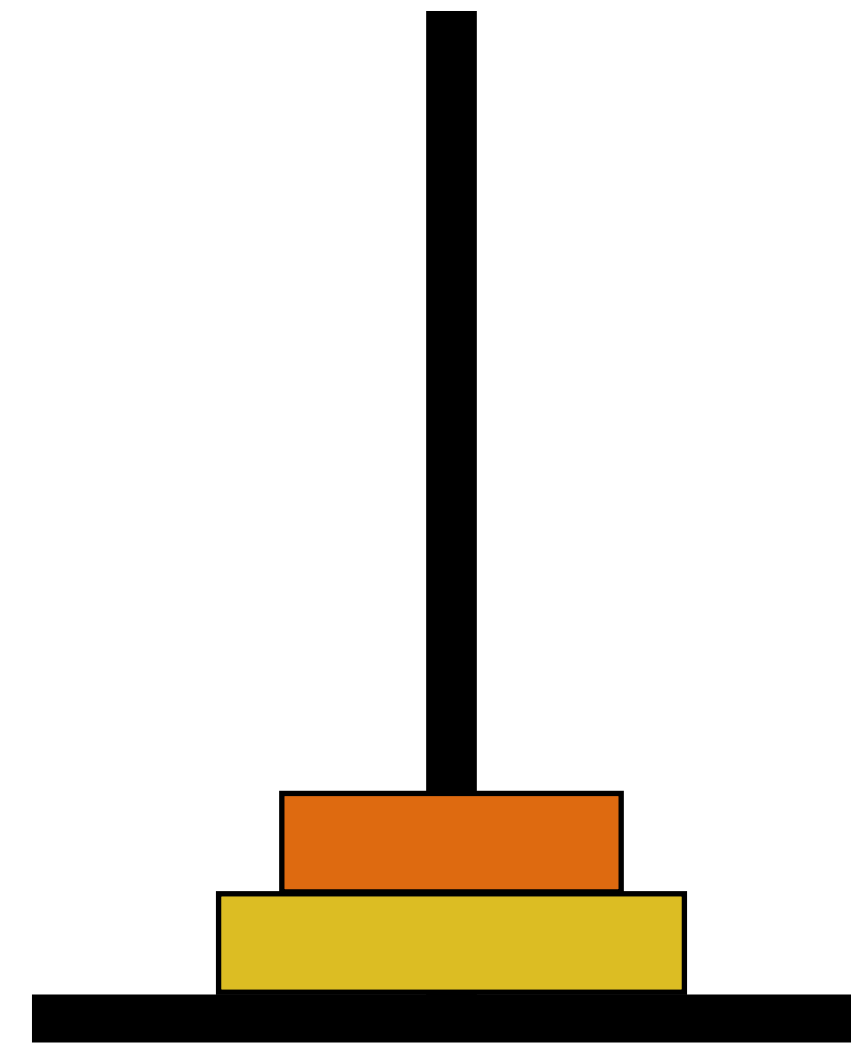
n=4



Stange A



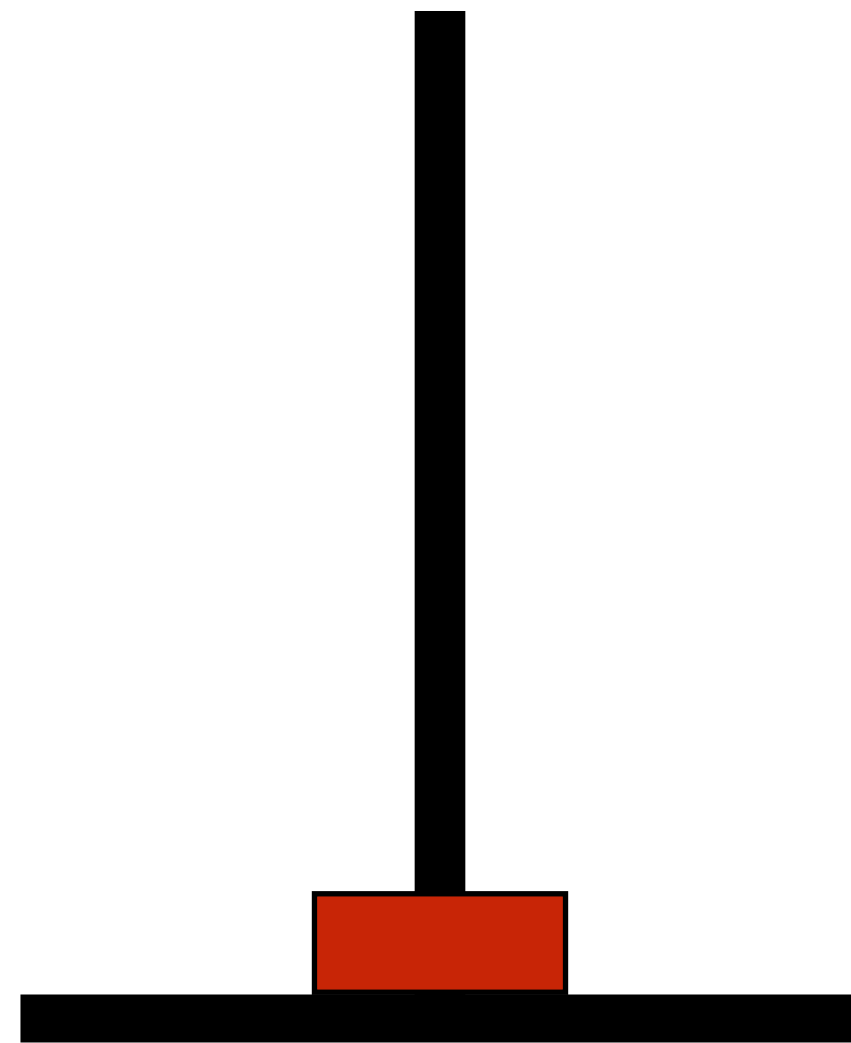
Stange B



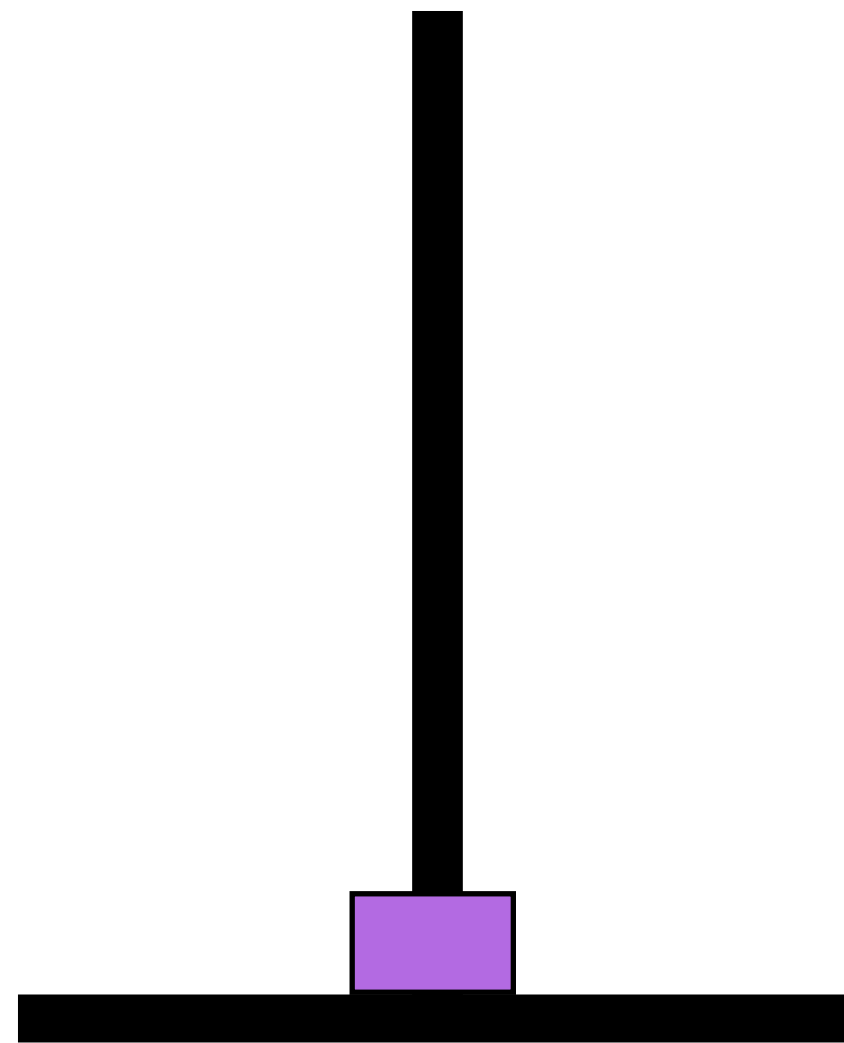
Stange C

Kleine Türme von Hanoi

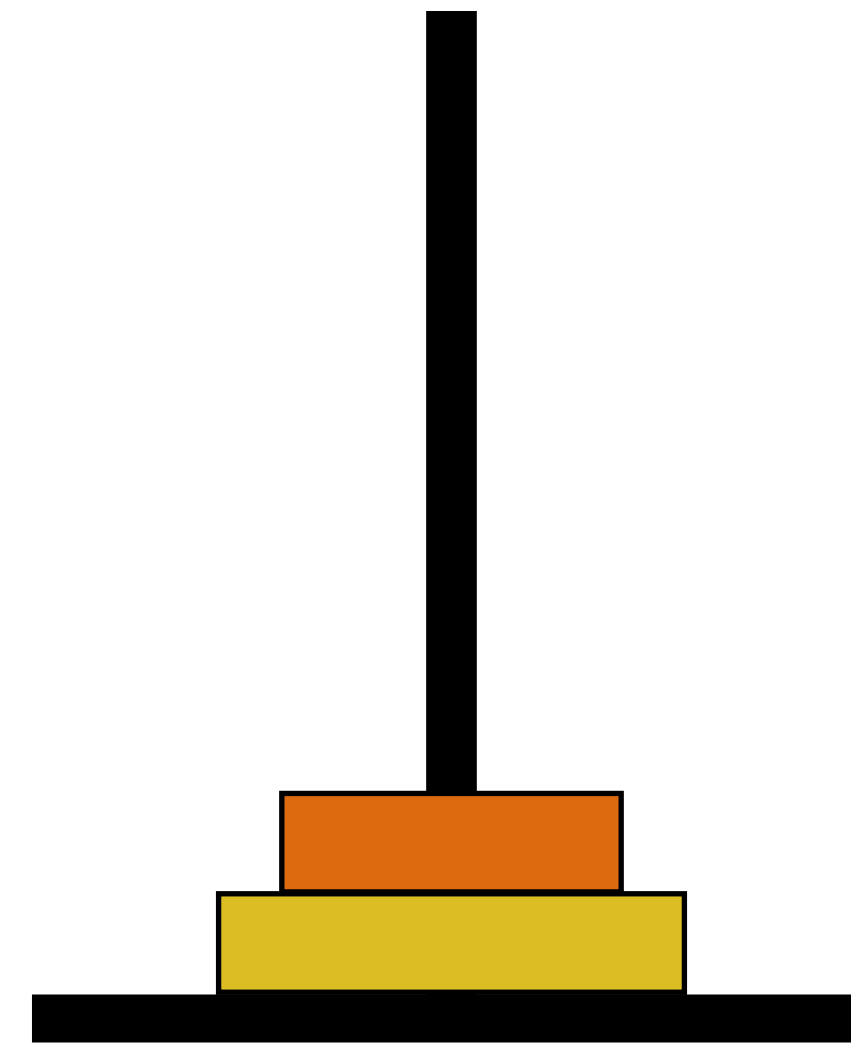
n=4



Stange A



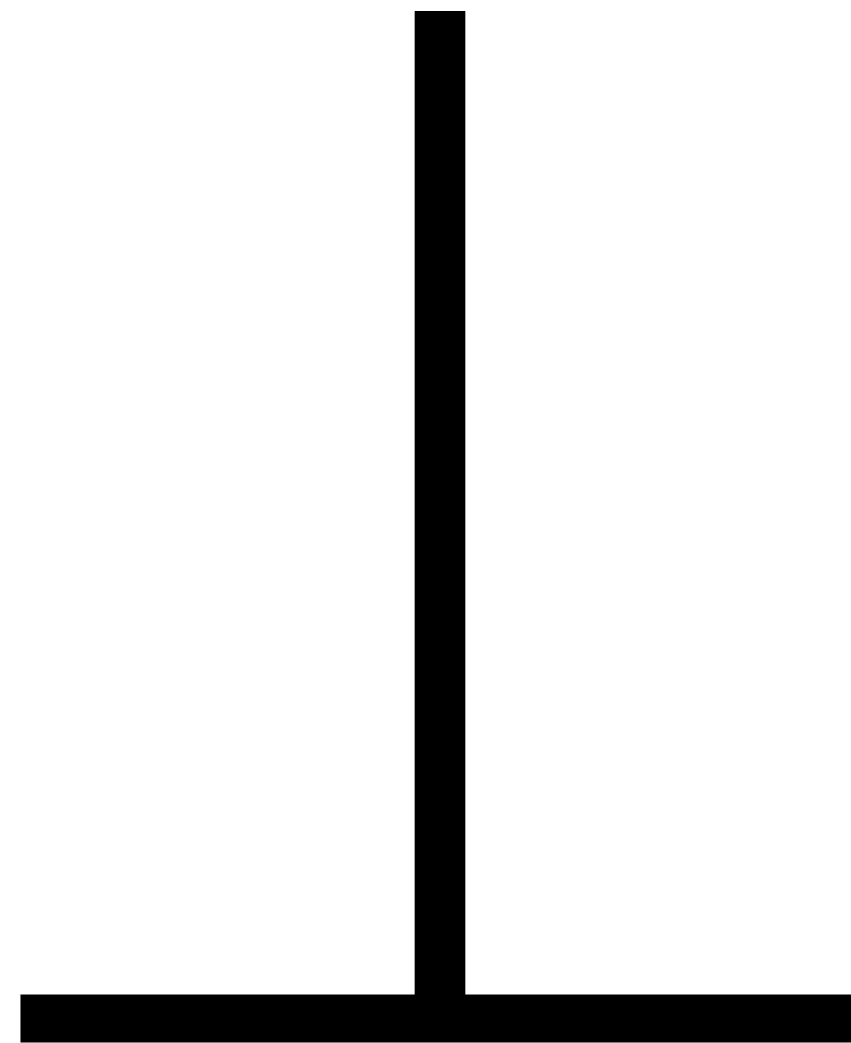
Stange B



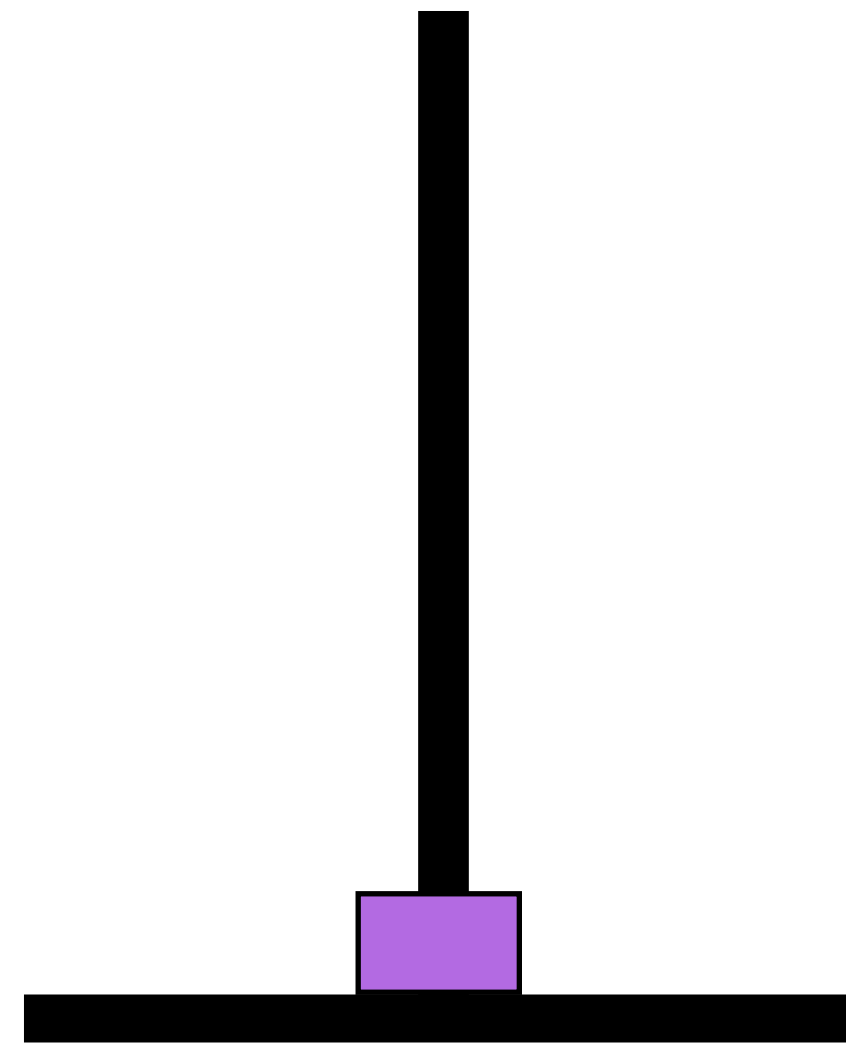
Stange C

Kleine Türme von Hanoi

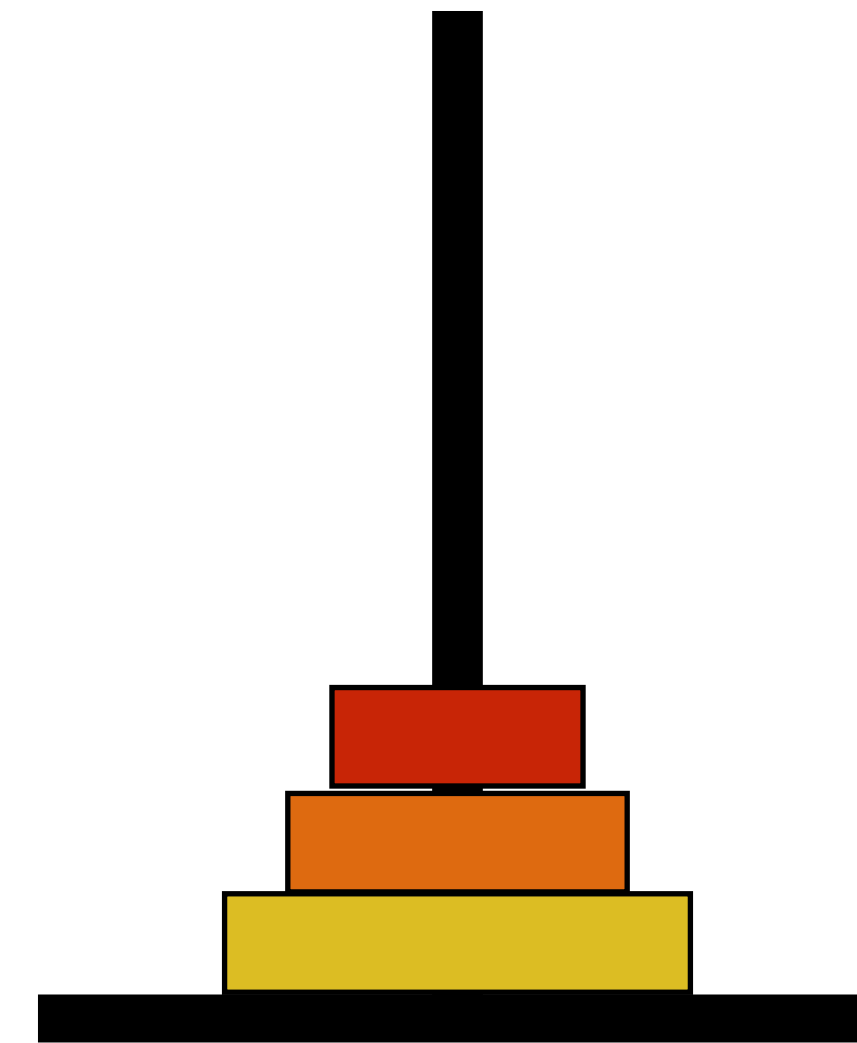
n=4



Stange A



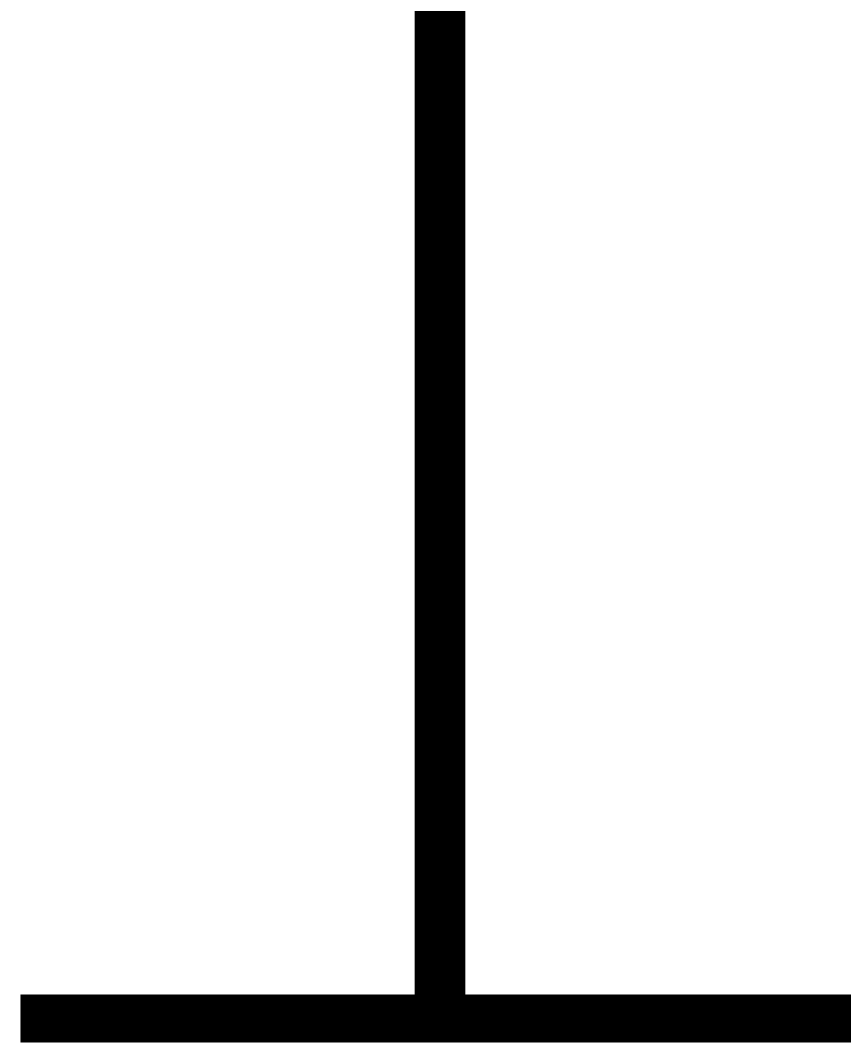
Stange B



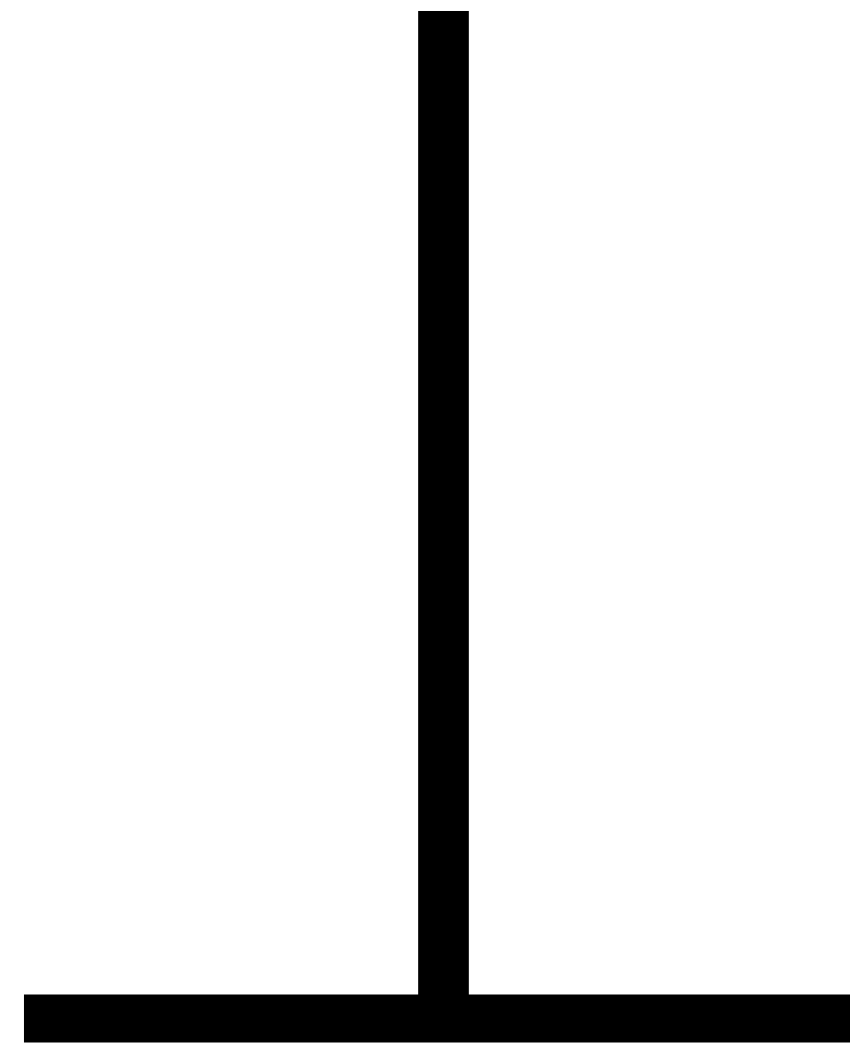
Stange C

Kleine Türme von Hanoi

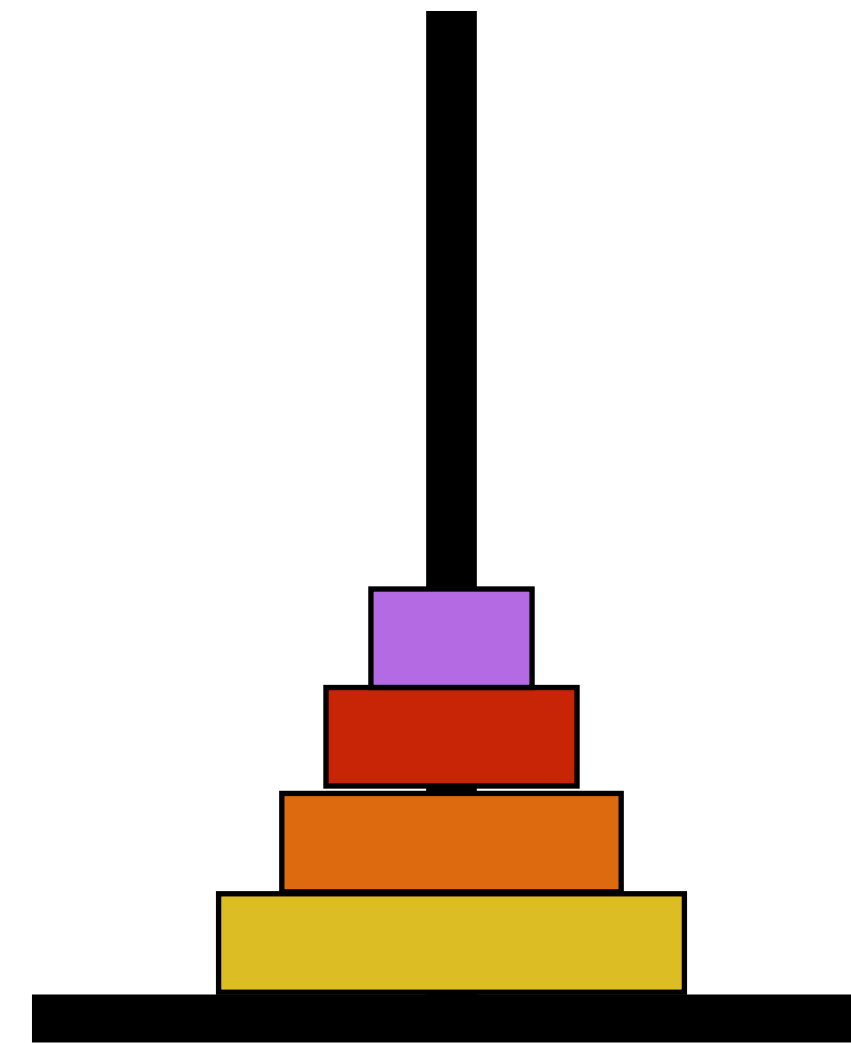
n=4



Stange A



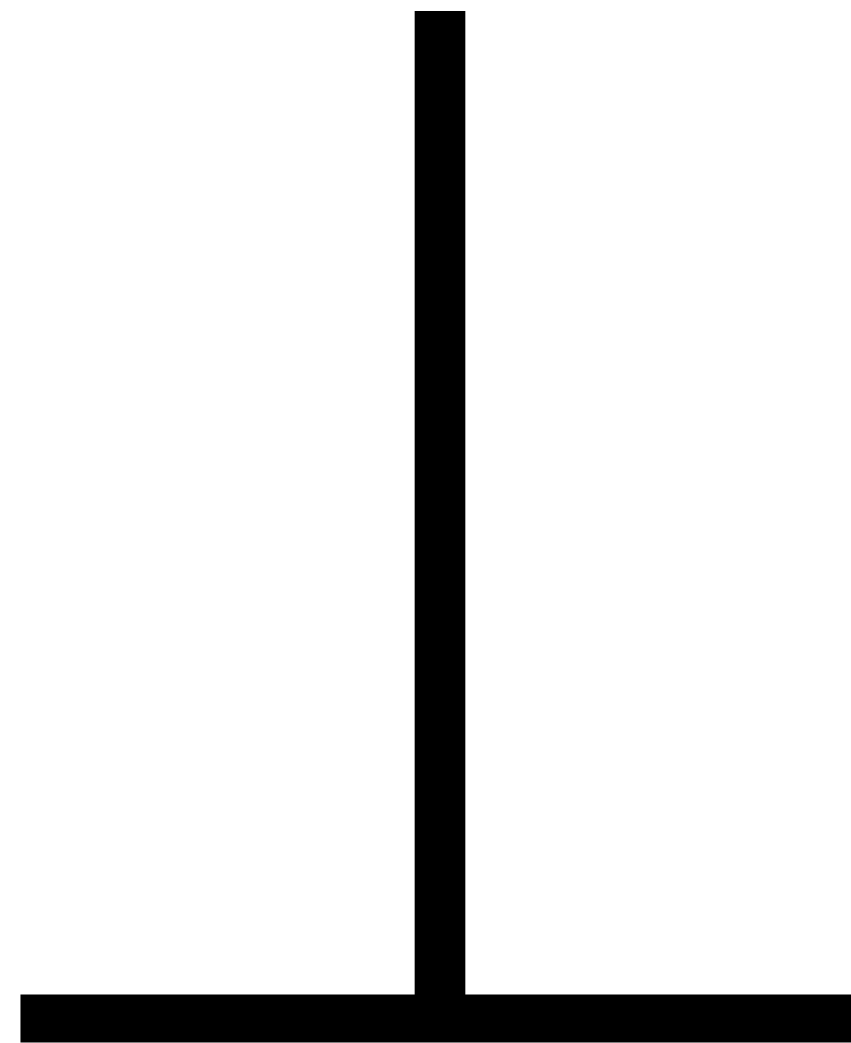
Stange B



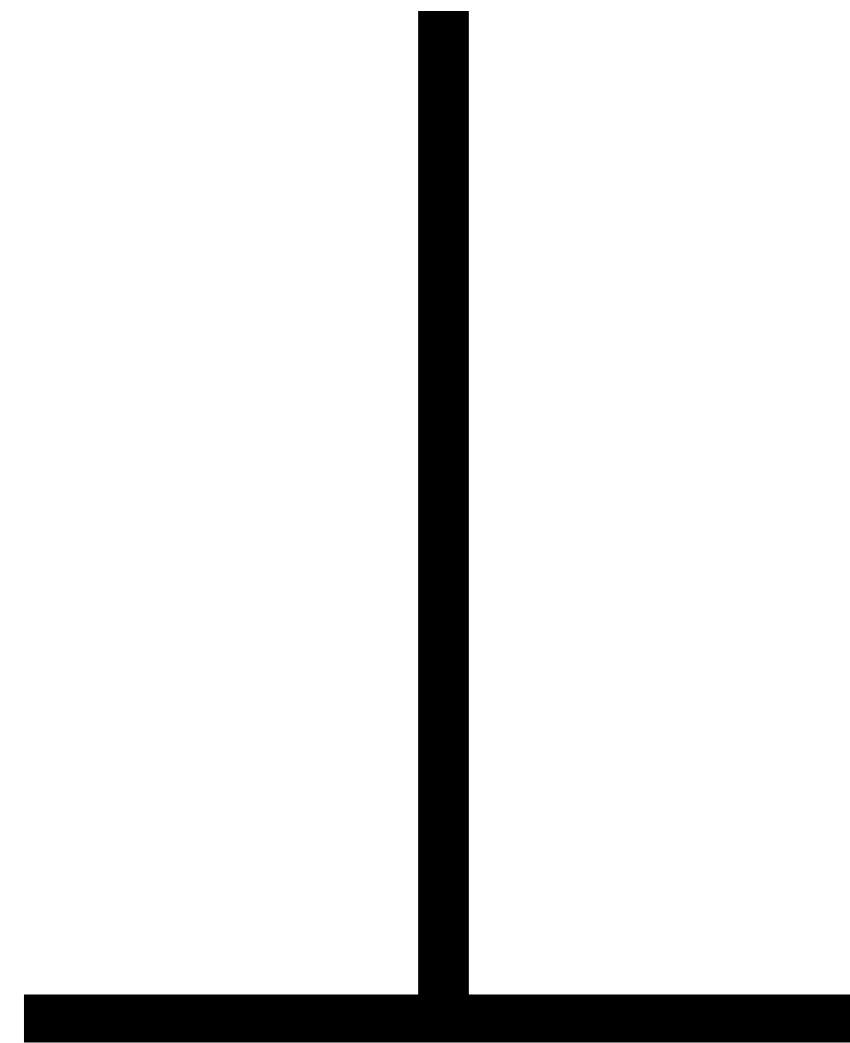
Stange C

Kleine Türme von Hanoi

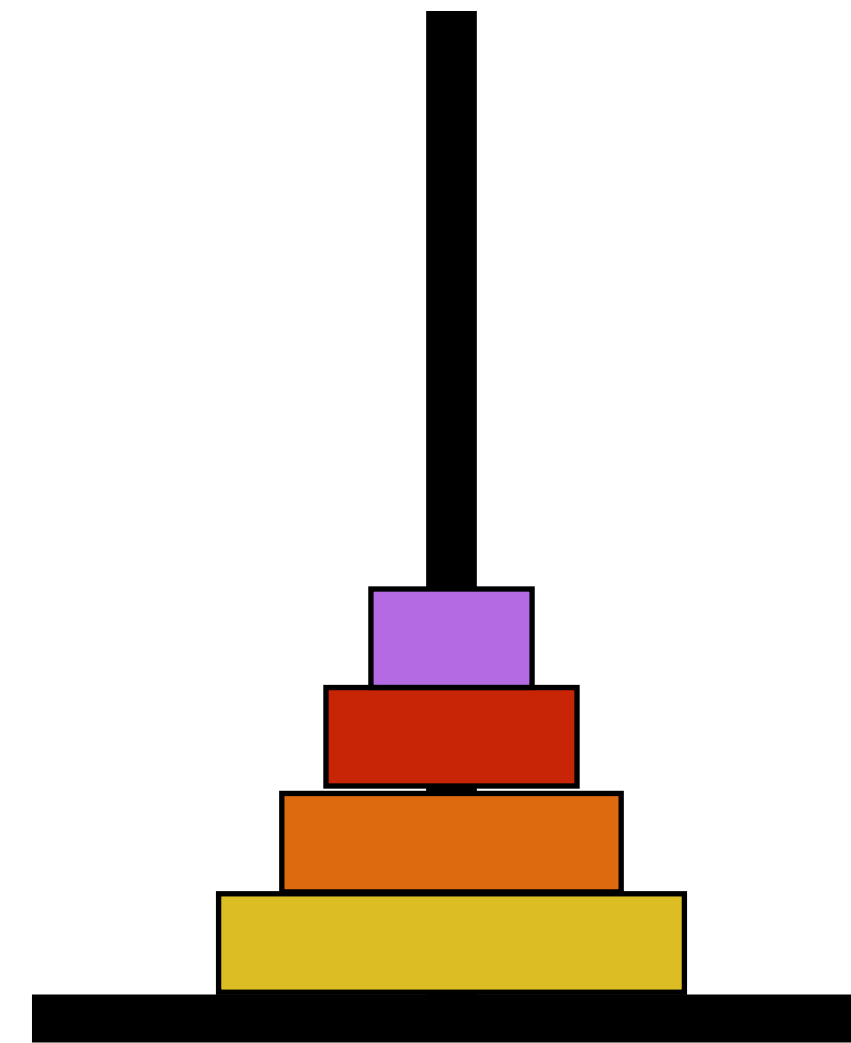
n=4



Stange A



Stange B

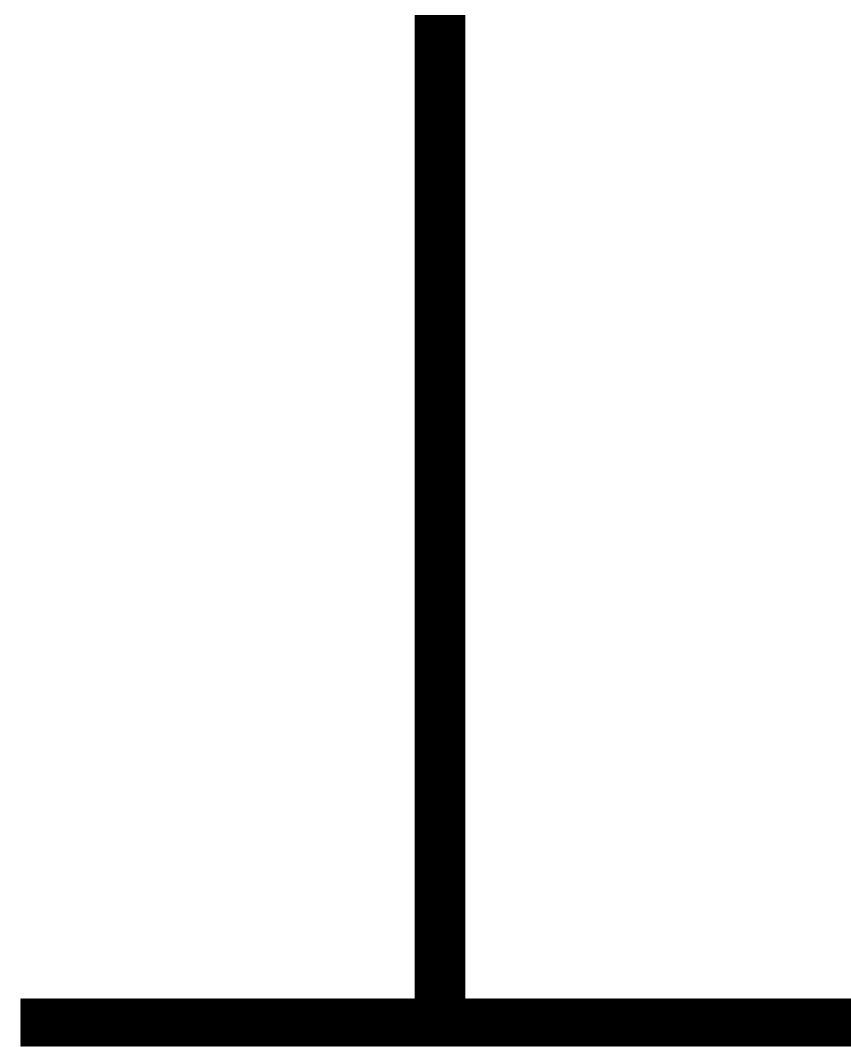


Stange C

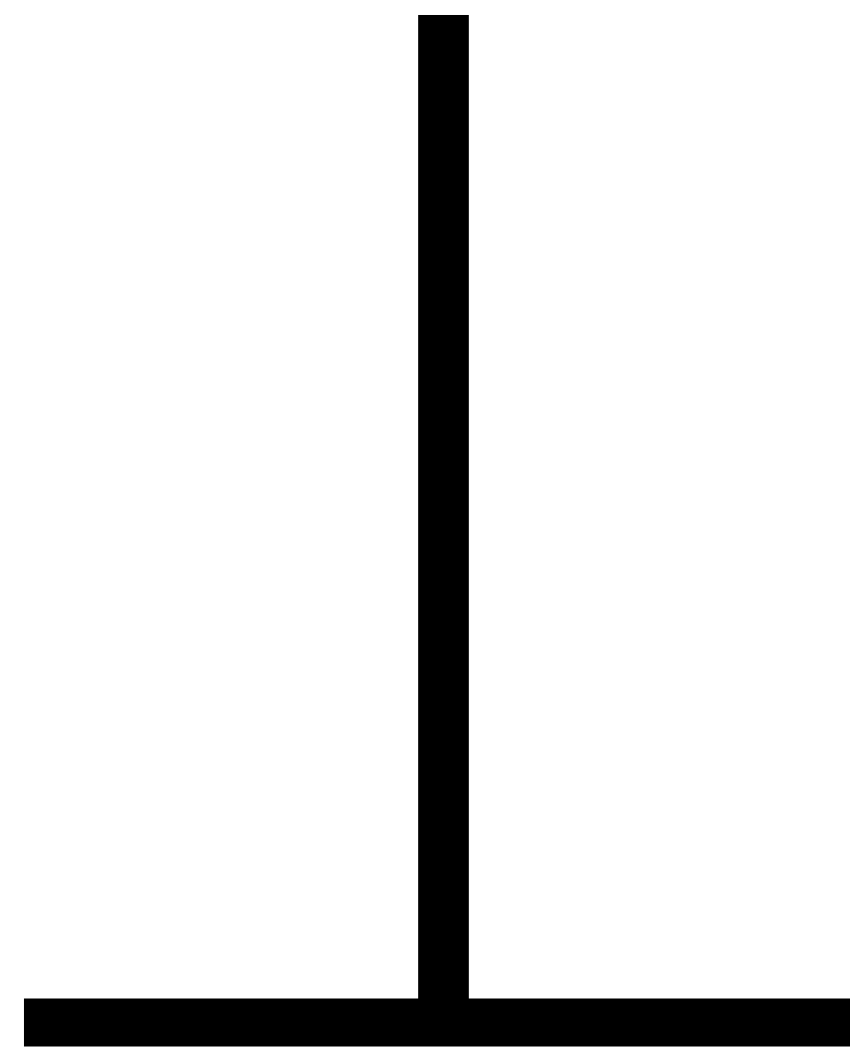


Kleine Türme von Hanoi

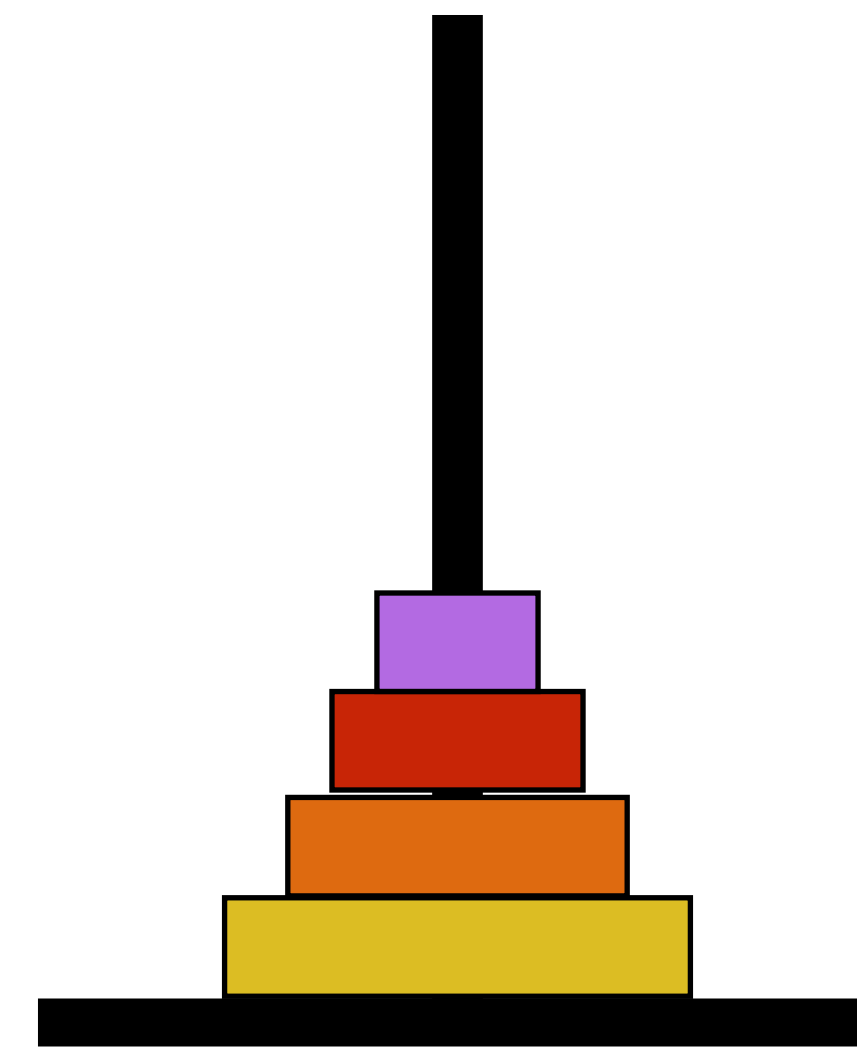
n=4



Stange A



Stange B

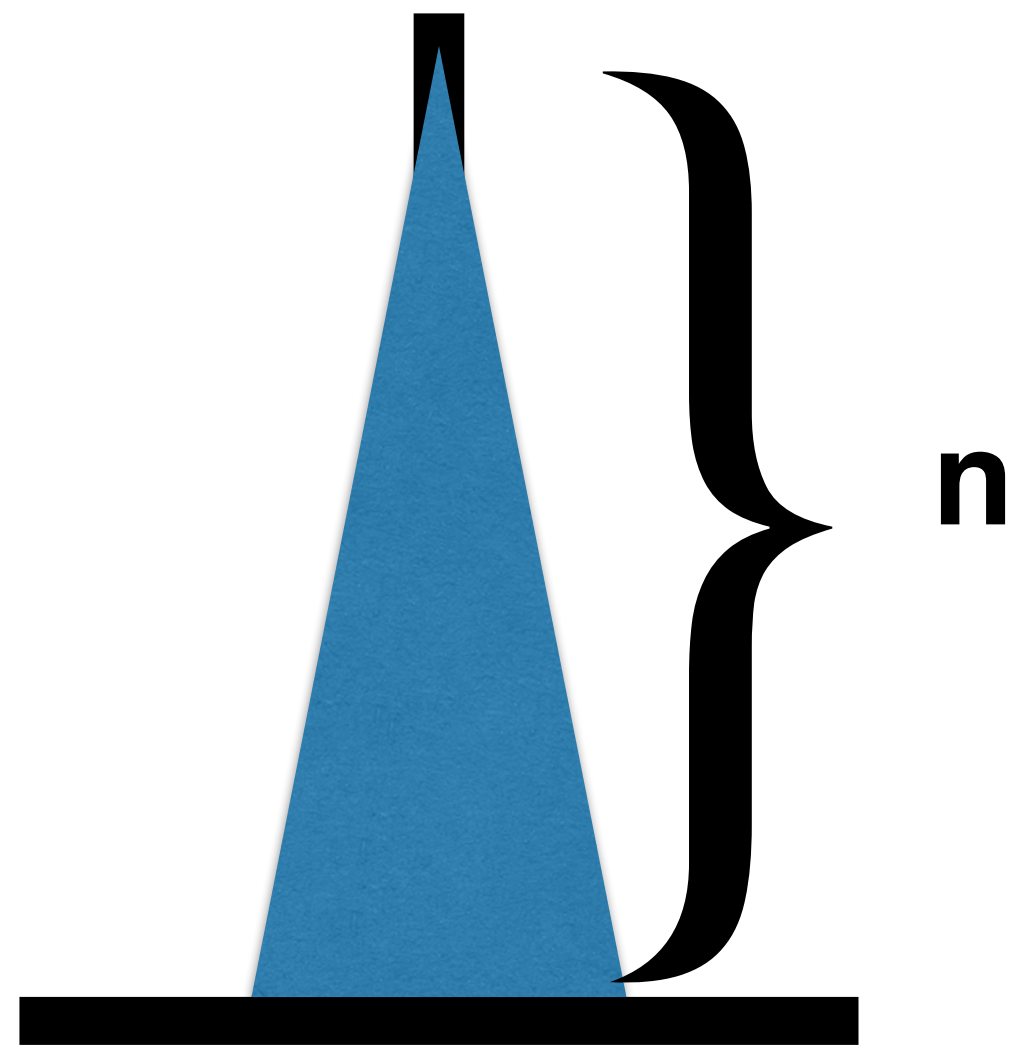


Stange C

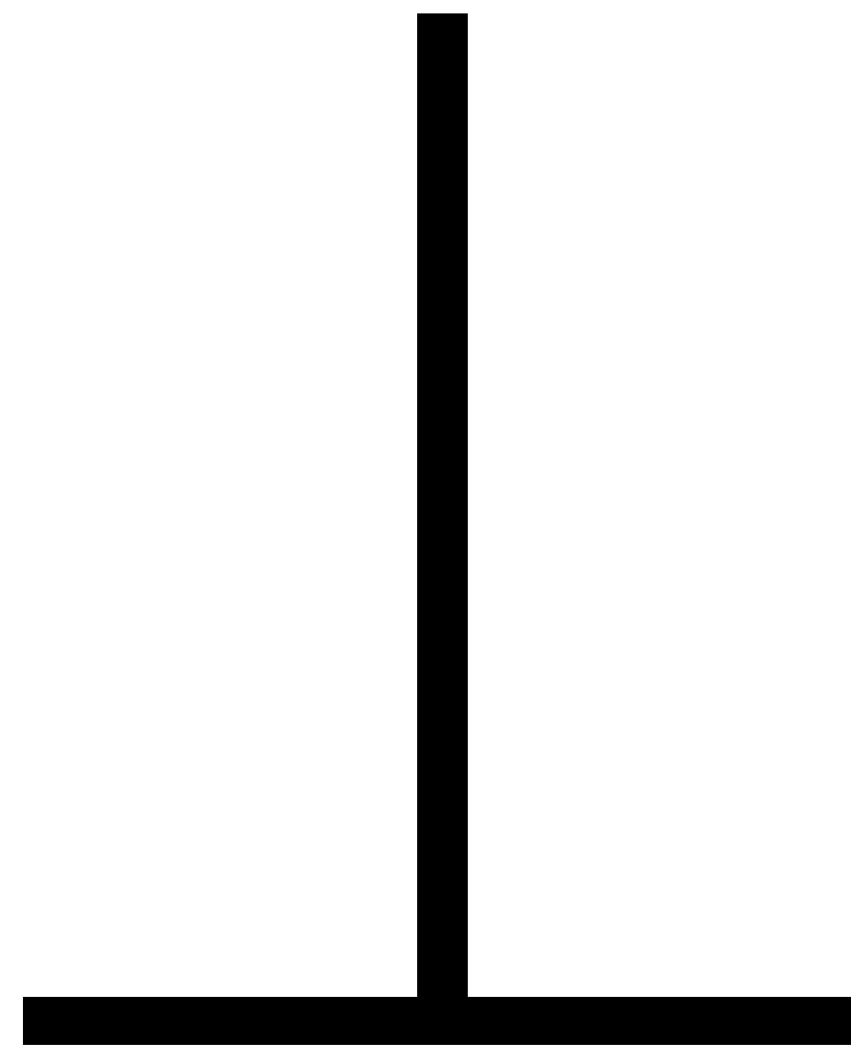
Kann dies für alle Turmhöhen **n** realisiert werden?

Leichte Verallgemeinerung

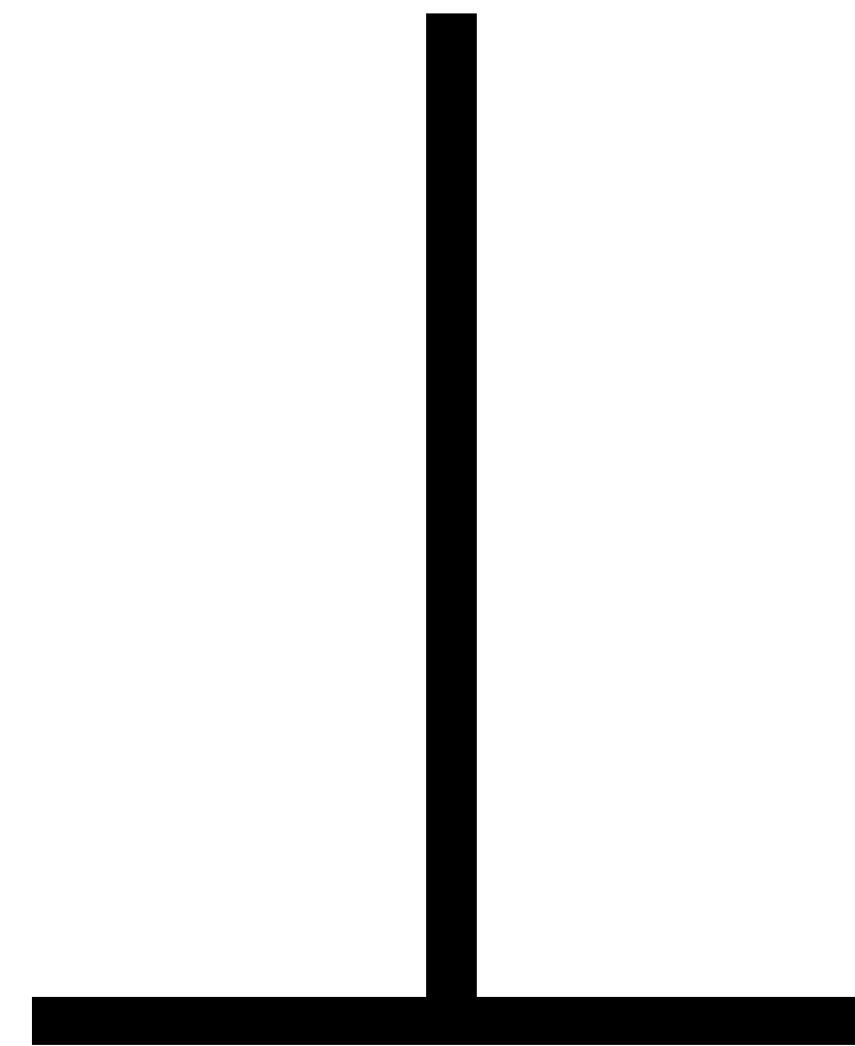
Wie bewegen wir (geordneten) Turm der Höhe **n** von beliebiger **Startstange** zu beliebiger **Zielstange**?



Startstange



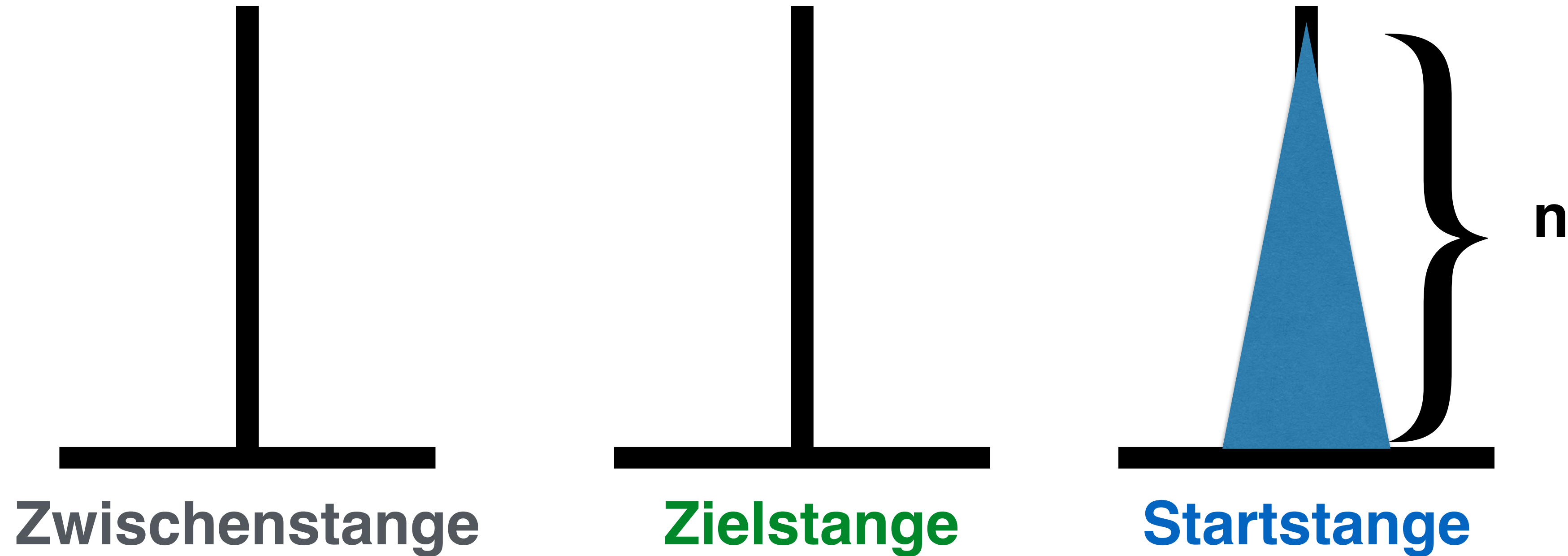
Zwischenstange



Zielstange

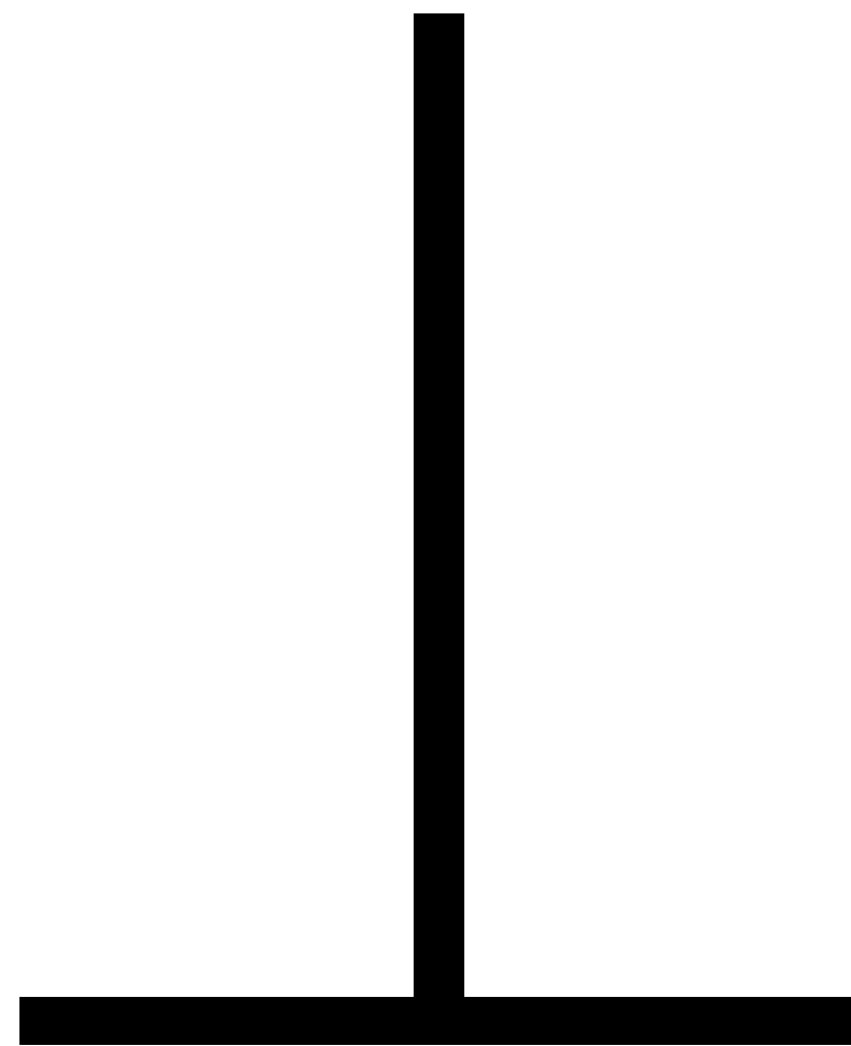
Leichte Verallgemeinerung

Wie bewegen wir (geordneten) Turm der Höhe **n** von beliebiger **Startstange** zu beliebiger **Zielstange**?

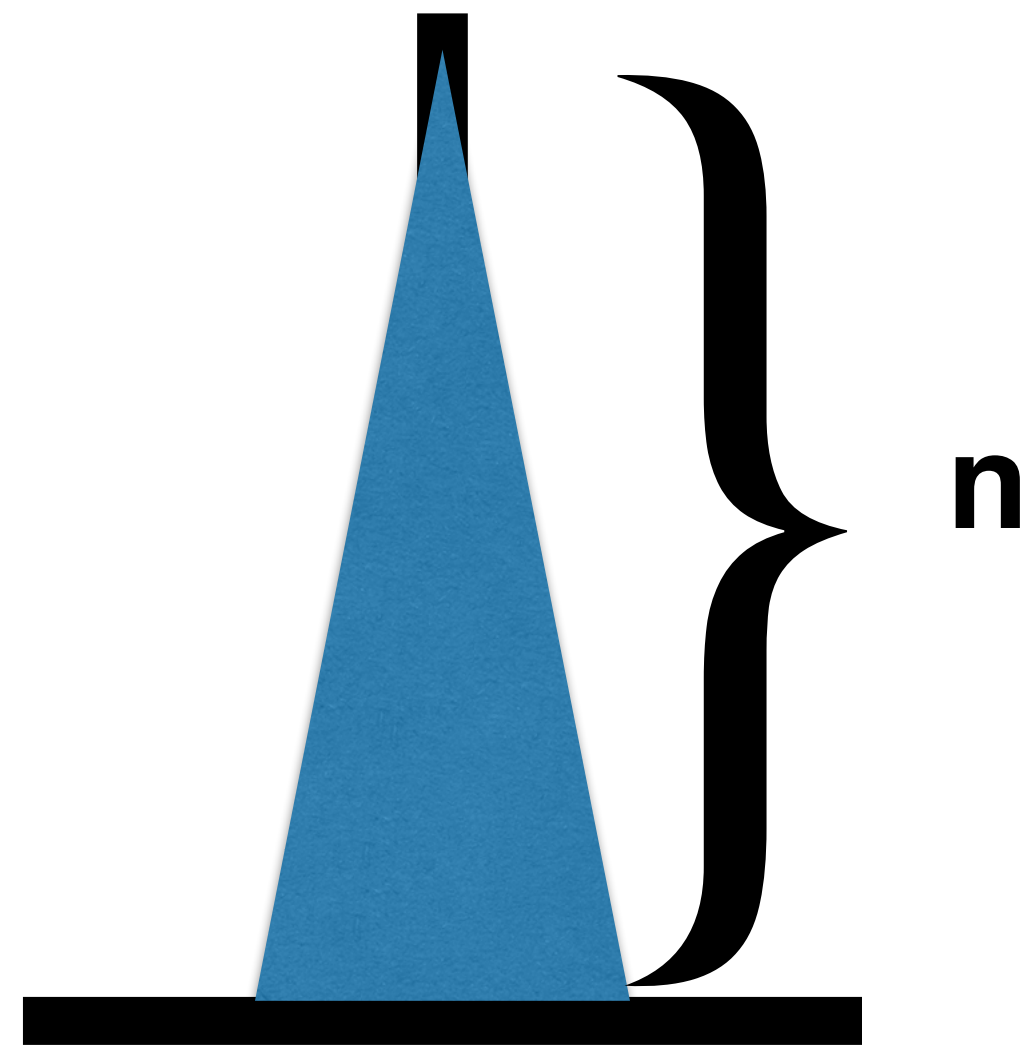


Leichte Verallgemeinerung

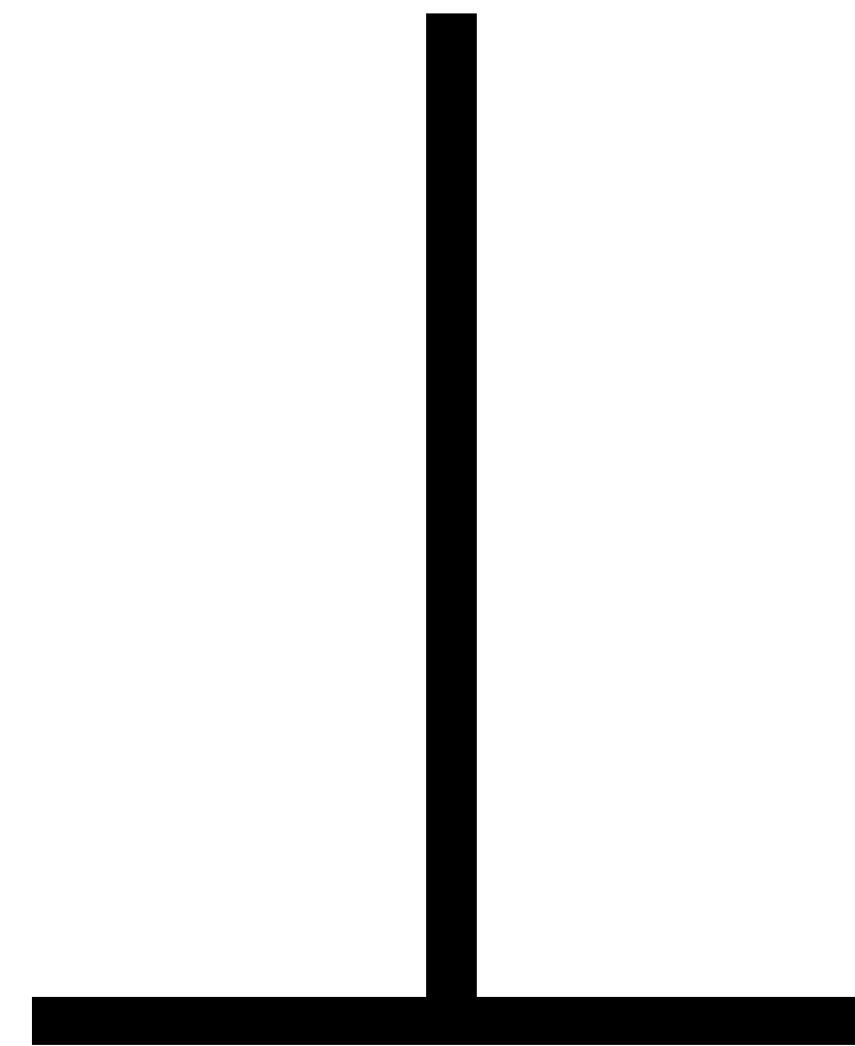
Wie bewegen wir (geordneten) Turm der Höhe **n** von beliebiger **Startstange** zu beliebiger **Zielstange**?



Zielstange



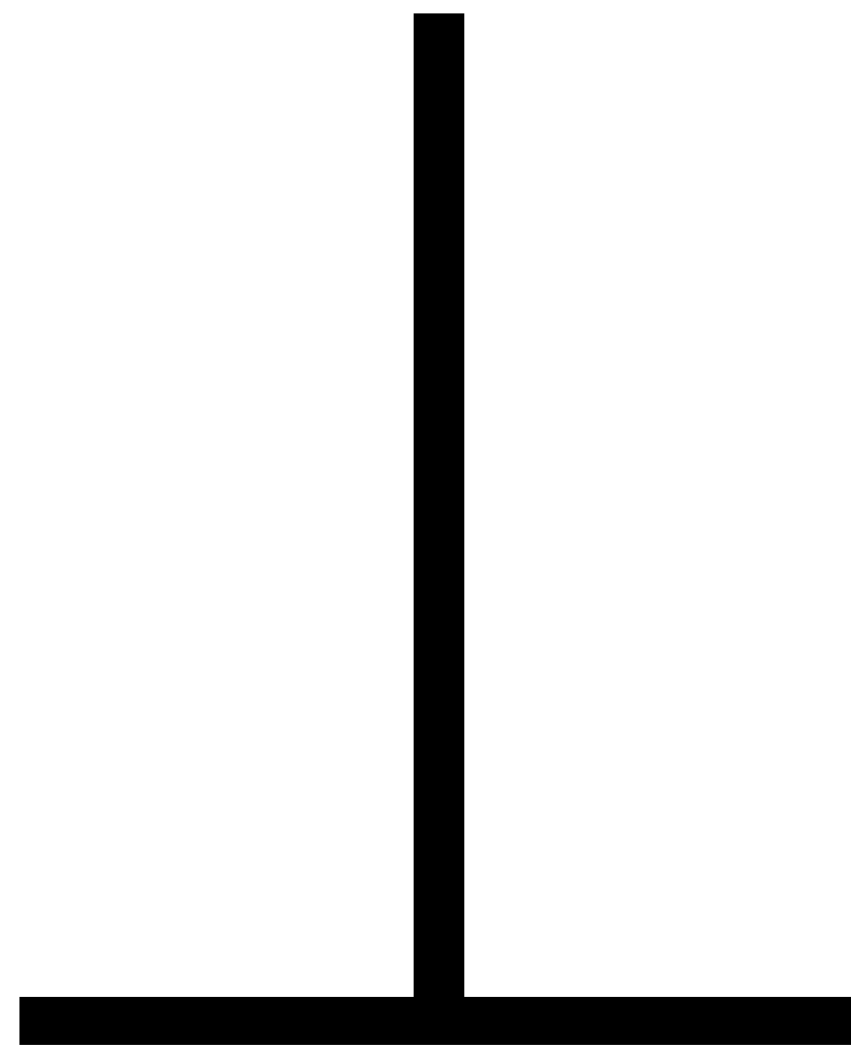
Startstange



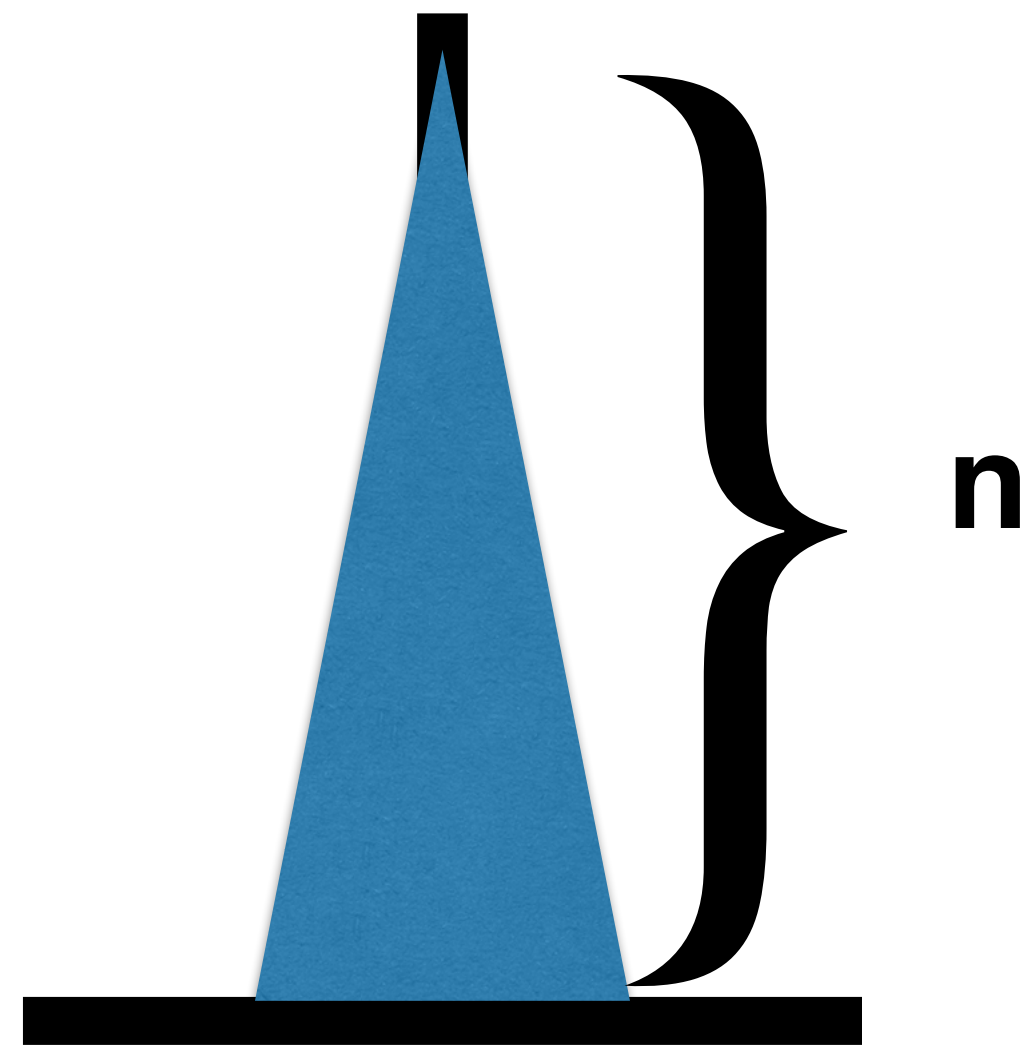
Zwischenstange

Leichte Verallgemeinerung

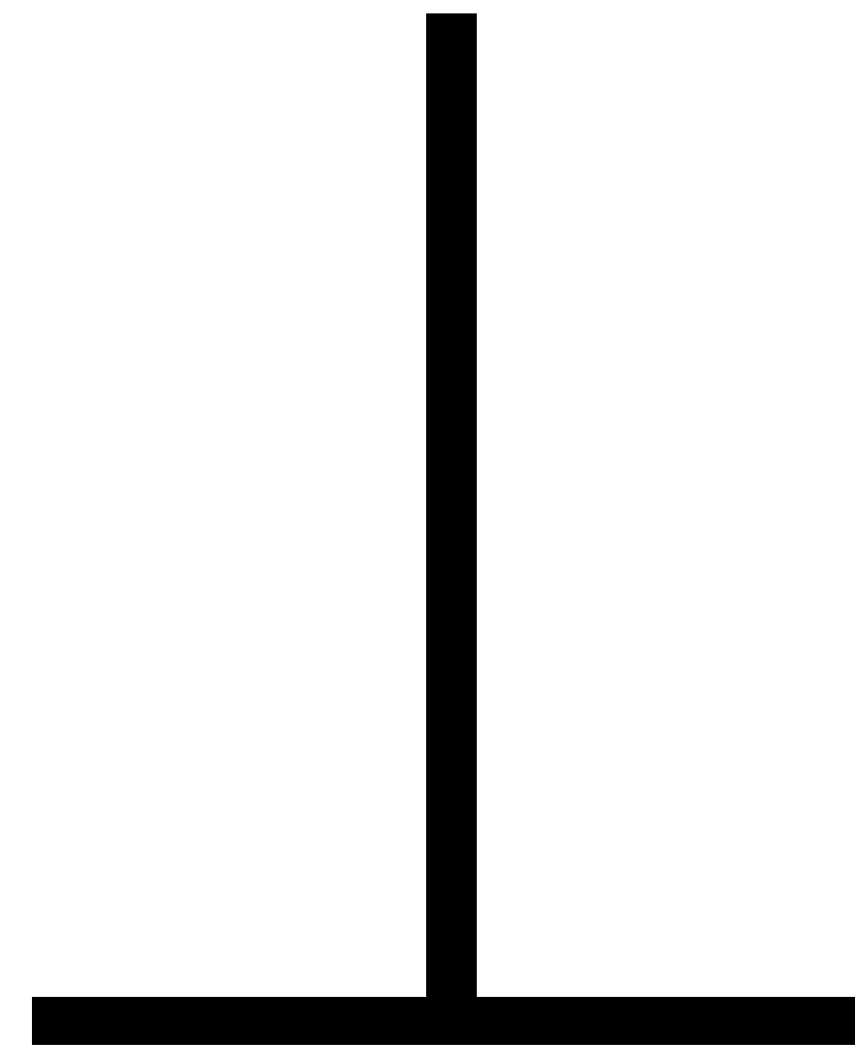
Wie bewegen wir (geordneten) Turm der Höhe **n** von beliebiger **Startstange** zu beliebiger **Zielstange**?



Zielstange



Startstange



Zwischenstange

Rekursion!

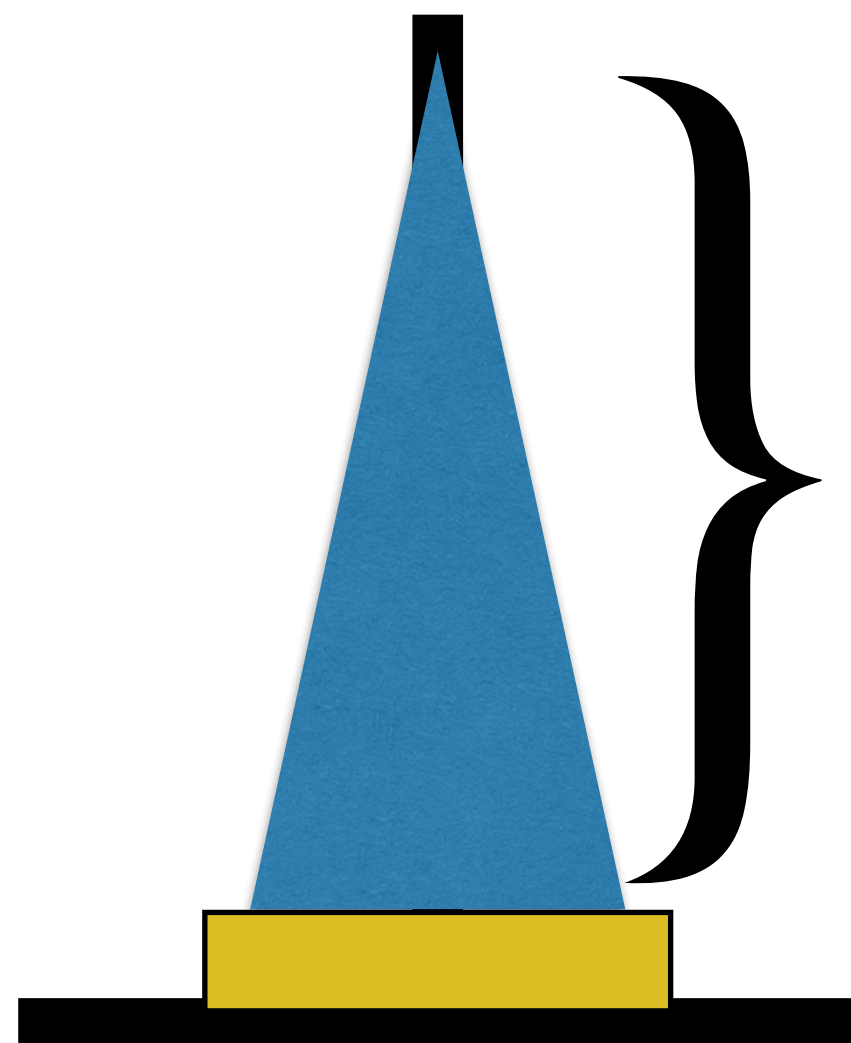
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

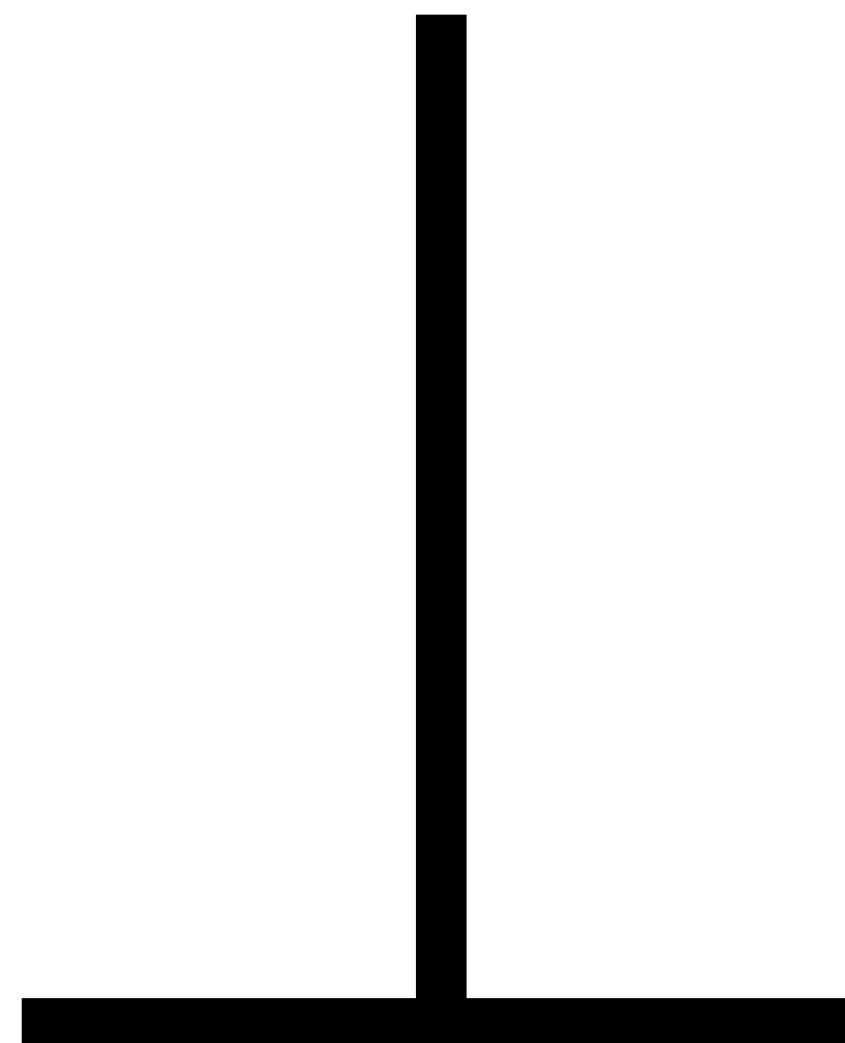
Leichte Verallgemeinerung

Die triviale Instanz **$n=1$** haben wir bereits gelöst!

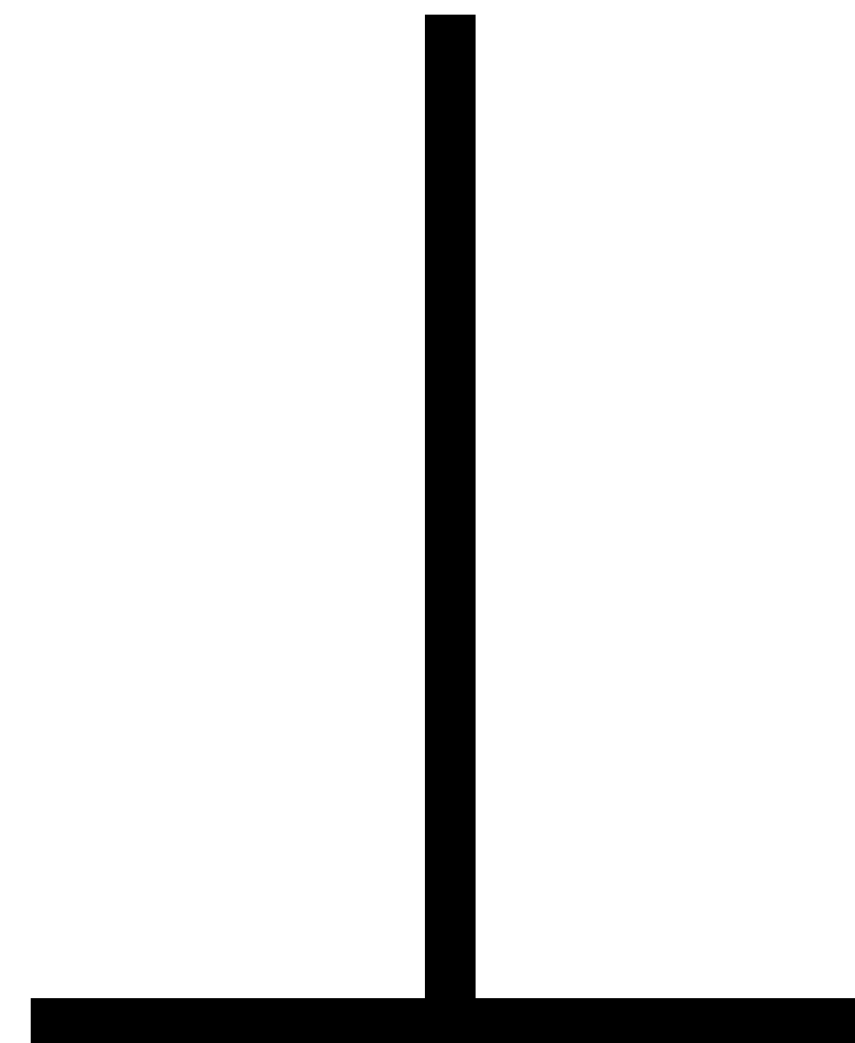
Sei also **$n > 1$** .



Startstange



Zwischenstange

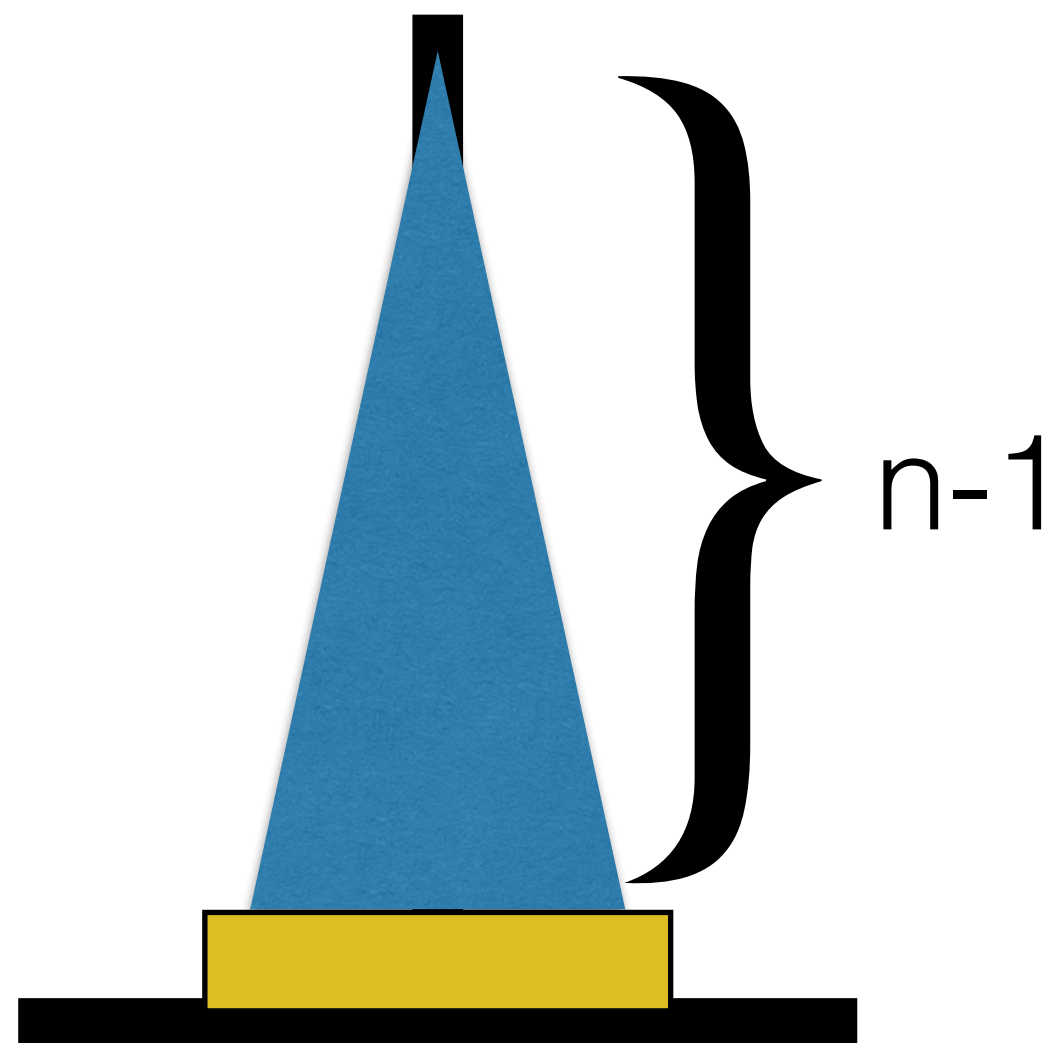


Zielstange

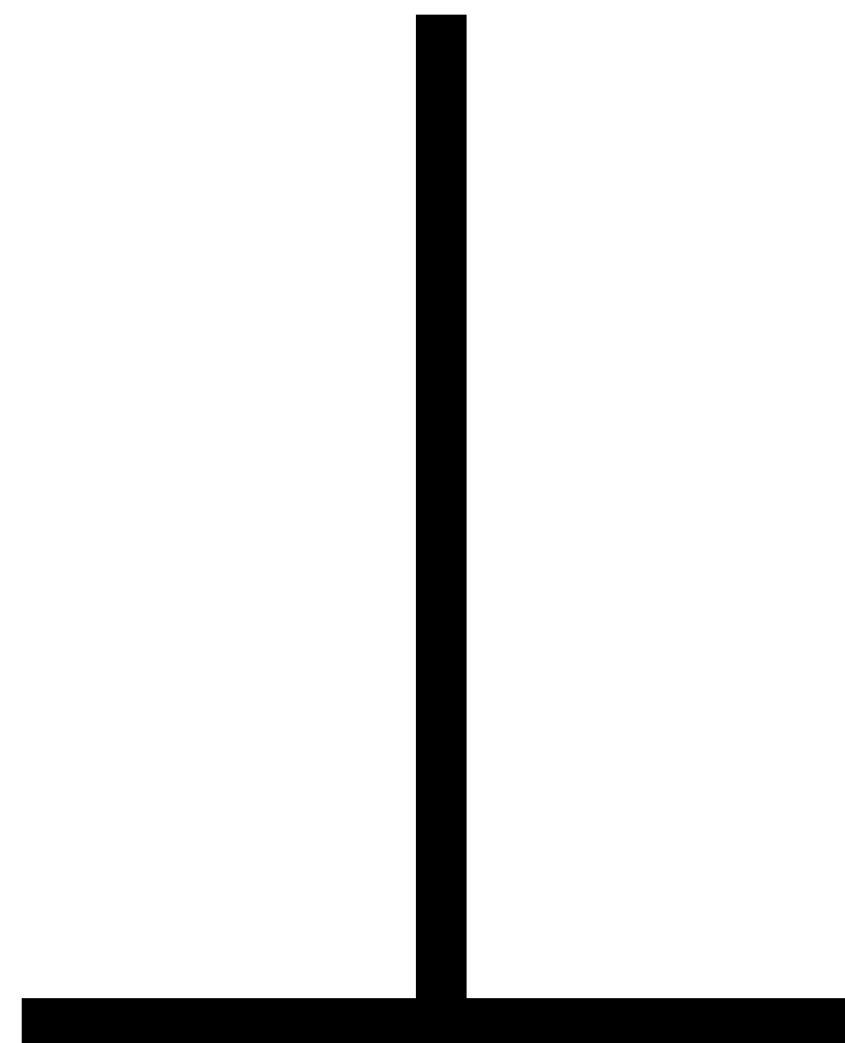
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

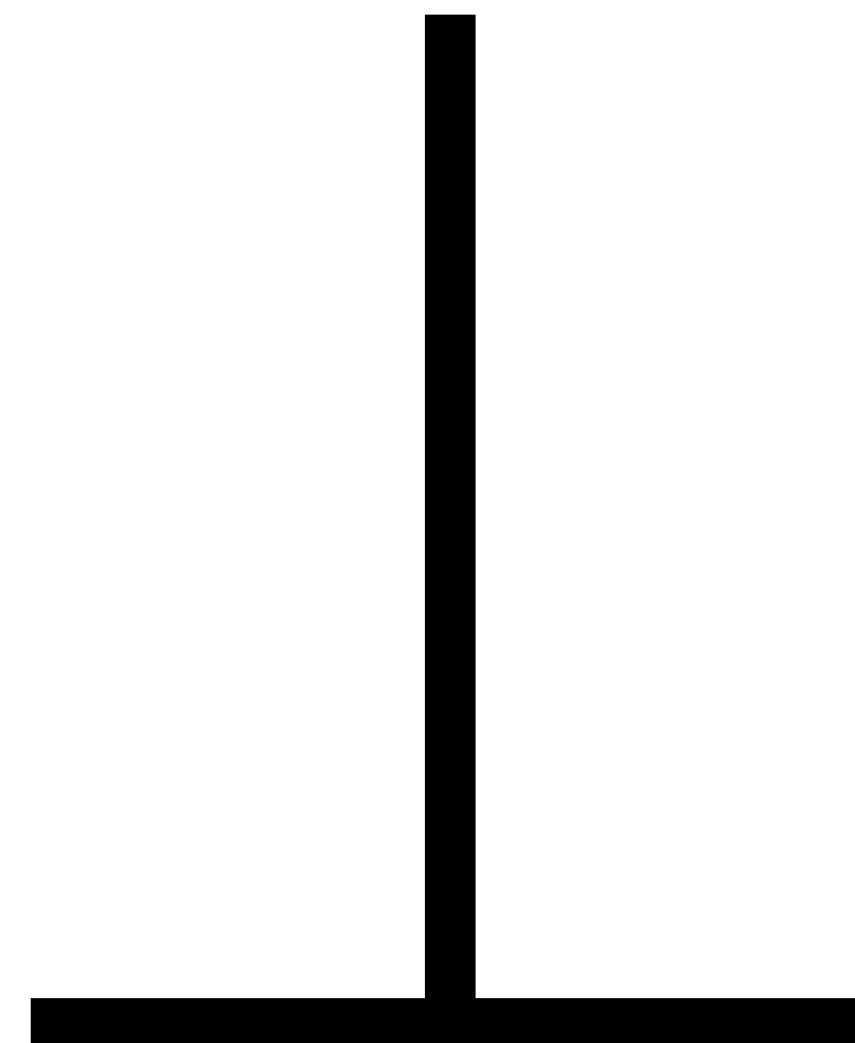
Sei also **n**>1.



Startstange



Zwischenstange



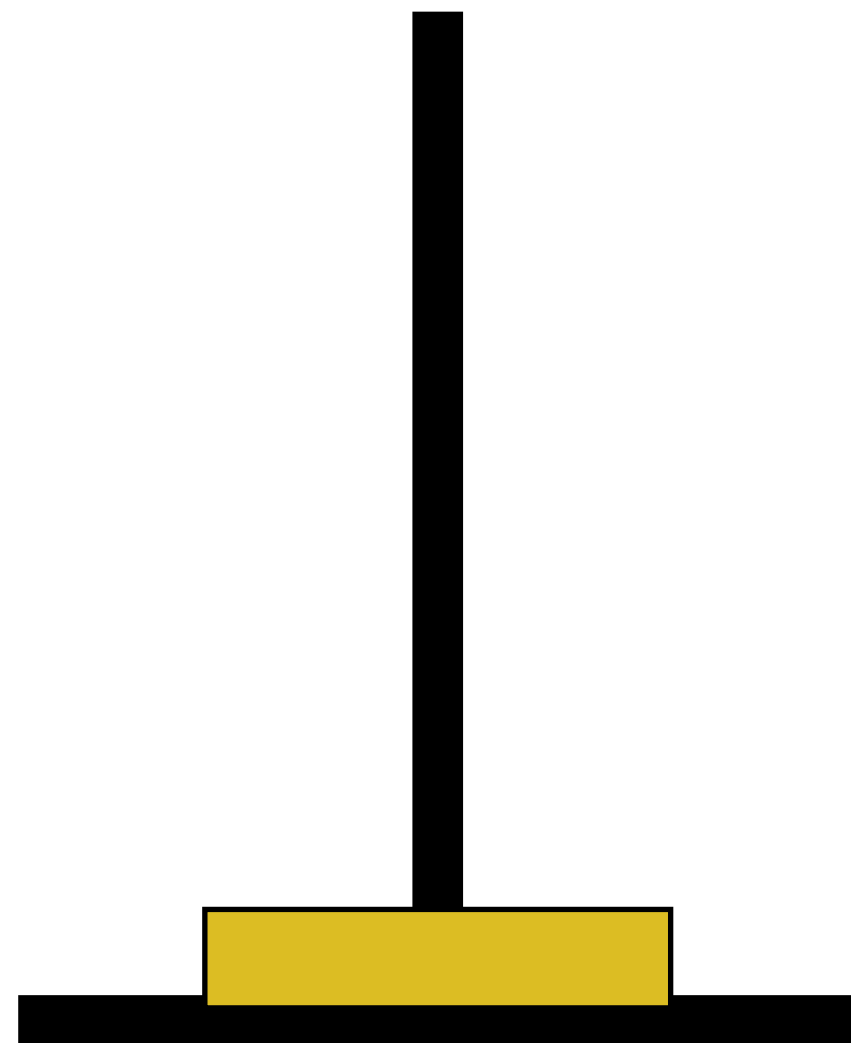
Zielstange

Verschiebe rekursiv  von **Startstange** zur **Zwischenstange**

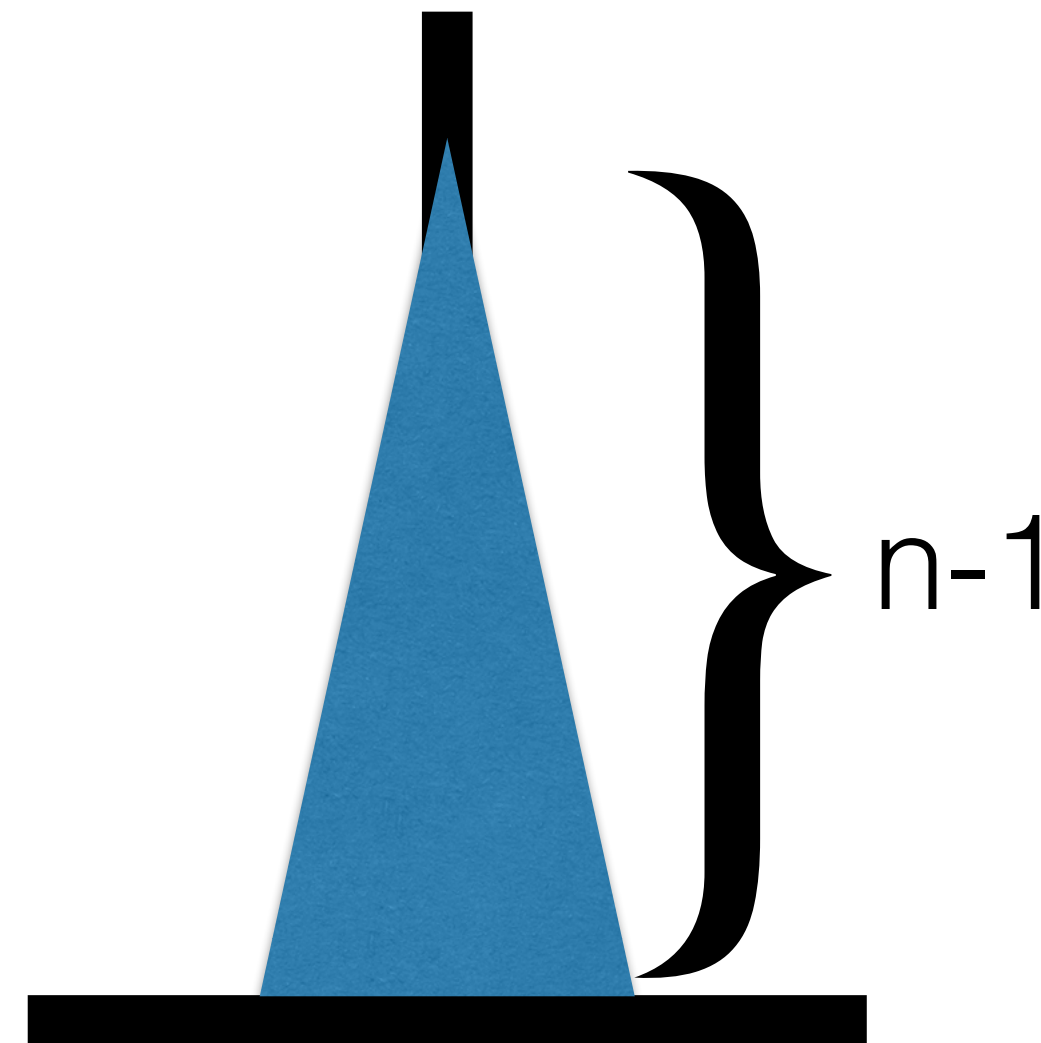
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

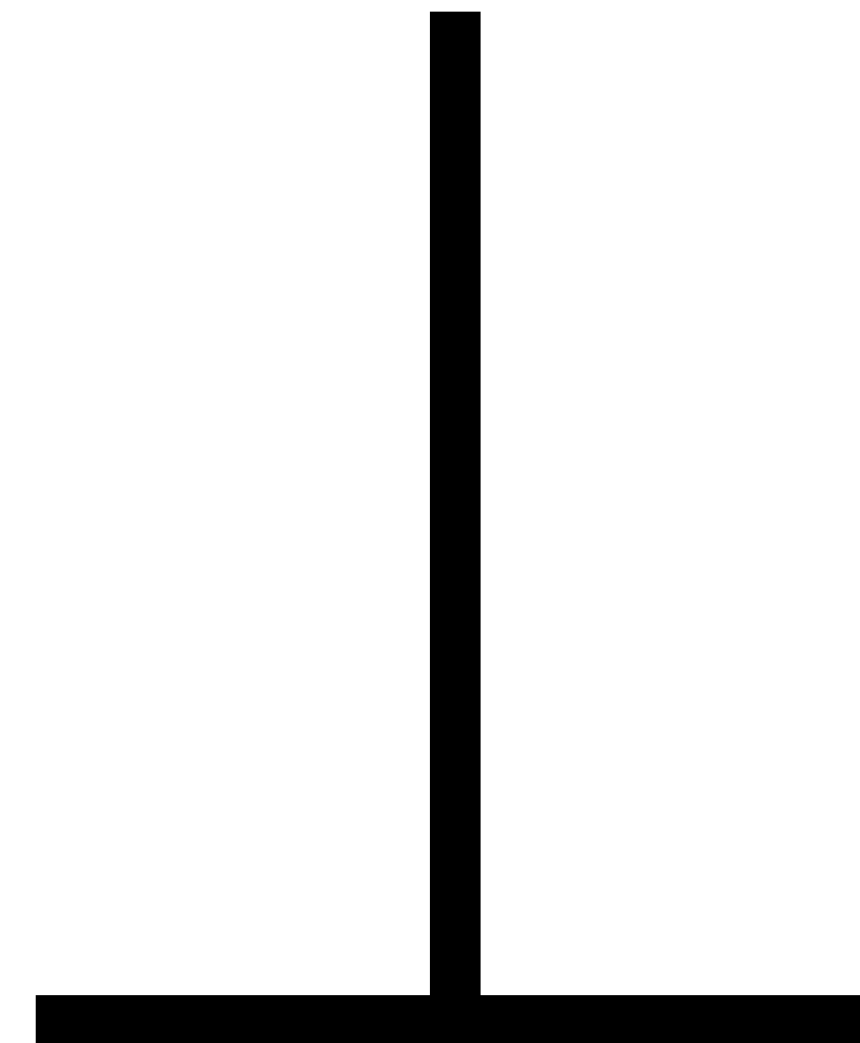
Sei also **n**>1.



Startstange



Zwischenstange



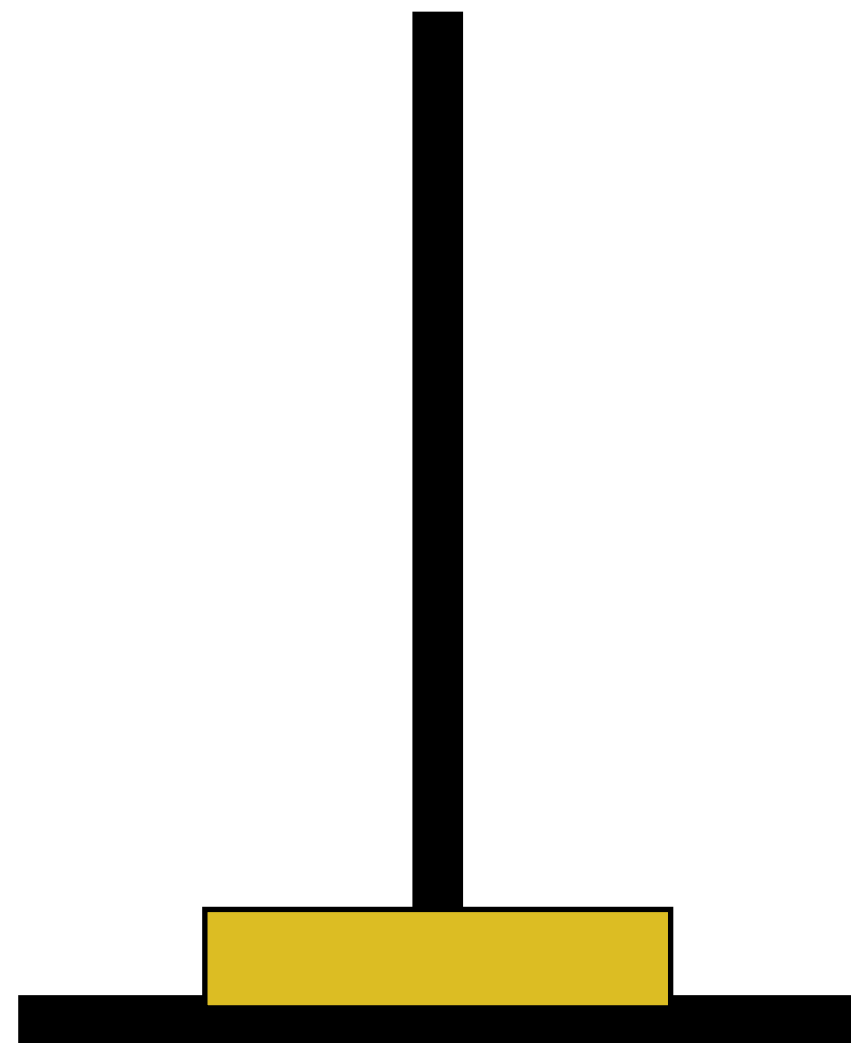
Zielstange

Verschiebe rekursiv  von **Startstange** zur **Zwischenstange**

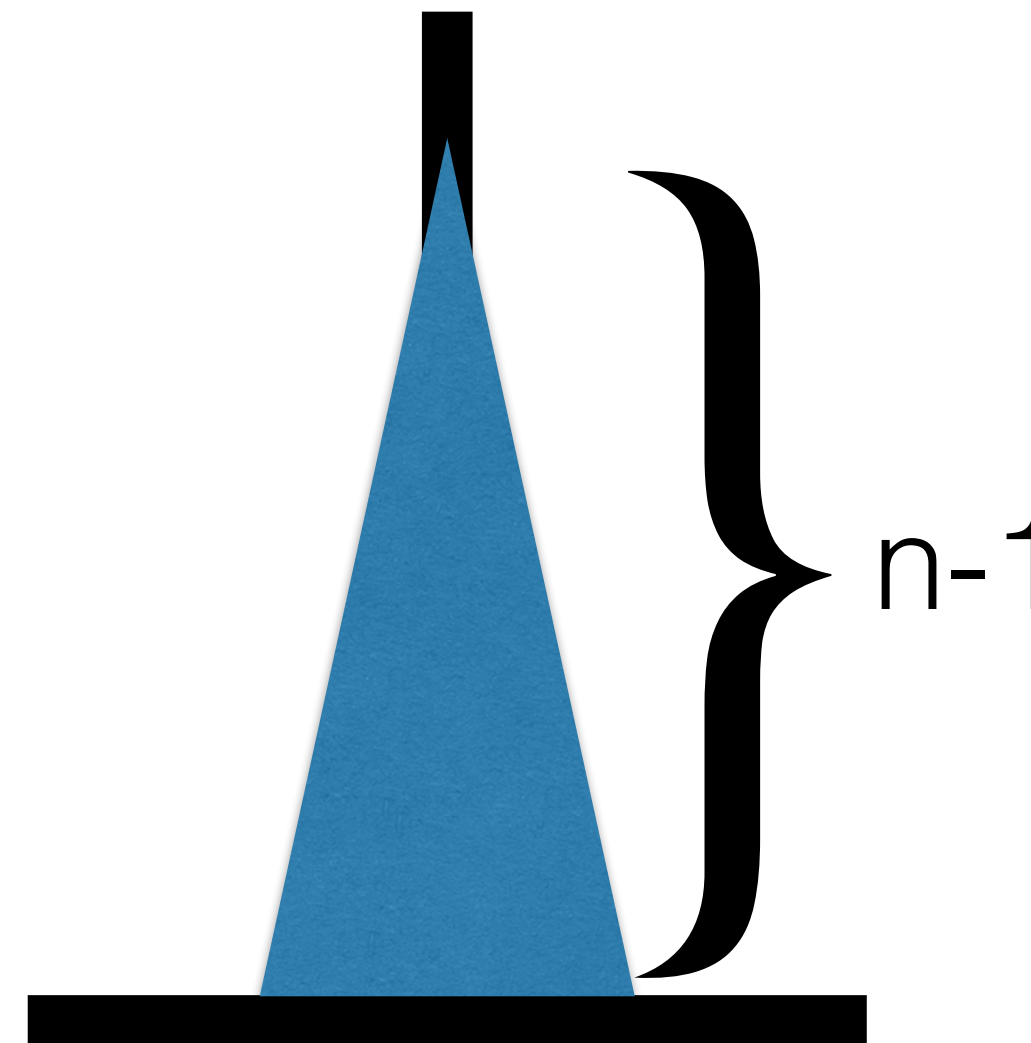
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

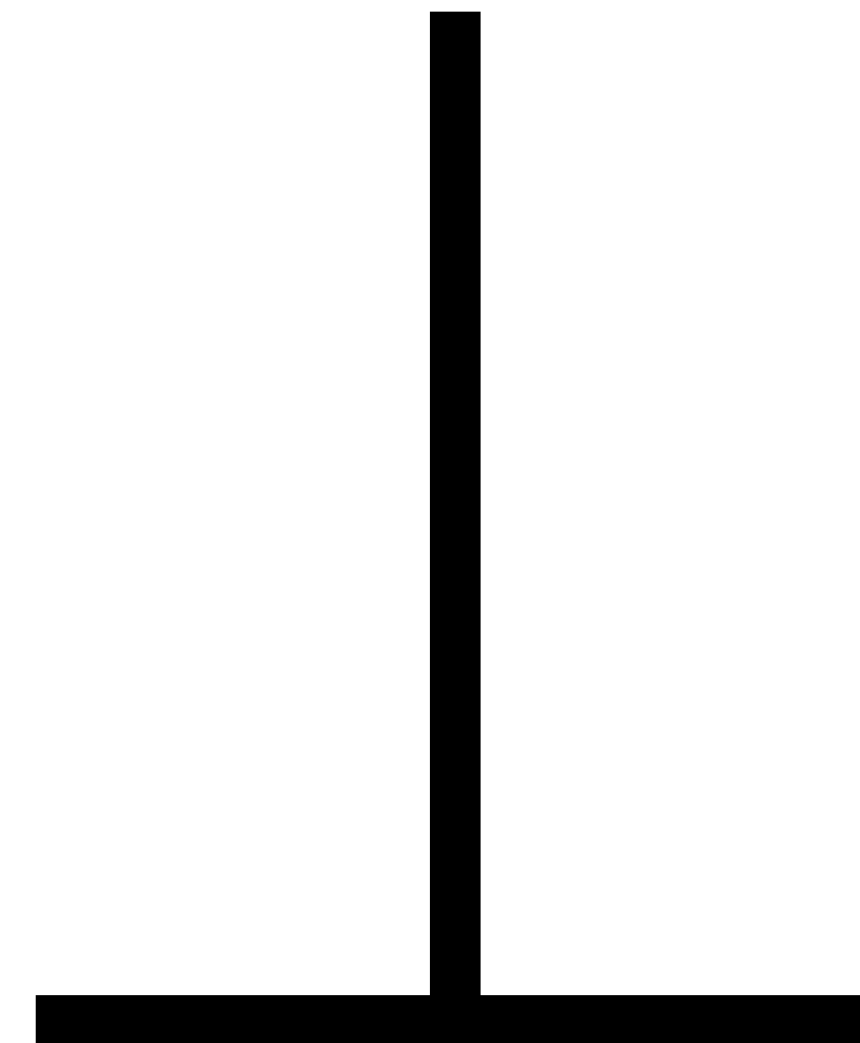
Sei also **n**>1.



Startstange



Zwischenstange



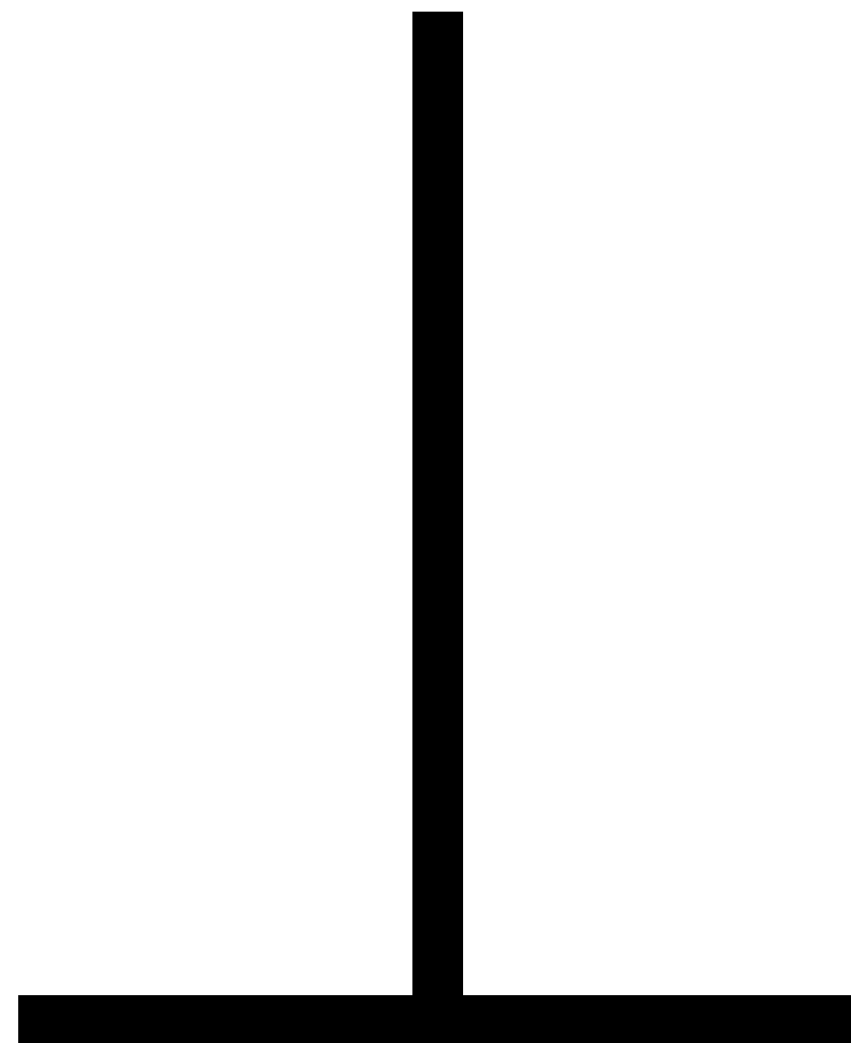
Zielstange

Verschiebe  direkt von **Startstange** zur **Zielstange**

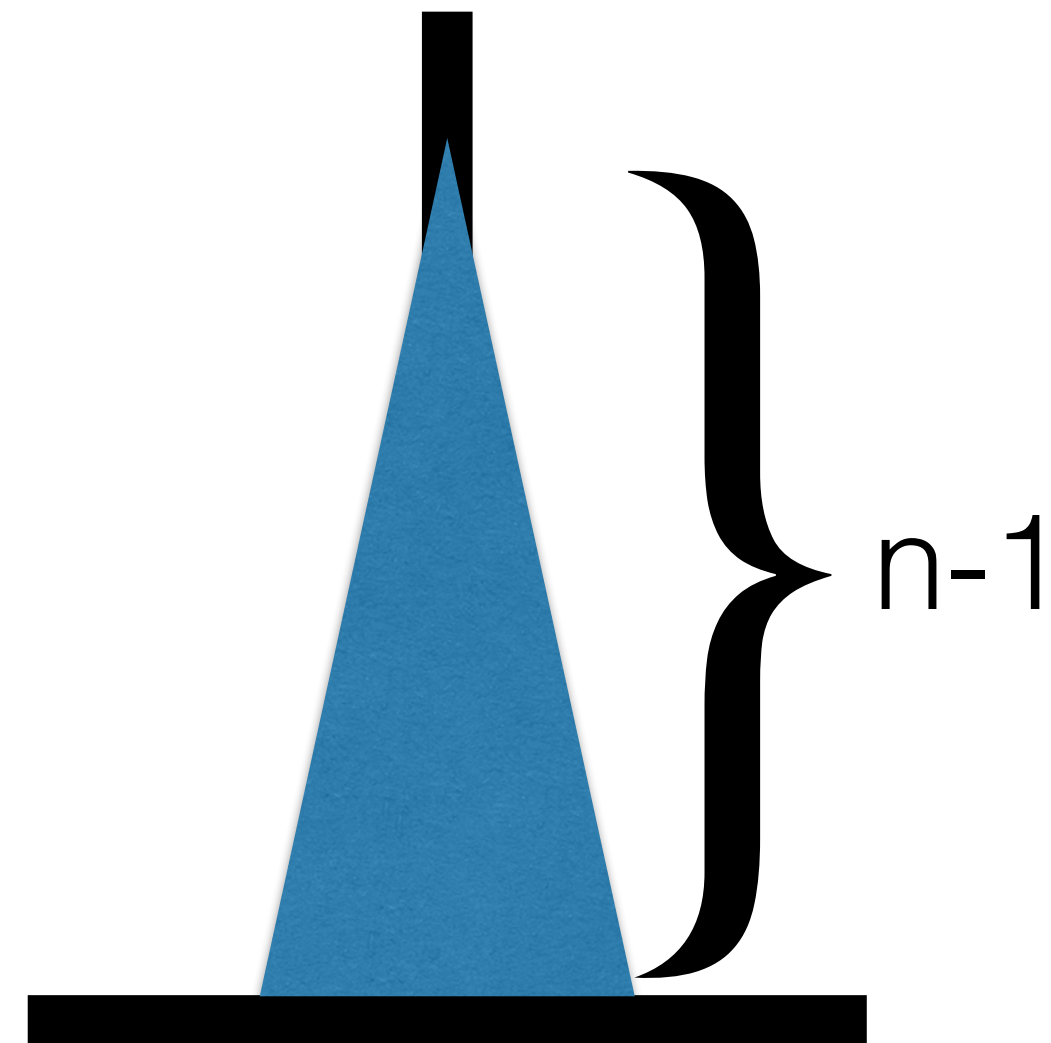
Leichte Verallgemeinerung

Die triviale Instanz **$n=1$** haben wir bereits gelöst!

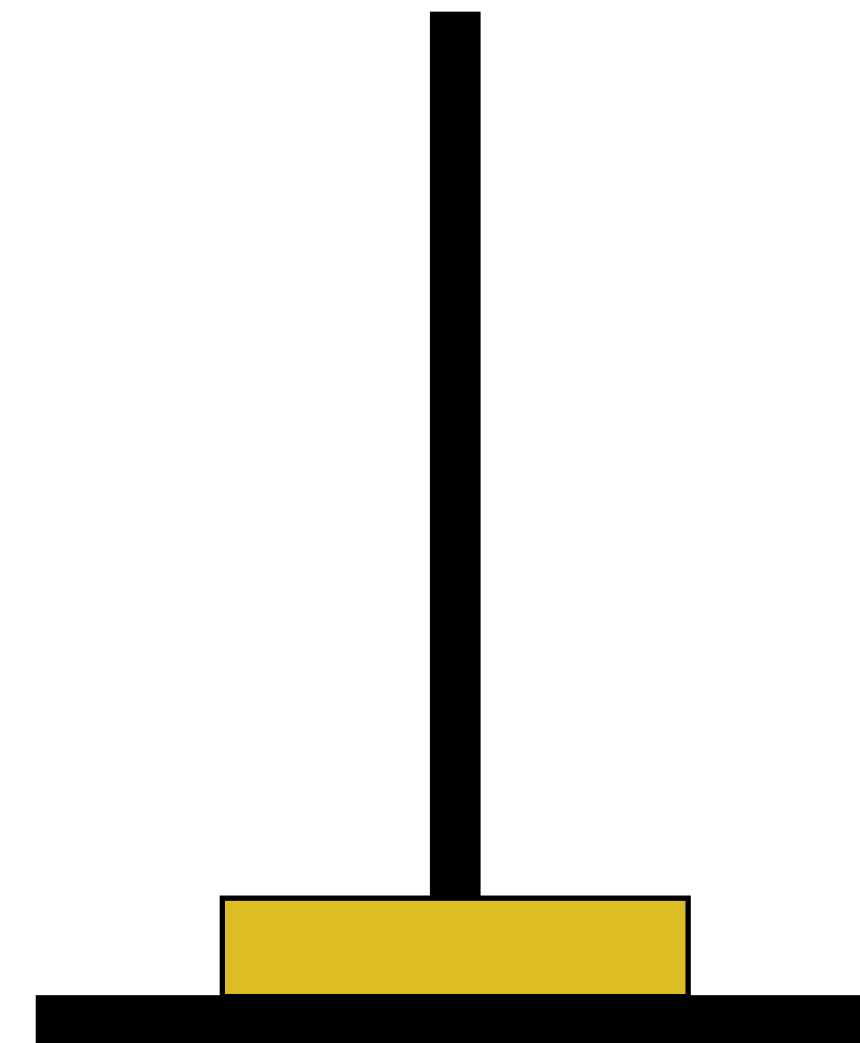
Sei also **$n > 1$** .



Startstange



Zwischenstange

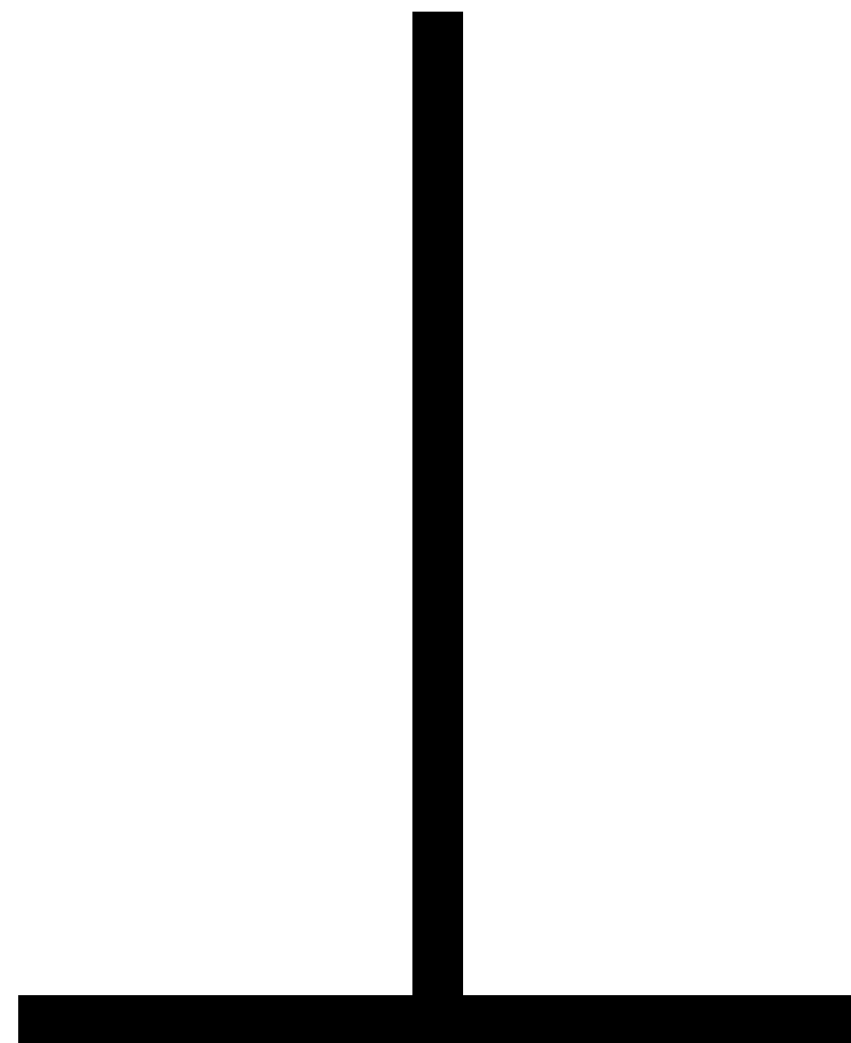


Zielstange

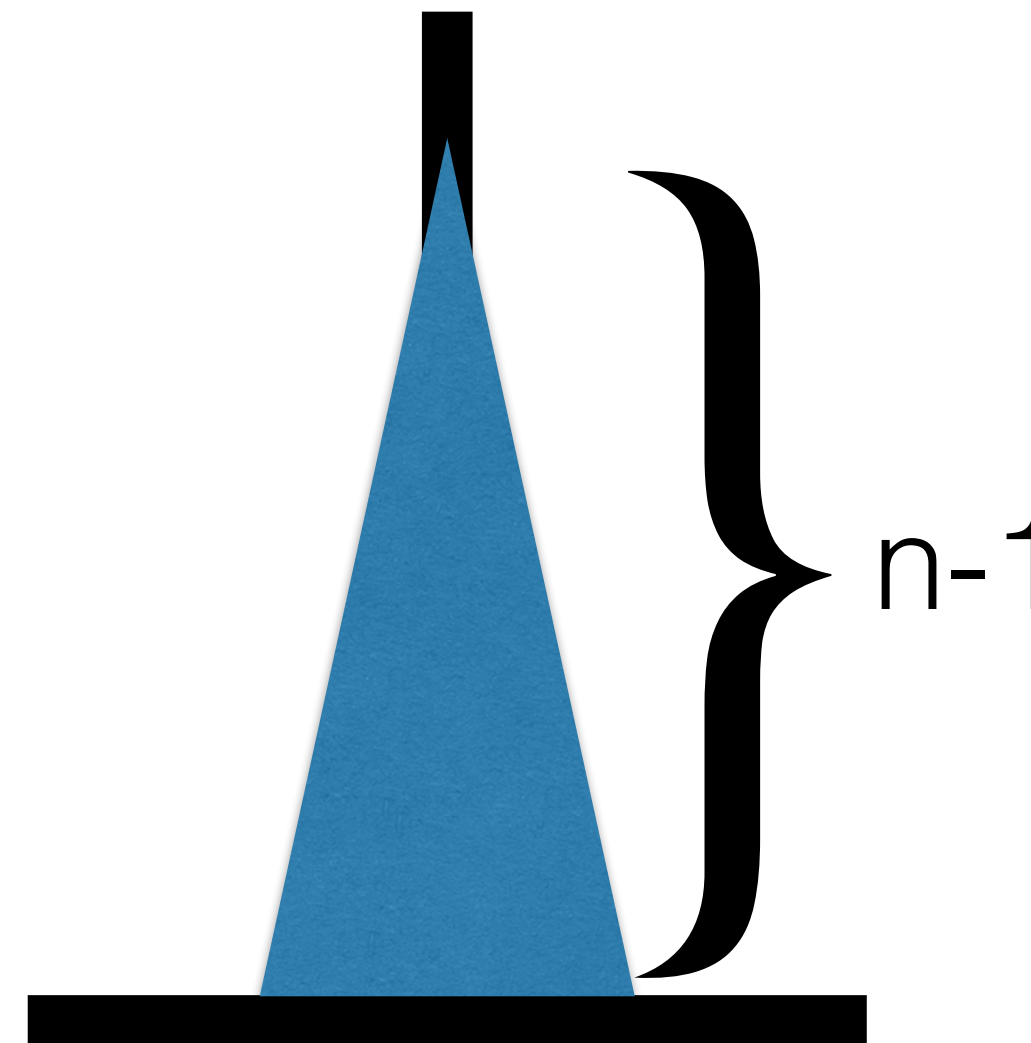
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

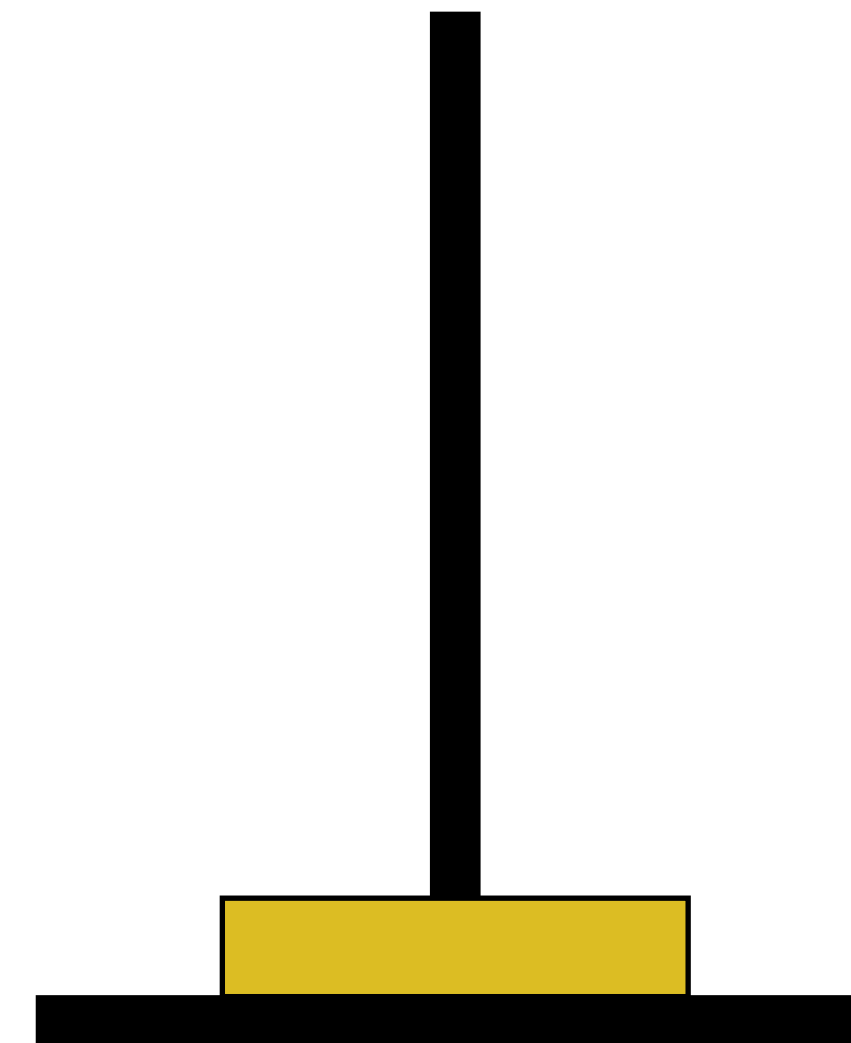
Sei also **n**>1.



Startstange



Zwischenstange



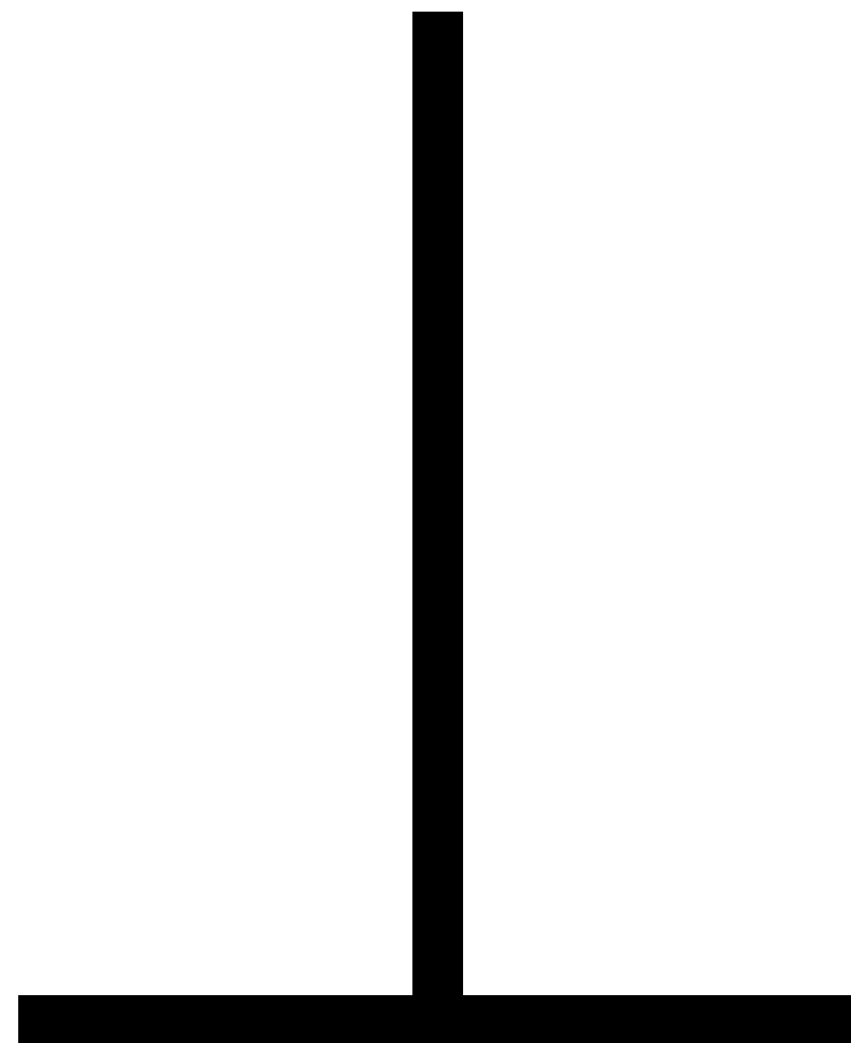
Zielstange

Verschiebe rekursiv  von **Zwischenstange** zur **Zielstange**

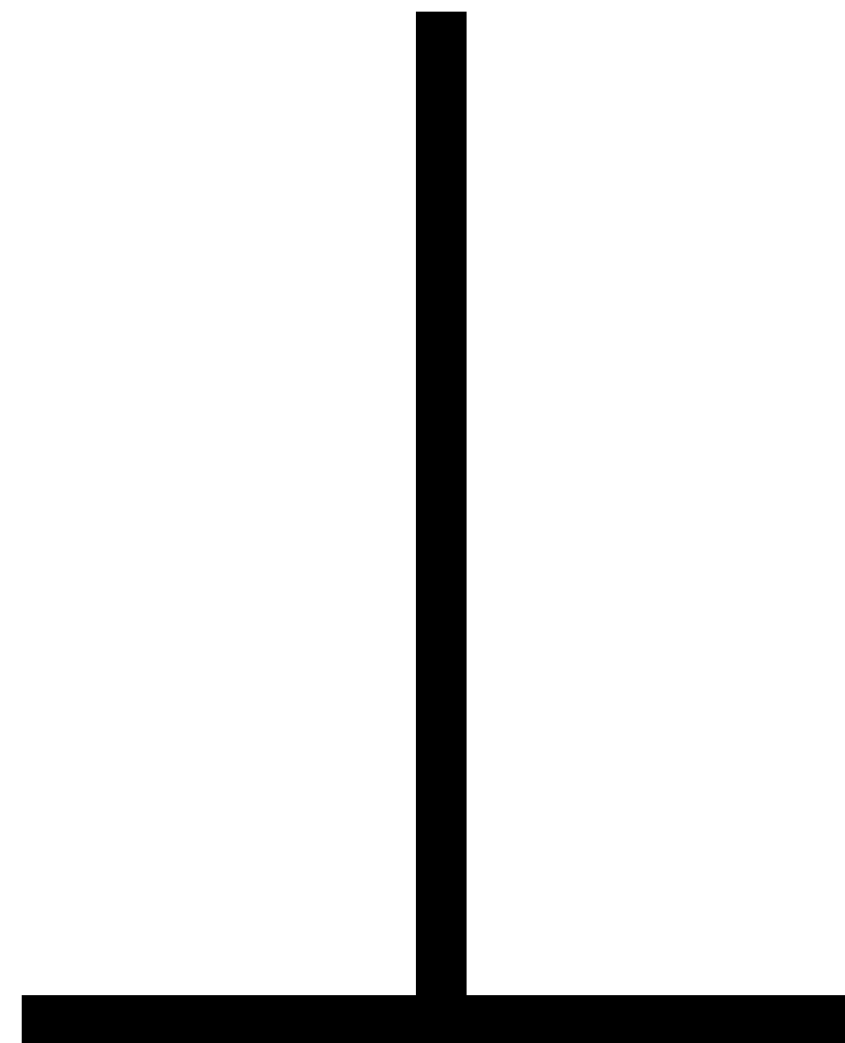
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

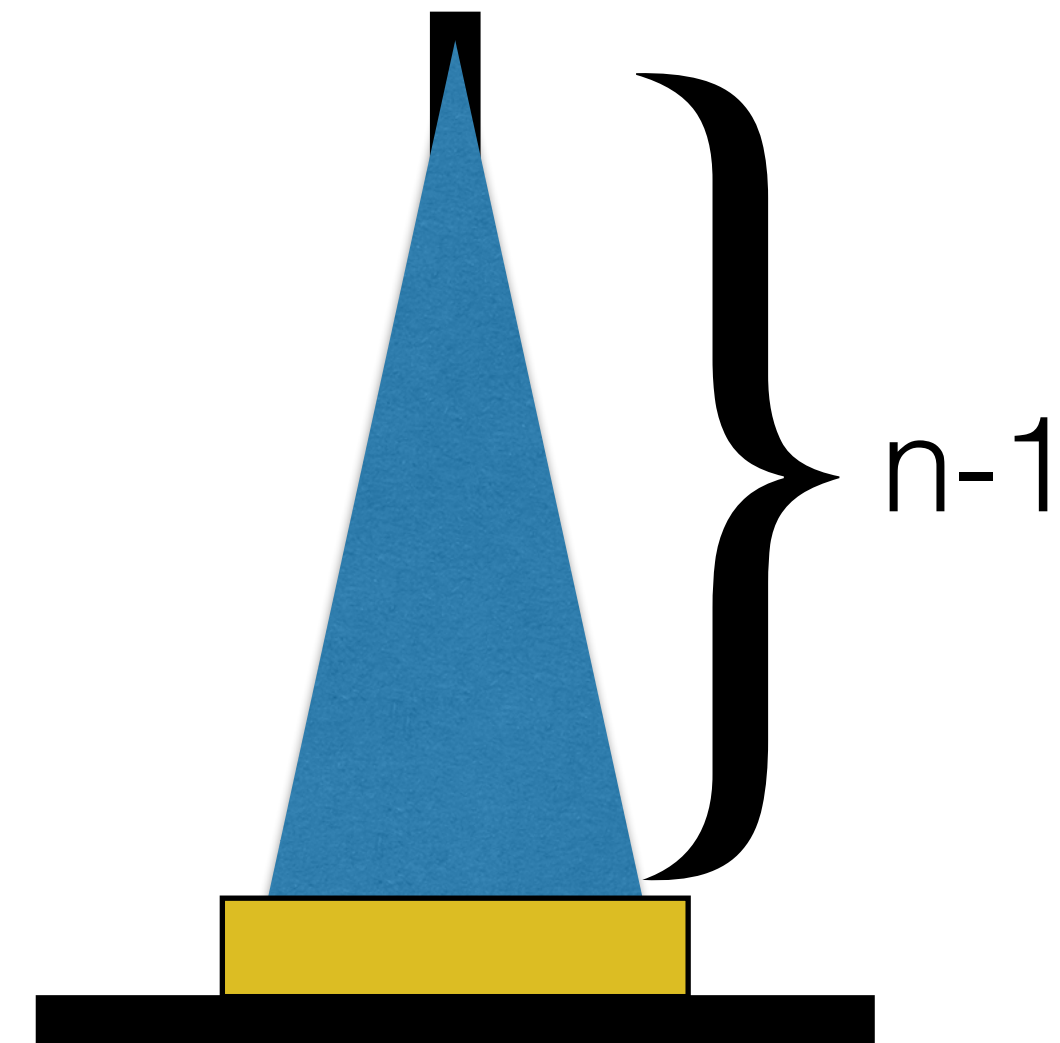
Sei also **n**>1.



Startstange



Zwischenstange



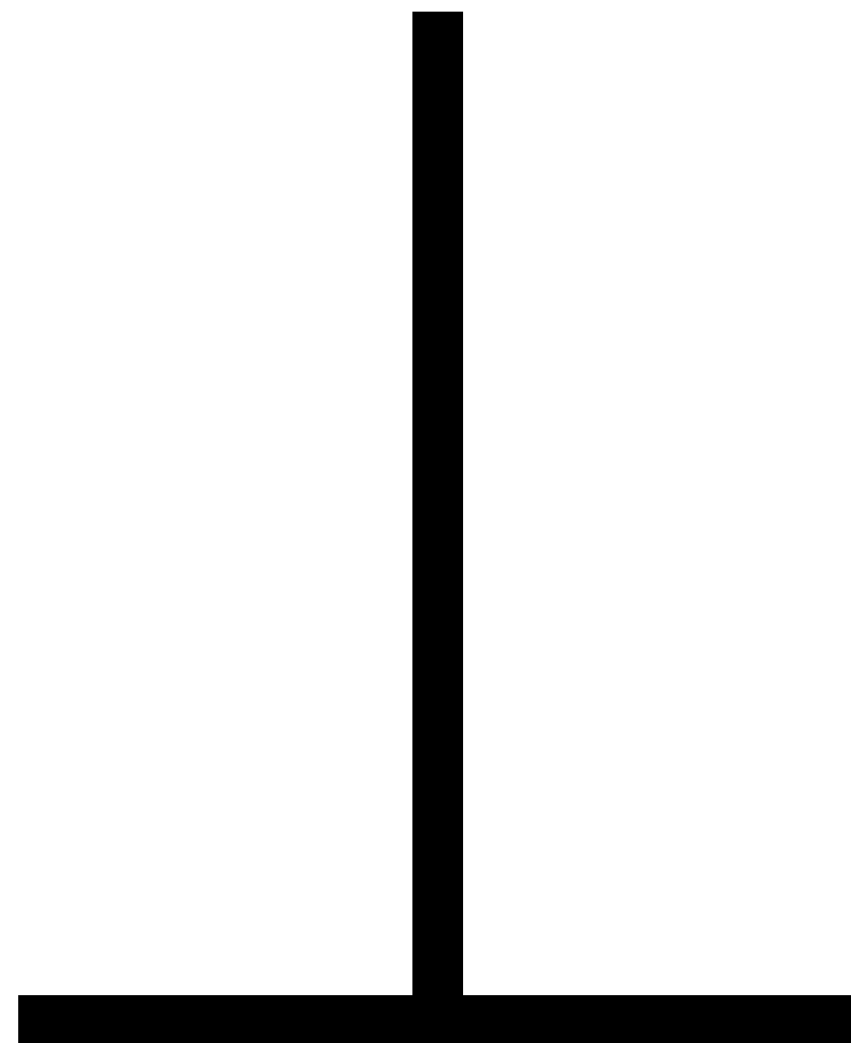
Zielstange

Verschiebe rekursiv  von **Zwischenstange** zur **Zielstange**

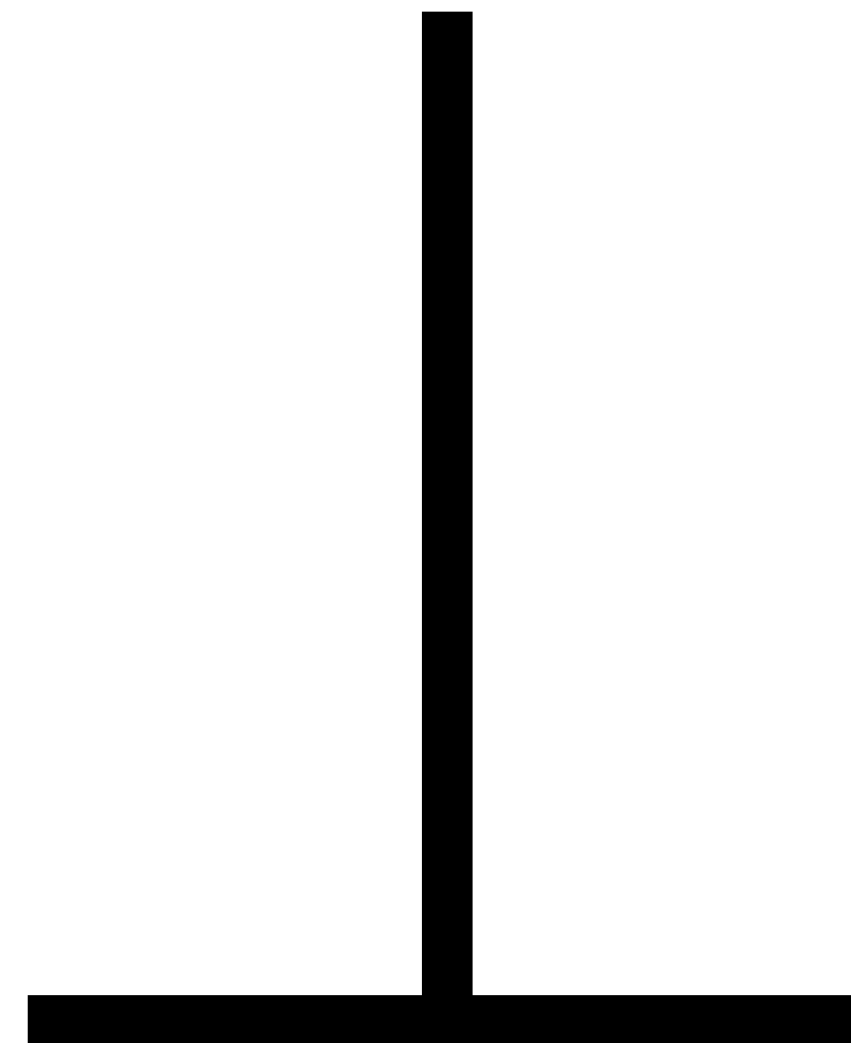
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

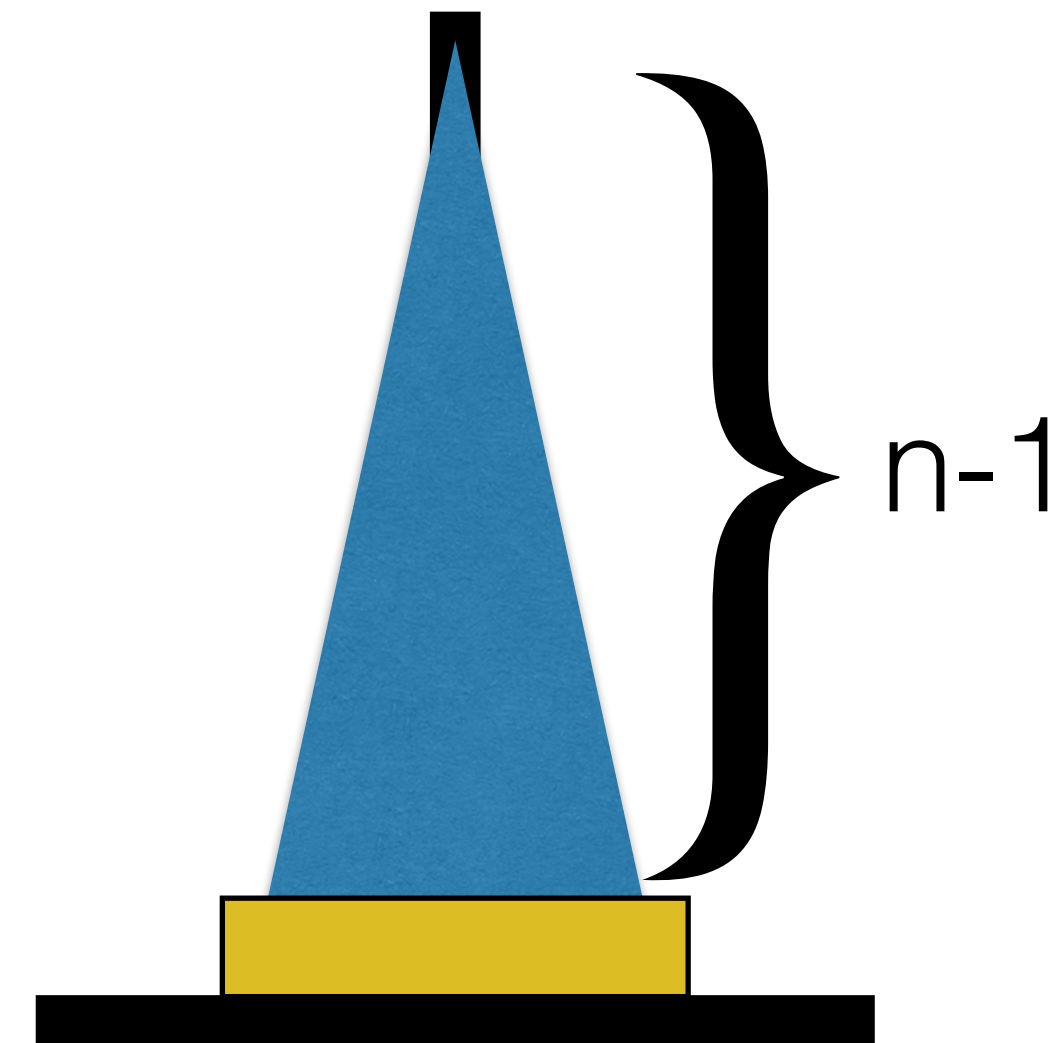
Sei also **n**>1.



Startstange



Zwischenstange



Zielstange

Wir sind fertig!

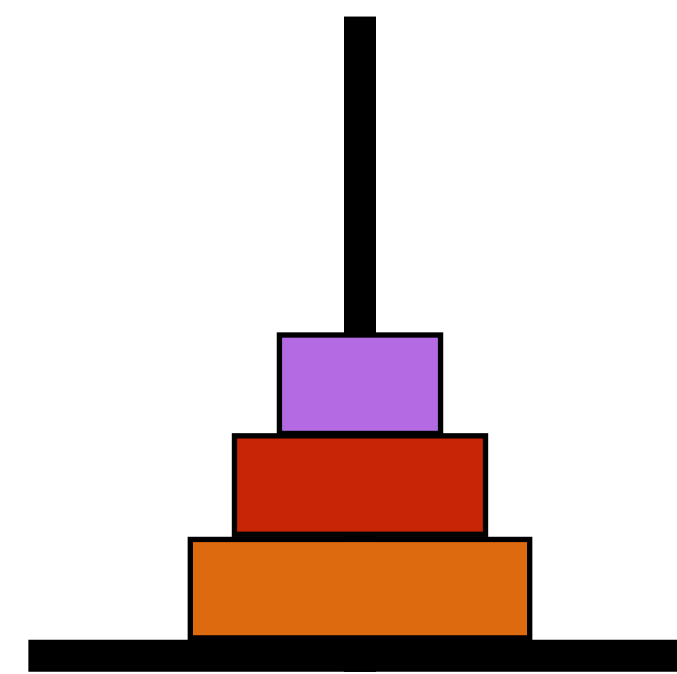
Lösung in Python

```
def Tuerme(n, von, nach, zwischen):  
    if n==1:  
        print(von+ " -> "+ nach)  
    else:  
        Tuerme(n-1, von, zwischen, nach)  
        Tuerme(1, von, nach, zwischen)  
        Tuerme(n-1, zwischen, nach, von)  
  
n=input("Geben Sie die Anzahl der Scheiben an ")  
Tuerme(int(n), "A", "C", "B")
```

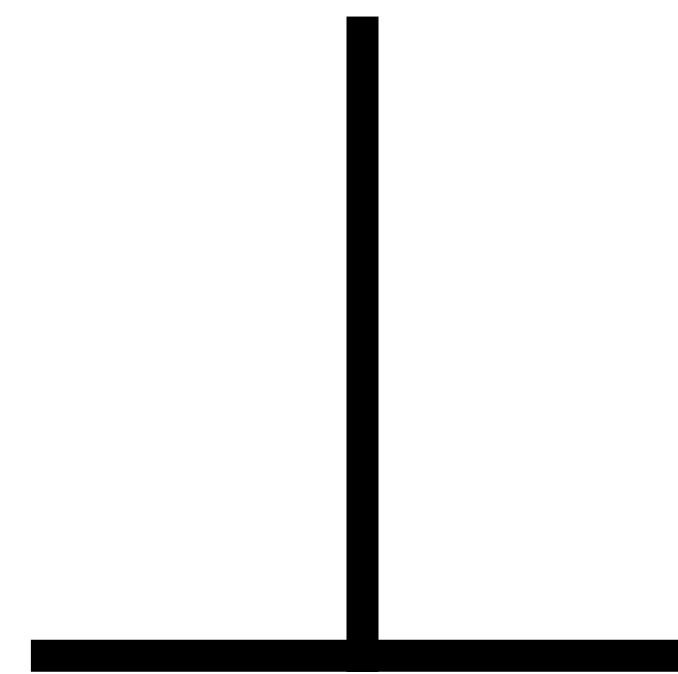
Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

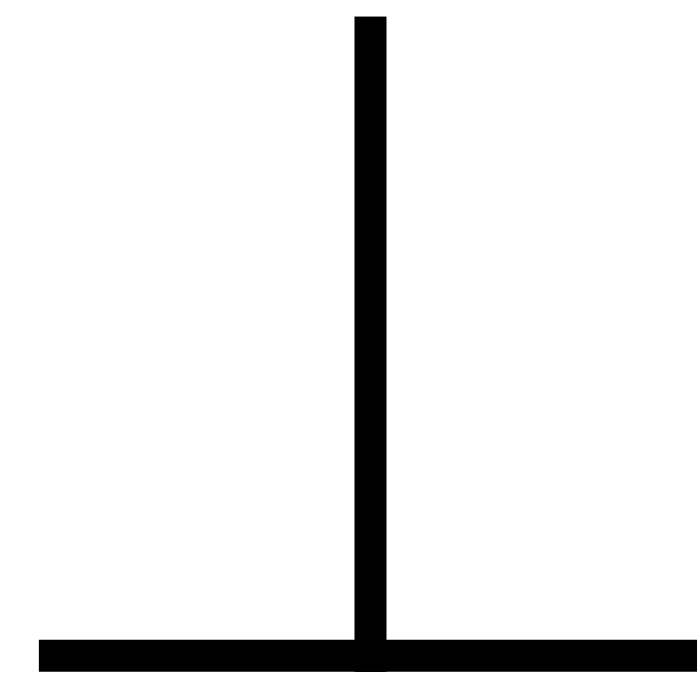
A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

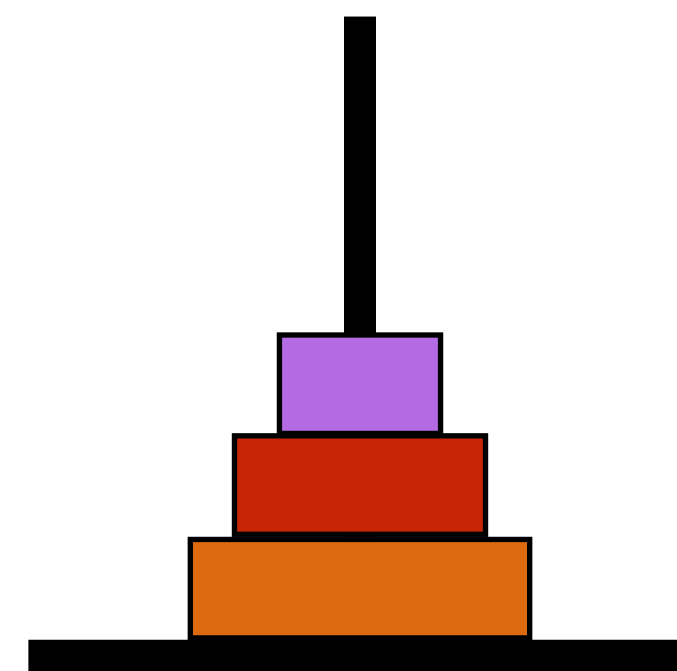
C -> B

A -> C

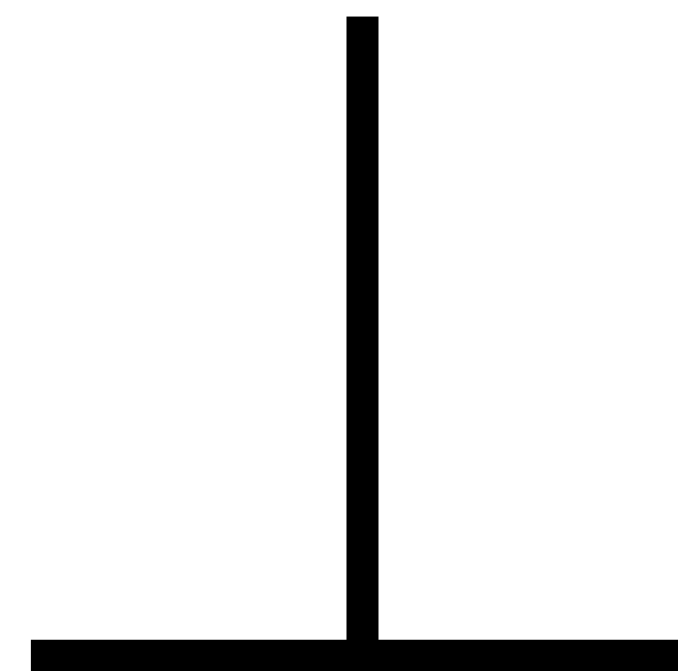
B -> A

B -> C

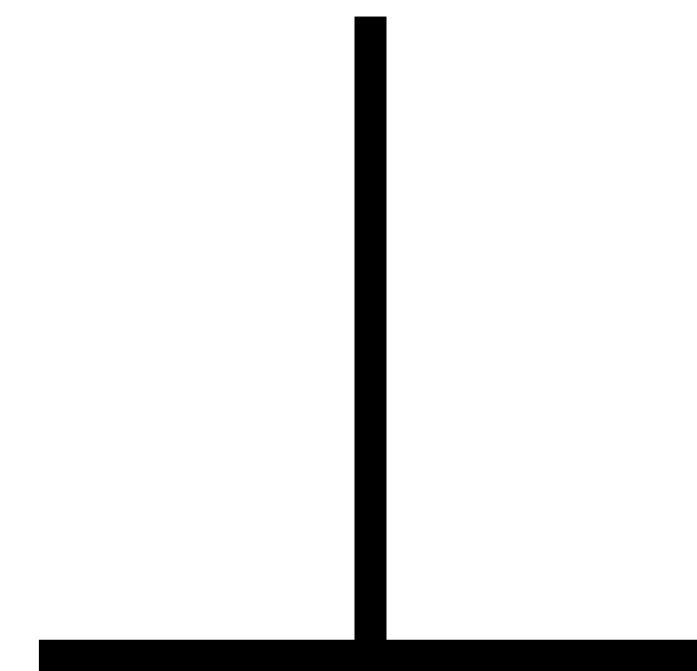
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

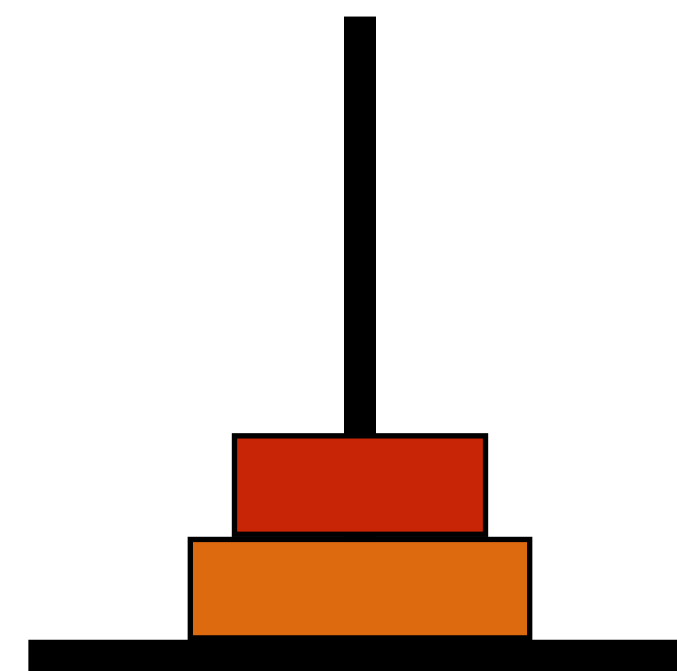
C -> B

A -> C

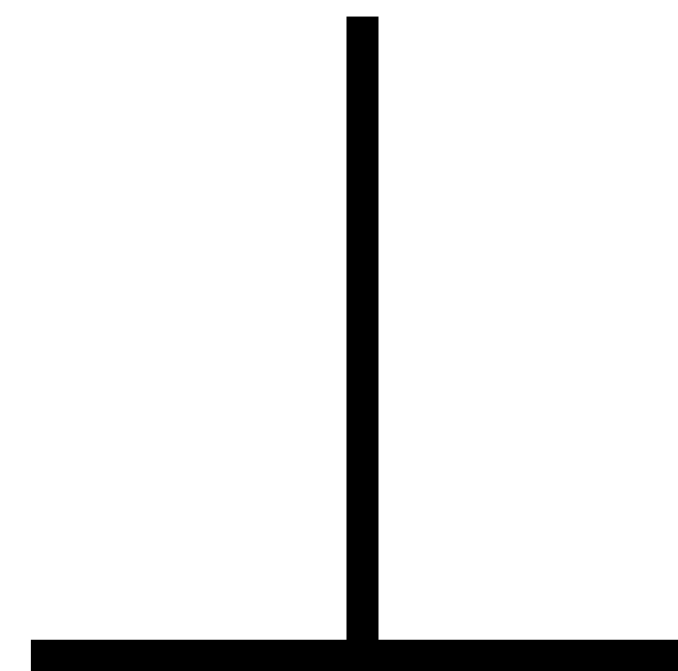
B -> A

B -> C

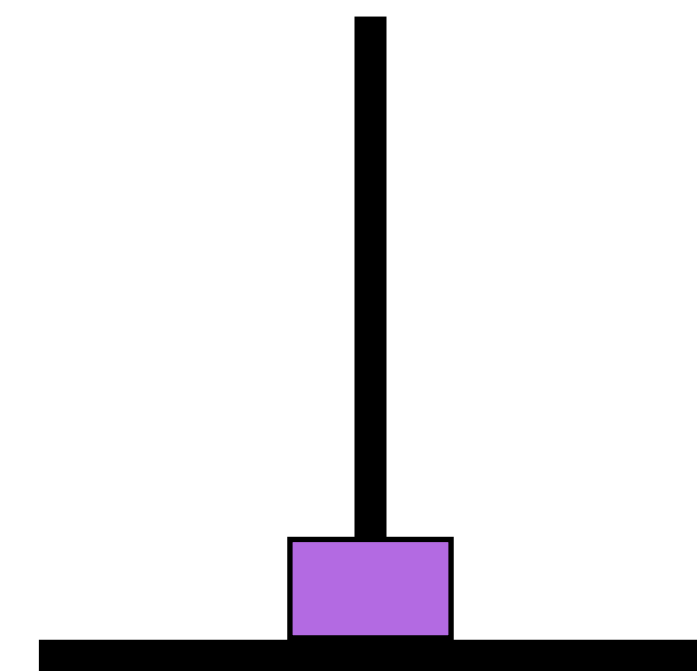
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

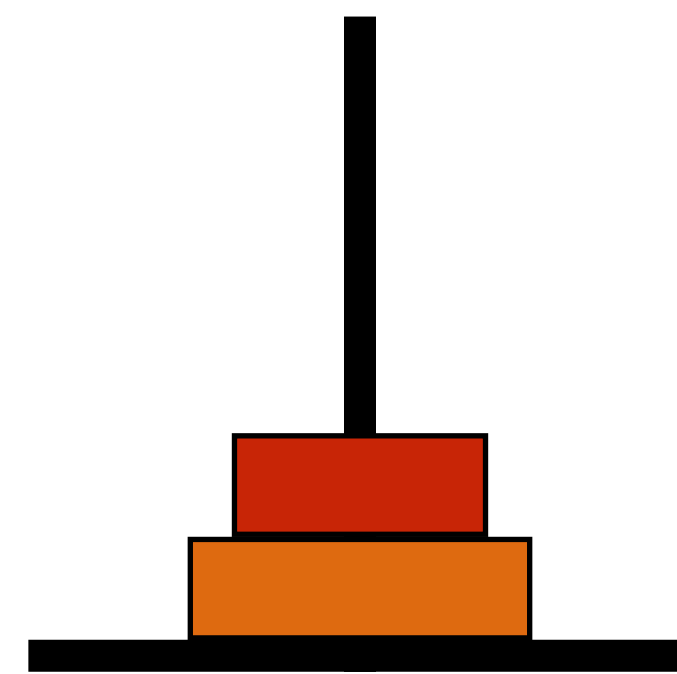
C -> B

A -> C

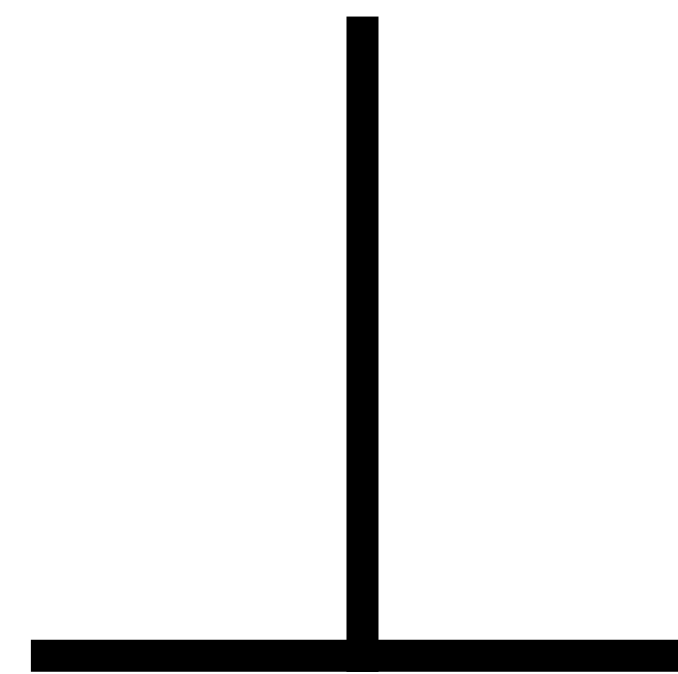
B -> A

B -> C

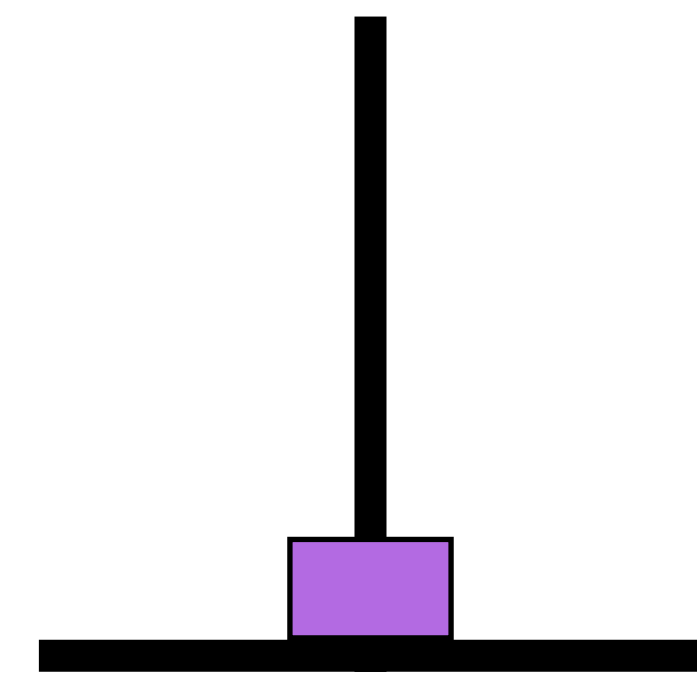
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

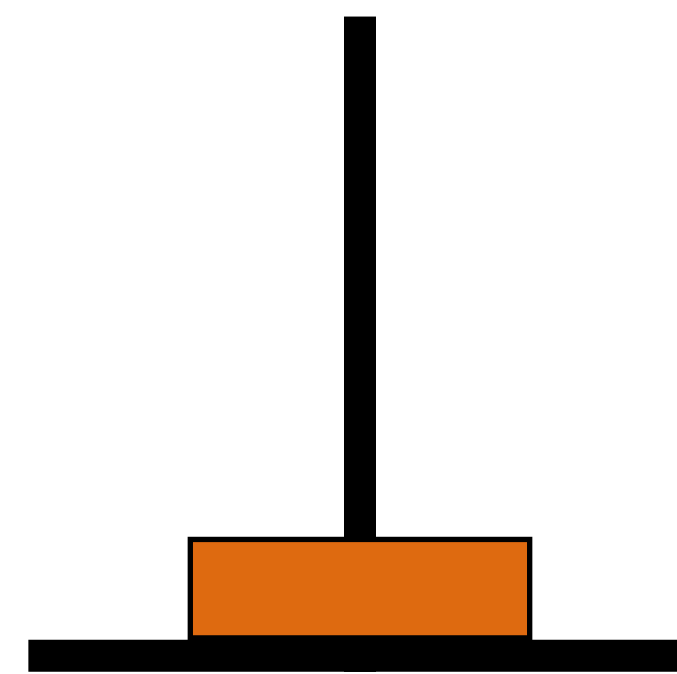
C -> B

A -> C

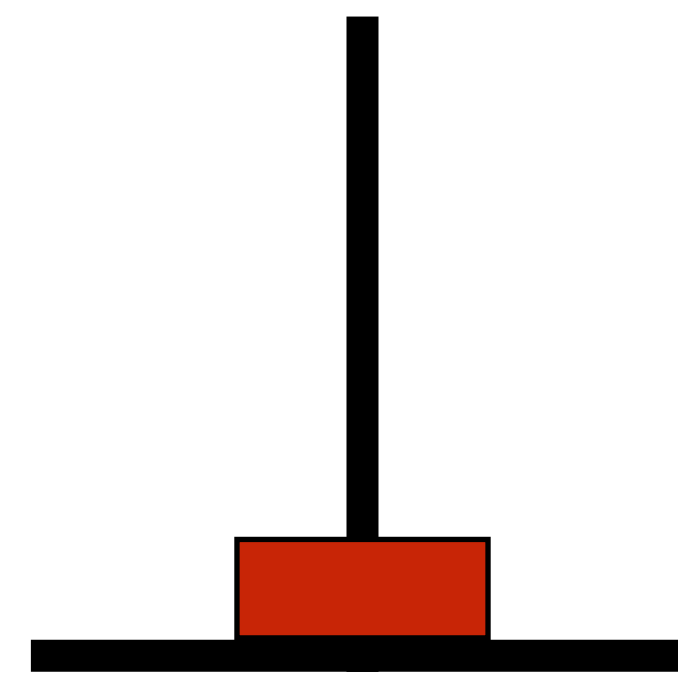
B -> A

B -> C

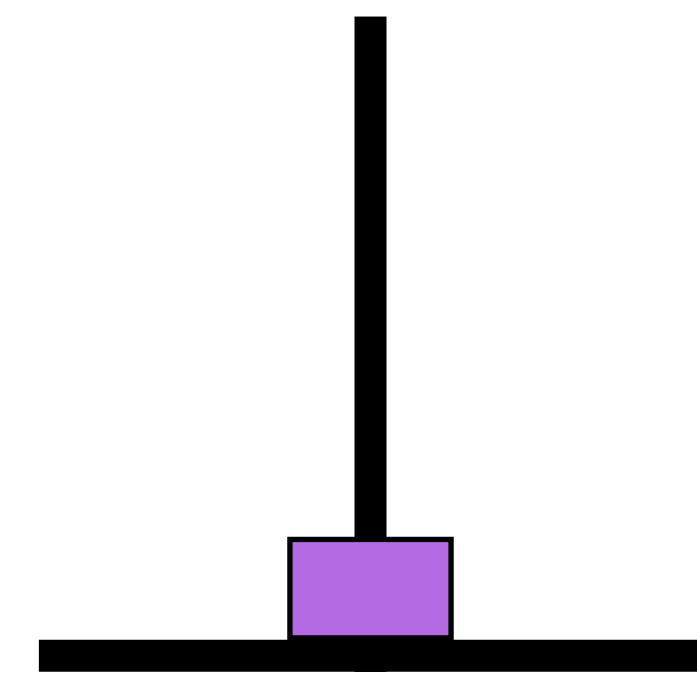
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

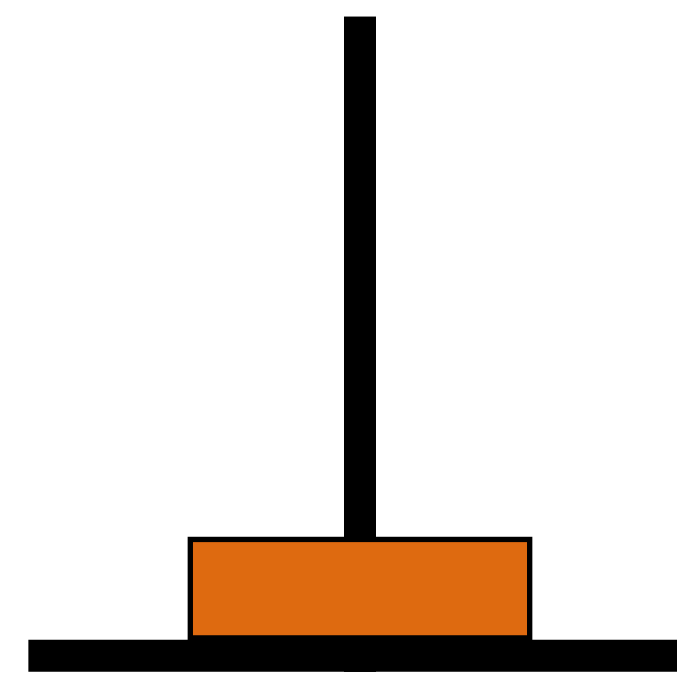
C -> B

A -> C

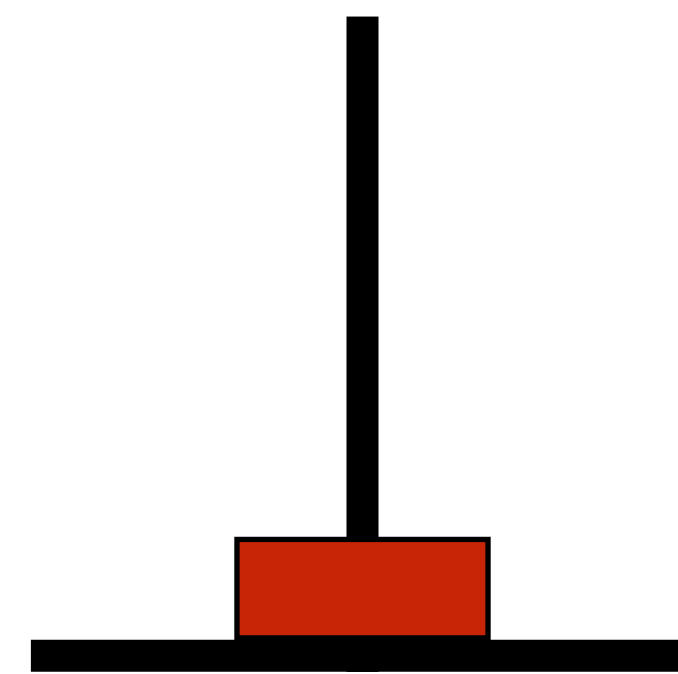
B -> A

B -> C

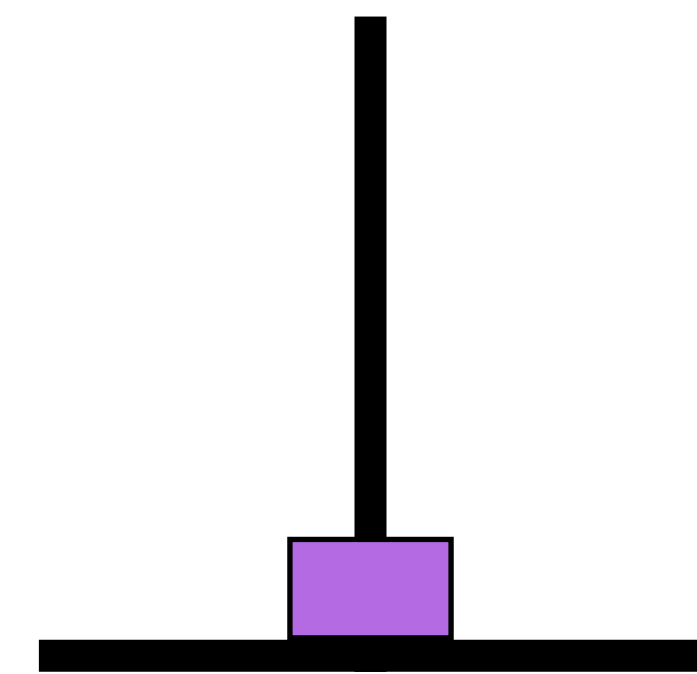
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

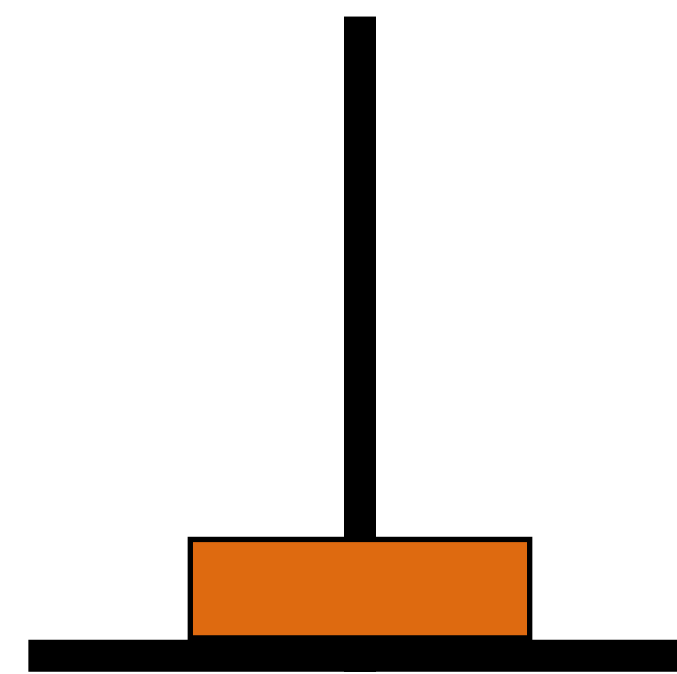
C -> B

A -> C

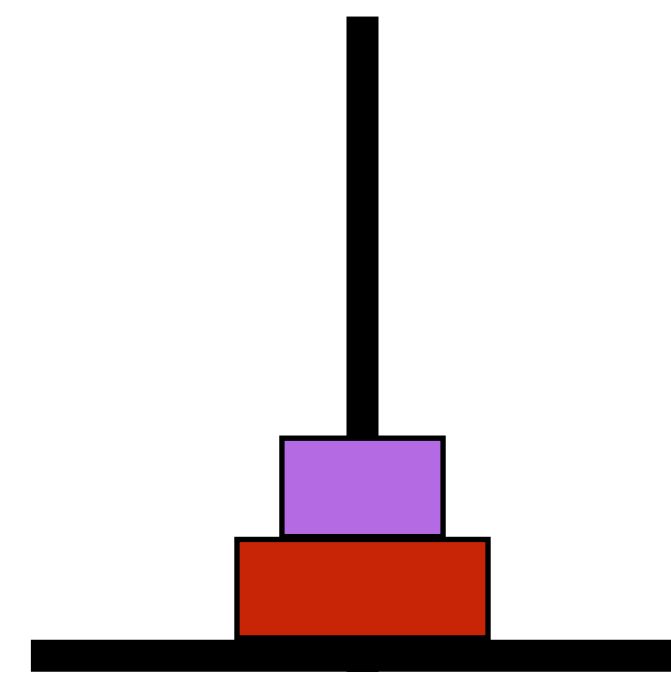
B -> A

B -> C

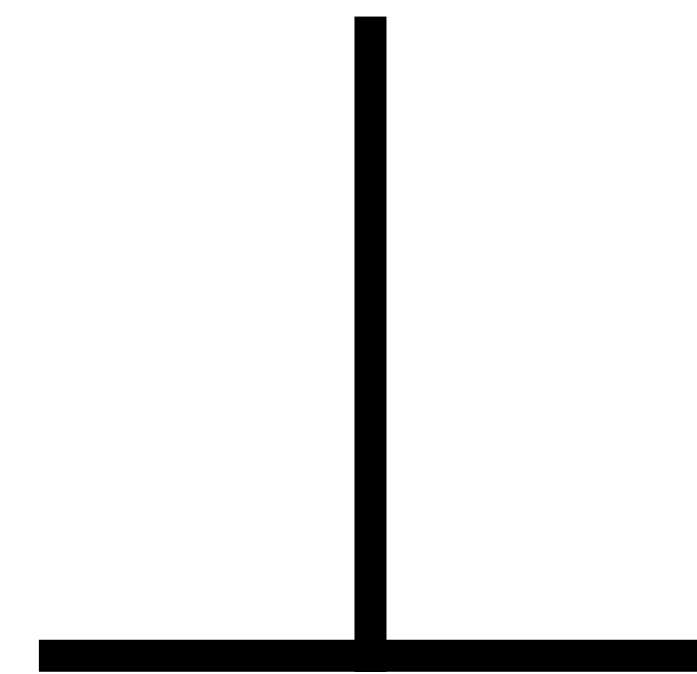
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

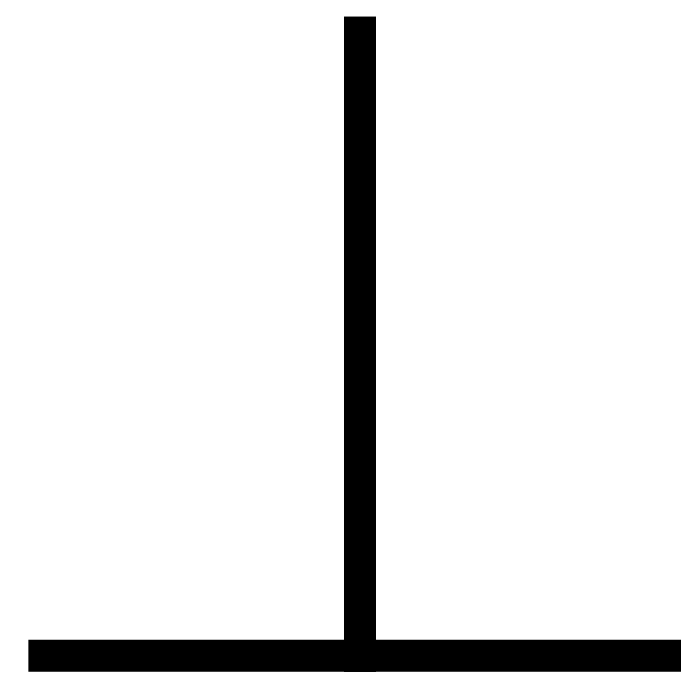
C -> B

A -> C

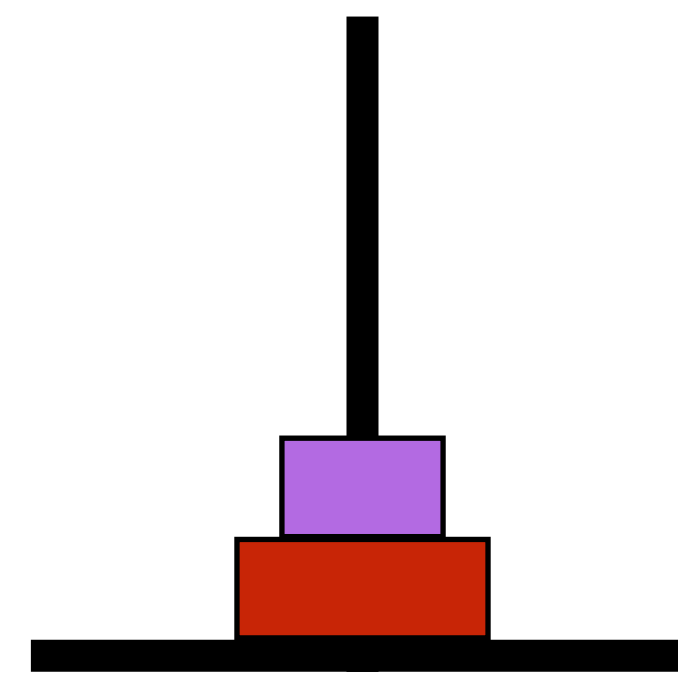
B -> A

B -> C

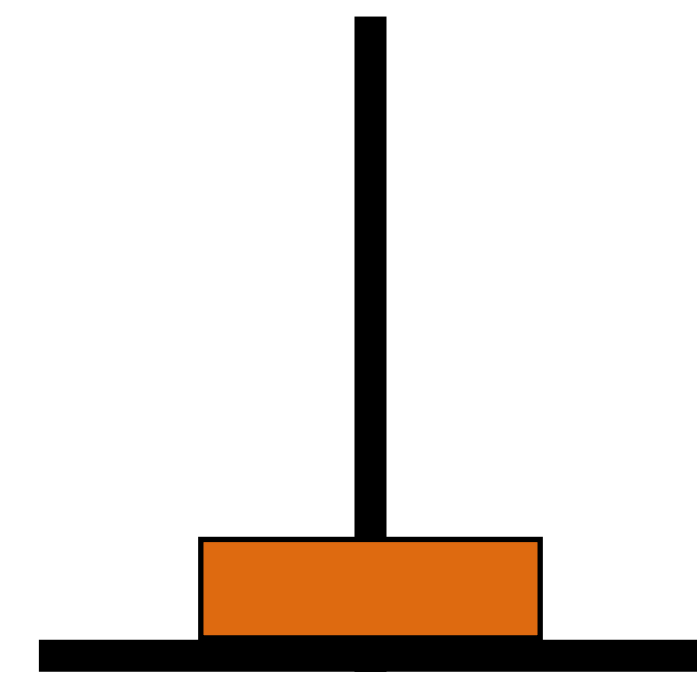
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

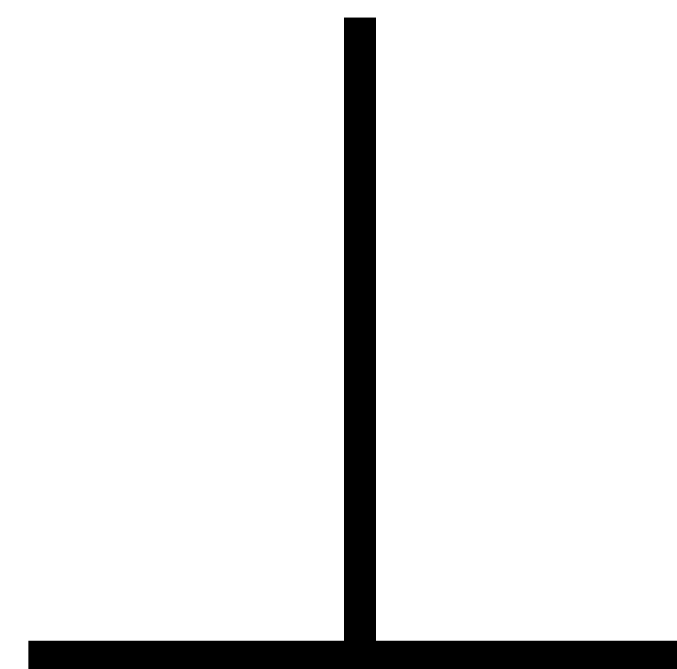
C -> B

A -> C

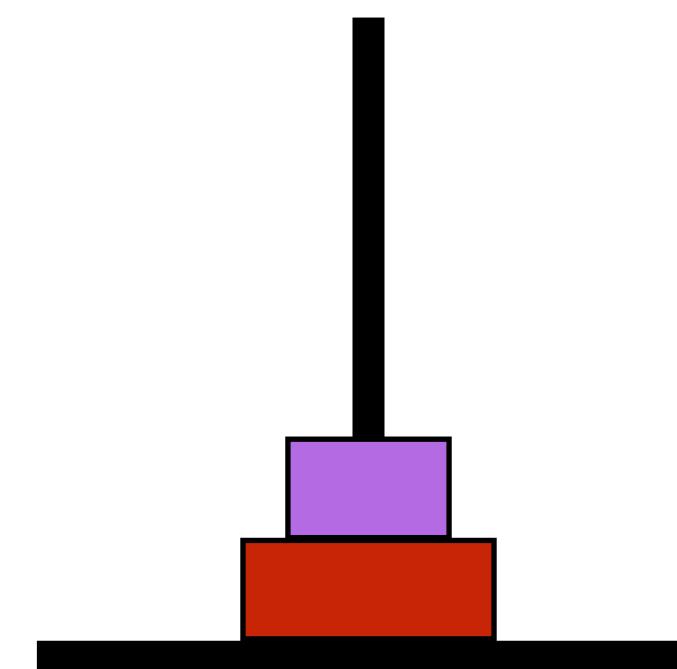
B -> A

B -> C

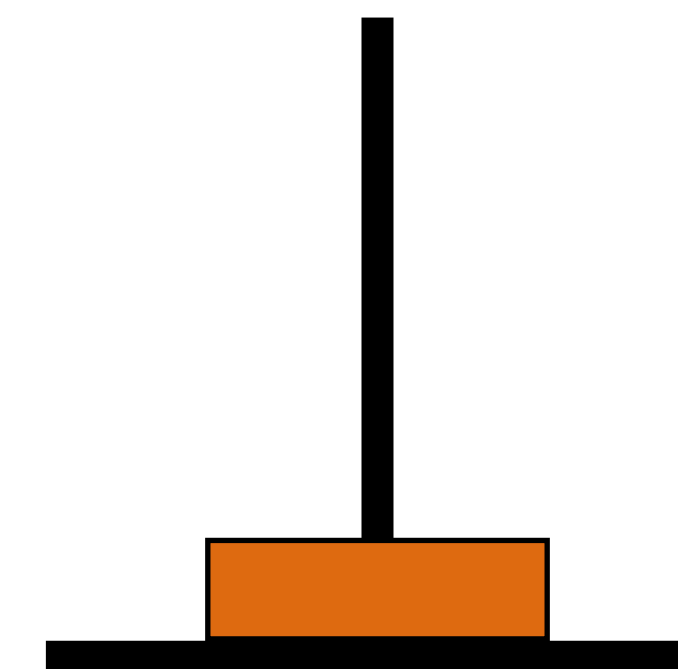
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

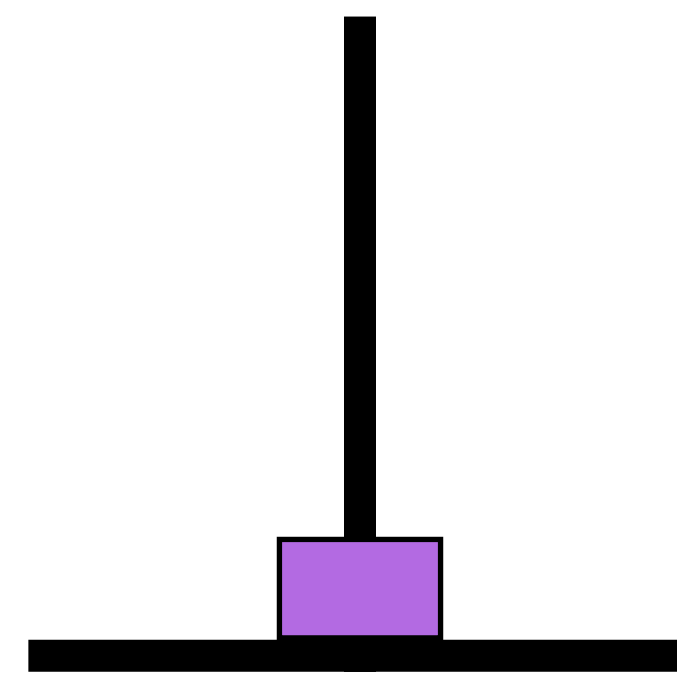
C -> B

A -> C

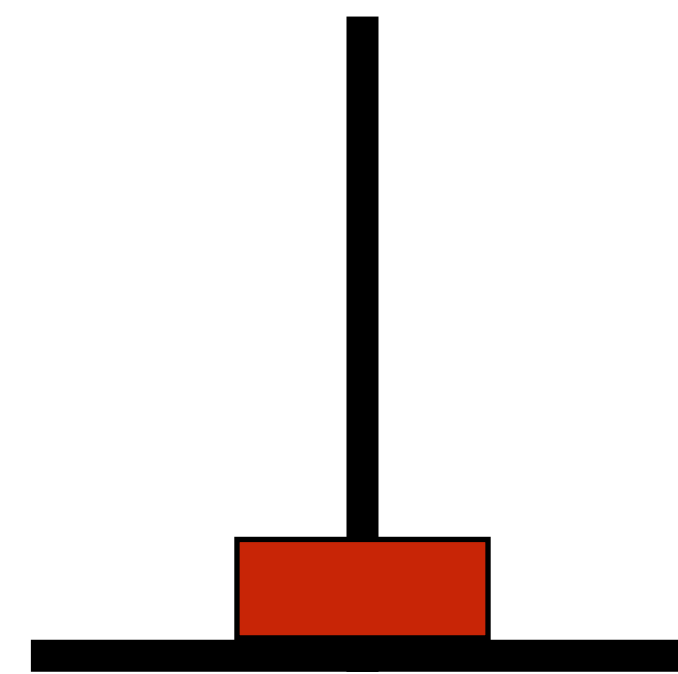
B -> A

B -> C

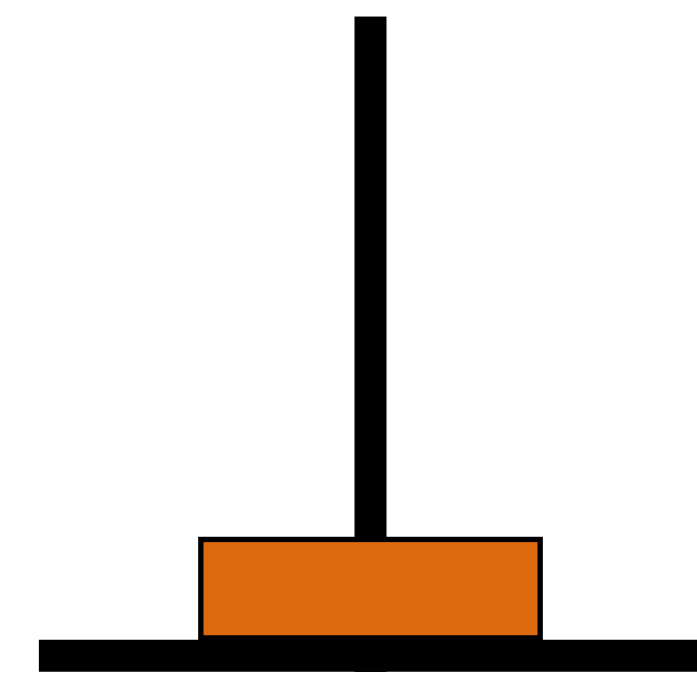
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

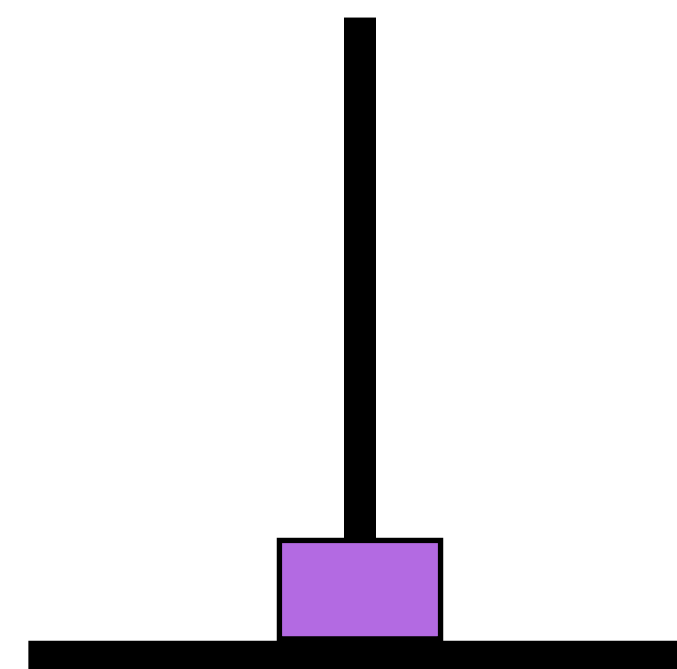
C -> B

A -> C

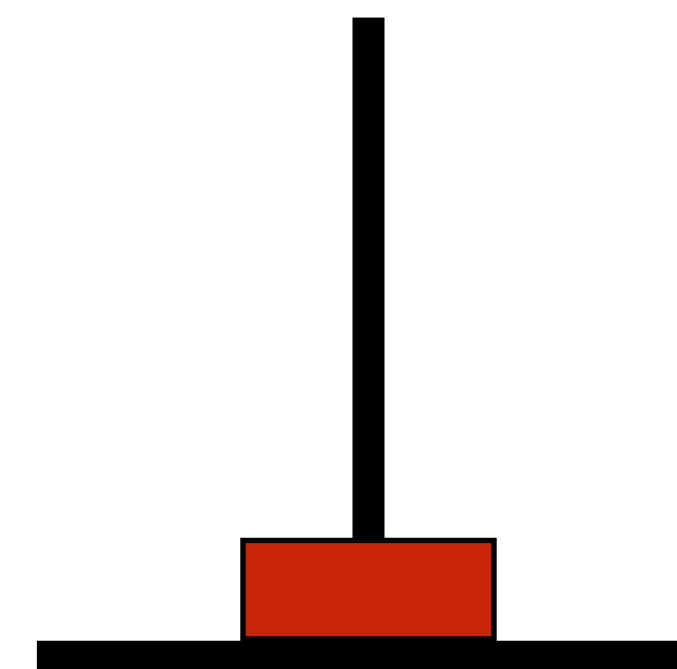
B -> A

B -> C

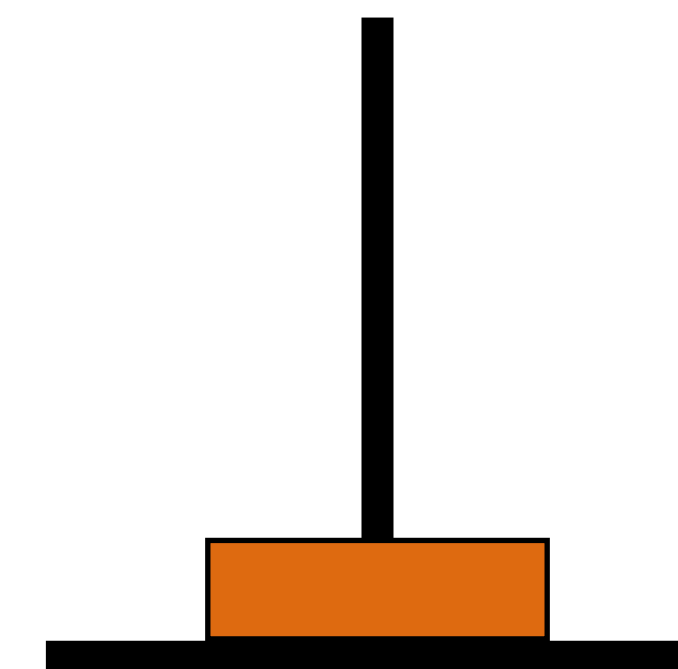
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

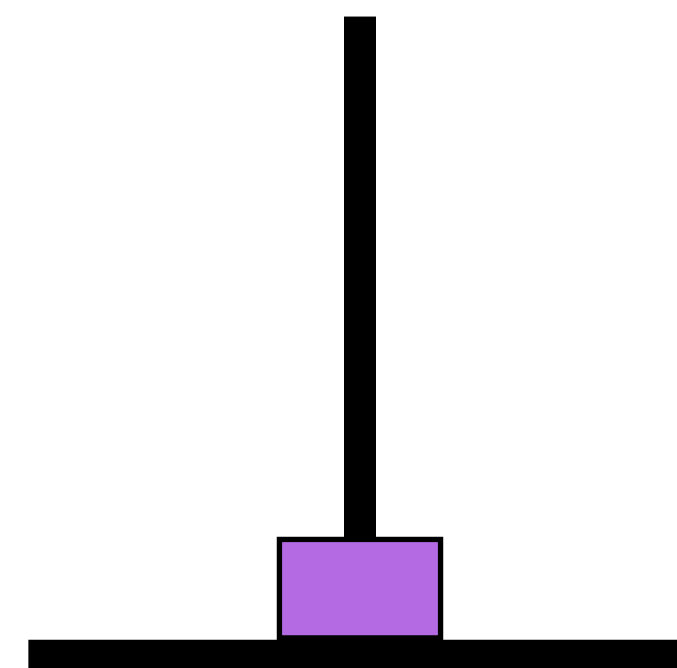
C -> B

A -> C

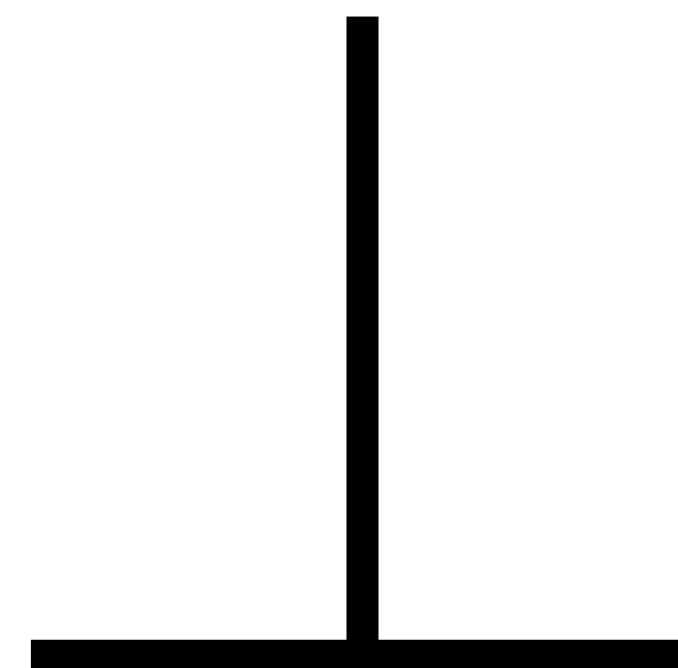
B -> A

B -> C

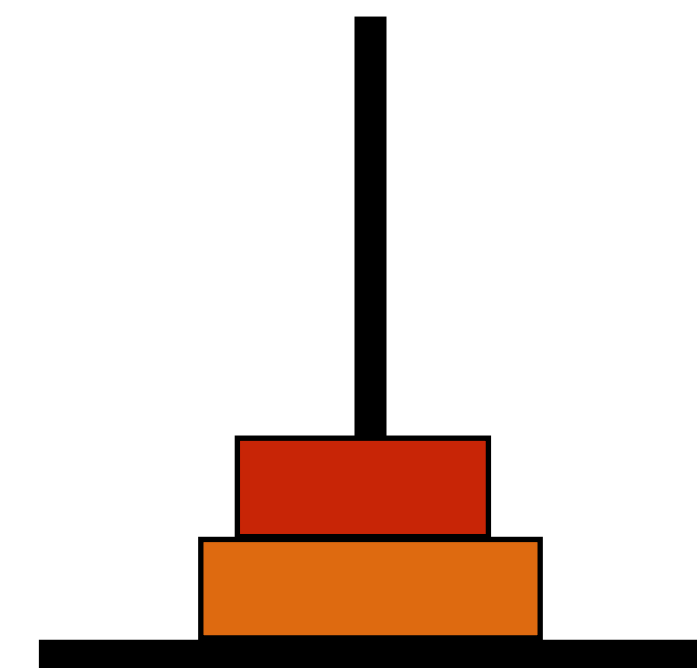
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

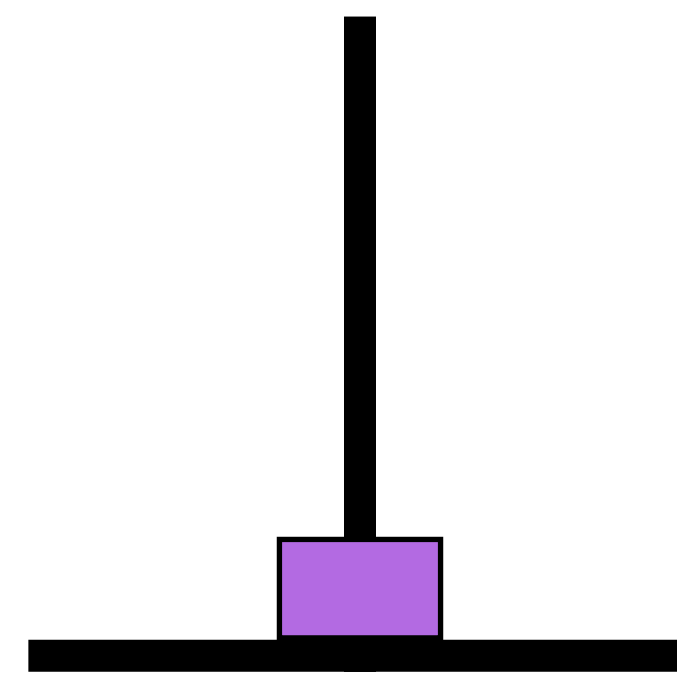
C -> B

A -> C

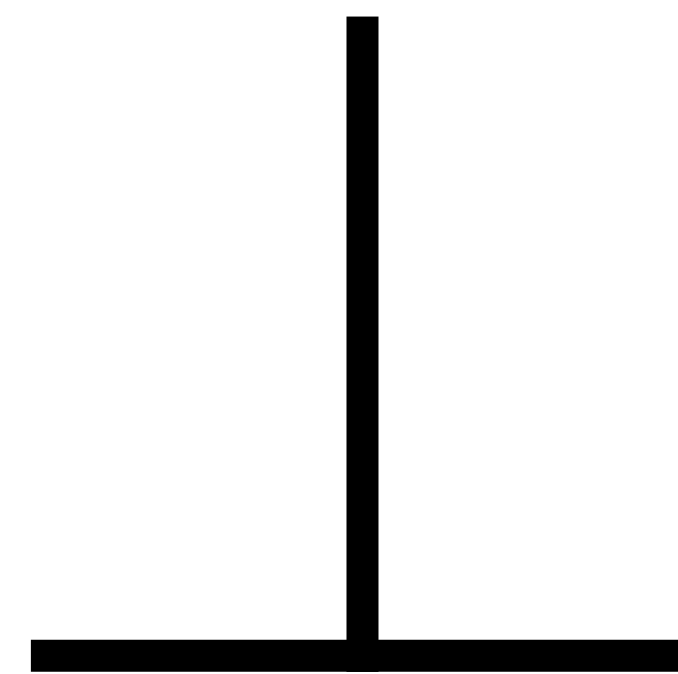
B -> A

B -> C

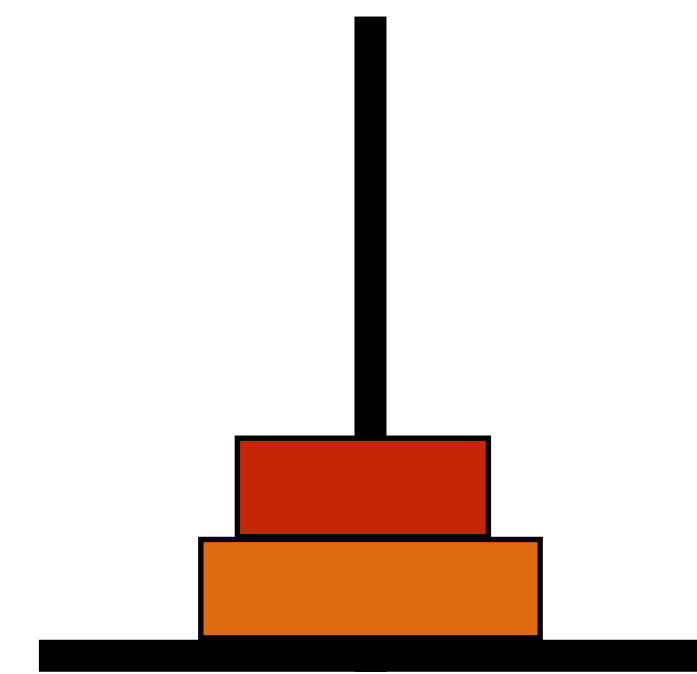
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

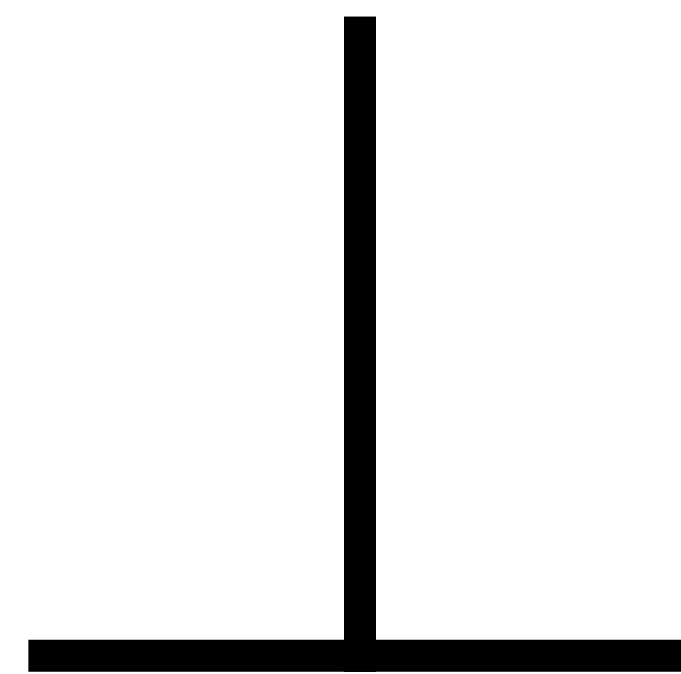
C -> B

A -> C

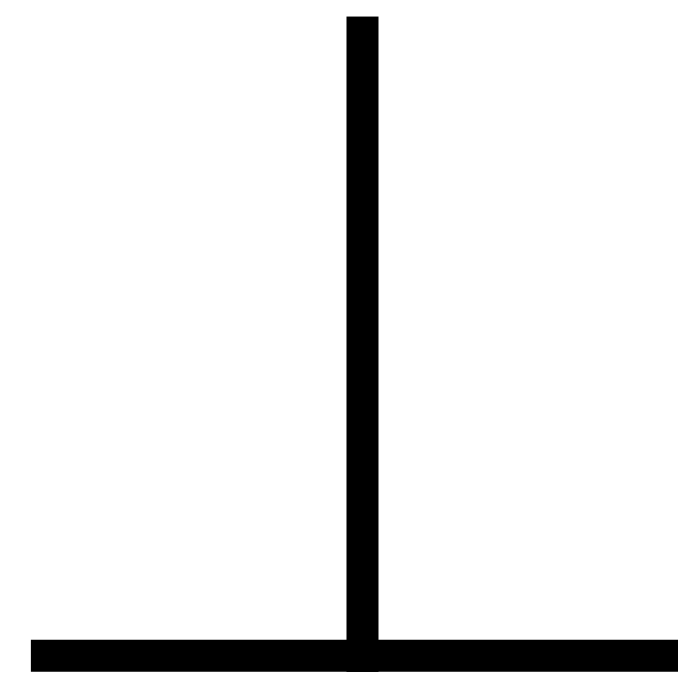
B -> A

B -> C

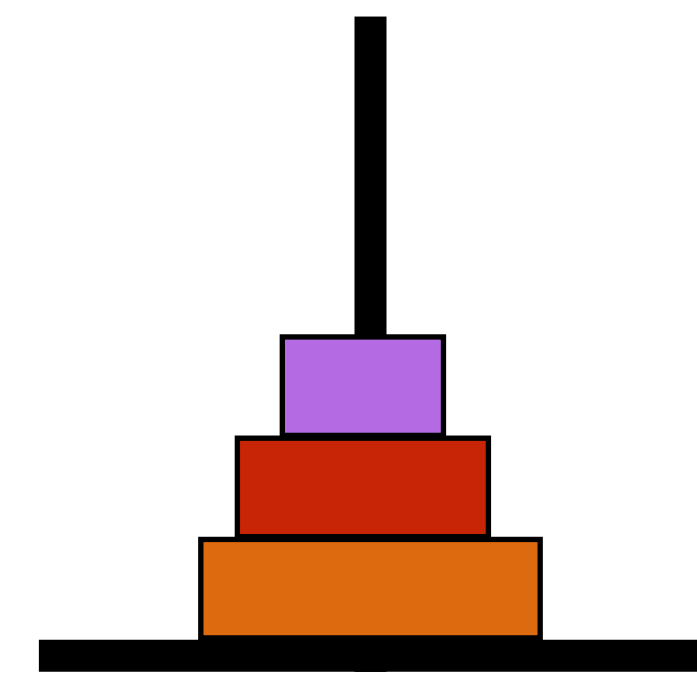
A -> C



Stange A

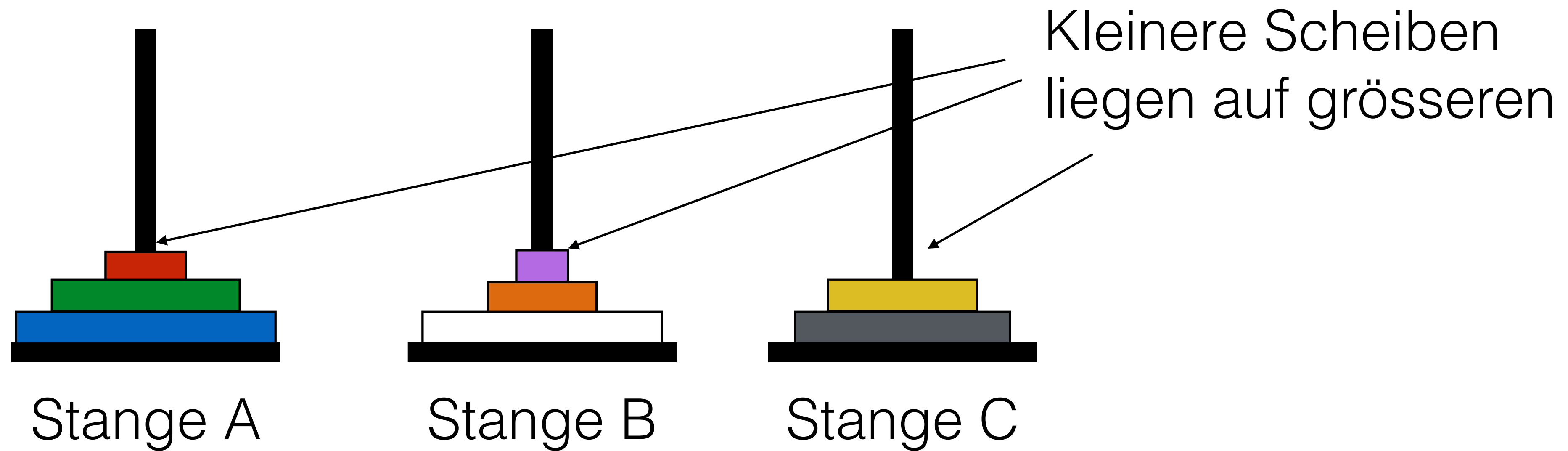


Stange B



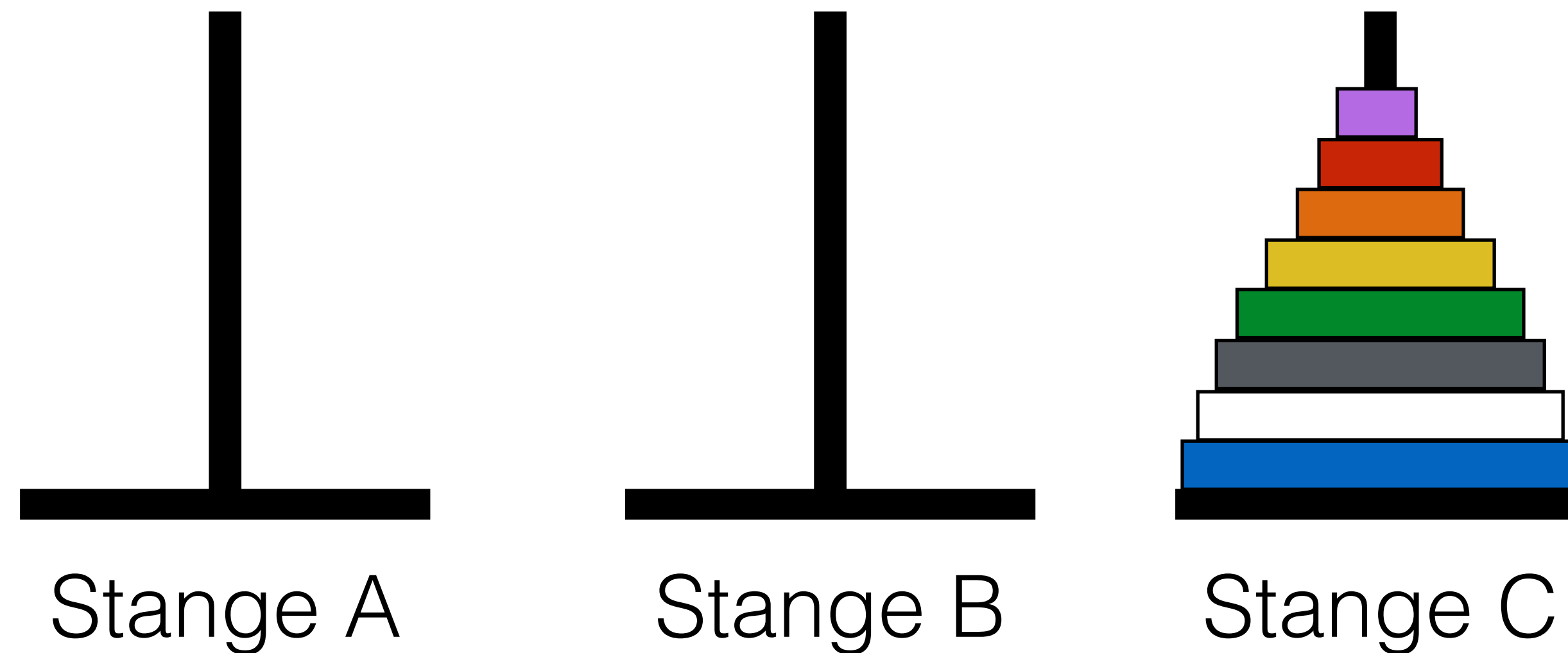
Stange C

Übungsaufgabe



Frage:

Können wir die
Scheiben auf
Stange C
bewegen?



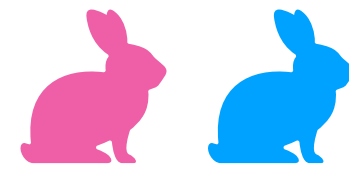
Rekursion mit mehreren Basisfällen



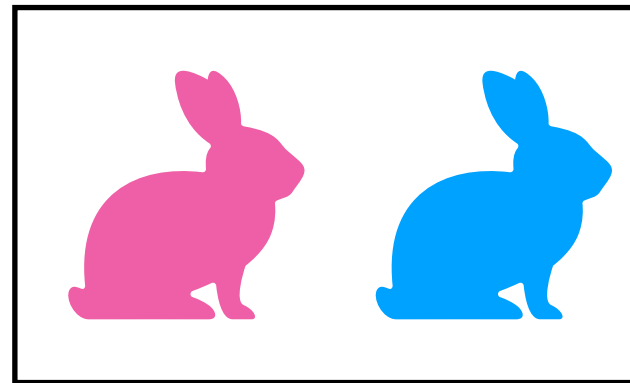
- Leonardo von Pisa (aka Fibonacci) hat folgendes Rätsel gestellt
 - Ein Paar neugeborener Hasen wird (ein Weibchen, ein Männchen) ins Gehege getan
 - Hasen sind nach einem Monat geschlechtsreif
 - Sie haben eine Trächtigkeitsdauer von einem Monat
 - Wir nehmen an, dass Hasen niemals (aus)sterben und jedes Weibchen (ab dem 2. Monat) monatlich zwei Hasen (ein Weibchen und ein Männchen) wirft.

Frage: Wieviele Hasen gibt es am Ende des Jahres?

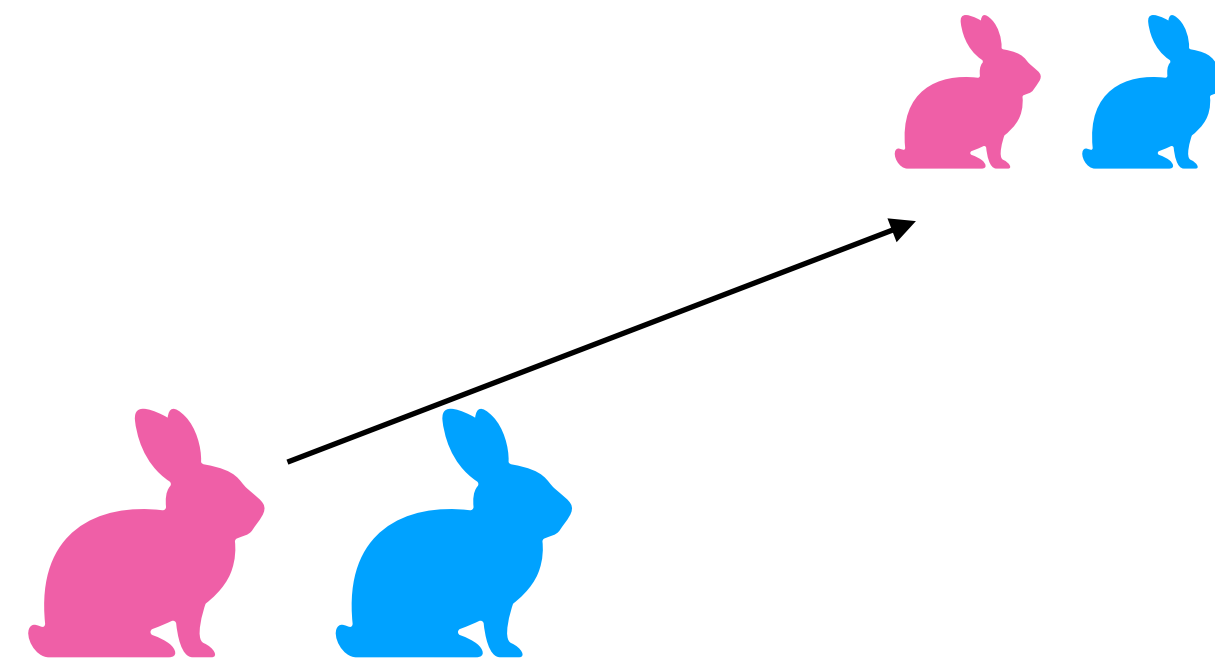
Vermehrung von Hasen



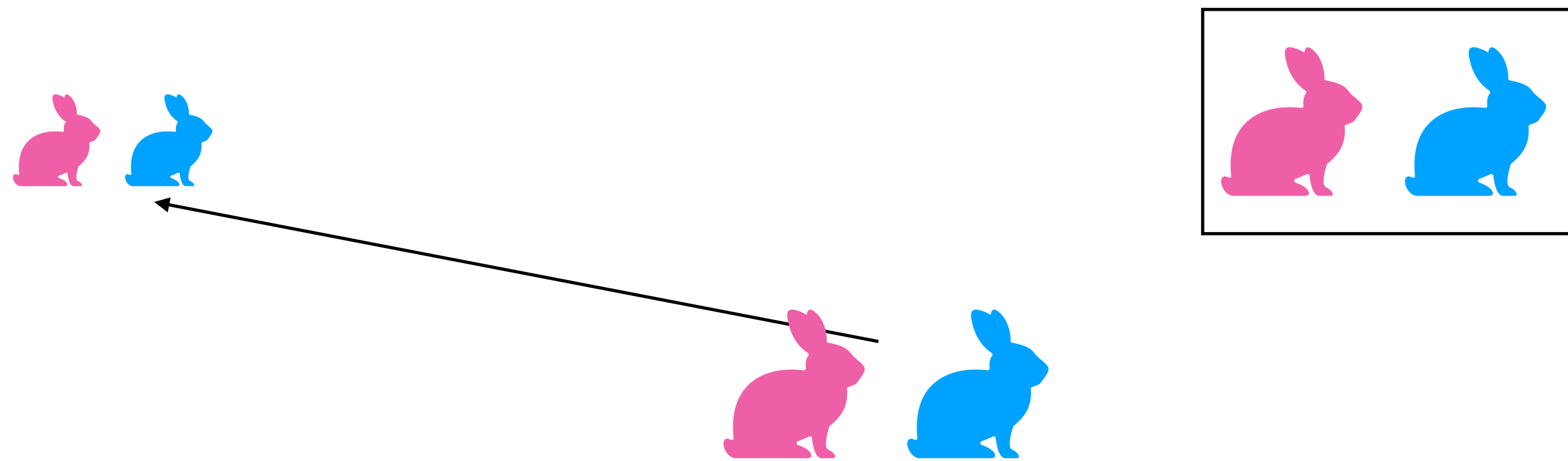
Vermehrung von Hasen



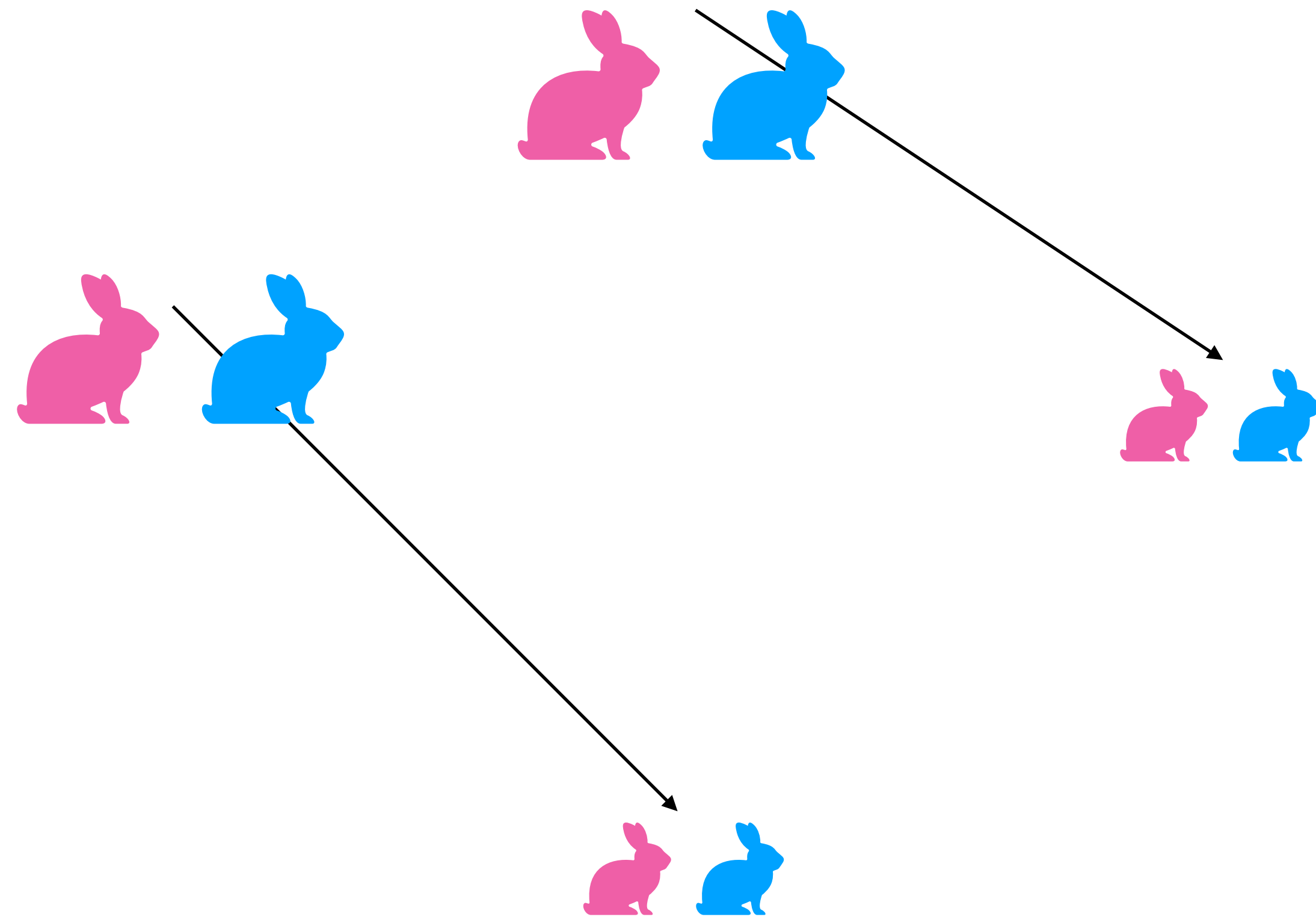
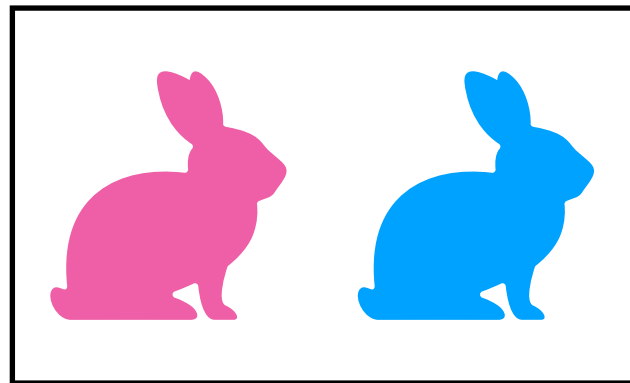
Vermehrung von Hasen



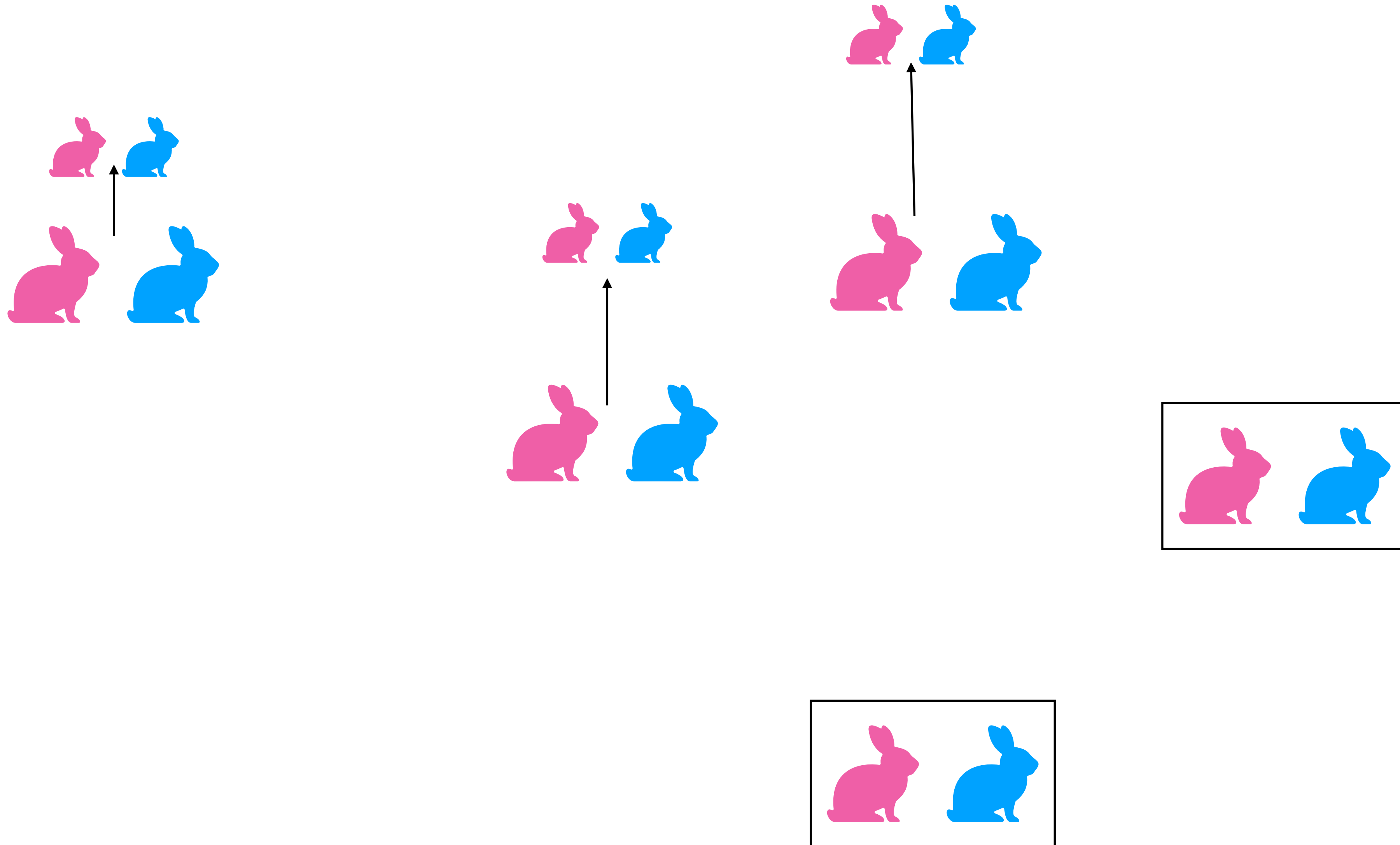
Vermehrung von Hasen



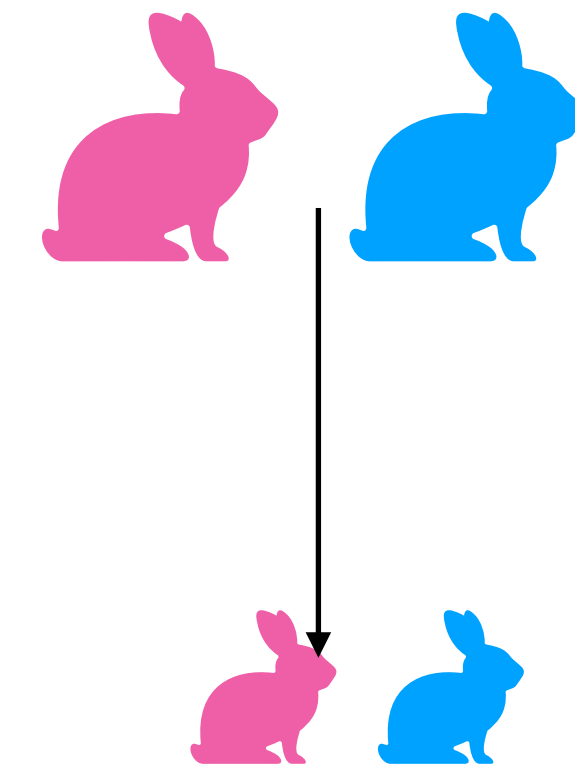
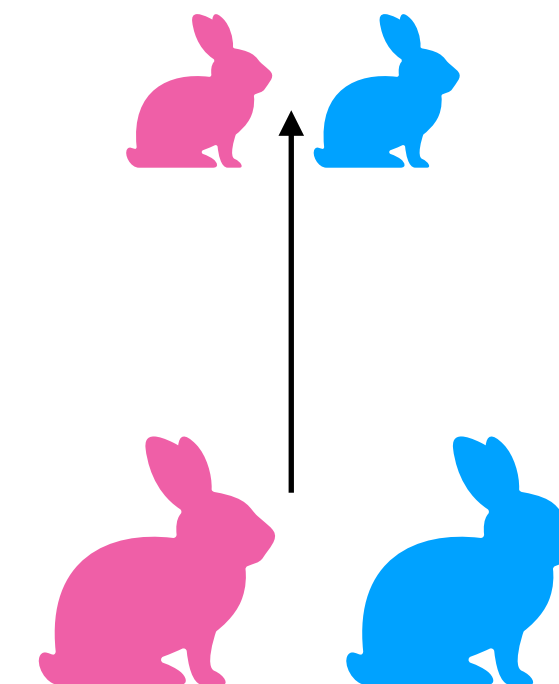
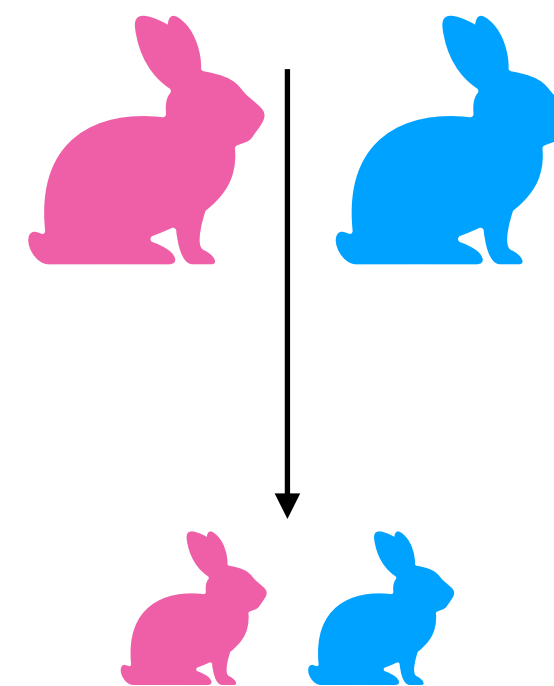
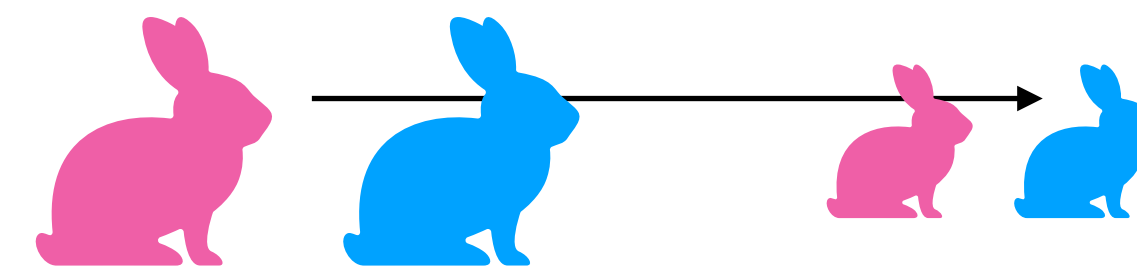
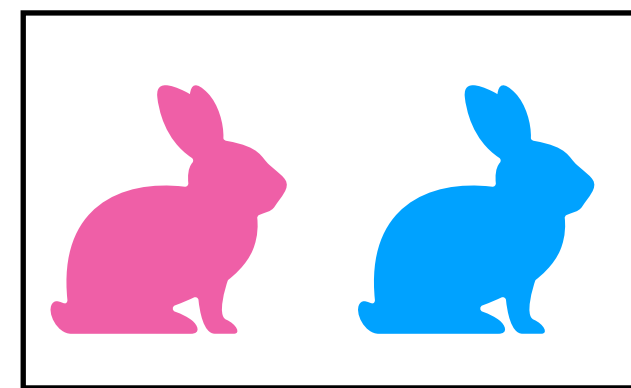
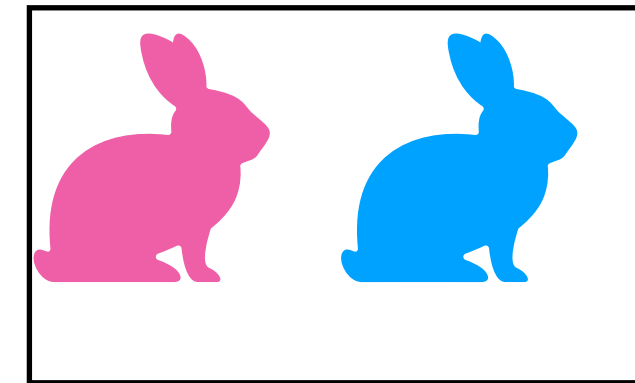
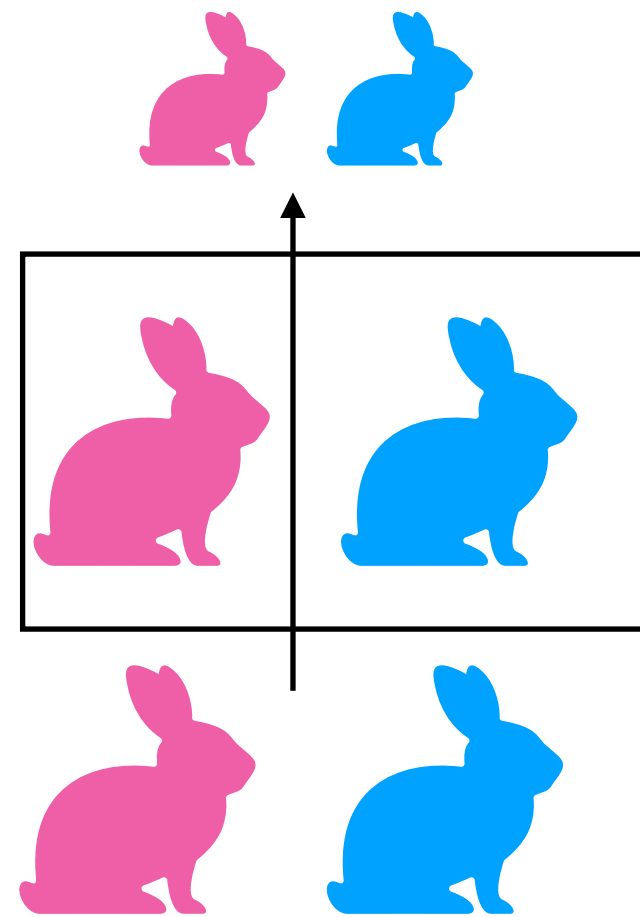
Vermehrung von Hasen



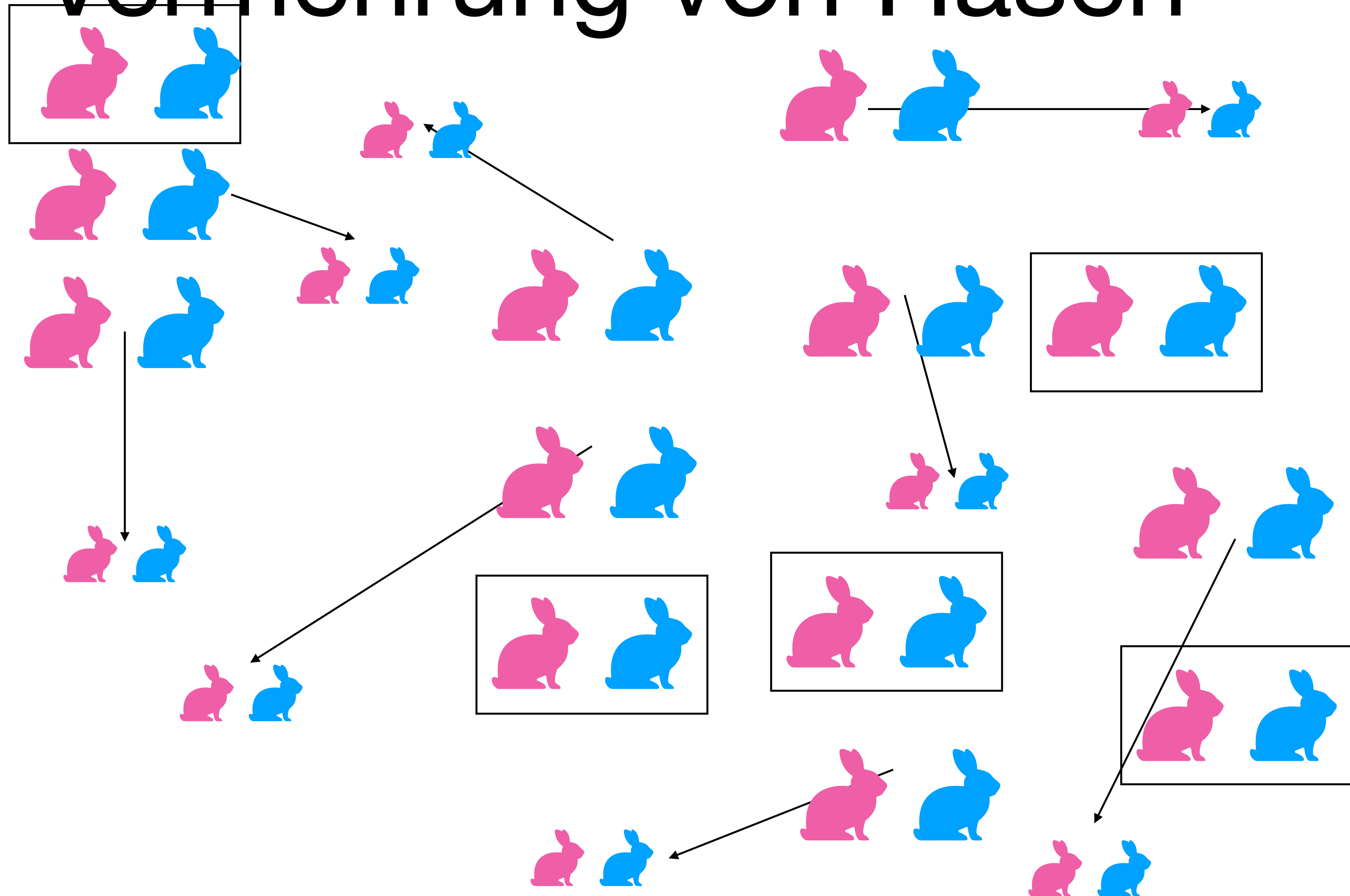
Vermehrung von Hasen



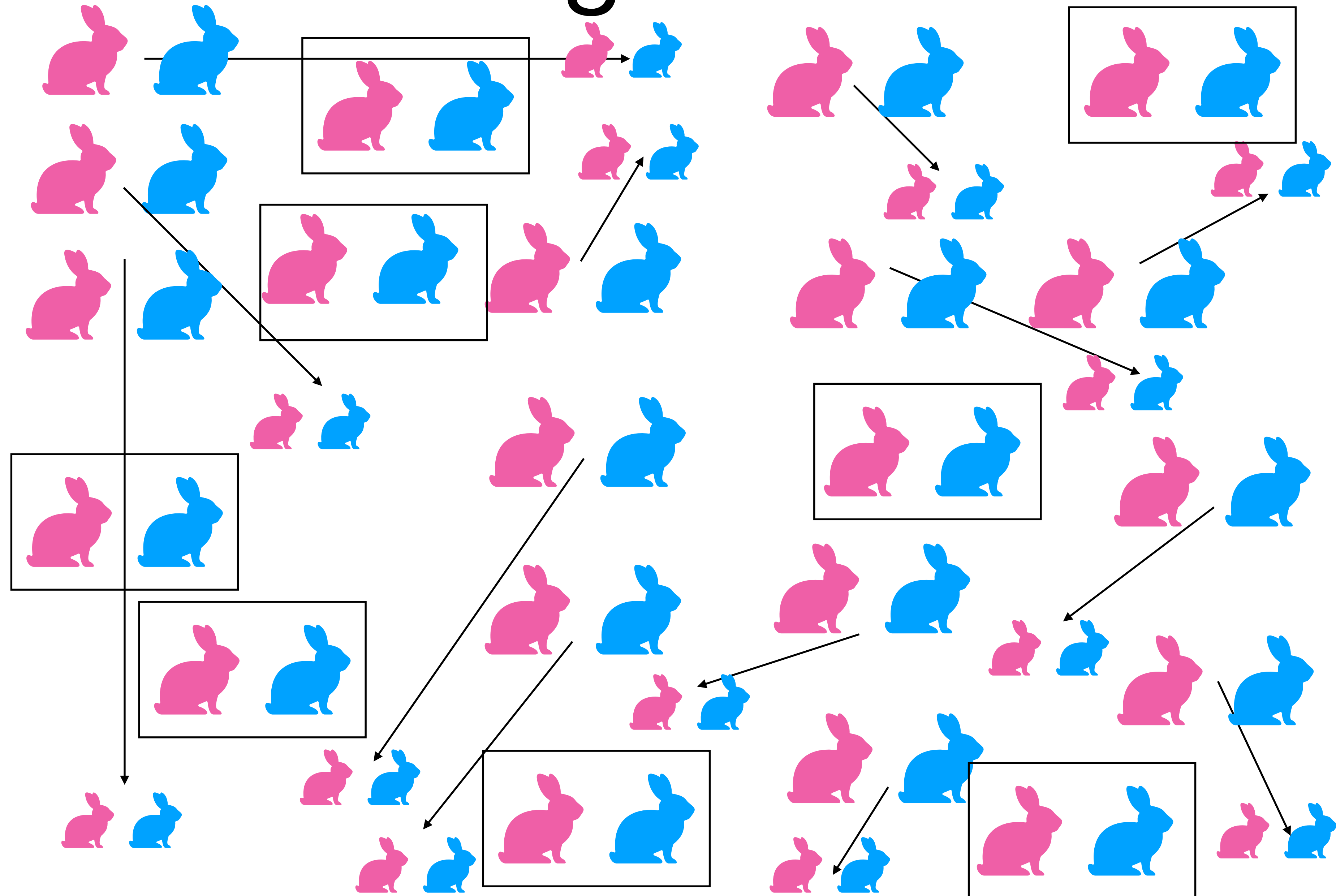
Vermehrung von Hasen



Vermehrung von Hasen



Vermehrung von Hasen



Fibonacci rekursiv

- Zu Beginn (nach 0 Monaten) gibt es ein Weibchen
- Nach einem Monat gibt es nach wie vor ein Weibchen (das nun trächtig ist)
- Nach dem zweiten Monat gibt es gibt es zwei Weibchen (eines das trächtig ist und eines das noch nicht trächtig ist)
- Bezeichne $\text{fib}(n)$ die Anzahl der Weibchen nach dem n -ten Monat
- Für alle $n > 1$ gilt
$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2), \quad \text{denn}$$
 - Jedes Weibchen, das im Monat $n-2$ lebt, wird ein junges Weibchen im Monat n werfen
 - Hinzu kommen die Weibchen, die bereits im Monat $n-1$ leben

Code Written By A CS 101 Student

```
public int fibonacci(int x) {  
    if (x == 1) {  
        return 1;  
    } else if (x == 2) {  
        return 1;  
    } else {  
        return fibonacci(x - 1) + fibonacci(x - 2);  
    }  
}
```


Code Written At A Hackathon

```
public int getFibonacciNumber(int n) {  
    switch(n) {  
        case 1: return 1;  
        case 2: return 1;  
        case 3: return 2;  
        case 4: return 3;  
        case 5: return 5;  
        case 6: return 8;  
        case 7: return 13;  
        default:  
            // good enough for the demo, lol  
            return -1;  
    }  
}
```

Code Written At A Startup

```
// TODO add Javadoc comments
/**
 * getFibonacciNumber
 */
// TODO Should we move this to a different file?
public int getFibonacciNumber(int n) {
    // TODO Stack may overflow with recursive implementation, switch over to
    // iteration approach at some point?
    if (n < 0) {
        // TODO This should probably throw an exception. Or maybe just print
        // a log message?
        return -1;
    } else if (n == 0) {
        // TODO Generalize the initial conditions?
        return 0;
    } else if (n == 1) {
        return 1;
    } else {
        // TODO Spend some time with my family and kids, I've been at work for
        // over 48 hours straight.
        return getFibonacciNumber(n - 1) + getFibonacciNumber(n - 2);
    }
}
```


}

Code Written At A Large Company

```
/**
 * getFibonacciNumber is a method that, given some index n, returns the nth
 * Fibonacci number.
 * @param n The index of the Fibonacci number you wish to retrieve.
 * @return The nth Fibonacci number.
 */
public CustomInteger64 getFibonacciNumber(CustomInteger64 n) {
    FibonacciDataViewBuilder builder =
        FibonacciDataViewBuilderFactory.createFibonacciDataViewBuilder(
            new FibonacciDataViewBuilderParams(n, null, null, 0, null));
    if (builder == FibonacciDataViewBuilderConstants.ERROR_STATE) {
        throw new FibonacciDataViewBuilderFactoryException();
    }
    FibonacciDataView dataView = builder.GenerateFibonacciDataView(this);
    if (dataView == FibonacciDataViewConstants.ERROR_STATE) {
        throw new FibonacciDataViewGenerationException();
    }
    return dataView.accessNextFibonacciNumber(null, null, null);
}
```


Code Written By A Math Ph.D.

```
public int getFibonacciNumber(int n) {  
    return (int) divide(subtract(exponentiate(phi(), n), exponentiate(psi(), n)),  
        subtract(phi(), psi()));  
}  
  
public double exponentiate(double a, double b) {  
    if (equal(b, zero())) {  
        return one();  
    } else {  
        return multiply(a, exponentiate(a, subtract(b, one())));  
    }  
}  
  
public double phi() {  
    return divide(add(one(), sqrt(add(one(), one(), one(), one(), one()))),  
        add(one(), one()));  
}  
  
public double psi() {  
    return subtract(one(), phi());  
}
```

Code Written By Your Cat

```
public static final int UNITE = 1;
public static final int UNITED = 2;

// meowwww meow
public int meow(int KITTENS_OF_THE_WORLD) {
    // MEOW
    if (KITTENS_OF_THE_WORLD < UNITED) {
        return KITTENS_OF_THE_WORLD;
    } else {
        // meeoowwwwwwww
        // meooooowwwwwwwwwwwwwwwwwwwwwww
        return meow(KITTENS_OF_THE_WORLD - UNITE)
            + meow(KITTENS_OF_THE_WORLD - UNITED);
    }
}
```

Fibonacci-Zahlen

Basisfälle

$\text{fib}(0)=1$

$\text{fib}(1)=1$

Rekursiver Fall, $n > 1$

$\text{fib}(n)=\text{fib}(n-1)+\text{fib}(n-2)$

n	fib(n)
0	1
1	1
2	2
3	3
4	5
5	8
6	13
7	21
8	34
9	55
10	89
11	144
12	233
13	377

Berechnung der Fibonacci-Zahlen

```
def fib(n):  
    if n==0 or n==1:  
        return 1  
    else:  
        return fib(n-1) + fib(n-2)
```

Rekursion auf nicht-numerischen Daten

- Wie kann man überprüfen, ob eine Zeichenkette ein Palindrom ist, d.h. es liest sich von hinten wie von vorne gleich, wie z.B.
 - ANNA
 - RENTNER
 - REGALLAGER
 - RETSINAKANISTER
 - Geist ziert Leben, Mut hegt Siege, Beileid trägt belegbare Reue, Neid dient nie, nun eint Neid die Neuerer, abgelebt gärt die Liebe, Geist geht, umnebelt reizt Sieg.
 - Die liebe Tote Beileid

Wenn man Leerzeichen, Groß- und Kleinschreibung und Kommasetzung vernachlässigt

Palindrome rekursiv erkennen

- **Schritt 1:** Wir bereinigen die eingegebene Zeichenkette bzgl. Groß- und Kleinschreibung und Zeichen außerhalb des Alphabets:
 - entferne alle Zeichen, die keinem Buchstaben entsprechen
 - ersetze jeden verbleibenden Großbuchstaben in den entsprechenden Kleinbuchstaben
- **Schritt 2:** Entscheide ob gegebene bereinigte Zeichenkette ein Palindrom ist rekursiv:
 - **Basisfall:** Jede Zeichenkette der Länge 0 oder 1 ist ein Palindrom
 - **Rekursiver Fall** (Länge mindestens 2):
 - Die Zeichenkette ist genau dann ein Palindrom, falls
 - erster und letzter Buchstabe gleich sind, und
 - das mittlere Wort (Wort das durch Löschen des ersten und letzten Zeichens entsteht) selbst ein Palindrom ist

Palindrom Beispiel

- Sei die Eingabe die Zeichenkette

"<In Nagold legen Hähne Geld> log Anni."

- Nach dem Bereinigungsschritt entsteht

"innagoldlegenhähnegeldloganni"

- Wir definieren eine Funktion `istPalindrom`, die eine Zeichenkette entgegennimmt und ein `bool` zurückliefert

- Es gilt `istPalindrom("innagoldlegenhähnegeldloganni")` genau dann, wenn
 - `i == i` und
 - `istPalindrom("nnagoldlegenhähnegeldlogann")` gilt

==true

Palindromerkennung rekursiv

```
def Palindrom(s):  
    def bereinige(s):  
        s=s.lower()  
        antwort=""  
        for c in s:  
            if c in "abcdefghijklmnopqrstuvwxyzäüö":  
                antwort = antwort + c  
        return antwort  
    def istPalindrom(s):  
        if len(s)<=1:  
            return True  
        else:  
            return s[0]==s[-1] and istPalindrom(s[1:-1])  
    return istPalindrom(bereinige(s))
```


Der euklidische Algorithmus

- gilt als der älteste nicht-triviale Algorithmus
- Berechnung des größten gemeinsamen Teilers zweier positiver natürlicher Zahlen a und b mit $a \geq b$ eindeutig bestimmt

- Setze $r_{-1} = a$ und $r_0 = b$ und finde $q_1, r_1 \geq 1$ mit $r_1 < r_0$ so, dass

$$a = q_1 \cdot r_0 + r_1 \quad \text{Falls } r_1 = 0 \text{ dann sind wir fertig.}$$

$$r_0 = q_2 \cdot r_1 + r_2 \quad \text{für } q_2, r_2 \geq 0 \text{ mit } 0 \leq r_2 < r_1$$

$$r_1 = q_3 \cdot r_2 + r_3 \quad \text{für } q_3, r_3 \geq 0 \text{ mit } 0 \leq r_3 < r_2$$

bis $\vdots$

$$r_{n-1} = q_{n+1} \cdot r_n + r_{n+1} \quad \text{für } q_{n+1}, r_{n+1} \geq 0 \text{ mit } r_{n+1} = 0$$

für ein $n \geq 0$ $\quad \quad \quad = 0$

- Ergebnis $\text{ggT}(a, b) = r_n$



um 300 v. Chr.

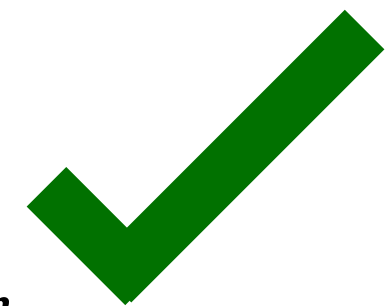
Terminierung/Korrektheit Euklidischer Alg.

- Terminierung klar wegen: $r_{i+1} < r_i$ für alle $1 \leq i < n$ Warum?
- In der $(i + 1)$ -ten Iteration werden $q_{i+1}, r_{i+1} \geq 0$ mit $0 \leq r_{i+1} < r_i$ gefunden so, dass $r_{i-1} = q_{i+1} \cdot r_i + r_{i+1}$

- Es gilt $\mathbf{ggT}(r_{i-1}, r_i) = \mathbf{ggT}(r_i, r_{i+1})$
für alle $0 \leq i < n$

Warum?

$$\mathbf{ggT}(a, b) = \mathbf{ggT}(r_{-1}, r_0) \boxed{=} \mathbf{ggT}(r_0, r_1) \boxed{=} \dots \boxed{=} \mathbf{ggT}(r_{n-1}, r_n) \boxed{=} r_n$$



- Es gilt sogar $r_{i+1} < r_{i-1}/2$ für alle $0 \leq i < n$

Warum?

Euklidischer Algorithmus rekursiv

```
def Euklid(a,b):  
    if a==0 or b==0:  
        return "undefiniert"  
    if a<b: return Euklid(b,a)  
    if a%b==0:  
        return b  
    else:  
        return Euklid(b,a%b)
```

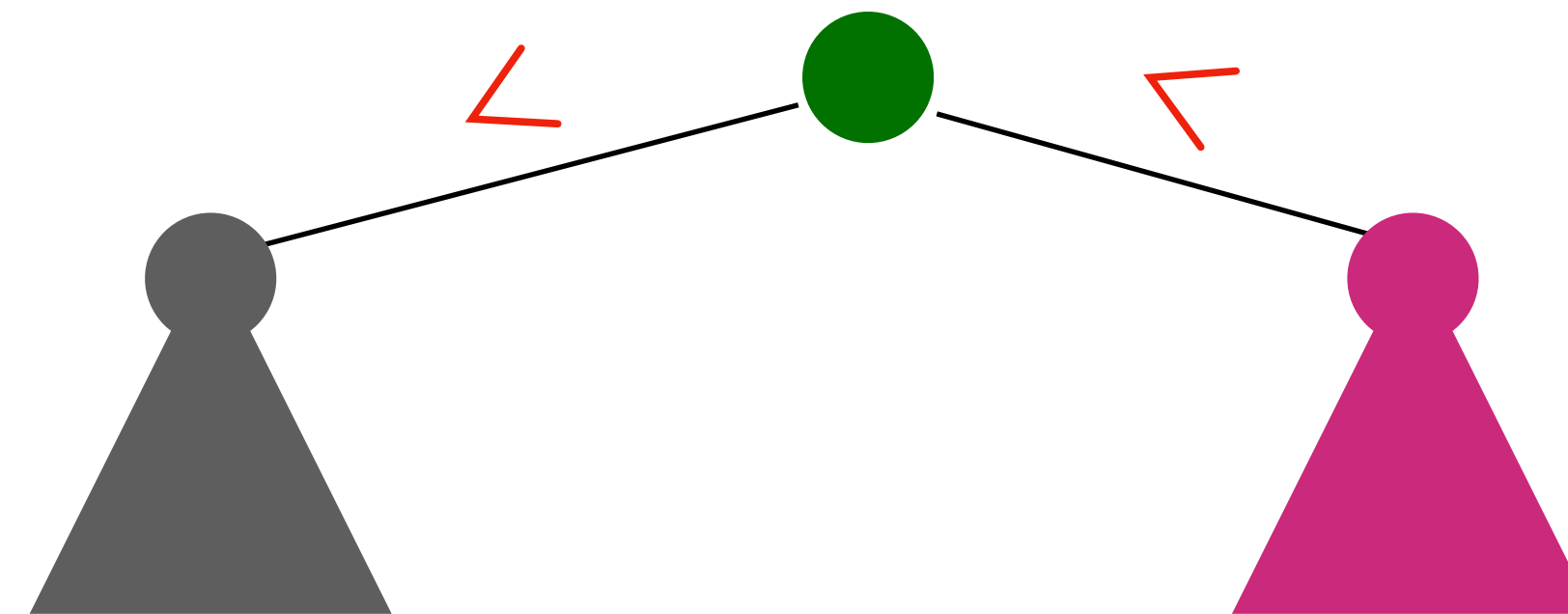

Betreiben einer Suchmaschine

- Stellen Sie sich vor Sie betreiben eine Suchmaschine
- Sie müssen sehr viele Suchbegriffe und die dazugehörigen Websites verwalten
- Sie müssen eine Datenstruktur anlegen, das diese angefragten Suchbegriffe verwaltet und jeweils mit Websites assoziiert
- Manche der Suchbegriffe kommen häufiger vor als andere und Sie kennen die Häufigkeit aller Suchbegriffe
- Die einem geg. Suchbegriff zugehörige Website muss so schnell wie möglich gefunden werden
- Sie ordnen die Suchbegriffe (mit Ihren Websites) in einem binären Suchbaum an (die einzige Möglichkeit, die Ihnen zur Verfügung steht)

Binäre Suchbäume

werden wir ab nun
vernachlässigen

- Sie verwalten Ihre Suchbegriffe in einem binären Suchbaum:
 - die Knoten in Ihrem Suchbaum entsprechen Ihren verwalteten Suchbegriffen (zugehörige Website ebenfalls im Knoten vorhanden)
 - alle Knoten im **linken Unterbaum** sind **kleiner oder gleich**, im **rechten Unterbaum größer oder gleich** dem **Wurzelwert** (für jeden Unterbaum)



kleiner bzw. größer
bzgl. String-Vergleich
oder jeder anderen
totalen Ordnung

- „je tiefer Sie den Baum nach unten traversieren um einen Suchbegriff zu suchen desto länger muss gewartet die dazugehörige Website aufzurufen“
- Suchbegriffe werden je nach angegebener Häufigkeit gesucht

Binäre Suchbäume: Gesamtgewicht

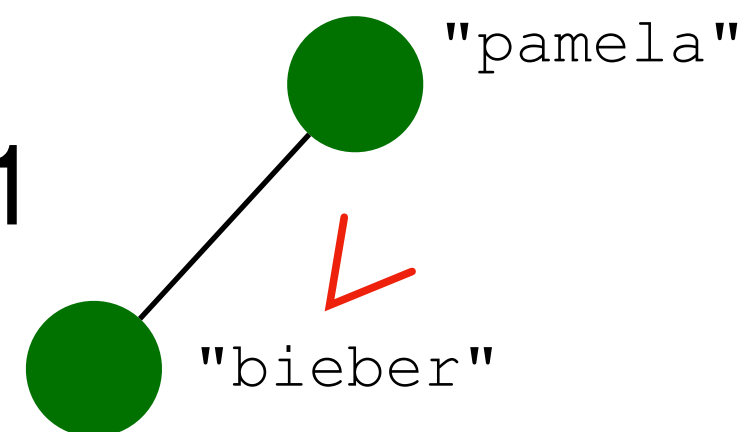
- nehmen wir an Sie haben folgende Begriffs- und Häufigkeitslisten:

begriffe = ["bieber", "pamela"]

Annahme:
nach String sortiert
"bieber" < "pamela"

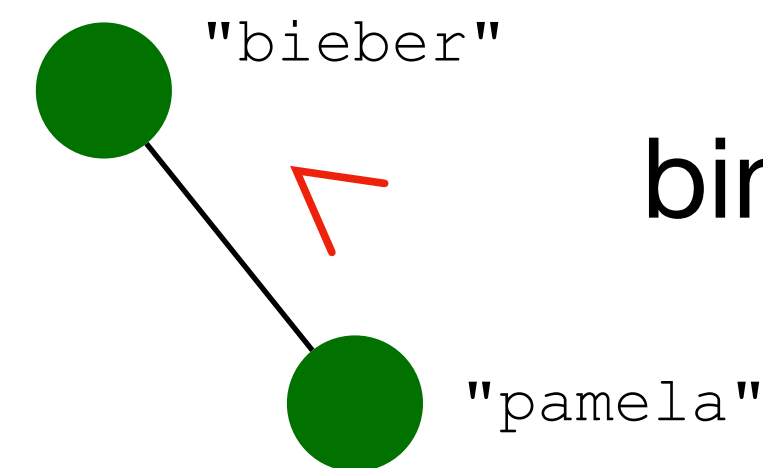
haeufigkeiten = [50, 34]

bin. Suchbaum 1



Gesamtgewicht $1 \cdot 34 + 2 \cdot 50 = 134$

bin. Suchbaum 2



Gesamtgewicht $1 \cdot 50 + 2 \cdot 34 = 118$
optimal!

- allgemeine Definition des **Gesamtgewicht (GG)** eines Suchbaums T

$$\mathbf{GG(T)} = \sum_{\substack{\text{Knoten } v \\ \text{in } T}} \text{Tiefe}(v) * \text{Häufigkeit}(\text{Beschriftung}(v))$$

Entfernung zur Wurzel + 1

hier: Zeichenkette, die an
Knoten v steht

Binäre Suchbäume: O(n) Zeit

- nehmen wir an Sie haben folgende

begriffe

haben

**Optimaler Suchbaum:
Finde binären Suchbaum
mit minimalem Gesamtgewicht**

Suchbaum 2

"anela"

Gesamtgewicht $1 \cdot 50 + 2 \cdot 34 = 118$

optimal!

Gesamtgewicht eines Suchbaums T

$$\text{Gewicht}(T) = \sum_{\text{Knoten } v \text{ in } T} \text{Tiefe}(v) * \text{Häufigkeit}(\text{Beschriftung}(v))$$

Knoten v
in T

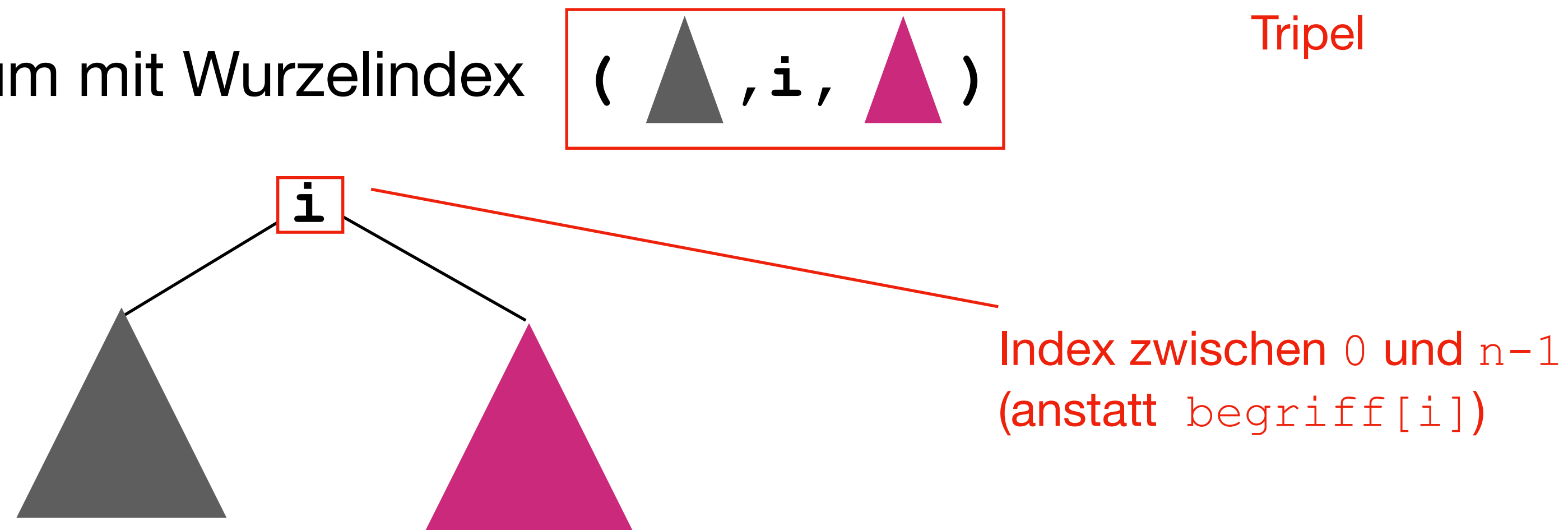
Entfernung zur Wurzel + 1

hier: Zeichenkette, die an
Knoten v steht

Präsentation von bin. Suchbäumen

begriffe = [, , ..., ]
 haeufigkeiten = [, , ..., ]

- Beschriftung der Knoten: Indizes anstatt Begriff an diesem Index
- Wir stellen binäre Suchbäume folgendermaßen rekursiv dar:
 - leerer Suchbaum: $()$
 - Suchbaum mit Wurzelindex $(\blacktriangle, i, \blacktriangle)$ Tripel



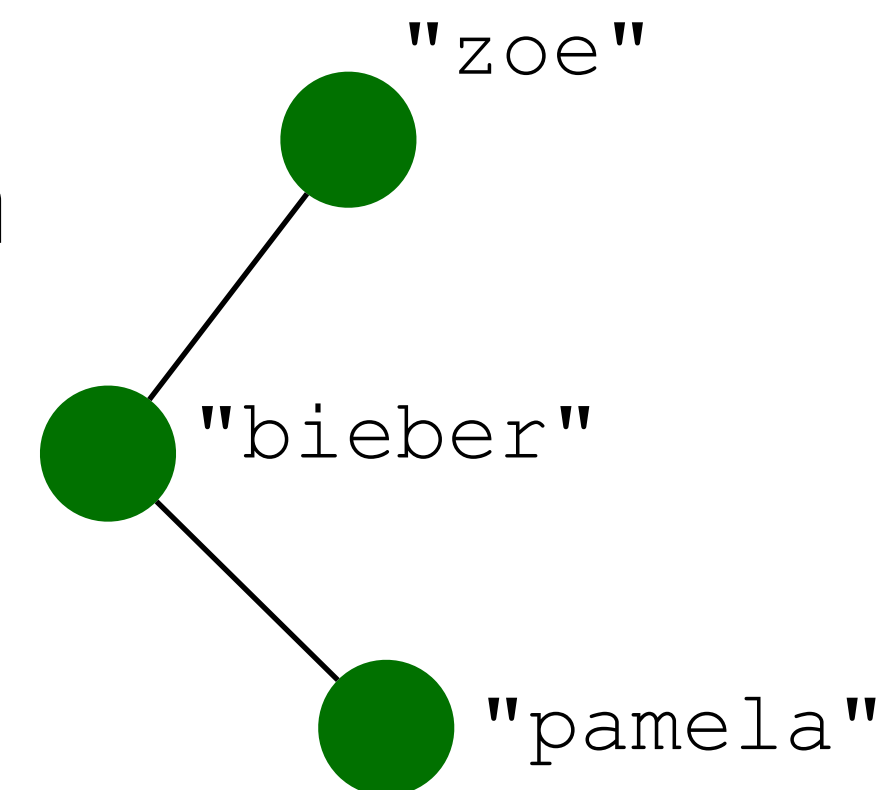
Darstellung von bin. Suchbäumen

Beispiel

begriffe = ["bieber", "pamela", "zoe"]

haeufigkeiten = [34, 8, 15]

Eigentlicher bin. Suchbaum



Interne Darstellung

(((), 0, ((), 1, ())), 2, ())

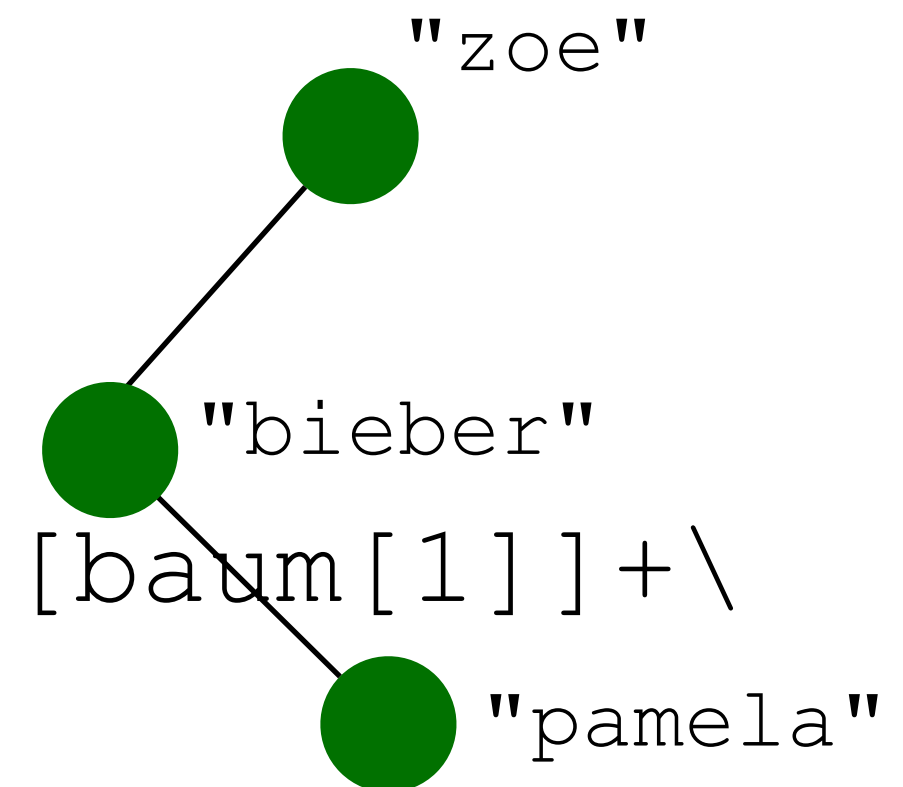
Index an der Wurzel

Geeignete Ausgabe der internen Präsentation

```
leerbaum=()
```

Eigentlicher bin. Suchbaum

```
def ausgabe(baum):  
    if not baum:  
        return ""  
    ## Baum ist nicht leer  
    return "("+ausgabe2(baum[0])+begriffe[baum[1]]+"\n        ausgabe2(baum[2])+")"
```



Interne Darstellung:

```
(((), 0, (((), 1, ()))), 2, ())
```

Rückgabe von ausgabe:

```
((bieber(pamela))zoe)
```

Berechnung des Gesamtgewichts

- allgemeine Definition des **Gesamtgewicht (GG)** eines Suchbaums T

$$\mathbf{GG(T)} = \sum_{\substack{\text{Knoten } v \\ \text{in } T}} \text{Tiefe}(v) * \text{Häufigkeit}(\text{Beschriftung}(v))$$

Entfernung zur Wurzel + 1

hier: Zeichenkette, die an Knoten v steht

def gg(baum) : Unterfunktion

```
def gewichtet(baum, faktor) :  
    if baum==leerbaum:  
        return 0  
    else:  
        return faktor*haeufigkeit[baum[1]] +\  
                gewichtet(baum[0], faktor+1) +\  
                gewichtet(baum[2], faktor+1)  
return gewichtet(baum, 1)
```

Wurzel des linken und rechten Unterbaums haben
einen um 1 erhöhten Abstand zur Wurzel des eigentlichen Baums

Problemstellung und Teillösungen

genau:

n-te Catalan-Zahl

Satz (ohne Beweis): Es gibt bei n Suchbegriffen rund $4^n / (n + 1) \sqrt{\pi n}$ verschiedene potentielle Suchbäume.

- Gegeben sei eine **Begriffsliste** und eine **Häufigkeitsliste** der Länge jeweils n
(Zeichenketten) (nicht-negative Integer)

	0	1		n-1	Indizes				
begriffe =	[	■	,	■	,	...	,	■	]
haeufigkeiten =	[	■	,	■	,	...	,	■	]

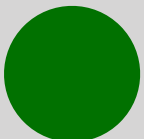
- Für jedes Unterintervall $[i, j]$ von $[0, n]$ bezeichne
 - $T[i, j]$ den (bzw. irgendeinen) optimalen bin. Suchbaum eingeschränkt auf $\{i, \dots, j-1\}$ und $haeufigkeiten[i, j]$
 - uns interessiert eigentlich nur $T[0, n]$

Rekursive Beschreibung

begriffe = [, , ..., ]
haeufigkeiten = [, , ..., ]

Basisfälle

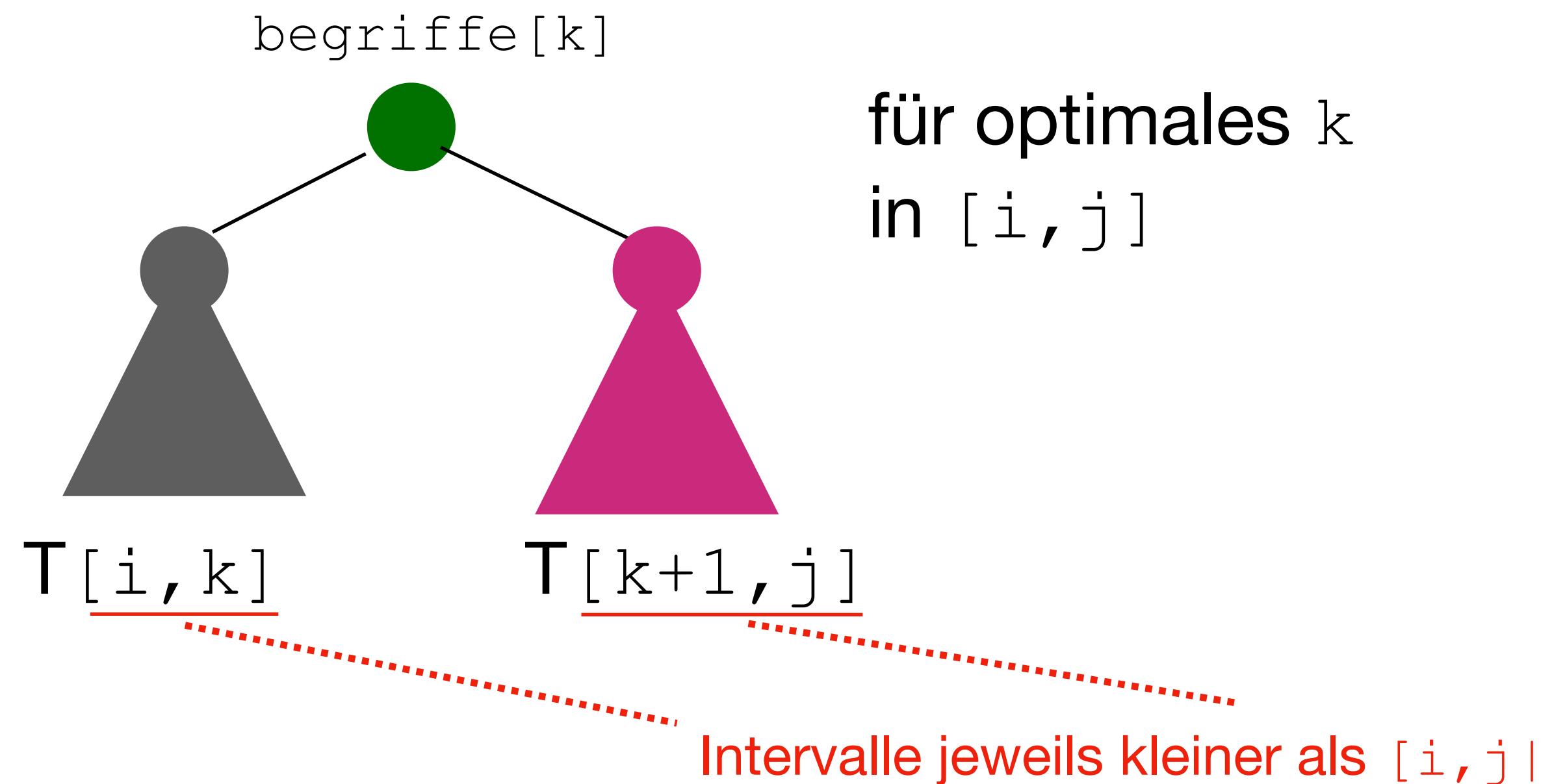
- $T[i, i] =$ **leerer Baum**
 $\parallel$
 j

- $T[i, i+1] =$ 
 $\parallel$
 j

Rekursive Fälle

- $T[i, j] =$

falls $j > i+1$



Rekursive Lösung

beginne mit erstem potentiellen optimalen Suchbaum für $[i, j]$: Nimm i als Wurzel, linker Teilbaum leer und rechter Teilbaum ist die beste Lösung für Intervall $[i+1, j]$

```
def loeseRekursiv(i, j):  
    if i==j:  
        return leerbaum  
    if j==i+1:  
        return (leerbaum, i, leerbaum)  
    else:  
        baum = (leerbaum, i, loeseRekursiv(i+1, j))  
        for k in range(i+1, j):  
            links = loeseRekursiv(i, k)  
            rechts = loeseRekursiv(k+1, j)  
            if gg((links, k, rechts)) < gg(baum):  
                baum = (links, k, rechts)  
    return baum
```

Ein Suchbaum im Paper von Donald Knuth



```
def init2():
    begriffe = ["a", "an", "and", \
                "by", "effects", "for", \
                "from", "high", "in", \
                "of", "on", "the", "to", \
                "with"]

    haeufigkeit=[32, 7, 69, 13, 6, 15,
                 10, 8, 64, 142, 22, 79, 18, 9]
    return (len(begriffe), begriffe, haeufigkeit)
```

Optimum Binary Search Trees*

D. E. KNUTH

Received June 22, 1970

One of the popular methods for retrieving information by its "name" is to store the names in a binary tree. To find if a given name is in the tree, we compare it to the name at the root, and four cases arise:

1. There is *no* root (the binary tree is empty): The given name is not in the tree, and the search terminates *unsuccessfully*.
2. The given name *matches* the name at the root: The search terminates *successfully*.
3. The given name is *less* than the name at the root: The search continues by examining the *left subtree* of the root in the same way.
4. The given name is *greater* than the name at the root: The search continues by examining the *right subtree* of the root in the same way.

Special cases of this method are the binary search and its variants (uncentered binary search; Fibonacci search) and the search-sort scheme of Wheeler-Berners Lee-Booth-Hibbard-Windley, *et al.* (see [1, 3, 7, 10]).

When all names in the tree are equally probable, it is not difficult to see that a best possible binary tree from the standpoint of average search time is one with minimum path length, namely the *complete binary tree* (see [9, pp. 400–401]). This is the tree which is implicitly present in one of the variants of the binary search method.

But when some names are known to be much more likely to occur than others, the best possible binary tree will not necessarily be balanced. For example, consider the following words and frequencies,

a	32
an	7
and	69
by	13
effects	6
for	15
from	10
high	8
in	64
of	142
on	22
the	79
to	18
with	9

* The research reported here was supported by IBM Corporation.

Das Hauptprogramm

```
leerbaum=()

def init2():
    ...

def ausgabe(baum):
    ...

def gg(baum):
    ...

def loeseRekursiv(i,j):
    ...

def loeseDP()
    ...

(n,begriffe,haeufigkeit)=init2()
baum1=loeseDP()
baum2=loeseRekursiv(0,n)
print(ausgabe(baum1))
print("Gesamtgewicht Baum 1: "+str(gg(baum1)))
print(ausgabe(baum2))
print("Gesamtgewicht Baum 2: "+str(gg(baum2)))
```

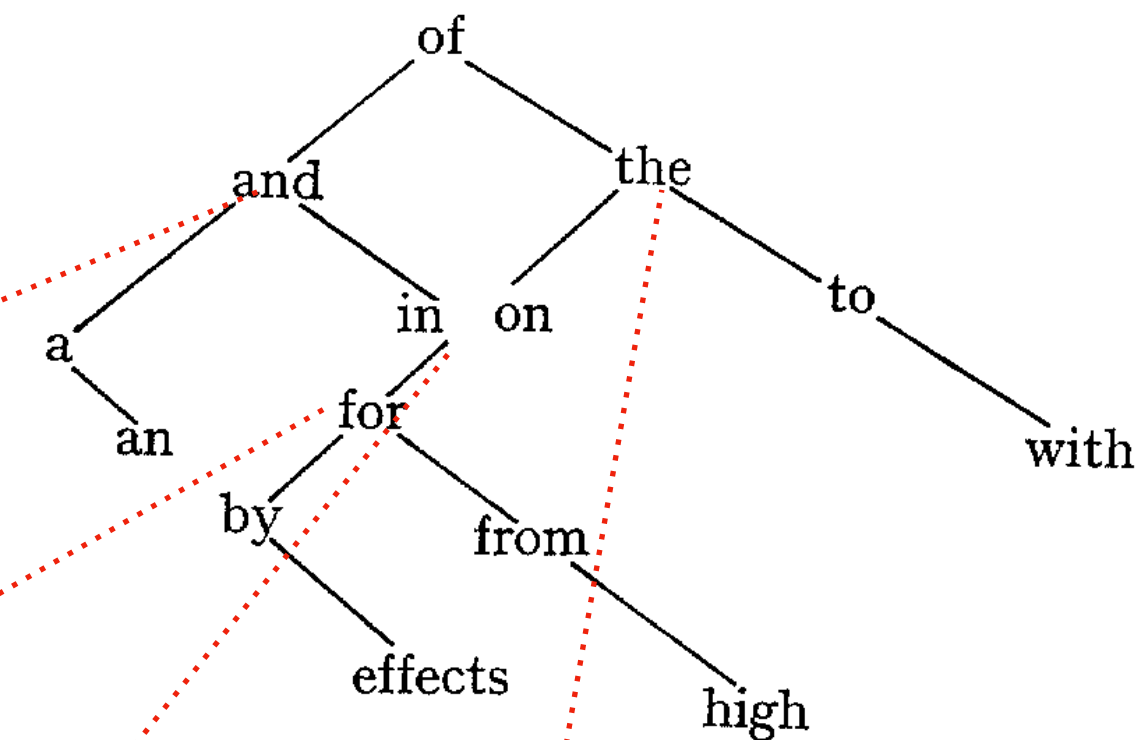

Ein Suchbaum im Paper von Donald Knuth



Optimum Binary Search Trees

15

showing words to be ignored in a certain KWIC indexing application [6, p.124].
The best possible tree in this case turns out to be



`((a(an))and(((by(effects))for(from(high)))in))of((on)the(to(with))))`

Gesamtgewicht dieses Baums:1169

Speichern von Studierendeninformation

- Bisher haben wir gelernt für jede einzelne Information eine Liste anzulegen

```
namen = ['Julia', 'Michael', 'Anna', 'Rose']
```

```
noten = ['1-', '2', '1', '4-5']
```

```
kurse = ['Mathe', 'Erdkunde', 'Deutsch', 'Kunst']
```

- Für jede Information einer Studierenden/eines Studierenden wird eine **separate Liste** angelegt
- Alle Listen müssen **diesselbe Länge** haben
- Information wird über mehrere Listen hinweg am **selben Index** gespeichert, jeder Index entspricht einer anderen Person

Zugriff/Modifikation von Studierendeninformation

```
def hole_note(studentIn, namenliste, notenliste, kursliste)
    i = namenliste.index(studentIn)
    note = notenliste[i]
    kurs = kursliste[i]
    return (kurs, note)
```

- recht **unübersichtlich** bis **choatisch** wenn man viel derartige Information zu verwalten hat
- **viele Listen** müssen verwaltet werden und viele Argumente müssen übergeben werden
- **stets** geschieht der Zugriff mittels **Integer-Indizes**
- mehrere Listen müssen angepasst werden (um bspw. dieselbe Längen zu garantieren), daher fehleranfällig

Die bessere Lösung: Wörterbücher/dictionaries

- **Indizierung allgemeiner**, nicht unbedingt mittels Integers
- angenehm eine einzelne Datenstruktur zu verwenden, anstatt separate Listen kompliziert zu verwalten

Eine Liste

0	Element 0
1	Element 1
2	Element 2
3	Element 3
...	...

Index

Element

Ein Wörterbuch

Schlüssel 1	Wert 1
Schlüssel 2	Wert 2
Schlüssel 3	Wert 3
Schlüssel 4	Wert 4
...	...

selbstdefinierter
Schlüssel

Element

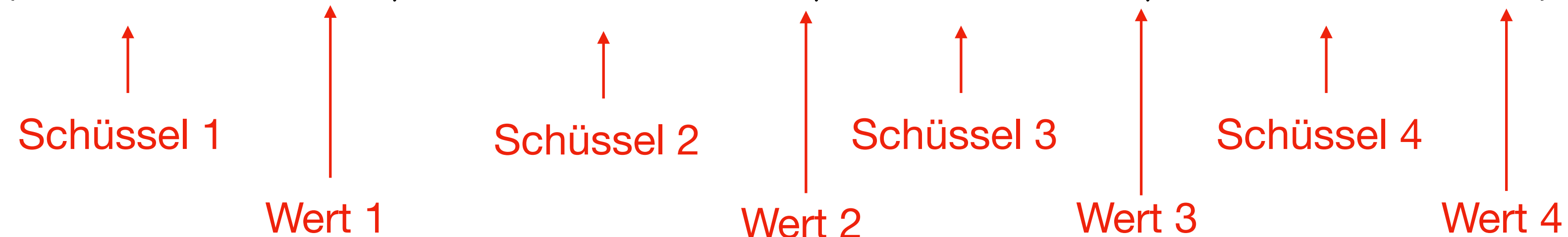
Ein Wörterbuch in Python

- speichert Paare von Daten, nämlich
 - einen Schlüssel
 - und seinen Wert

'Julia'	'1-'
'Michael'	'2'
'Anna'	'1'
'Rose'	'4-5'

```
mein_buch = {}
```

```
noten={'Julia':'1-','Michael':'2','Anna':'1','Rose':'4-5'}
```



Wörterbuchzugriff

- ähnlich zu Indizierung bei Listen
- entspricht Funktionen in der Mathematik (Urbildbereich=Schlüssel, Wertebereich=Werte)
- Zugriff mittels Schlüssel
- Wörterbuch gibt den, mit diesem Schlüssel assoziierten Wert zurück
- Fehlermeldung, wenn Schlüssel nicht gefunden werden kann

'Julia'	'1-'
'Michael'	'2'
'Anna'	'1'
'Rose'	'4-5'

```
noten={'Julia':'1-', 'Michael':'2', 'Anna':'1', 'Rose':'4-5'}
```

```
noten['Anna']  wertet sich zu '1' aus
```

```
noten['Horst']  liefert einen KeyError
```

Wörterbuch- operationen

'Julia'	'1-'
'Michael'	'2'
'Anna'	'1'
'Rose'	'4-5'
'Horst'	'3-'

```
noten={'Julia':'1-','Michael':'2','Anna':'1','Rose':'4-5'}
```

- **füge** einen Eintrag **hinzu**

```
noten['Horst']='3-'
```

- **teste** ob es einen Schlüssel im Wörterbuch gibt

'Elvira' in noten  wertet sich zu False aus

'Horst' in noten  wertet sich zu True aus

- **lösche** einen Eintrag

```
del(noten['Rose'])
```

Wörterbuch- operationen

'Julia'	'1-'
'Michael'	'2'
'Anna'	'1'
'Rose'	'4-5'
'Horst'	'3-'

```
noten={'Julia':'1-','Michael':'2','Anna':'1','Rose':'4-5'}
```

Ordnung nicht garantiert

- hole einen **Iterator**, der sich wie ein **Tupel aller Schlüssel** verhält

```
noten.keys() ————— liefert ['Rose', 'Michael', 'Anna', 'Julia']
```

- hole einen **Iterator**, der sich wie ein **Tupel aller Werte** verhält

```
noten.values() ————— liefert ['4-5', '2', '1', '1-']
```

Ordnung nicht garantiert

Wörterbuchschlüssel und -werte

- **Werte**
 - können beliebigen Typ haben (**veränderbar** oder **unveränderbar**)
 - können **mehrfach vorkommen**
 - können sogar Listen und weitere Wörterbücher sein!
- **Schlüssel**
 - müssen **eindeutig** sein
 - müssen von **unveränderbaren** Typ sein (also `int, float, string, tuple, bool`)
 - es reicht, wenn sie **hashable** sind, aber der Einfachheit halber denken wir unveränderbar entspricht hashable
- Weder auf Schlüsseln noch Werten muss eine Ordnung bestehen

```
wb = {4: (1, 0), (1, 3): 'zwölf', 'konst': [3.14, 2.7, 8.44]}
```

list vs dict

- **geordnete** Sequenz von Elementen
- Zugriff auf Elemente mittels Index
- Indizes haben eine **Ordnung**
- Index ist ein **Integer**
- **ordnet** Schlüsseln Wert **zu**
- Zugriff auf Elemente mittels Schlüssel
- **keine** garantierte **Ordnung**
- Schlüssel kann jeder **unveränderbare** Typ sein

Ein Wortzähler im Songtext

- Finde diejenigen Wörter, die in Datei `lied.txt` **mindestens x mal vorkommen**
 - Lies Zeilen der Datei `lied.txt` ein und bereinige diese
 - Benutzer gibt x selbst als **Parameter** an
 - Wir kreieren ein **Häufigkeitswörterbuch** (`str` nach `int`)
 - Gib eine **Liste von Paaren** der Form (`list, int`) zurück, wobei die erste Komponente eine Liste von Wörtern und die zweite Komponente die Häufigkeit aller in dieser Liste vorkommenden Wörter beschreibt
 - **Informelle Idee:** Ausgehend vom Lieder-Wörterbuch, **erhalte die größte Häufigkeit**, entferne alle solch häufigen Wörter, entferne sie aus Wörterbuch und füge Sie dem Ergebnis hinzu. Wiederhole dies. Funktioniert, da man Wörter aus Lieder-Wörterbuch entfernt.

Häufigkeitswörterbuch

```
def liedtext_haeufigkeit(liedtext):  
    '''  
    Erwartet einen Liste von Zeichenketten,  
    die am Besten um Sonderzeichen und Gross- und  
    Kleinschreibung bereinigt ist.  
    '''  
    meinwb = {}  
    for wort in liedtext:  
        if wort in meinwb:  
            meinwb[word] += 1  
        else:  
            meinwb[word] = 1  
    return meinwb
```

Häufigkeitswörterbuch

```
def haeufigste_woerter(haeufige) :  
    werte = haeufige.values()  
    maximum = max(werte)  
    woerter = []  
    for s in haeufige:  
        if haeufige[s] == maximum:  
            woerter.append(s)  
    return (woerter,maximum)
```


Häufigkeitswörterbuch

```
def haeufige_woerter(woerterbuch, mindestens):  
    ergebnis = []  
    fertig = False  
    while not fertig and woerterbuch:  
        temp = haeufigste_woerter(woerterbuch)  
        if temp[1] >= mindestens:  
            ergebnis.append(temp)  
            for w in temp[0]:  
                del (woerterbuch[w])  
        else:  
            fertig = True  
    return ergebnis
```

Hauptprogramm (1)

```

lied=""
datei = open("lied.txt")
for line in datei:
    lied+=" " + line.rstrip()
datei.close()

lied=lied.lower()

lied=lied.replace(",","")
lied=lied.replace(")","")
lied=lied.replace("(","")

l=lied.split()

x=input("Geben Sie eine Häufigkeit >= 1 ein : ")

woerterbuch=liedtext_haeufigkeit(l)

liste=haeufige_woerter(woerterbuch,int(x))
```

Öffne Datei `lied.txt`

entferne Zeilenumbruch `\n` am Ende der eingelesenen Zeile

Schließe Datei

Alle Grossbuchstaben werden in Kleinbuchstaben umgewandelt

Alle Kommata, "(" und ")" werden durch den leeren String ersetzt, also entfernt

Erzeuge Liste aller Wörter im bereinigten Liedtext, die nach Leerzeilen getrennt werden

Erzeuge Wörterbuch aller im Lied vorkommenen Wörter und ordne ihnen ihre Häufigkeit zu

Hauptprogramm (2)

Ausgabe der berechneten Liste (von `(list, integer)`-Paaren)

```
for paar in liste:
    if paar[1]== 1:
        w=paar[0].pop()
        print("Das Wort '"+w+"' kommt einmal vor")
    else:
        satz=str(paar[1])+" mal kommen vor: "+ "\n"
        for w in paar[0]:
            satz = satz + " "*20 +w+"\n"
        print(satz)
```

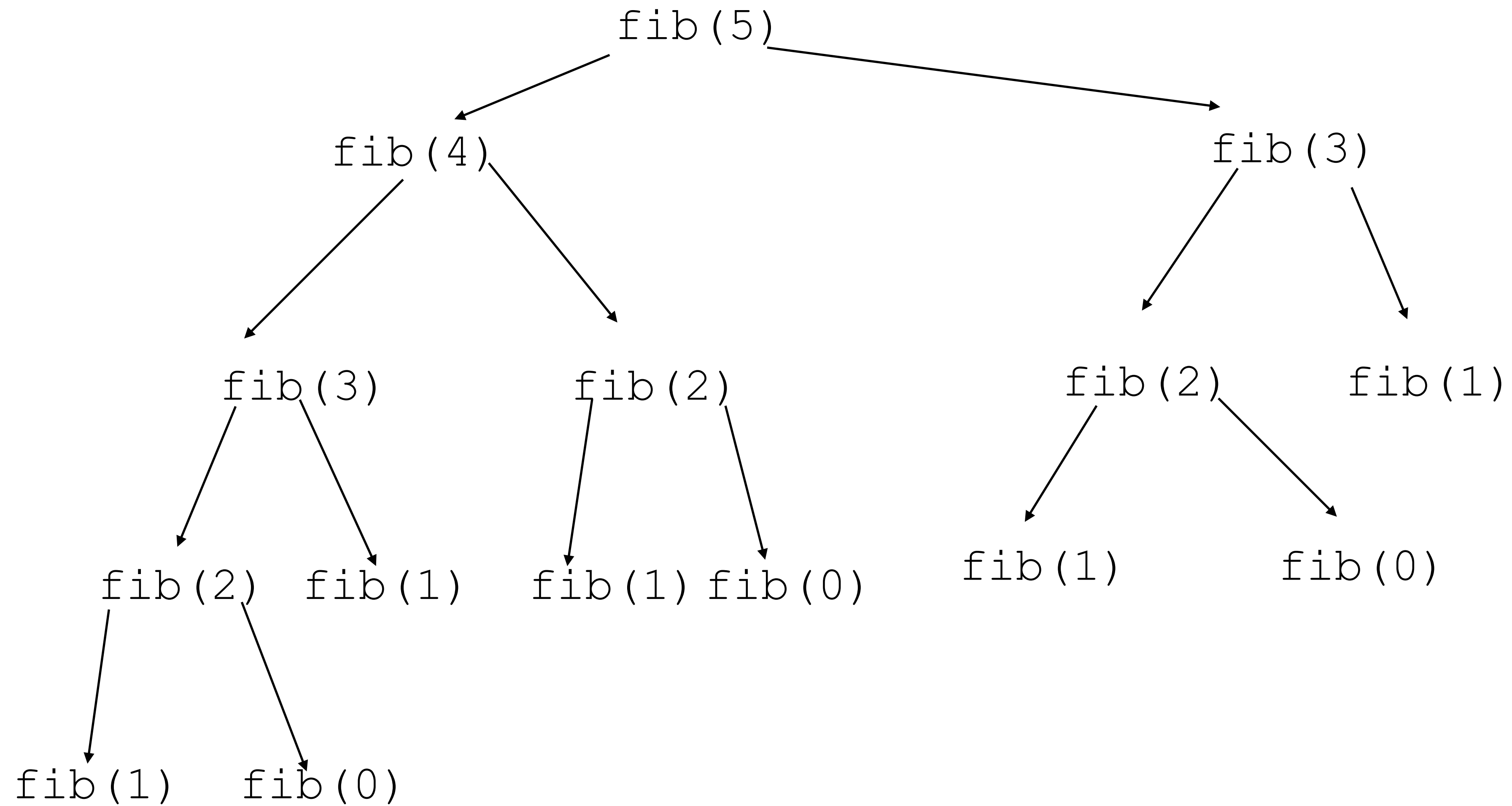
Kopiere Leerzeichen 20 Mal (für Darstellungszwecke)

Fibonacci-Zahlen rekursiv

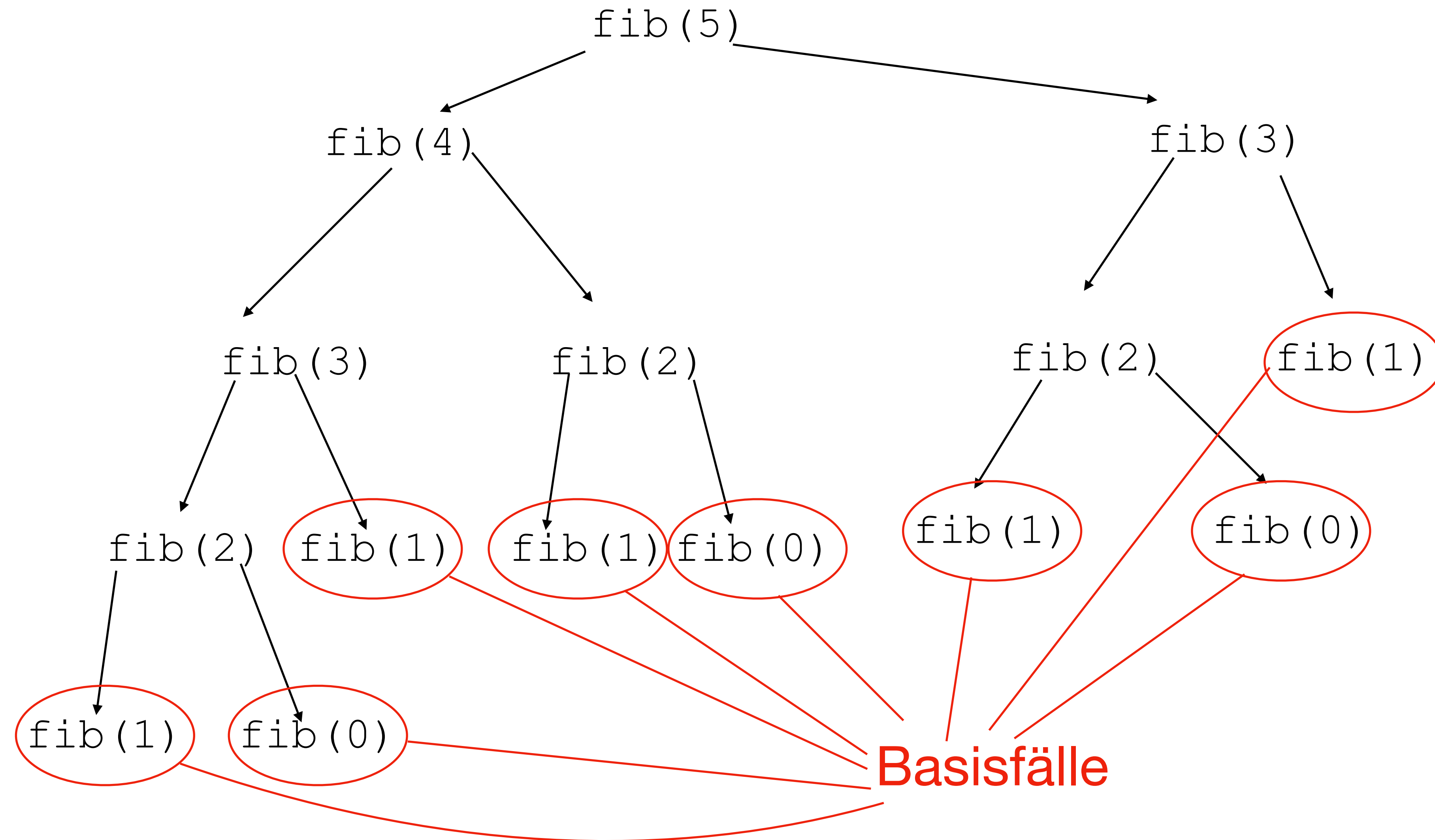
```
def fib(n):  
    if n==0 or n==1:  
        return 1  
    else:  
        return fib(n-1) + fib(n-2)
```

- zwei Basisfälle (n=0 und n=1)
- ruft sich zweimal rekursiv auf
- dieser Code ist ineffizient *Warum?*

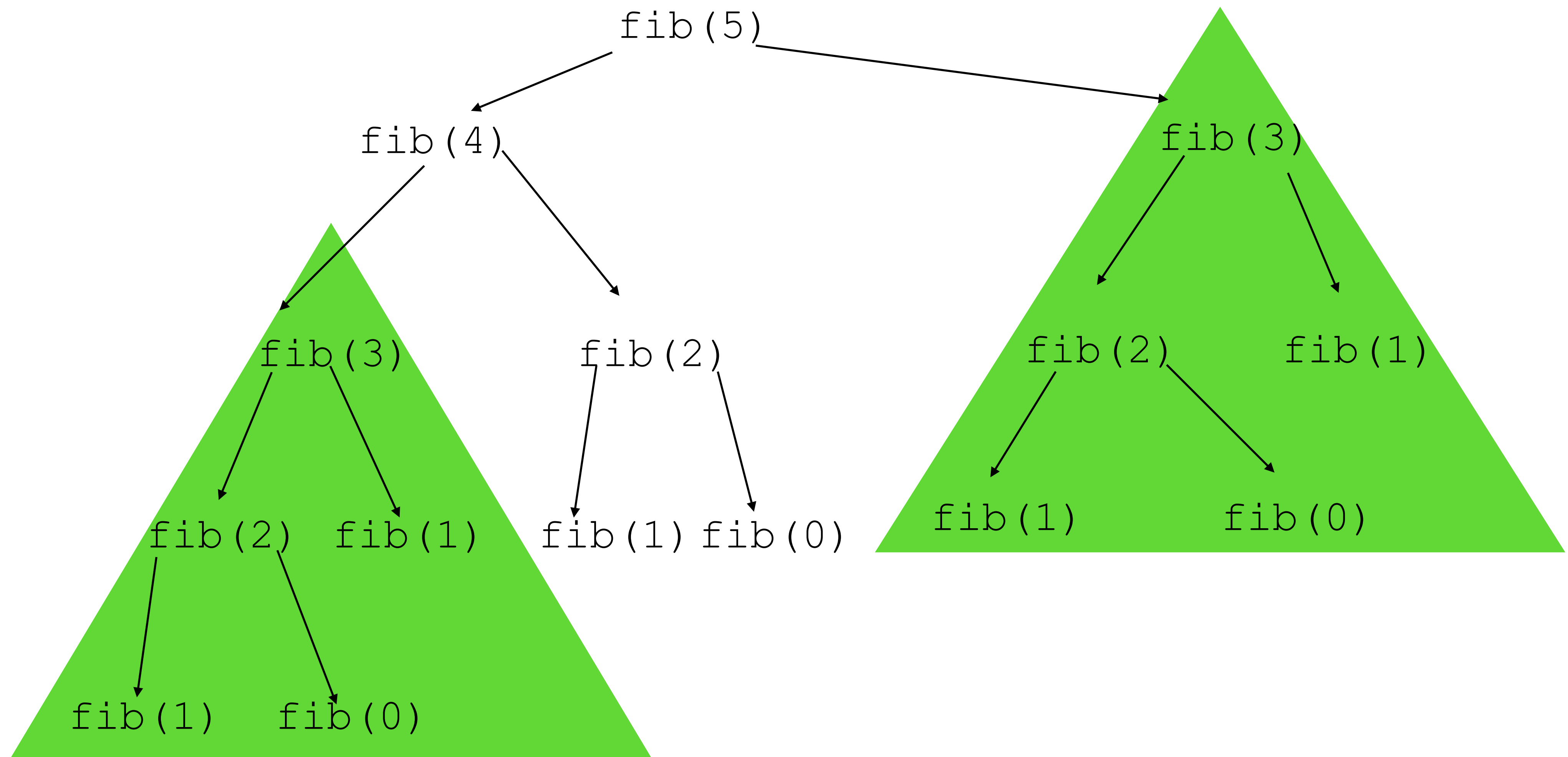
Ineffizienter Code



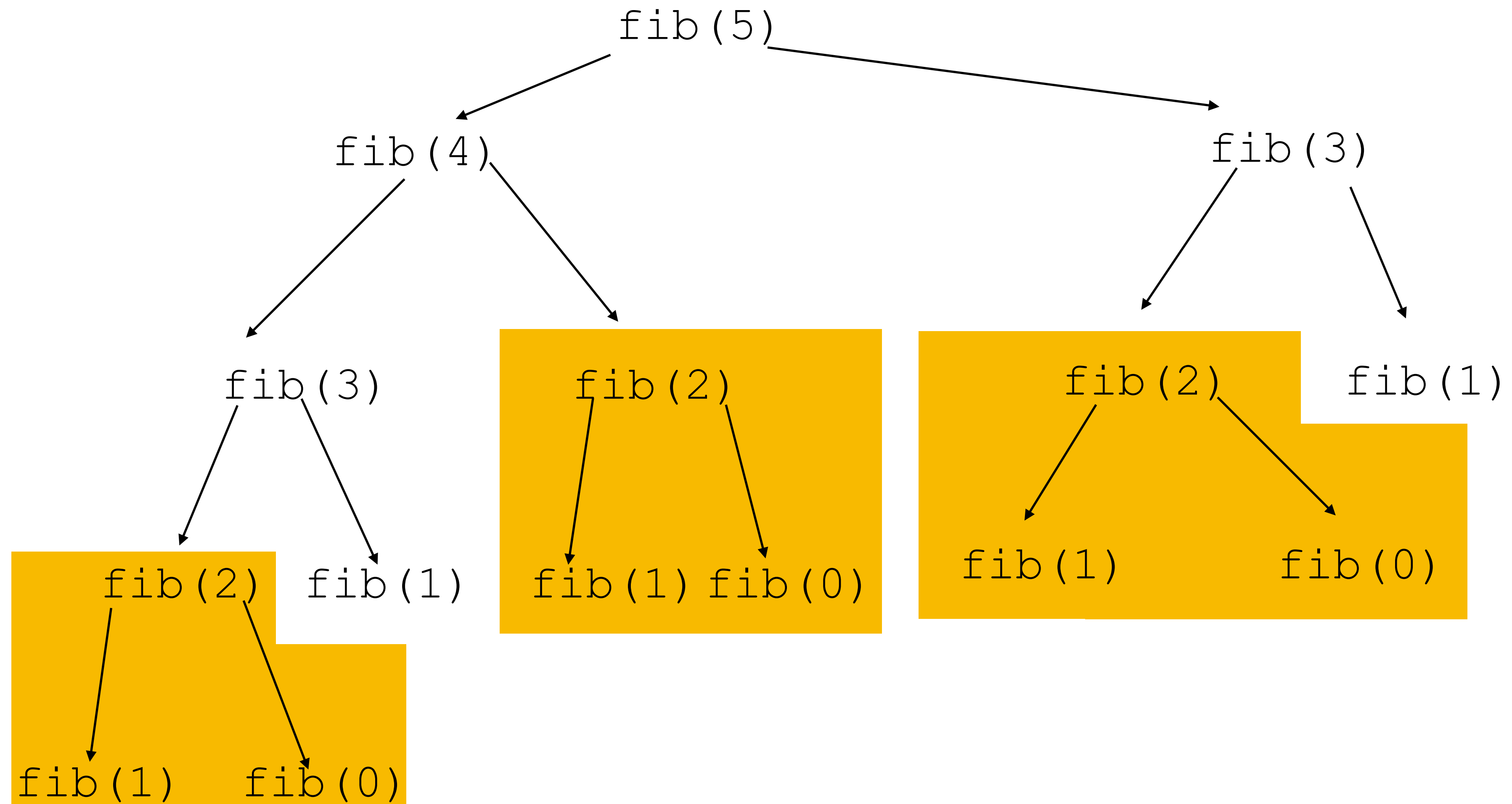
Ineffizienter Code



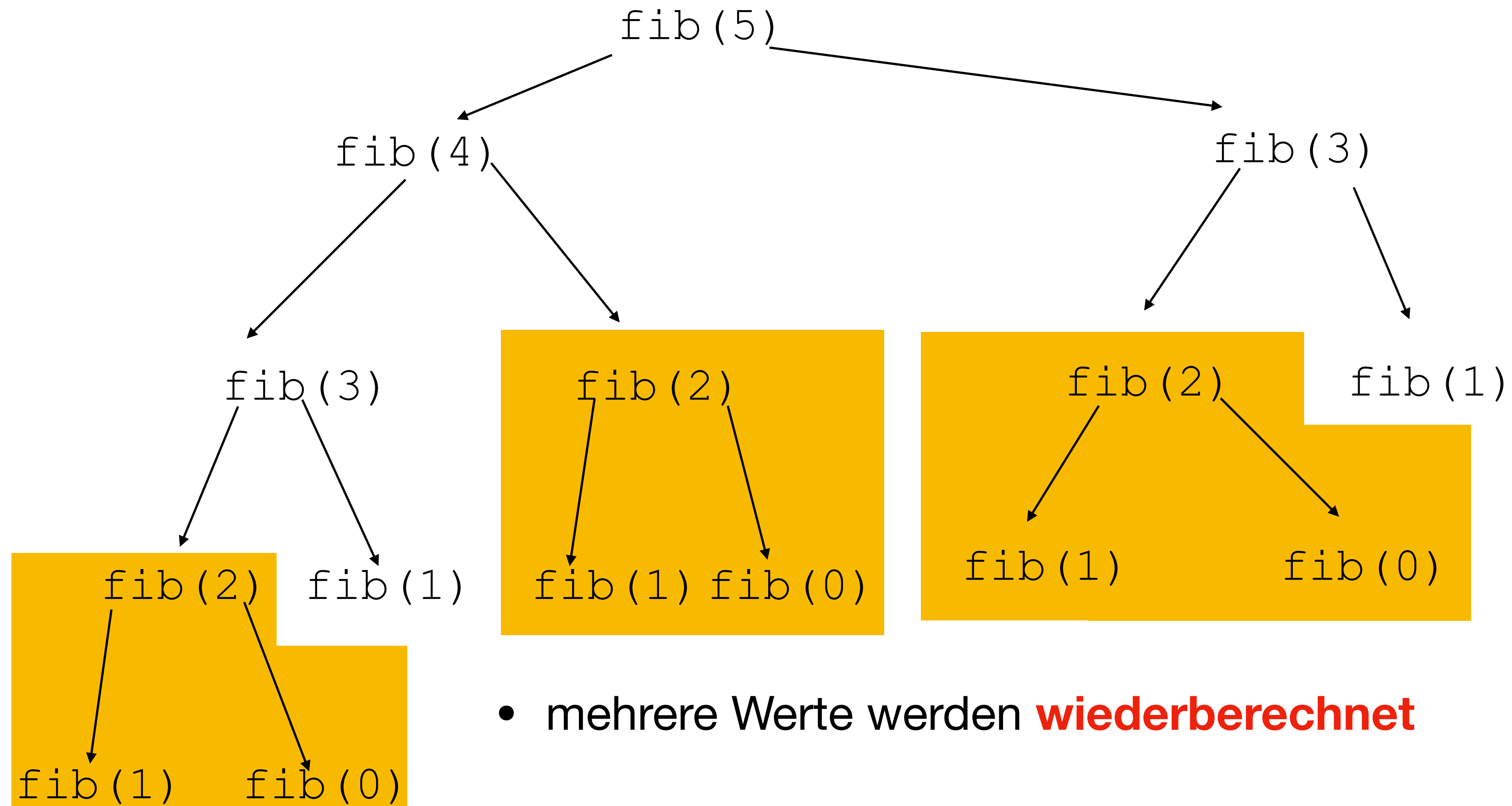
Ineffizienter Code



Ineffizienter Code



Ineffizienter Code



- mehrere Werte werden **wiederberechnet**
- besser wäre es zu **speichern** welche Werte bereits berechnet wurden

Fibonacci mit einem Wörterbuch

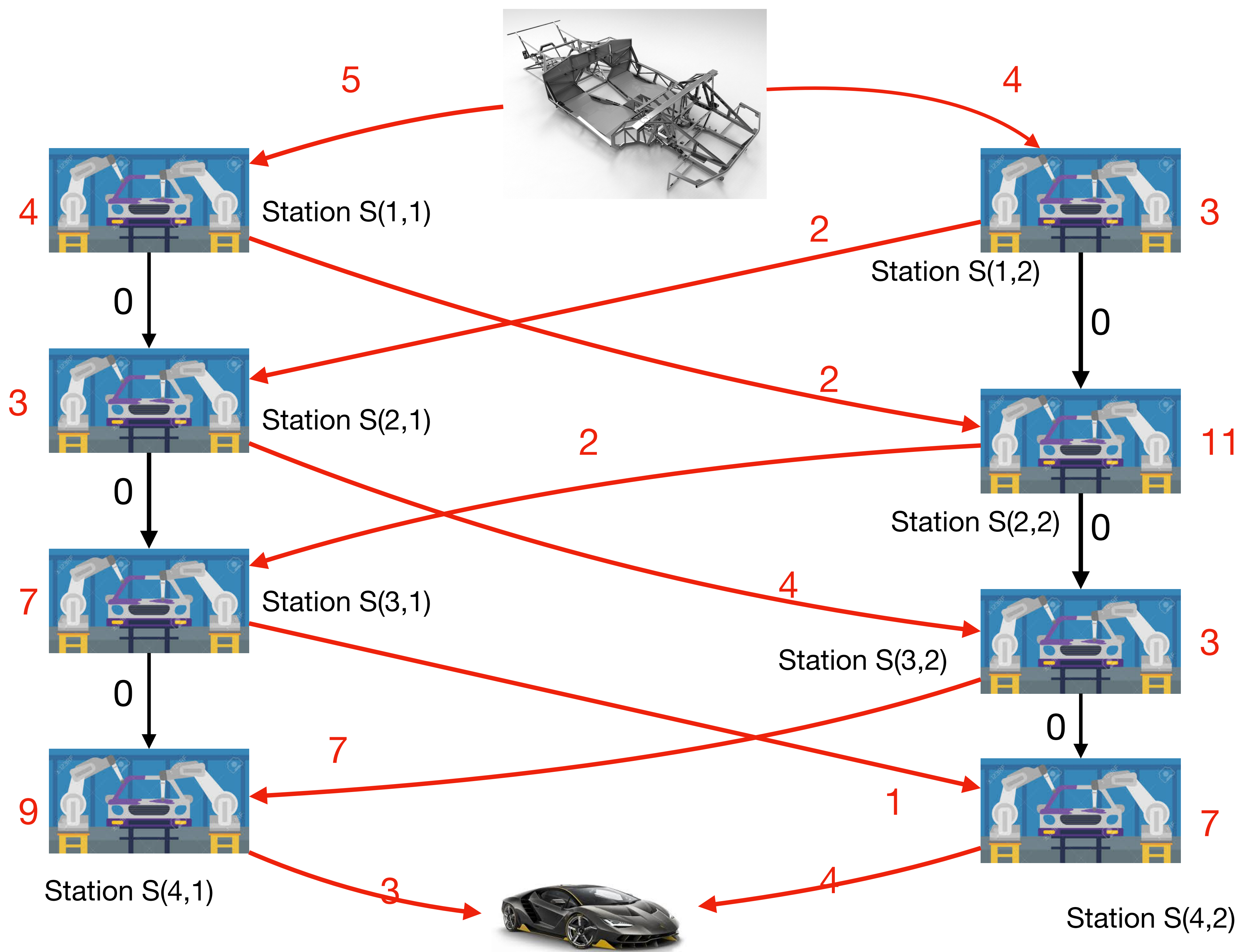
```
def fib_effizient(n,wb):  
    if n in wb:  
        return wb[n]  
    else:  
        antwort = fib_effizient(n-1,wb) + fib_effizient(n-2,wb)  
        wb[n] = antwort  
        return antwort
```

Achtung: Da Wörterbücher veränderbar sind, hat diese Funktion Nebeneffekte auf das Wörterbuch *initial* im Hauptprogramm (aber genau das wollen wir!)

```
initial={0:1,1:1}  
n=input("Geben sie n ein: ")  
print(fib_effizient(int(n),initial))
```

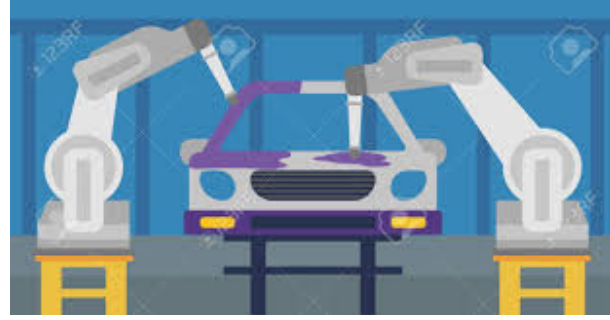
Initialisiere Wörterbuch

- **zuerst** im Wörterbuch **nachschlagen**
- falls Schlüssel nicht vorhanden, **trage** entsprechenden Wert **ins Wörterbuch ein**

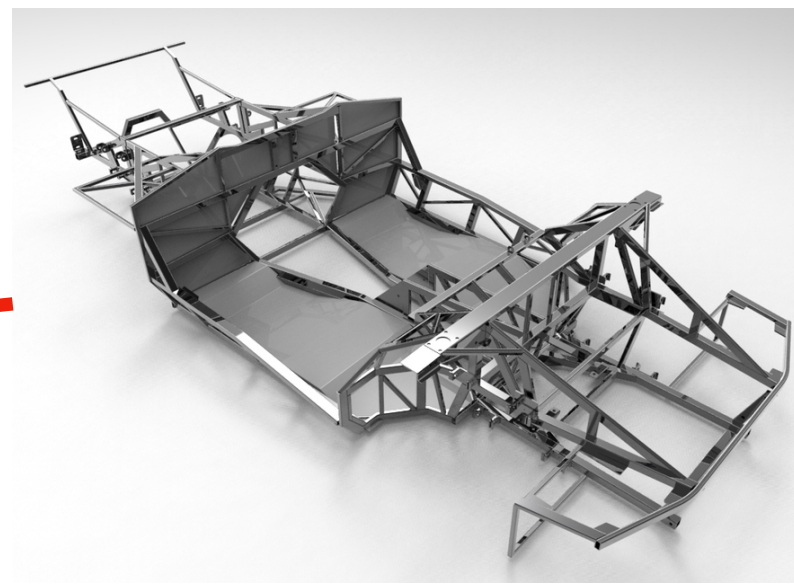




4



5



4



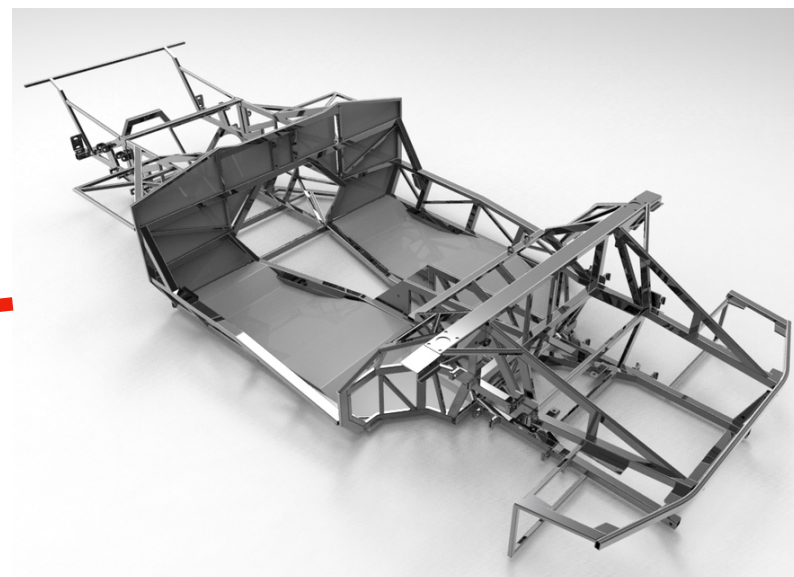
3

4

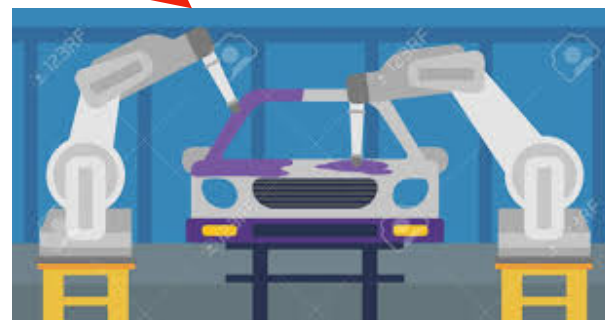


9

5

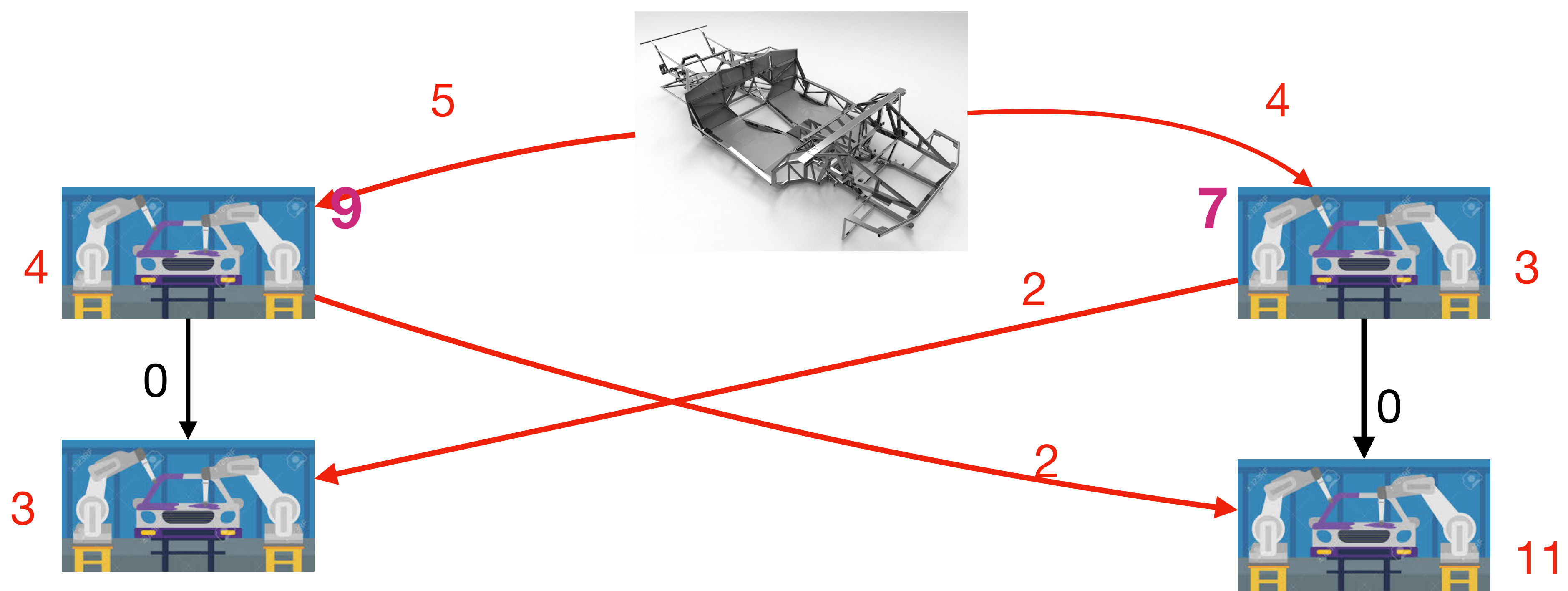


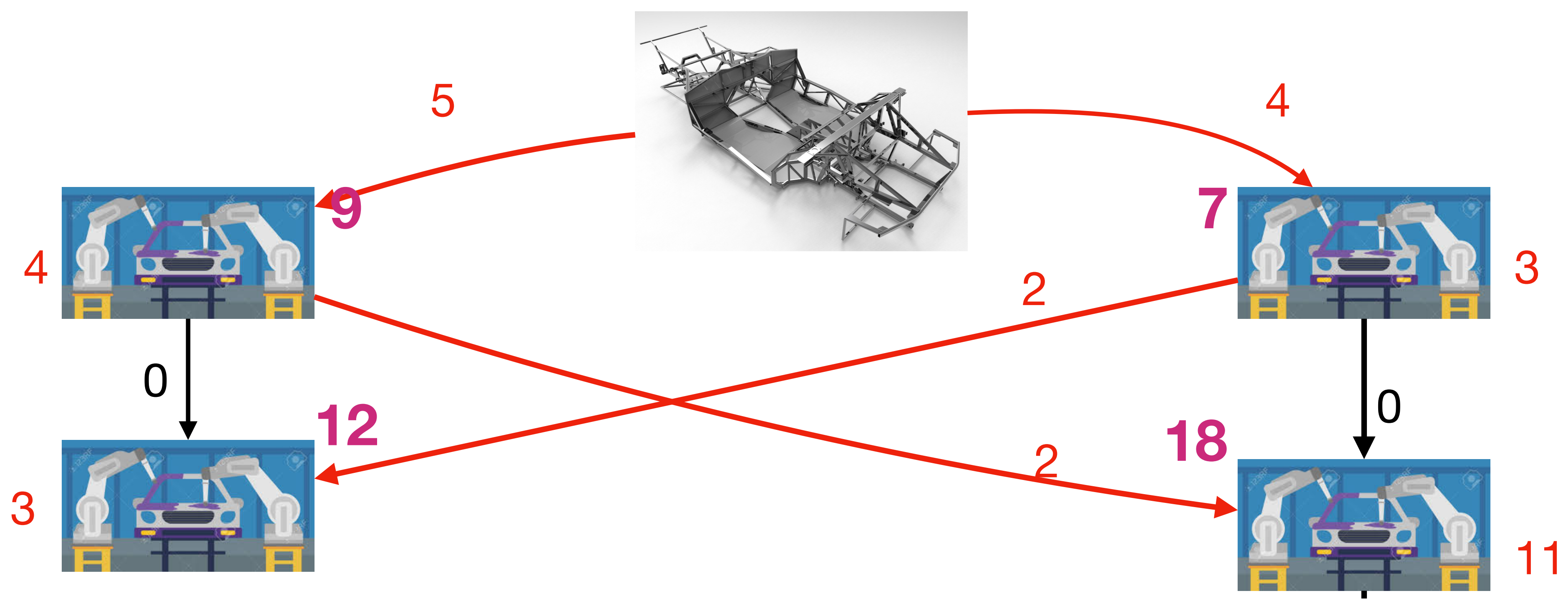
7

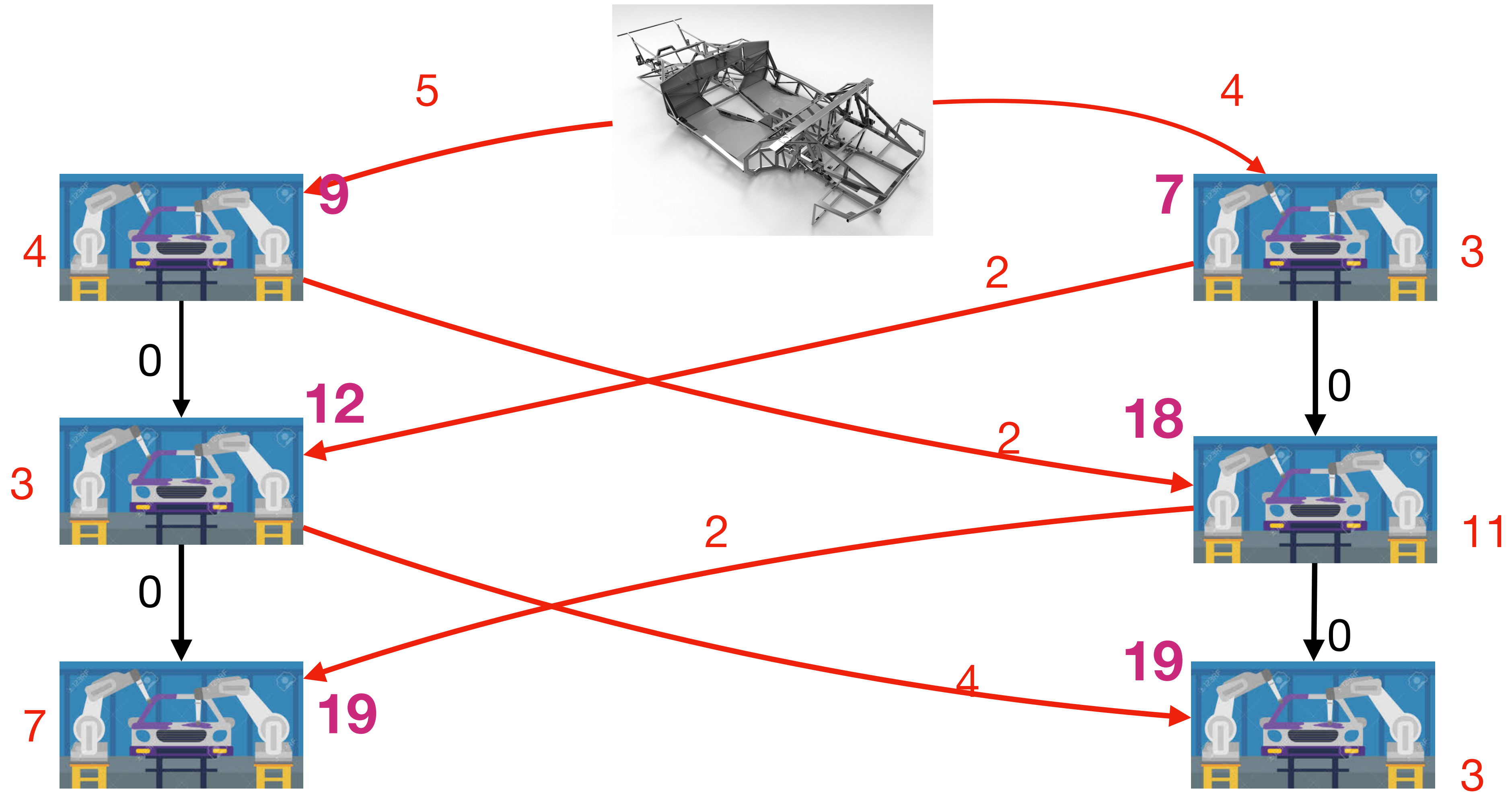


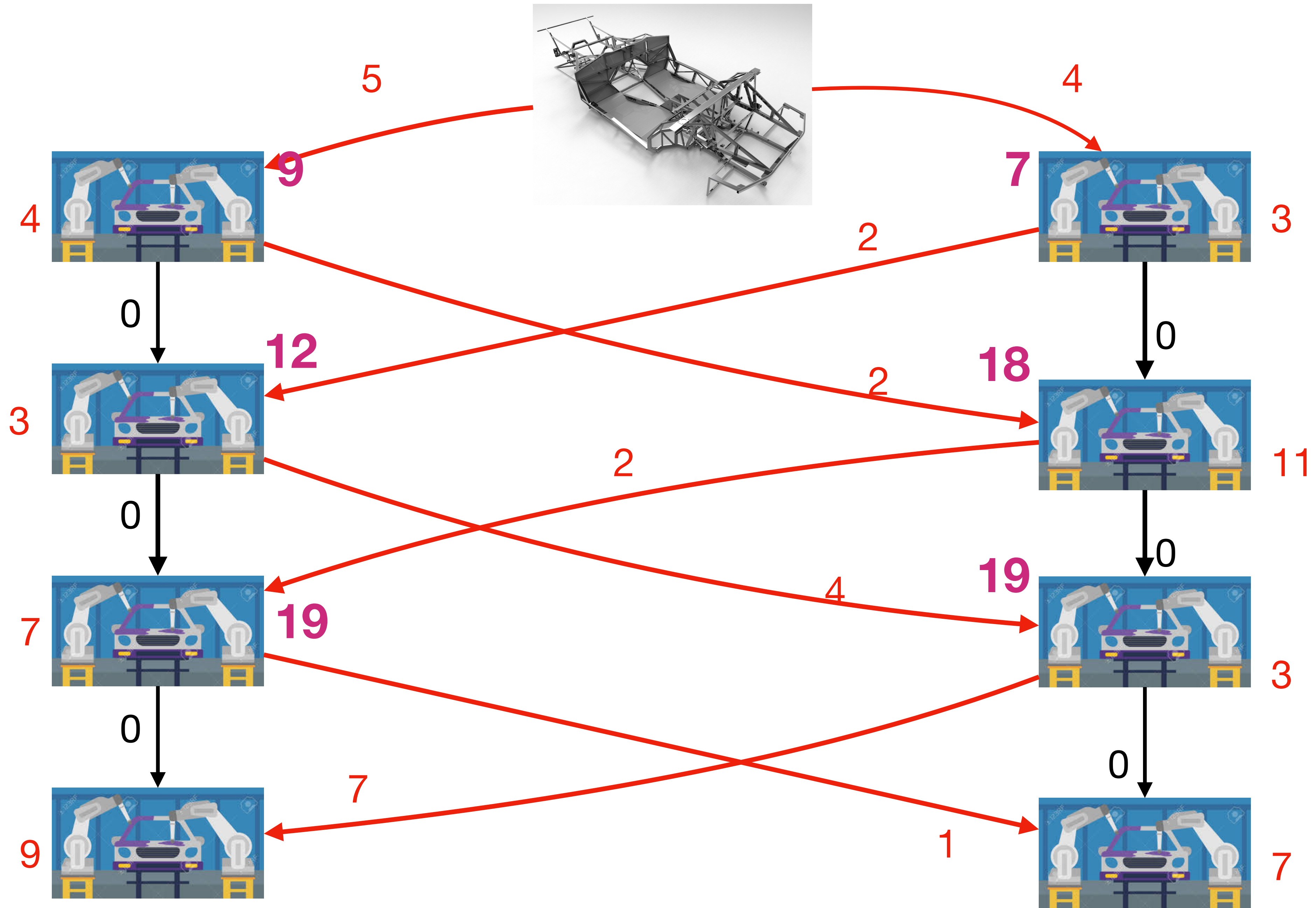
3

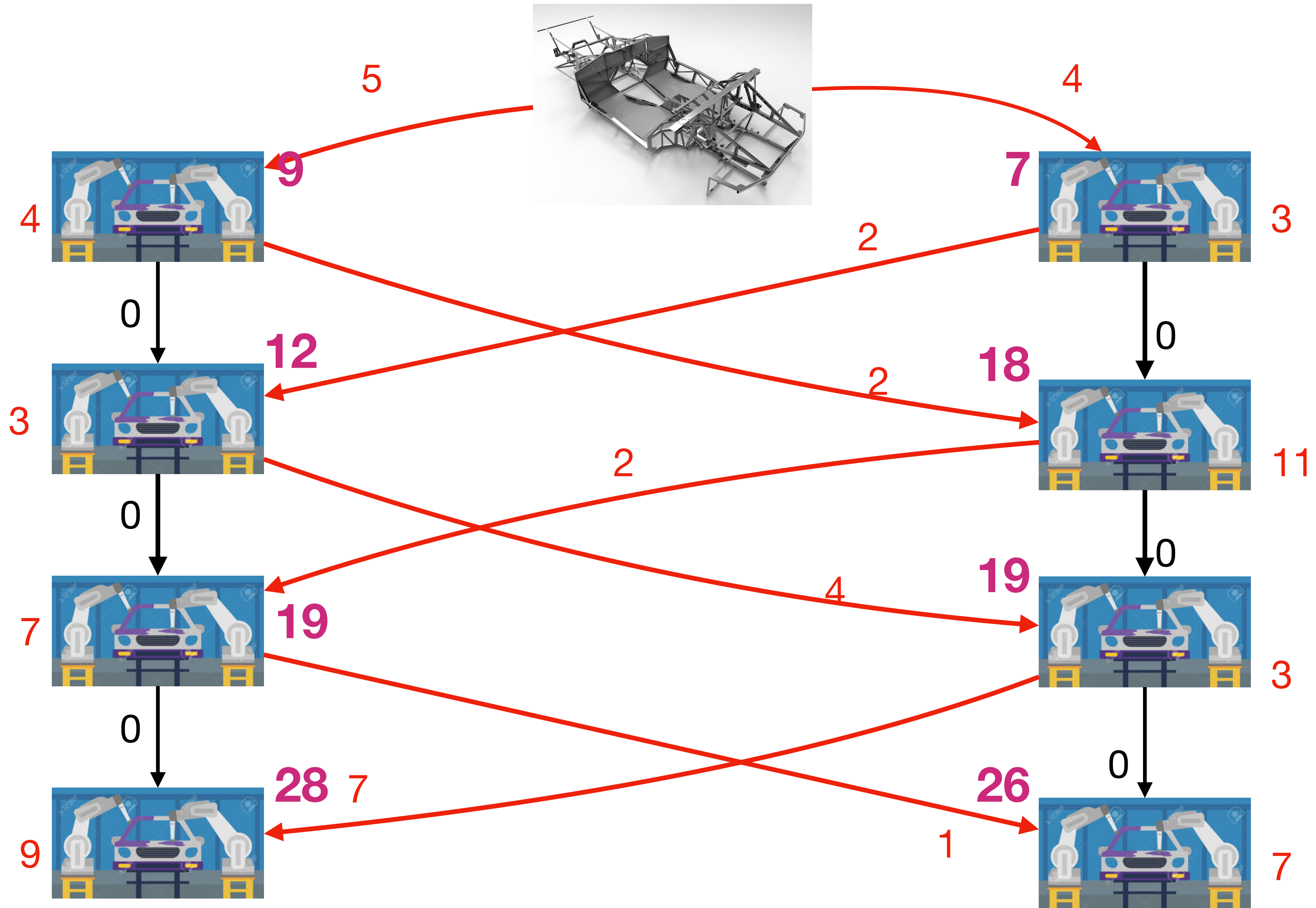
4

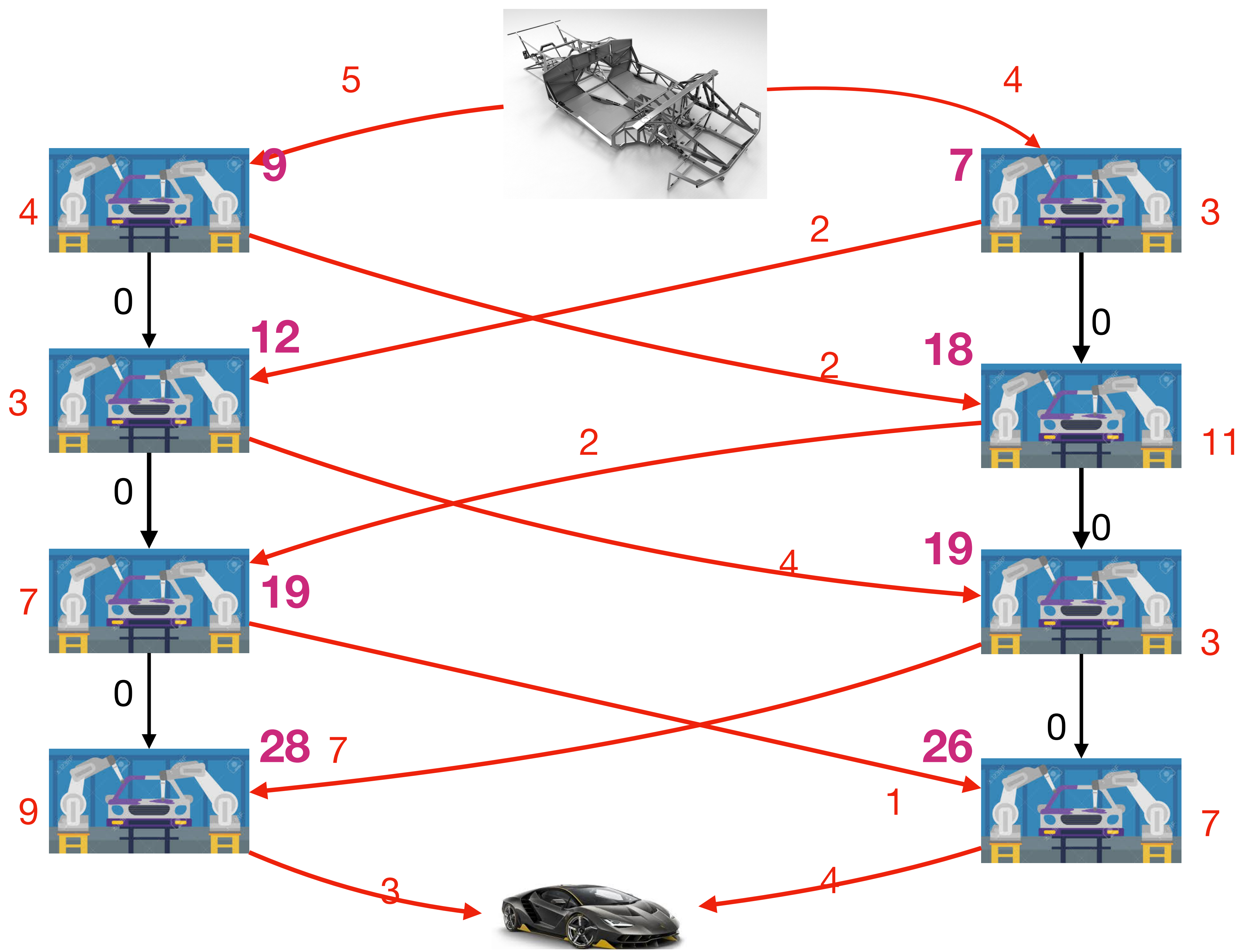


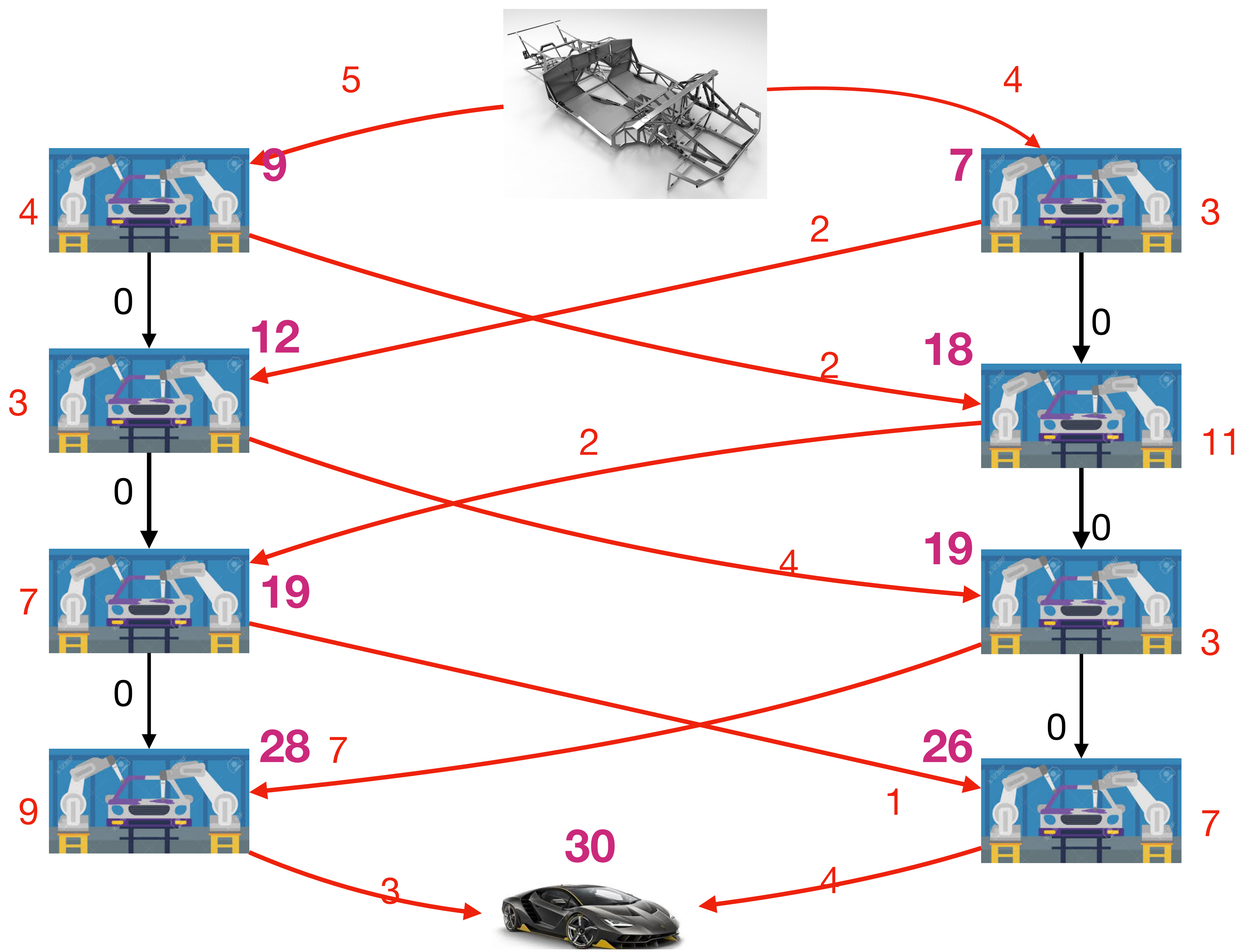


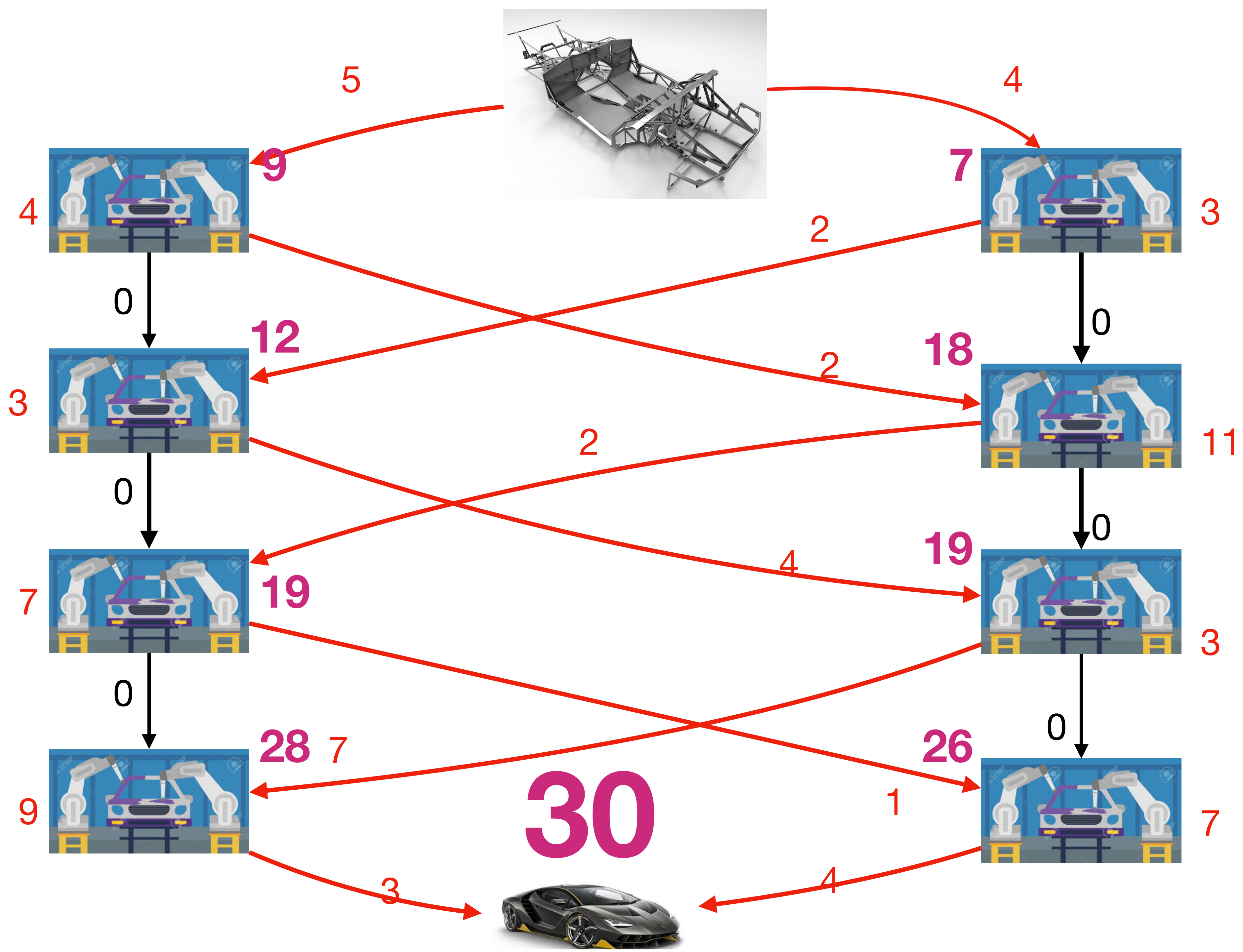






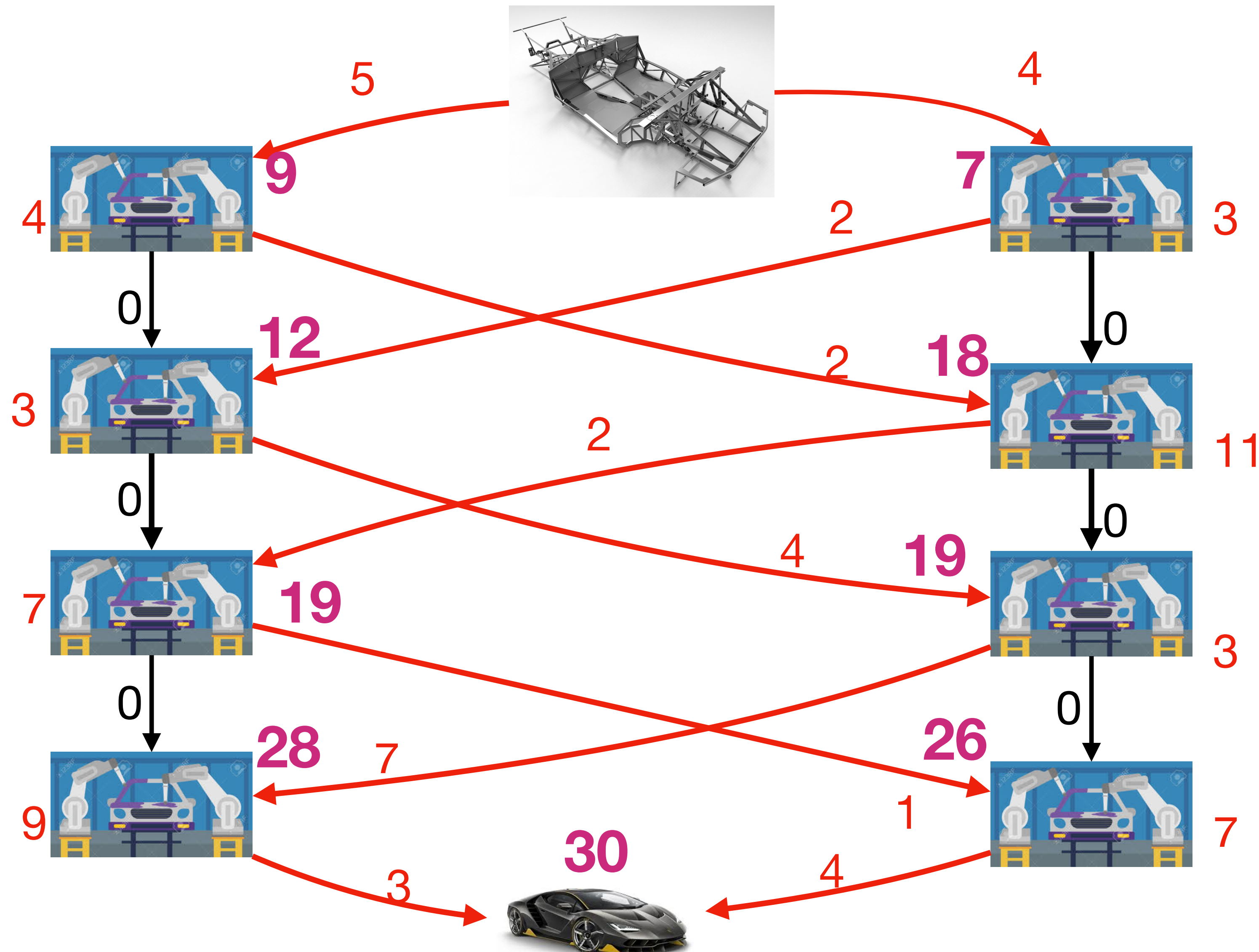






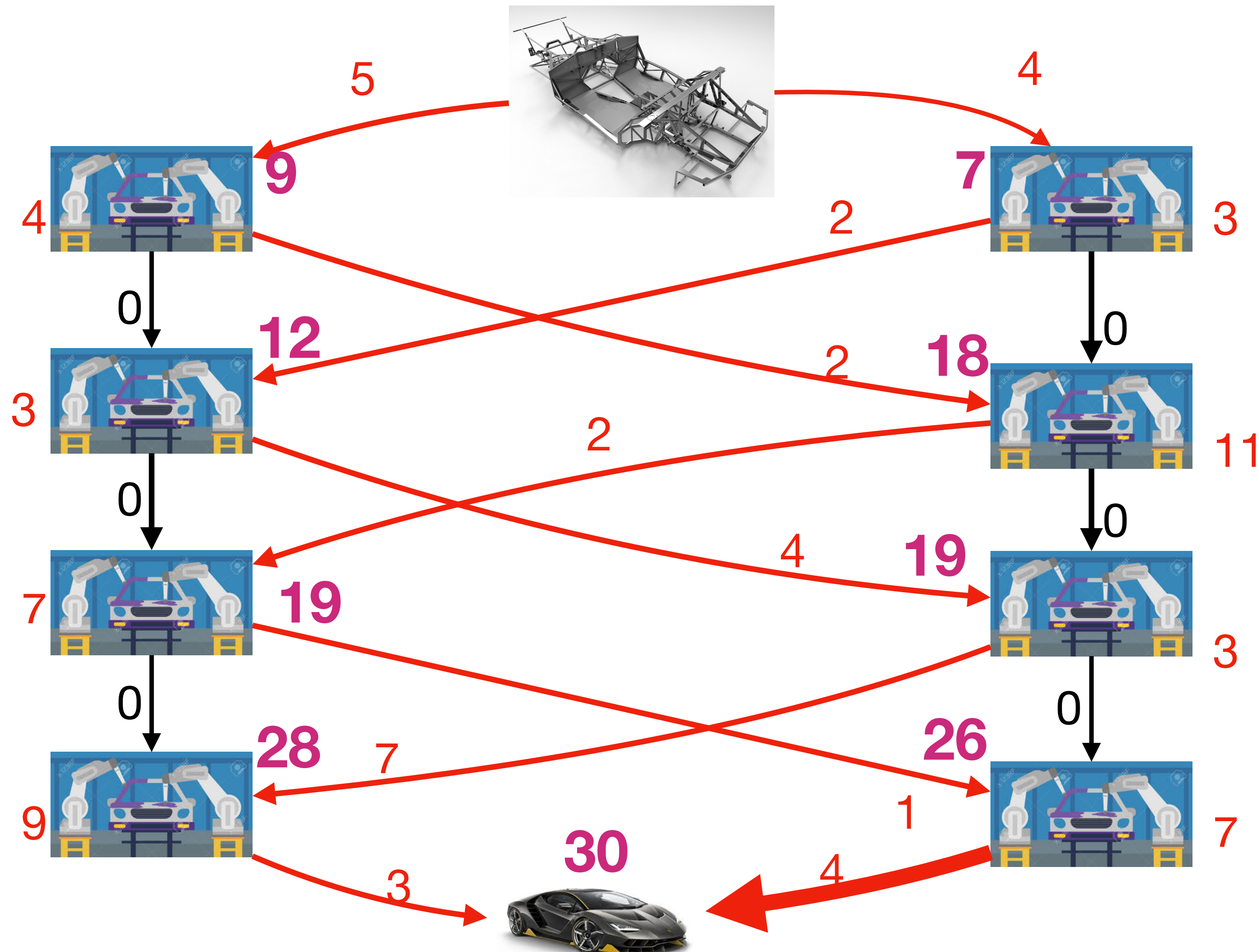
Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



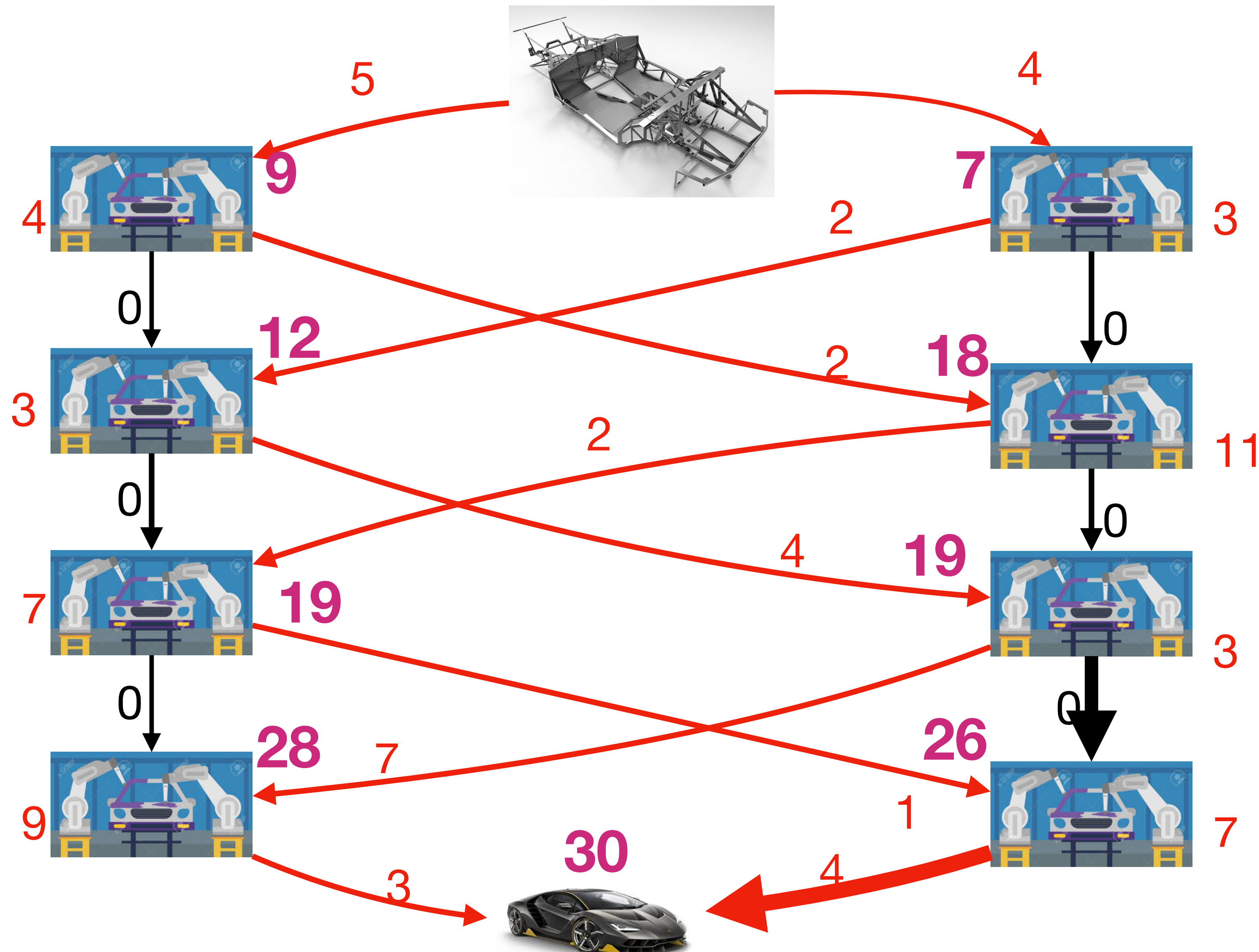
Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



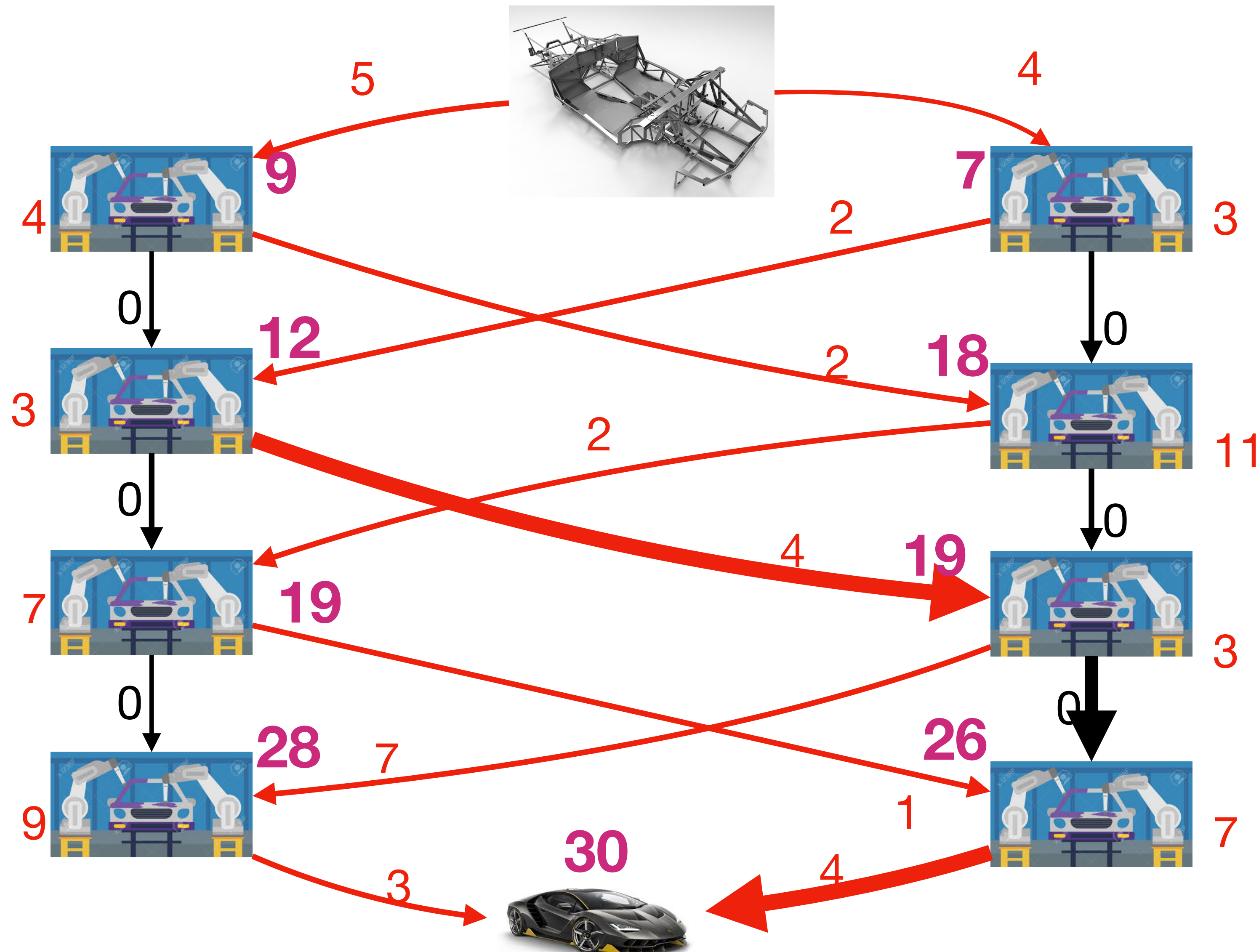
Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



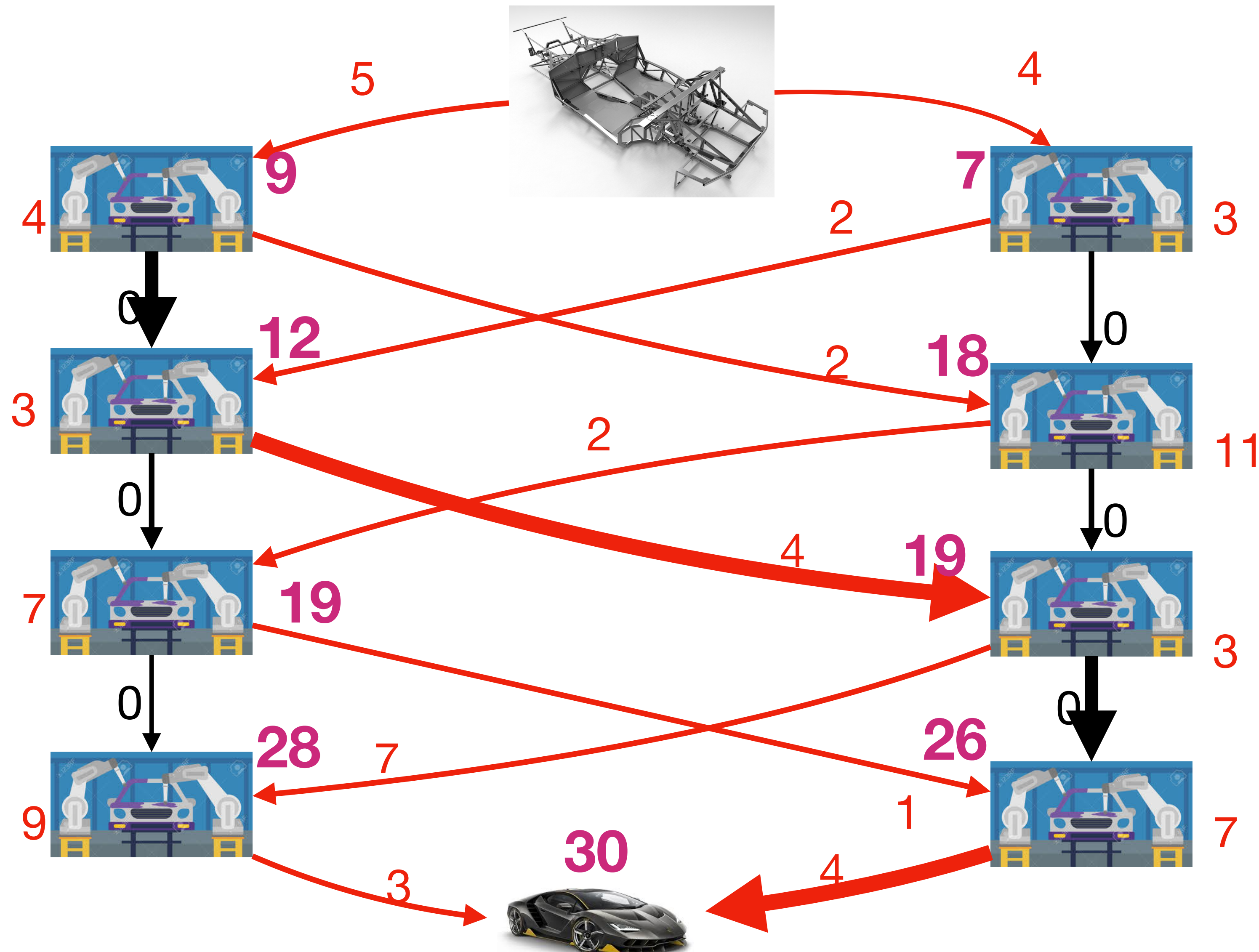
Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



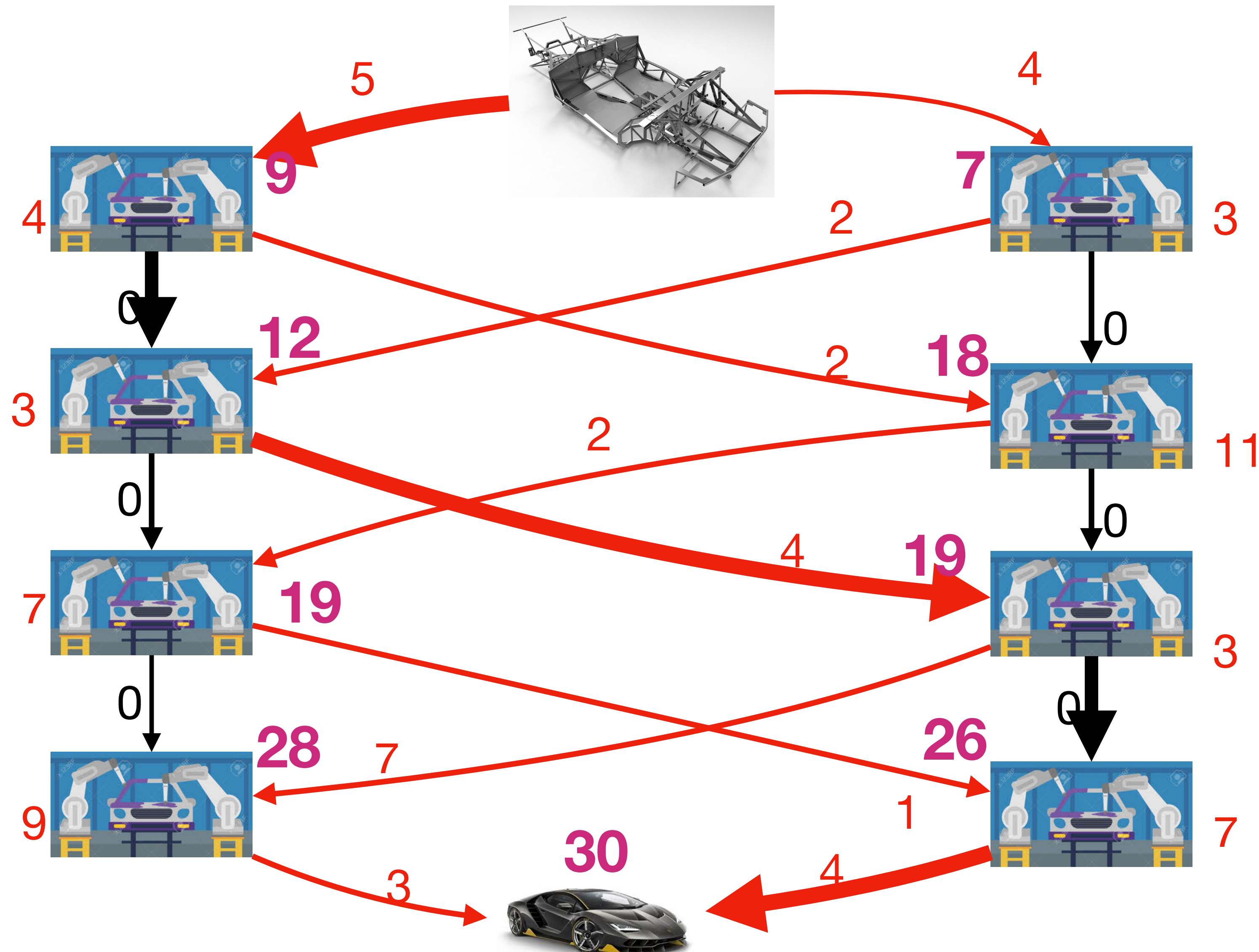
Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



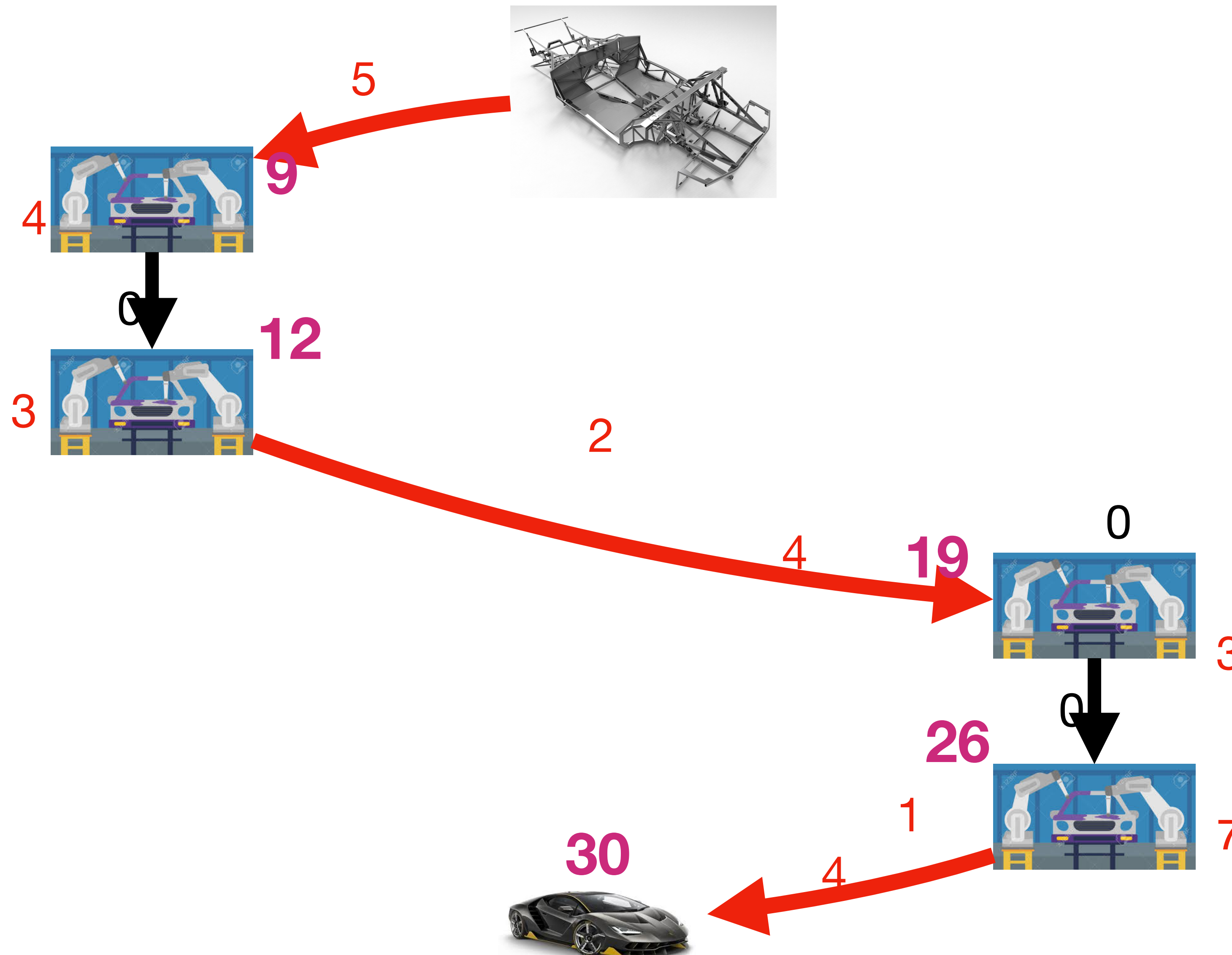
Struktur einer optimalen Lösung

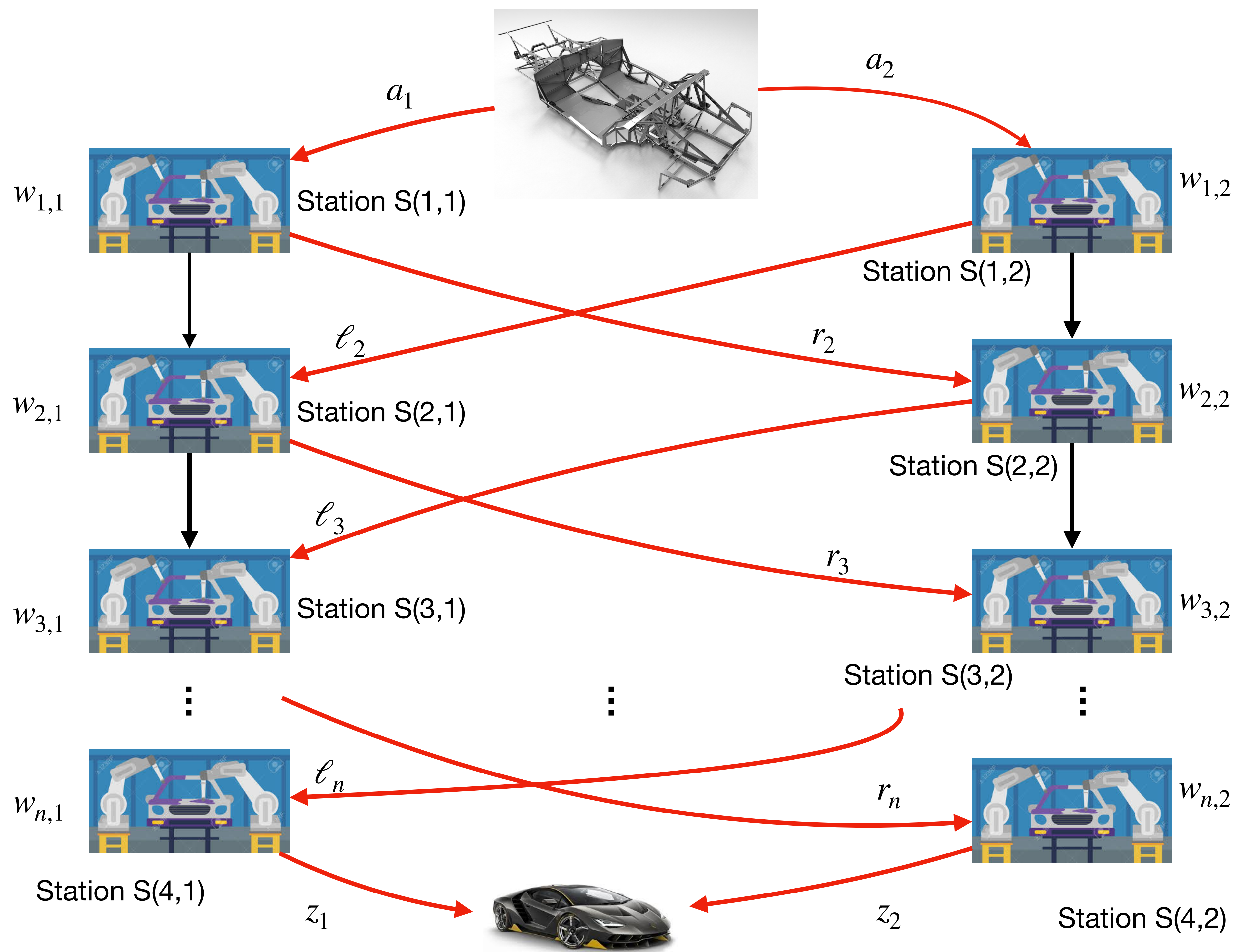
- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**





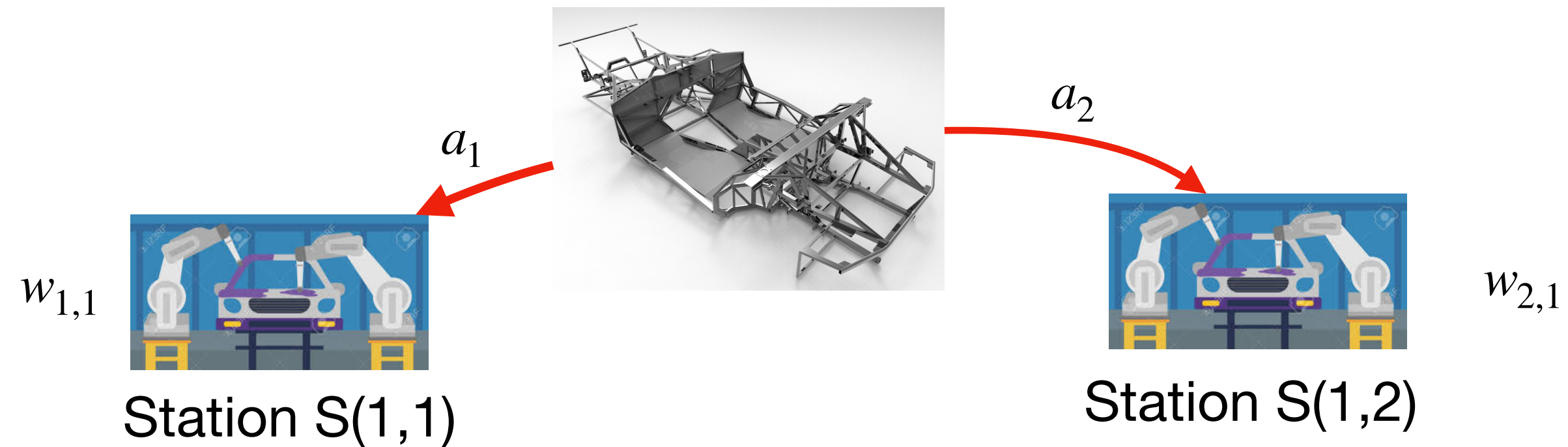
Lösung rekursiv beschreiben

- Bezeichne $m_{i,j}$ die Minimalzeit, bis Chassis an Stelle $S(i,j)$ zu fertig ist.

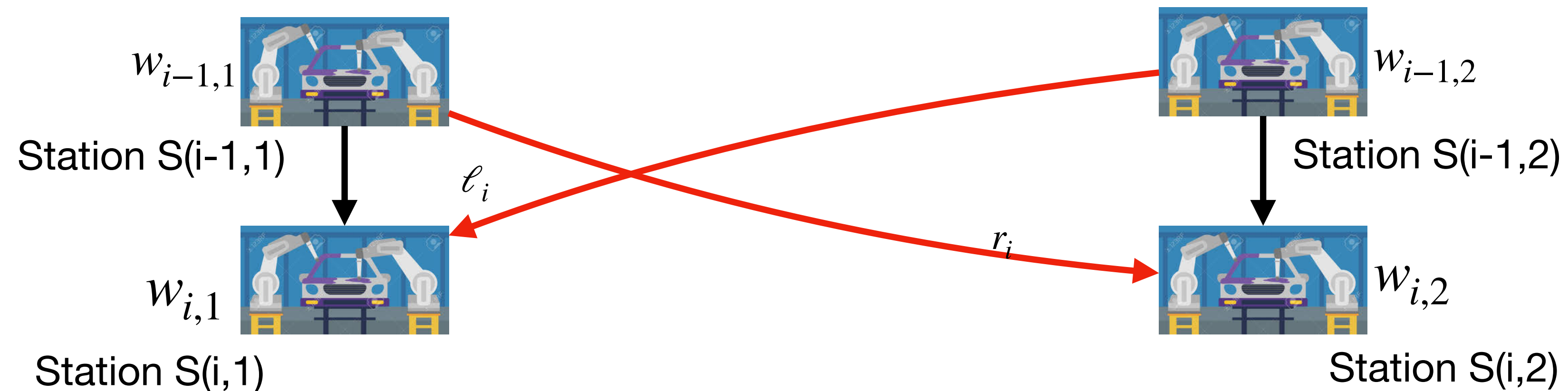
- Es gilt folgendes:

$$m_{1,1} = a_1 + w_{1,1}$$

$$m_{1,2} = a_2 + w_{1,2}$$



- Für $1 < i \leq n$ gilt:



$$m_{i,1} = \min (w_{i,1} + m_{i-1,1}, \quad w_{i,1} + \ell_i + m_{i-1,2})$$

$$m_{i,2} = \min (w_{i,2} + m_{i-1,2}, \quad w_{i,2} + r_i + m_{i-1,1})$$

Zufällige Eingabe erzeugen

m=10

n=3

Maximalwert für Zeitwerte

Anzahl der Stationen

a={ }

z={ }

l={ }

r={ }

w={ }

def init(m,n):

a={k:random.choice(range(1,m)) **for** k **in** range(1,3)}

z={k:random.choice(range(1,m)) **for** k **in** range(1,3)}

l={i:random.choice(range(1,m)) **for** i **in** range(1,n+1)}

r={i:random.choice(range(1,m)) **for** i **in** range(1,n+1)}

w={(i,k):random.choice(range(1,m)) **for** i **in** range(1,n+1) \

for k **in** range(1,3)}

return (a,z,l,r,w)

Zufällig erzeugte Eingabe ausgeben

```
def ausgabe():  
  
    print("a[1] = " + str(a[1]))  
    print("a[2] = " + str(a[2]))  
  
    for x in range(1,n+1):  
        print("l["+str(x)+"] = " + str(l[x]))  
        print("r["+str(x)+"] = " + str(r[x]))  
        print("w[("+str(x)+",1)] = " + str(w[(x,1)]))  
  
    print("z[1] = " + str(z[1]))  
    print("z[2] = " + str(z[2]))
```

Rekursive Berechnung

```
def loeseRekursiv(i,k):  
    if i==1:  
        return a[k] + w[(i,k)]  
    else:  
        h = l if k==1 else r  
        return w[i,k] + min(loeseRekursiv(i-1,k),  
                             h[i] + loeseRekursiv(i-1,3-k))
```

wir wollen entweder auf
Wörterbuch `l` oder auf
Wörterbuch `r` zugreifen

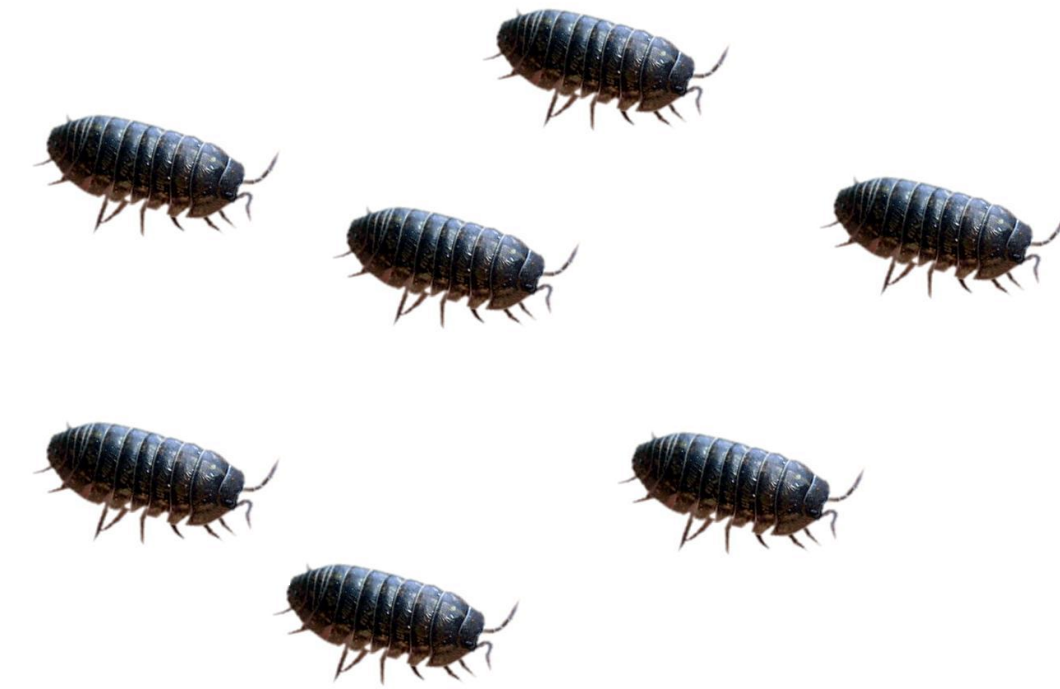
Beschleunigte rekursive Berechnung mit Wörterbuch

```
def loeseRekursivmitwb(wb,i,k):  
    if i==1:  
        return a[k] + w[(i,k)]  
    elif (i,k) in wb:  
        return wb[(i,k)]  
    else:  
        h = 1 if k==1 else r  
        wb[(i,k)] = w[i,k] + min(loeseRekursivmitwb(wb,i-1,k), \  
                                h[i] + loeseRekursivmitwb(wb,i-1,3-k))  
        return wb[(i,k)]
```


Qualitätssicherung, eine Analogie

Sie bereiten eine Suppe zu und es fallen Käfer (Bugs, auf englisch) von der Decke. Was machen Sie?

- Sie überprüfen, ob die Suppe Käfer enthält
 - entspricht Testen
- Sie schließen den Deckel
 - entspricht defensiver Programmierung
- Sie säubern die Küche
 - Sie entfernen die Plage direkt



DEFENSIVE PROGRAMMIERUNG

- Man schreibt **Spezifikationen** für Funktionen
- Man **modularisiert** die Programme
- Man überprüft **Bedingungen** auf Eingabe/Ausgabe-Ebene (Assertions)

DEBUGGING

- Man **untersucht** Ereignisse, die zu Fehlern führen
- „Warum funktioniert das Programm nicht?“
- „Wie kann ich mein Programm anpassen“?

TEST/VALIDIERUNG

- Man **vergleicht** konkrete Eingabe/Ausgabe-Paare bzgl. Spezifikation
- „Es funktioniert nicht!“
- „Wie kann ich mein Programm zum Absturz bringen“?

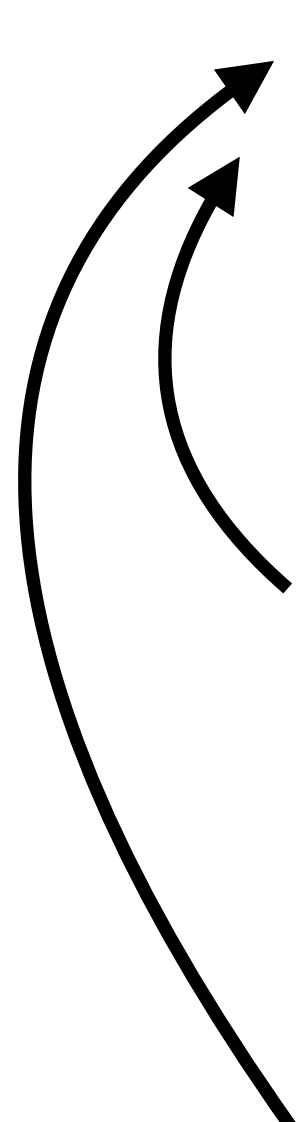
Testen und Debugging, von Anfang an

- man sollte sich **von Anfang an** darauf einstellen, dass Testen und Debuggen Teil der Entwicklung ist
 - die Struktur des Programms und der Code sollte dies auch reflektieren
- man unterteilt das Programm in **Module**, die individuell getestet und debugged (entwurmt) werden können
- man **dokumentiert Bedingungen** an die Module
 - wie genau soll die Eingabe aussehen, was wird erwartet?
 - wie genau soll/wird die Ausgabe aussehen?
- **Annahmen** sollen **dokumentiert** werden (Teil des Programm-Designs)

Ab wann kann getestet werden?

- der Code muss **ausführbar** sein
 - man entfernt Syntax-Fehler
 - man entfernt statische semantische Fehler
 - der Python-Interpreter ist beim Auffinden solcher Fehler meistens sehr behilflich
- man hat bereits eine **Menge erwarteter Ergebnisse** parat
 - eine Eingabemenge
 - für jede Eingabe eine erwartete Ausgabe

Formen des Tests

- 
- **Einheitstest** (unit testing)
 - jedes Programmstück wird einzeln überprüft
 - d.h. jede einzelne Funktion/Methode
 - **Regressionstest** (regression testing)
 - Wiederholung von Testfällen, um bei bereits getesteten Teilen der Software nach Modifikation keine neuen Fehler zu verursachen
 - **Integrationstest** (integration testing)
 - funktioniert das **Gesamtprogramm**?
 - abgestimmte Einzeltests, die dazu dienen voneinander abhängige Komponenten als Ganzes zu testen
 - wird oftmals zu schnell durchgeführt bzw. vernachlässigt

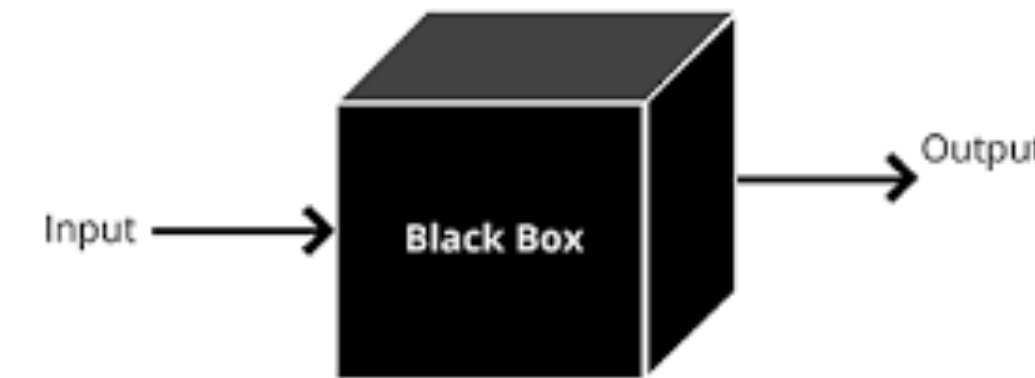
Testen: Ansätze

- **Intuition** über natürliche Randfälle des Problems

```
def istgroesser(x,y):  
    """Erwartet, dass x und y jeweils vom Typ int sind  
    Gibt True zurueck falls x groesser als y ist, ansonsten False"""
```

- Gibt es natürliche Partitionierungen der Eingaben?
- falls keine natürlichen Partitionierungen der Eingabe existieren, könnte man Zufallseingaben würfeln
 - Die Wahrscheinlichkeit, dass der Code korrekt ist, steigt mit der Anzahl der durchgeführten Tests
 - es gibt jedoch bessere Vorgehensweisen (siehe unten)
- **Black Box Testing**
 - man exploriert gewisse Pfade (nur) anhand der Spezifikation, insbesondere ohne den Code zu kennen
- **Glass Box Testing**
 - man exploriert gewisse Pfade des Codes

Black Box Testing

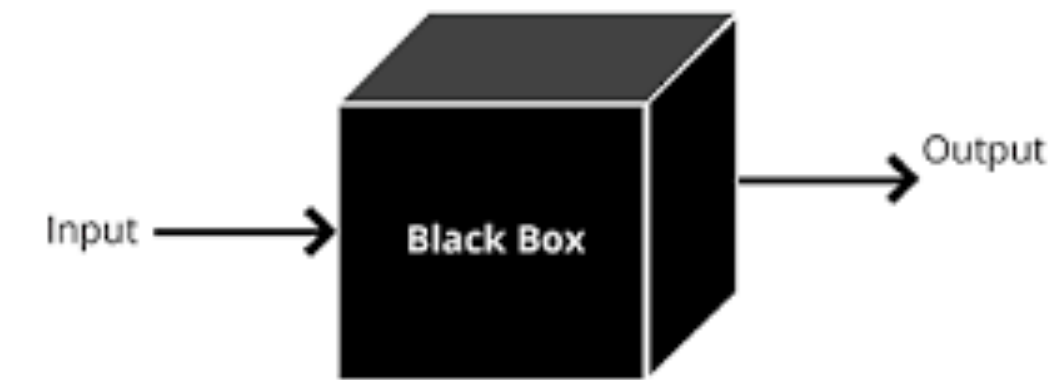


```
def wurzel(x,eps):
```

```
    """Erwartet, dass x und eps jeweils vom Typ float sind  
    Gibt erg zurueck so, dass  $x - \text{eps} \leq \text{erg}^2 \leq x + \text{eps}$ """
```

- wird durchgeführt **ohne** den **Code** zu kennen
- kann von jemandem durchgeführt werden, die/der nichts von der Implementierung weiss, um **unbefangen** zu urteilen
- kann **wiederholt** durchgeführt werden, auch dann wenn sich die Implementierung ändert
- **Pfade** durch die Spezifikation werden betrachtet
 - Testfälle entstehen durch möglichst aussagekräftige Partitionierung
 - Randfälle sollten auch abgedeckt werden (leere Listen, Liste der Länge 1, zu kleine/große Zahlen, zu viele Argumente)

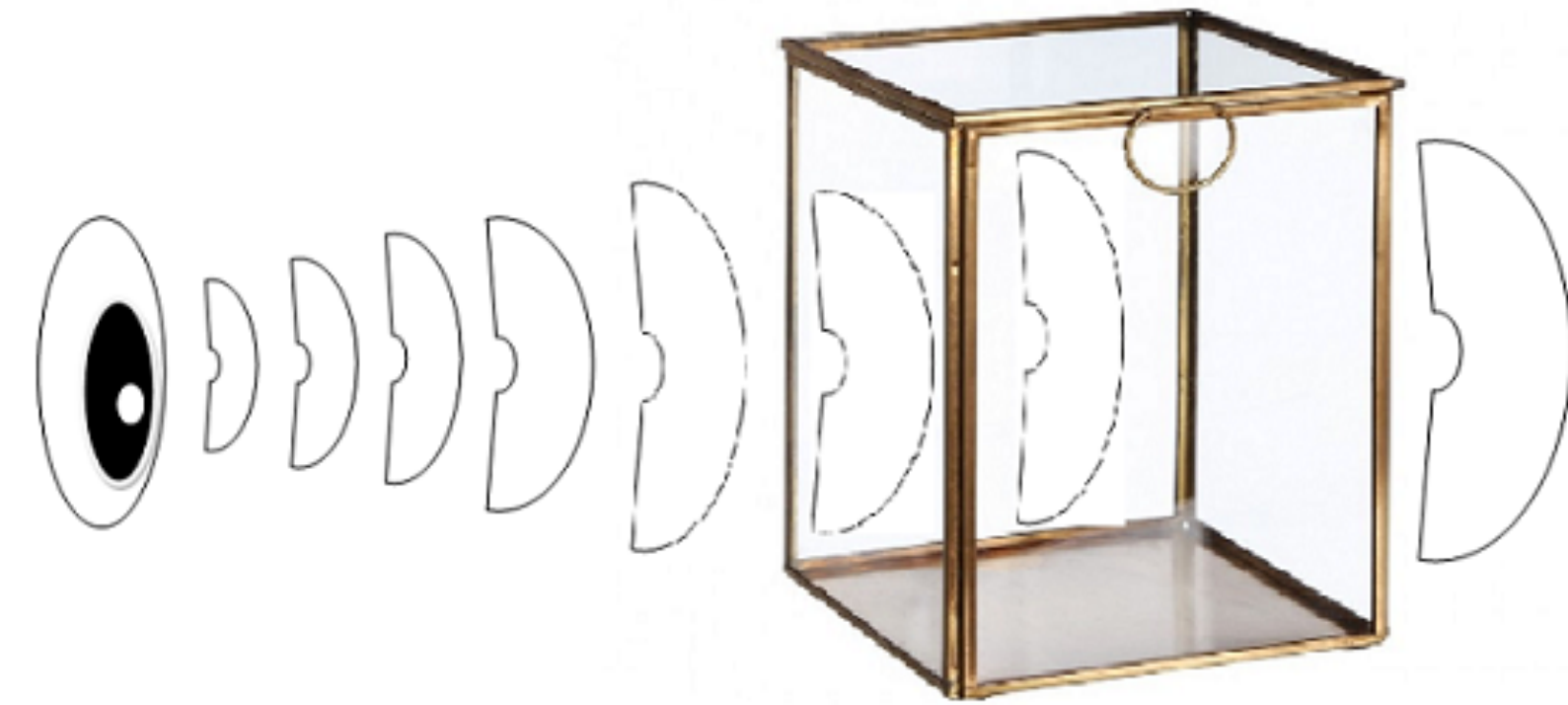
Black Box Testing



```
def wurzel(x,eps):  
    """Erwartet, dass x und eps jeweils vom Typ float sind  
    Gibt erg zurueck so, dass x-eps <= erg**2 <= x+eps"""
```

Fall	x	eps
Grenzfall	0	0.0001
ganzzahlige Quadratzahl	25	0.0001
kleiner 1	0.05	0.0001
irrationale Quadratwurzel	2	0.0001
Extremfall 1	2	1.0/2.0**64
Extremfall 2	2.0**64	1.0/2.0**64
Extremfall 3	1.0/2.0**64	2.0**64
Extremfall 4	2.0**64	2.0**64

Glass Box Testing



- man **verwendet den Code** direkt für den Entwurf der Testfälle
 - wird **Pfad-vollständig** genannt, wenn jeder potentielle Programmpfad durch mindestens einen Testfall abgedeckt wird
 - was sind die **Nachteile** dieses Test-Ansatzes?
 - durch eine Schleife kann theoretisch beliebig oft gelaufen werden
 - dadurch werden gewisse Pfade nicht abgedeckt
 - Typische **Richtlinien**
 - Verzweigungen
 - **for**-Schleifen
 - **while**-Schleifen
- Alle Verzweigungen sollen abgedeckt werden*
- Schleife wird nicht betreten
Schleife wird einmal betreten
Schleife wird mehr als einmal betreten*
- die drei Fälle der **for**-Schleife plus alle Möglichkeiten
die **while**-Schleife zu verlassen*

Debugging

- benötigt einiges an Erfahrung
- Ziel ist es ein fehlerfreies Programm zu erhalten
- Werkzeuge
 - **eingebaut** in IDLE und Anaconda
 - **Python Tutor**
 - **print**-Anweisung
 - man verwende das eigene Gehirn, dabei ist ein **systematisches** Vorgehen sinnvoll

`print`-Anweisungen

- liefern eine gute Methode um **Hypothesen zu testen**
- was soll der `print`-Anweisung übergeben werden?
 - repräsentative Eingaben (Parameter) für Funktionen/Methoden
 - Überprüfung ob erwartete Ausgabe zurückgegeben wird
 - einzelne Variablen
 - ...
- **Binäre Suche** oft hilfreich:
 - `print`-Anweisung in der Mitte des Programms
 - wenn Fehler davor auftritt, ersetze/ergänze `print`-Anweisung davor, ansonsten danach

Vorgehen beim Debugging

- man **studiert** den Programmcode
 - man fragt nicht was falsch am Programm ist
 - die Frage ist eher: „Wie habe ich diesen Fehler erhalten“?
 - war es ein einzelner Fehler oder liegt eine Familie von Fehlern vor?
- **wissenschaftlicher Ansatz**
 - man untersucht die verfügbaren Daten
 - man stellt eine Vermutung/Behauptung auf
 - man führt Experimente durch, die wiederholt reproduzierbar durchgeführt werden können
 - man beginnt damit die „einfachsten“ Eingaben zu testen

Fehlermeldungen (einfach)

- Versuch auf einen ungültigen Index einer Liste zuzugreifen

```
test = [1, 2, 3]
```

```
test[4]
```

IndexError

- Versuch auf ungültige Konvertierung durchzuführen

```
int(test)
```

TypeError

- Referenzierung einer nichtexistierenden Variablen

```
a
```

NameError

- Mischen von Datentypen ohne geeignete Konvertierung

```
'3' / 4
```

TypeError

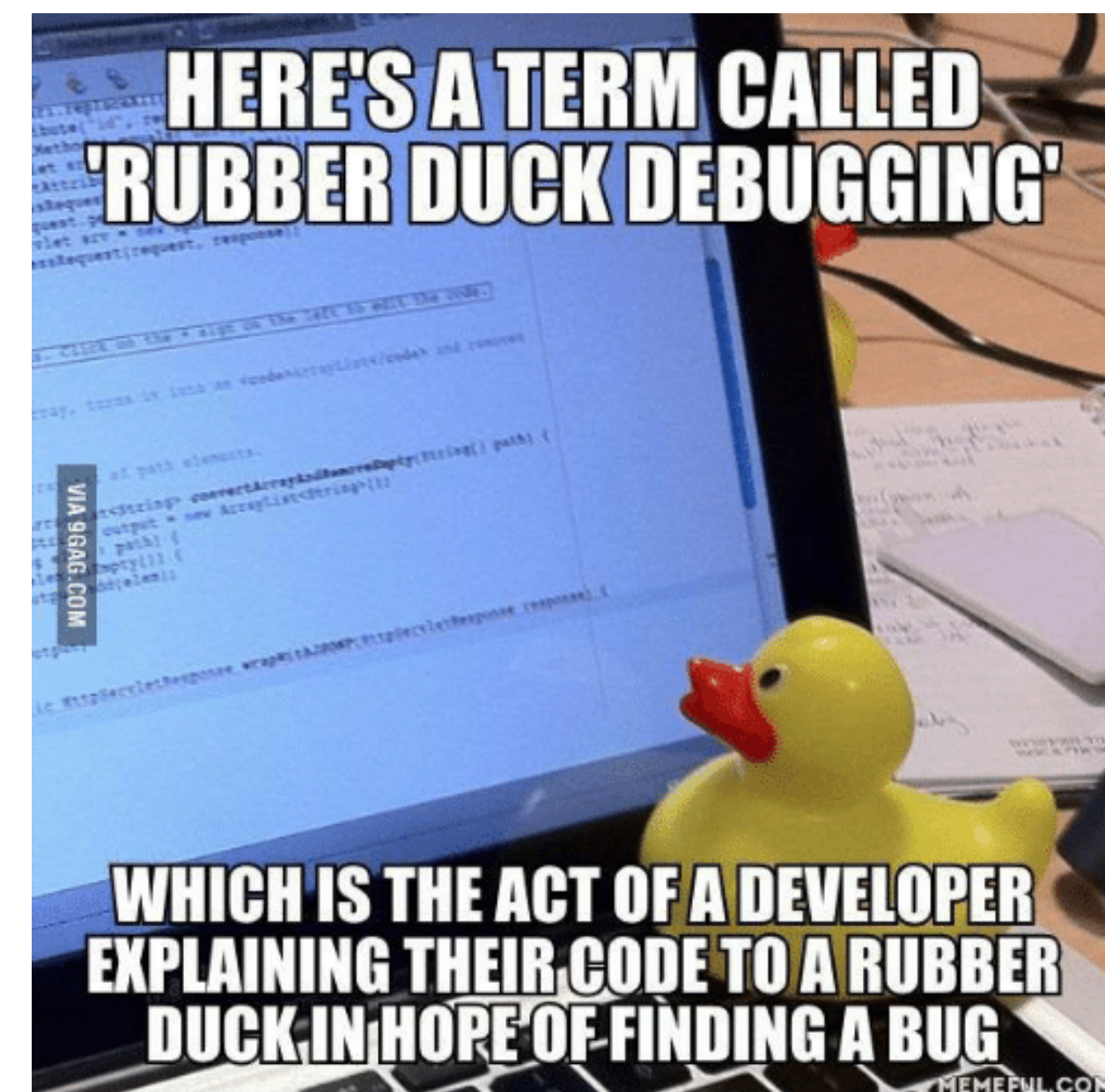
- schließende Klammern fehlen, Anführungszeichen fehlen, etc.

```
a=len ([1, 2, 3]
```

SyntaxError

Logische Fehler (schwierig)

- man **denkt nach** bevor man ein neues Programm schreibt
- man **zeichnet** Bilder, macht eine Pause
- man **erkläre** das Programm
 - einer/einem Mitstudierenden
 - jemandem, der absolut keine Ahnung hat

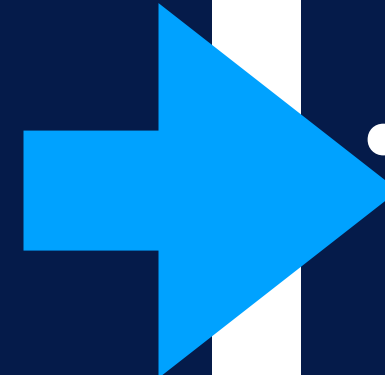


I guess his job is to... Quack the code

Was ist gut, was ist schlecht

SCHLECHT

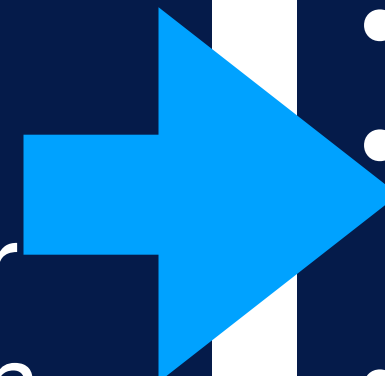
- Man schreibt das komplette Programm
- Man testet das komplette Programm
- Man entwirmt (debugged) das komplette Programm



GUT

- Man schreibt eine Funktion
- Man testet und entwirmt die Funktion
- Man schreibt die nächste Funktion
- Man testet und entwirmt die Funktion
- Integrationstest

- Man ändert den Code
- Man weiss wo der Fehler war
- Man testet den Code
- Man vergisst wo der Fehler war und wie er entstanden/behoben wurde
- Programmieren auf gut Glück



- Man legt eine Sicherheitskopie an
- Man ändert den Code
- Man schreibt einen potentiellen Fehler in Programmcode
- Man testet den Code
- Man vergleicht die neue Version mit der alten

weitere Arten von Exceptions

- wir haben bereits folgende Fehlermeldungen erhalten:
 - `SyntaxError`: Python kann das Programm nicht interpretieren
 - `NameError`: Lokaler oder globaler Name nicht bekannt
 - `AttributeError`: Attribut-Referenz schlägt fehl
 - `TypeError`: Operand passt nicht mit Typ des anderem Operanden zusammen
 - `ValueError`: Typ des Operanden ist korrekt, aber Werte sind nicht erlaubt
 - `IOError`: Fehler beim Lesen/Schreiben von Daten, Datei bspw. nicht gefunden, kein Speicher mehr frei

Handhabung von Exceptions

- wir haben unterschiedliche Anweisungen, um unterschiedliche Exceptions abzufangen

try:

```
a=int(input("Nennen Sie mir eine Zahl: "))  
b=int(input("Nennen Sie mir eine weitere Zahl: "))  
print("a/b = ",a/b)  
print("a+b = ",a+b)
```

except ValueError:

wird ausgeführt, nur falls dieser Fehler eintritt

```
print("Ihre Eingabe konnte nicht in eine Zahl konvertiert werden")
```

except ZeroDivisionError:

wird ausgeführt, nur falls dieser Fehler eintritt

```
print("Durch 0 darf nicht geteilt werden")
```

except:

wird bei allen anderen Fehlern ausgeführt

```
print("Irgendetwas ist schief gelaufen")
```

weitere Exceptions

- **else:**
 - Rumpf hiervon wird ausgeführt, falls die Ausführung des **try**-Zweigs **beendet** wird **ohne Exceptions** erhalten zu haben
- **finally:**
 - Rumpf hiervon wird immer nach den **try**-, **else**- und **except**-Anweisungen ausgeführt, sogar falls weitere Fehler, **break**-, **continue**- oder **return**-Anweisungen ausgeführt werden
 - eignen sich gut für Aufräumarbeiten, die stets ausgeführt werden sollen (Daten ausschreiben, Dateien schließen, etc.)

was macht man mit Exceptions?

- was soll getan werden, wenn ein Fehler auftritt?
- „**fail silently**“ (d.h. keine Fehlermeldung)
 - man ersetzt die diversen Daten mit Standardwerten oder macht einfach heiter weiter
 - ist oftmals keine gute Idee: Der Benutzer würde dadurch nicht informiert werden, was falsch lief
- Rückgabe eines „**Fehlerwerts**“
 - welcher konkrete Wert soll gewählt werden?
 - kann Wartung des Programmcodes erschweren nach speziellen Werten zu suchen
- Beendigung der Ausführung, **Meldung** eines Fehlers
 - in Python: **Werfen einer Exception**
`raise Exception("falsches Eingabeformat")`

Exceptions als Kontrollfluß

- man gibt keine speziellen Werte zurück sobald ein Fehler auftritt um später zu überprüfen, ob „Fehlerwert“ zurückgegeben wurde
- Anstattdessen sollte man eine **Exception werfen**, wenn es nicht möglich ist ein Ergebnis zu liefern, das mit der Spezifikation übereinstimmt

```
raise <exceptionName> (<Argumente>)
```

```
raise ValueError("Irgendwas stimmt hier nicht")
```

Schlüsselwort
Name des
Fehlers/
Exception, der
geworfen wird

ist optional, aber typischerweise eine
Zeichenkette mit einer Nachricht

Beispiel: Werfen und Abfangen einer Exception

```
def teile_komponentenweise(L1,L2):  
    """Wir nehmen an, dass L1 und L2 Listen gleicher Laenge sind,  
    die jeweils Zahlen enthalten  
    Rueckgabe: Eine Liste gleicher Laenge, die die Zahlen  
    L1[i]/L2[i] enthaelt """  
    ergebnis = []  
    for index in range(len(L1)):  
        try:  
            ergebnis.append(L1[index]/L2[index])  
        except ZeroDivisionError:  
            ergebnis.append(float('nan')) # nan=not a number  
        except:  
            raise ValueError('teile_komponentenweise mit \  
                                unpassenden Argumenten aufgerufen')  
    return ergebnis
```

Kontrollfluß des Programms wird
durch Werfen von eigener Exception
gehandhabt

Beispiele für Exceptions

- nehmen wir an wir haben eine Klassenliste, wobei wir jeweils zwei Einträge haben
- Erster Eintrag: Eine Liste bestehend aus dem Vornamen und dem Nachnamen der Schülerin/des Schülers
- Zweiter Eintrag: Eine Liste von Noten (als Prozentsatz)

```
test_noten = [[['Horst', 'Sebert'], [80.0, 70.0, 85.0]],  
              [['Angela', 'Merk'], [100.0, 80.0, 74.0]]]
```

- wir erzeugen eine **neue Klassenliste**, die die Namen, die Noten und den Notendurchschnitt enthält

```
[[['Horst', 'Sebert'], [80.0, 70.0, 85.0], 78.3333],  
 [['Angela', 'Merk'], [100.0, 80.0, 74.0], 84.6667]]
```

Code-Beispiel

```
def hole_statistik(klassenliste):  
    neue_statistik = []  
    for s in klassenliste:  
        neue_statistik.append\  
            ([s[0], s[1], schnitt(s[1])])  
  
    return neue_statistik  
  
def schnitt(noten):  
    return sum(noten) / len(noten)
```

Fehler, falls keine Note für Schüler vorhanden ist

- falls ein oder mehrere Schüler keine Noten haben, gibt es einen Fehler

```
test_noten=[[['Horst', 'Sebert'], [80.0, 70.0, 85.0]],  
            [['Angela', 'Merk'], [100.0, 80.0, 74.0]],  
            [['Gregor', 'Giese'], [98.0, 100.0, 56.0]],  
            [['Alice'], []]]
```

- wir erhalten `ZeroDivisionError: division by zero`
da wir beim Aufruf der Funktion `schnitt` folgendes machen
`return sum(noten) / len(noten)`

Länge 0

Option 1: Fehlermarkierung

- man entscheidet sich dazu eine Benachrichtigung zu versenden, die mitteilt, dass etwas nicht geklappt hat

```
def schnitt2(noten):  
    try:  
        return sum(noten)/len(noten)  
    except ZeroDivisionError:  
        print("Achtung: leere Notenliste")
```

- wir erhalten folgende Ausgabe

Achtung: leere Notenliste

Fehler markiert

```
[[['Horst', 'Sebert'], [80.0, 70.0, 85.0], 78.33333333333333], [['Ang  
'Merk'], [100.0, 80.0, 74.0], 84.66666666666667], [['Gregor', 'Giese'  
100.0, 96.0], 98.0], [['Alice'], [], None]]
```

*da schnitt2 nichts
zurückgibt bei Exception*

Option 2: Ändern der Richtlinien

- man entscheidet sich dazu, dass ein notenloser Schüler einen Schnitt von **Null** erhält

```
def schnitt3(noten):  
    try:  
        return sum(noten) / len(noten)  
    except ZeroDivisionError:  
        print("Achtung: leere Notenliste")  
        return 0.0
```

- wir erhalten folgende Ausgabe

Achtung: leere Notenliste

[[['Horst', 'Sebert'], [80.0, 70.0, 85.0], 78.33333333333333], [['Ang',
'Merk'], [100.0, 80.0, 74.0], 84.66666666666667], [['Gregor', 'Giese'],
100.0, 96.0], 98.0], [['Alice'], [], 0.0]]

Fehler immer noch markiert

*Rückgabe von
schnitt3*

Assertions



- man will sich vergewissern, dass gewisse **Annahmen**, von denen im Laufe des Programms ausgegangen wird, auch eingehalten werden
- man verwende eine **assert-Anweisung** um eine `AssertionError`-Exception zu werfen, falls die Annahmen nicht zutreffen
- ist ein Beispiel guter **defensiver Programmierung**

Beispiel für eine Assertion

```
def schnitt4(noten):  
    assert len(noten) != 0, 'leere Noteliste'  
    return sum(noten) / len(noten)
```

Funktion wird sofort abgebrochen, wenn
Annahme (Assertion) nicht zutrifft

- wirft eine `AssertionError`-Exception wenn eine leere Liste übergeben wird
- andernfalls wird das Programm wie gewohnt ausgeführt

Assertions als Umsetzung defensiver Programmierung

- Assertions erlauben den Entwicklern die Handhabung unerwarteter Annahmen zu kontrollieren
- stellt sicher, dass die **Ausführung angehalten** wird immer dann wenn Annahmen nicht zutreffen
- typischerweise werden Assertions verwendet, **Eingaben** für Methoden/Funktionen **zu überprüfen**, können jedoch überall eingesetzt werden
- können natürlich auch dafür verwendet werden, dass **Ausgaben wohlgeformt** sind, damit sich unerwünschte Werte nicht propagieren

Wo sollen Assertions eingesetzt werden?

- das Ziel ist Fehler so schnell wie möglich zu erkennen und zu kennzeichnen wo sie passieren
- kann für das **Testen unterstützend** eingesetzt werden
- man wirft üblicherweise **Exceptions**, wenn der Benutzer ungewollte Eingaben getätigt hat
- man verwendet **Assertions**, um
 - zu überprüfen, dass die **Typen** der Argumente korrekt sind
 - **Invarianten** eingehalten werden
 - **Einschränkungen** auf den Ausgabewerten eingehalten werden
 - **Vorbedingungen** von Funktionen/Methoden zu gewährleisten (bspw. keine Duplikate in übergebener Liste vorhanden)

Backtracking

- Nehmen wir an wir haben einen endlichen Suchraum, den wir traversieren wollen und dieser habe folgende Gestalt:
 - Wir haben endliche (geordnete) Menge von **Positionen P** an denen man eine Auswahl treffen kann
- **Backtracking-Prinzip:**
 - Zu Beginn haben wir an keiner Position etwas probiert
 - Solange wir noch **Positionen p** im Suchraum setzen können, wähle eine Position **p**
 1. Probiere **erste/nächste Möglichkeit aus** an **p** und **modifiziere Daten** dazu
 2. Löse **kleineres Problem rekursiv**
 3. Mache **Modifikation aus 1. (u.U.) rückgängig**

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```


Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```


i=0

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```


i=0

- Wenn **i==len(liste)-1** gib liste aus

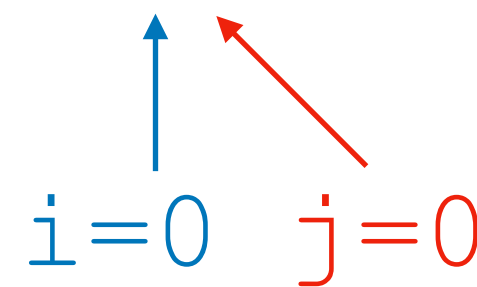
Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```



- Wenn **i==len(liste)-1** gib liste aus
- Wenn **i<len(liste)-1**
 - Für alle **Indizes j>=i**:

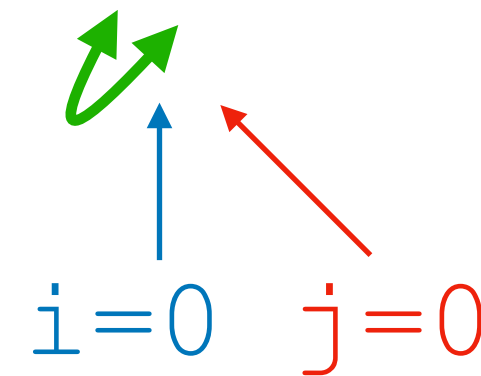
Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus
- Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 1. **vertausche** `liste[i]` **und** `liste[j]`

Backtracking
Schritt 1

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```

↑
i=1

- Wenn **i==len(liste)-1** gib liste aus
 - Wenn **i<len(liste)-1**
 - Für alle **Indizes j** **j>=i**:
 - Backtracking**
 - Schritt 1** 1. **vertausche** liste[i] **und** liste[j]
 - Schritt 2** 2. verfare **rekursiv** mit **i+=1**

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```


i=0

- Wenn **i==len(liste)-1** gib liste aus

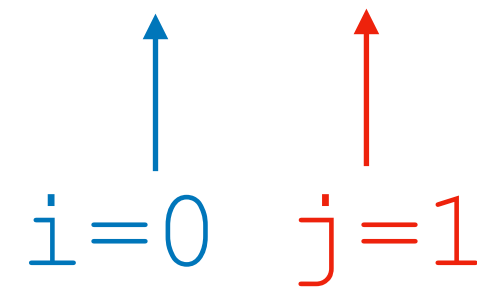
Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```


i=0 j=1

- Wenn **i==len(liste)-1** gib liste aus
- Wenn **i<len(liste)-1**
 - Für alle **Indizes j** **j>=i**:

Permutationen einer Liste

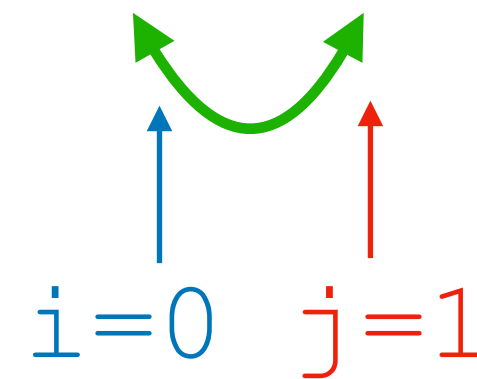
- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus
- Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 1. **vertausche** `liste[i]` **und** `liste[j]`

Backtracking
Schritt 1

Permutationen einer Liste

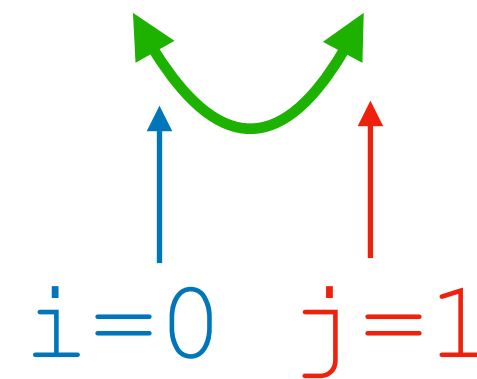
- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[2, 3, 1, 17, 5]
```



- Backtracking
Schritt 1

 - Wenn `i==len(liste)-1` gib liste aus
 - Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 1. **vertausche** `liste[i]` **und** `liste[j]`

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[2, 3, 1, 17, 5]
```

↑
i=1

- Wenn **i==len(liste)-1** gib liste aus
 - Wenn **i<len(liste)-1**
 - Für alle **Indizes j** **j>=i**:
 - Schritt 1** 1. **vertausche** liste[i] **und** liste[j]
 - Schritt 2** 2. verfare **rekursiv** mit **i+=1**

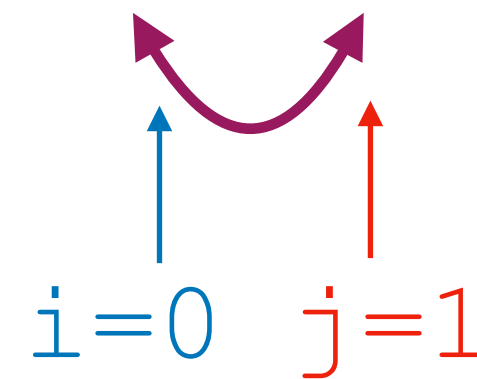
Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[2, 3, 1, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus
- Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 - Backtracking**
 - Schritt 1** 1. **vertausche** `liste[i]` **und** `liste[j]`
 - Schritt 2** 2. verfare **rekursiv** mit `i+=1`
 - Schritt 3** 3. mache Vertauschung **rückgängig**

Permutationen einer Liste

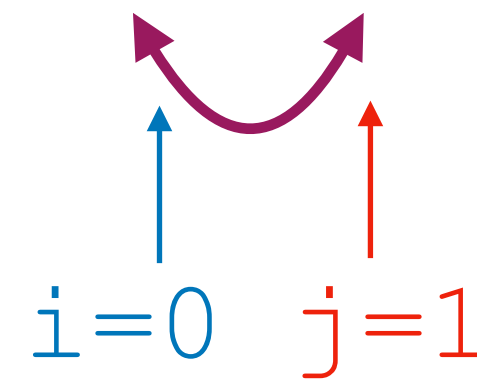
- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus

- Wenn `i<len(liste)-1`

- Für alle **Indizes** `j>=i`:

Schritt 1

1. **vertausche** `liste[i]` **und** `liste[j]`

Schritt 2

2. verfare **rekursiv** mit `i+=1`

Schritt 3

3. mache Vertauschung **rückgängig**

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```


i=0 j=2

- Wenn **i==len(liste)-1** gib liste aus
- Wenn **i<len(liste)-1**
 - Für alle **Indizes j >= i**:
 - Backtracking**
 - Schritt 1** 1. **vertausche** liste[i] **und** liste[j]
 - Schritt 2** 2. verfare **rekursiv** mit **i+=1**
 - Schritt 3** 3. mache Vertauschung **rückgängig**

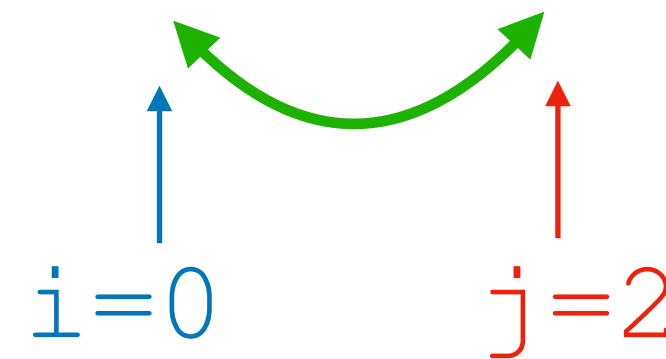
Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**
- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[3, 2, 1, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus
- Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 - Schritt 1** 1. **vertausche** `liste[i]` **und** `liste[j]`
 - Schritt 2** 2. verfare **rekursiv** mit `i+=1`
 - Schritt 3** 3. mache Vertauschung **rückgängig**

Permutationen einer Liste

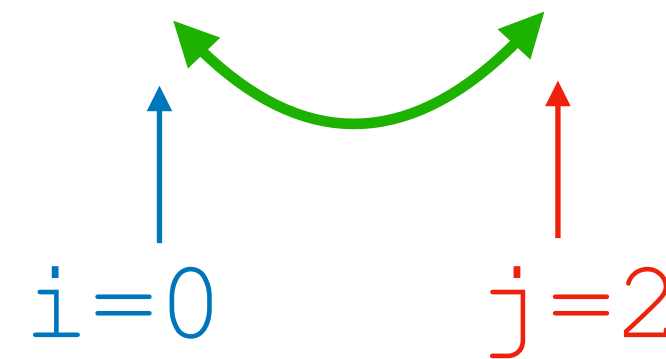
- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[1, 2, 3, 17, 5]
```



- Wenn `i==len(liste)-1` gib liste aus
- Wenn `i<len(liste)-1`
 - Für alle **Indizes** `j>=i`:
 - Backtracking**
 - Schritt 1** 1. **vertausche** `liste[i]` **und** `liste[j]`
 - Schritt 2** 2. verfare **rekursiv** mit `i+=1`
 - Schritt 3** 3. mache Vertauschung **rückgängig**

Permutationen einer Liste

- Gib alle Permutationen einer gegebenen Liste aus

```
liste=[3, 2, 1, 17, 5]
```

- **Positionen: Indizes der Liste**

- Gib alle Permutationen beginnend vom Startindex **i=0** (bis zu **len(liste)-1**)

```
liste=[1, 2, 3, 17, 5]
```



i=1

- Wenn **i==len(liste)-1** gib liste aus

- Wenn **i<len(liste)-1**

- Für alle **Indizes j** $\geq i$:

Backtracking

Schritt 1

1. **vertausche** liste[i] **und** liste[j]

Schritt 2

2. verfare **rekursiv** mit **i+=1**

Schritt 3

3. mache Vertauschung **rückgängig**

Permutationen einer Liste in Python

```
def permutiere(liste, i):  
    if (i == len(liste)-1):  
        print(liste)  
    else:  
        for j in range(i, len(liste)):  
            liste[i], liste[j] = liste[j], liste[i]  
            permutiere(liste, i+ 1)  
            liste[i], liste[j] = liste[j], liste[i]  
  
permutiere([1, 2, 3, 4, 5, 6], 0)
```

Backtracking

Schritt 1

Schritt 2

Schritt 3

Alle Binärzeichenketten einer gewissen Länge

```
def binaerwoerter(n,wb):  
    if n==0:  
        ausgabe=""  
        keys=list(wb.keys())  
        keys.sort()  
        for i in keys:  
            ausgabe=str(wb[i])+ausgabe  
        print(ausgabe)  
    else:  
        for i in [0,1]:  
            wb[n]=i  
            binaerwoerter(n-1,wb)
```

Schritt 1 auf vorheriger Folie

Schritt 2 auf vorheriger Folie

```
wb={ }  
n=input("Geben Sie eine Wortlänge ein: ")  
binaerwoerter(int(n),wb)
```

Mengen in Python

- Listen erlauben uns eine Sequenz von Elementen zu speichern
- insbesondere dürfen Elemente mehrfach auftauchen
- Listen sind geordnet

- Eine **Menge** ist eine **ungeordnete Sammlung von Objekten**

- Python bietet einen speziellen Mengentyp bereit

```
meineMenge = {1, 2, 3, 5}
```

- Typkonvertierung von einer Liste in eine Menge

```
set([1, 2, 3, 2]) liefert Menge {1, 2, 3}
```

nicht geordnet

- Typkonvertierung von Menge in eine Liste

```
list({1, 2, 5, 4, 3, 17, 1222}) liefert Liste  
[1, 2, 3, 4, 5, 1222, 17]
```

nicht unbedingt geordnet

Modifikation von Mengen

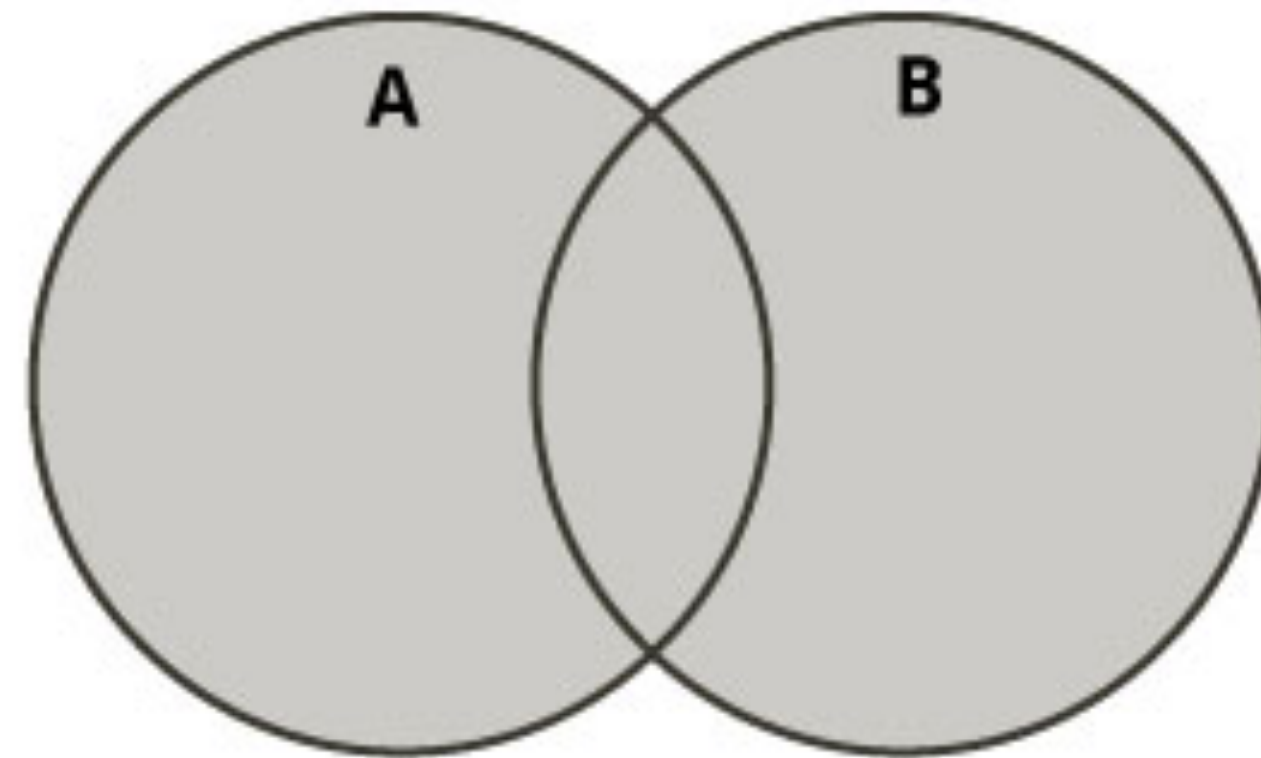
- Die leere Menge kann mittels `meineMenge={ }` initialisiert werden
- Die Elemente in einer Menge müssen nicht vom selben Typ sein
 - `{ 'A' , 1 , 1.2 }` ist also eine gültige Menge in Python
- Ein Element `element` kann mittels `meineMenge.add(element)` hinzugefügt werden
- Es kann sogar eine Liste mittels `meineMenge.update(L)` kann eine Liste `L` hinzugefügt werden
- Mittels `meineMenge.discard(element)` kann ein Element `element` der Menge entfernt werden (keine Fehlermeldung/Exception, wenn Element nicht vorhanden)
- mittels `meineMenge.pop()` wird irgendein Element der Menge `meineMenge` entfernt und zurückgegeben

Operationen auf Mengen

$$A = \{1, 2, 3, 4, 5\}$$

$$B = \{4, 5, 6, 7, 8\}$$

- Die **Vereinigung** zweier Mengen A und B ist die Menge aller Elemente die in **mindestens einer der beiden Mengen** vorkommen



- In Python werden mittels `|` zwei Mengen vereinigt

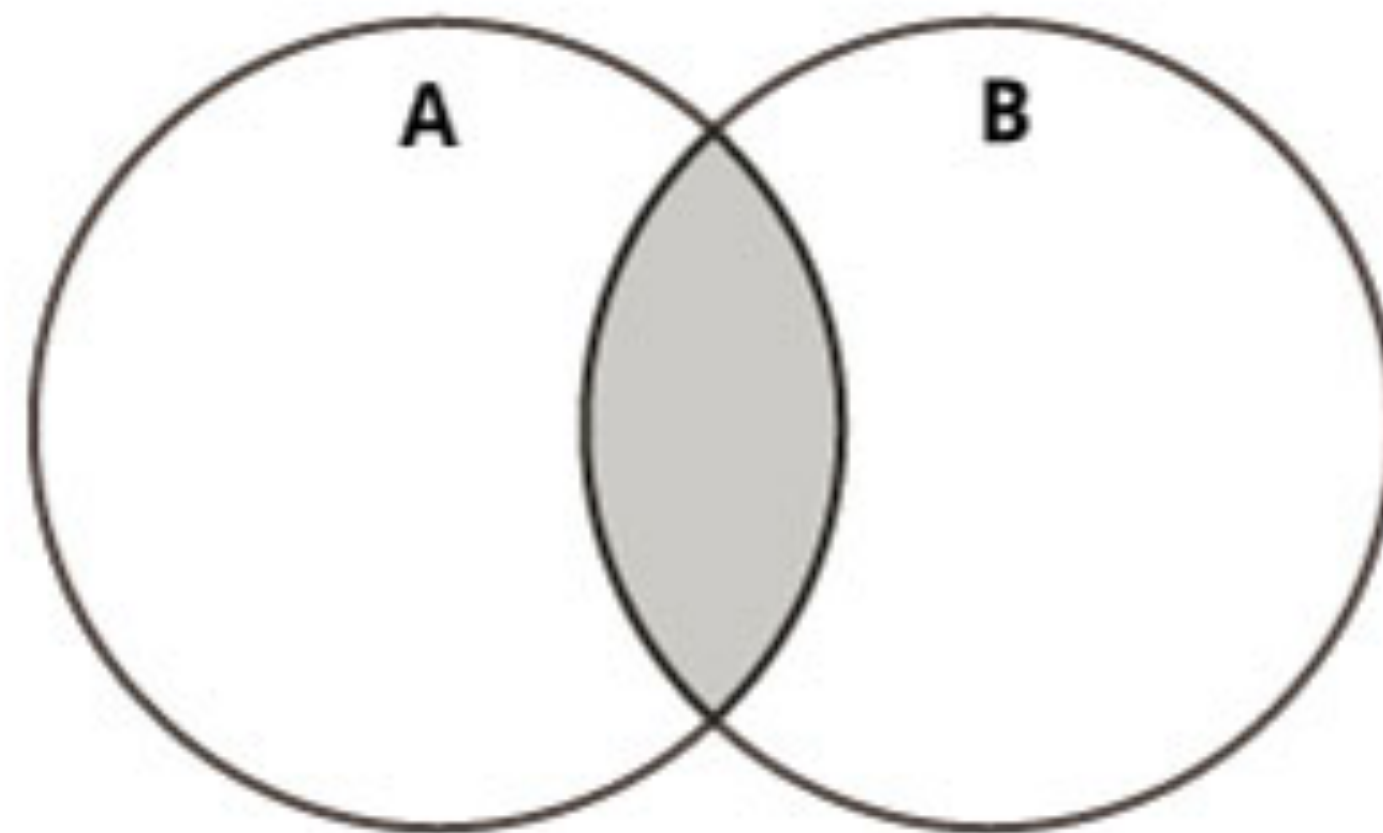
`A | B` liefert also $\{1, 2, 3, 4, 5, 6, 7, 8\}$

Operationen auf Mengen

$A = \{1, 2, 3, 4, 5\}$

$B = \{4, 5, 6, 7, 8\}$

- Die **Durchschnitt** zweier Mengen A und B ist die Menge aller Elemente die in **beiden Mengen** vorkommen



- In Python werden mittels **&** zwei Mengen geschnitten

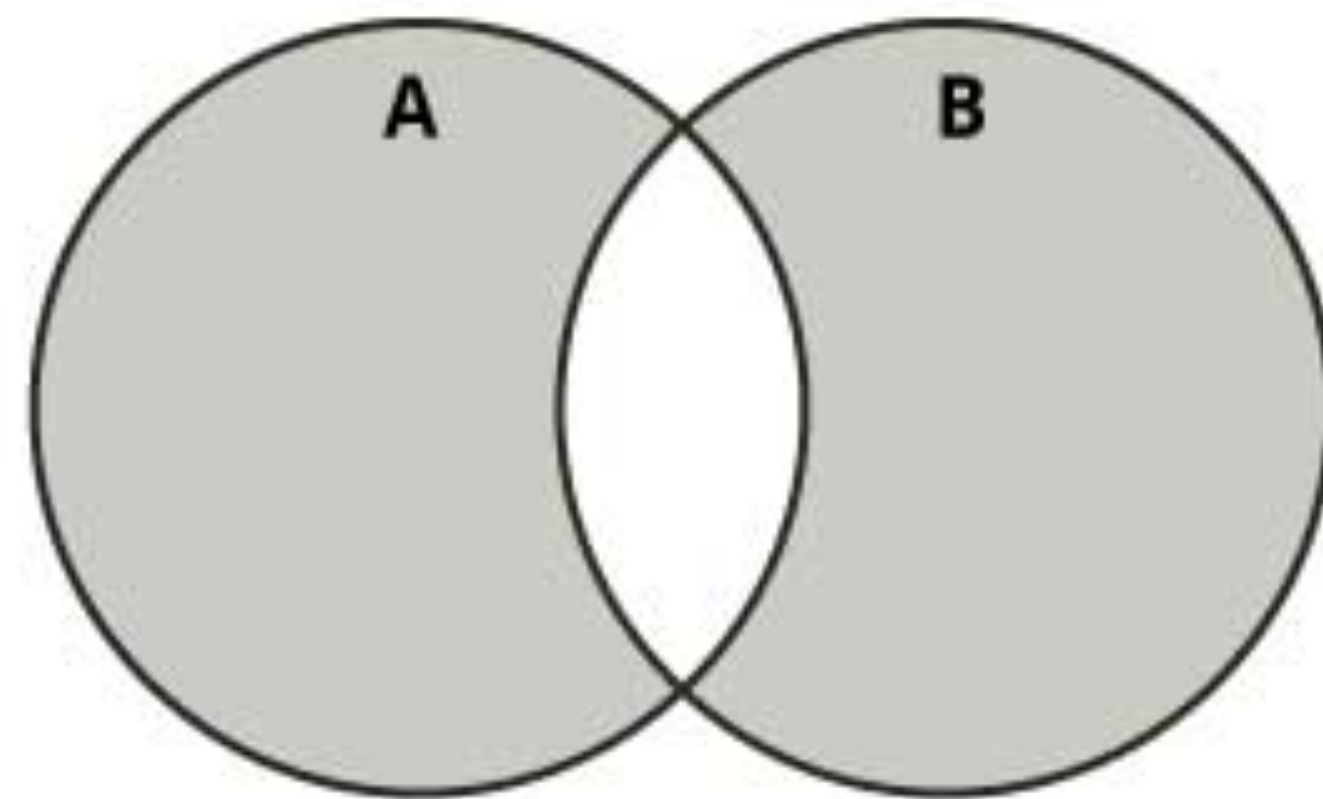
$A \ \& \ B$ liefert also $\{4, 5\}$

Operationen auf Mengen

$$A = \{1, 2, 3, 4, 5\}$$

$$B = \{4, 5, 6, 7, 8\}$$

- Die **symmetrische Differenz** zweier Mengen A und B ist die Menge aller Elemente die **in genau einer der beiden Mengen** vorkommen



- In Python wird dies mittels **$\wedge$** realisiert

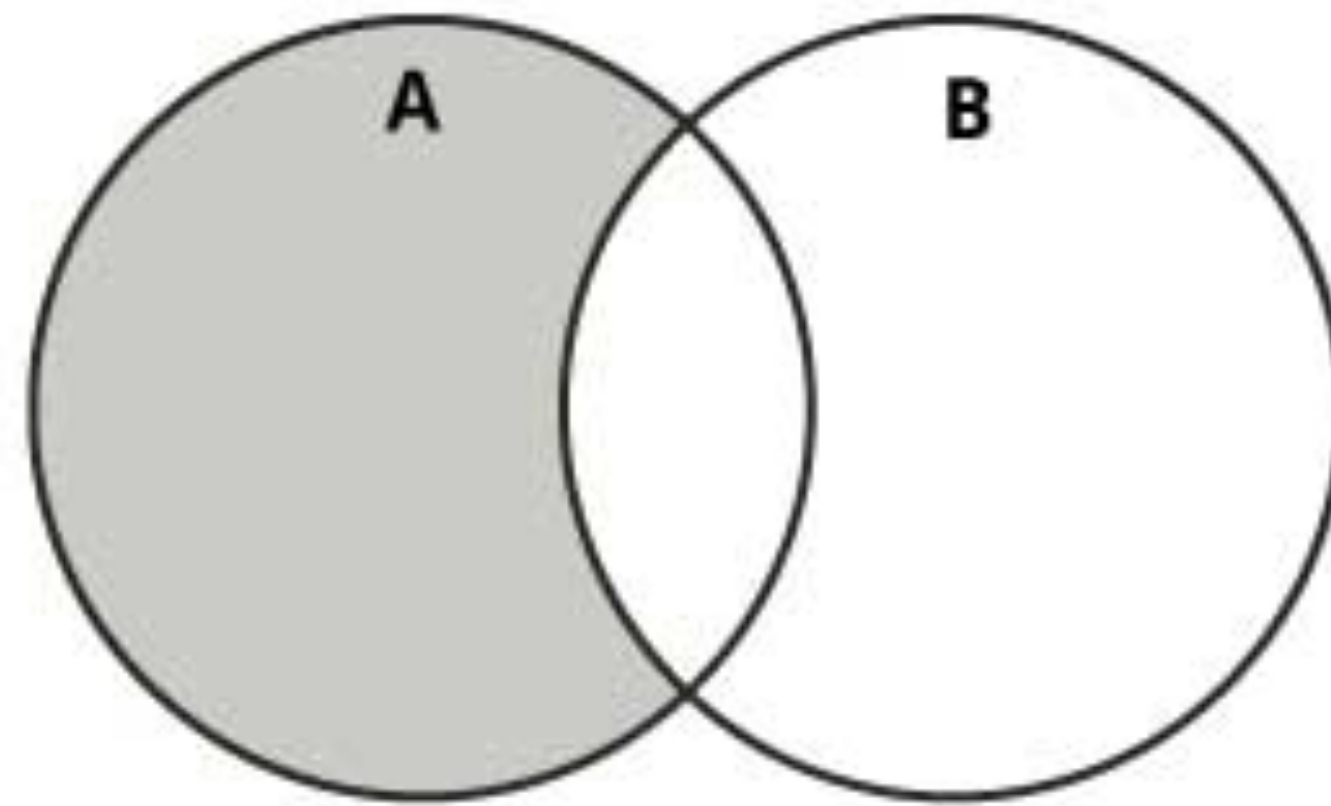
$$A \wedge B \text{ liefert also } \{1, 2, 3, 6, 7, 8\}$$

Operationen auf Mengen

$$A = \{1, 2, 3, 4, 5\}$$

$$B = \{4, 5, 6, 7, 8\}$$

- Die **Differenz** der Menge A von der Menge B ist die Menge aller Elemente die **in A aber nicht in B** vorkommen



- In Python wird dies mittels `-` realisiert

$A - B$ liefert also $\{1, 2, 3\}$

weitere Operationen

- Mitgliedschaft eines Elements `element` in der Menge `meineMenge` kann getestet werden mittels

`element in meineMenge`

vom Typ `bool`

- es kann über eine Menge iteriert werden mittels:
`for e in meineMenge:`

...

Mehr unter:

<https://docs.python.org/2/library/sets.html>

Ein paar nette Abkürzungen: **lambda**

- oftmals will man
 - kurz eine Funktion definieren, die nur einmal verwendet wird
 - Dinge generisch implementieren (d.h. derart implementieren, dass verschiedene Varianten dieser Implementierung verwendet werden)
- Mittel $f = \text{lambda } x, y: x+y$ wird ein Funktionsobjekt der Variablen f zugewiesen - es hat (hier) zwei Argumente x und y und gibt den Wert $x+y$ zurück
- $f(3, 7)$ würde bspw. den Wert 10 liefern

erster Parameter

zweiter Parameter

Rückgabewert

Ein paar nette Abkürzungen:

lambda

```
def sortiere(kleiner,L):  
    trenner = 0  
    while trenner != len(L):  
        for i in range(trenner,len(L)):  
            if kleiner(L[i],L[trenner]):  
                L[trenner], L[i] = L[i], L[trenner]  
        trenner += 1  
    return L
```

```
L = [5,2,1,232,2122,1123,12112121,12,2,434]  
meinKleiner = lambda x,y : x<y  
L=sortiere(meinKleiner,L)
```


Ein paar nette Abkürzungen:

map

- oftmals will man über eine Sequenz (wie z.B. eine Liste) iterieren, Funktion anwenden und das Ergebnis zuweisen
- dies lässt sich mittels

`map (funktion, sequenz)`

realisieren

*liefert ein Objekt
vom Typ `map`*

- Objekte vom Typ `map` lassen sich wiederum in eine Liste oder eine Menge konvertieren

```
def fahrenheit(T):  
    return ((9/5)*T + 32)  
def celsius(T):  
    return (5/9)*(T-32)  
temp = [36.5, 37, 37.5, 39]  
print(temp)  
F = list(map(fahrenheit, temp))  
print(F)  
C = list(map(celsius, F))  
print(C)
```

Ein paar nette Abkürzungen:

map + lambda

- das Programm auf der vorigen Seite hätte man auch knappe schreiben können, indem man **map** und **lambda** kombiniert:

```
Celsius = [39.2, 36.5, 37.3, 37.8]
Fahrenheit = map(lambda x: (9/5)*x + 32, Celsius)
print Fahrenheit
C = map(lambda x: (5/9)*(x-32), Fahrenheit)
print C
```

- es sind auch noch mehr Parameter möglich

```
a = [1, 2, 3, 4]
b = [17, 12, 11, 10]
c = [-1, -4, 5, 9]
print(list(map(lambda x, y: x+y, a, b) ))
print(list(map(lambda x, y, z: x+y+z, a, b, c) ))
print(list(map(lambda x, y, z: x+y-z, a, b, c) ))
```

Ein paar nette Abkürzungen:

`filter`

- recht knapp kann man auch gewisse Elemente einer Liste entfernen für die die Funktion `true` liefert

`filter` (funktion, sequenz)

*liefert ein Objekt
vom Typ `filter`*

- Objekte vom Typ `filter` lassen sich wiederum in eine Liste oder eine Menge konvertieren

```
L = [0,1,1,2,3,5,8,13,21,34,55]
ergebnis = list(filter(lambda x: x % 2, L))
print(ergebnis)
ergebnis = list(filter(lambda x: x % 2 == 0, L))
print(ergebnis)
```


List Comprehensions

- liefern eine elegante Möglichkeit Listen zu erzeugen
- sie kombinieren Iteration, Listenerzeugung und Kontrollfluß in einem
- die generelle Form

```
neuesL = [ ausdr(var) for var in L if bed(var) ]
```

liefert eine Liste `neuesL` aller Elemente der Form `ausdr(var)` in der Liste `L`, die die Bedingung `bedingung(var)` erfüllen (`var` ist Laufvariable über Elemente in `L`), ist also zu folgendem Programm äquivalent:

```
neuesL=[]  
for var in L:  
    if bed(var) :  
        neuesL.append(ausdr(var) )
```

List Comprehensions

- Der Ausdruck

```
[i*i*i for i in [0,1,2,3,4,5] if i%2==0]
```

erzeugt die Liste [0, 8, 64]

- Der Ausdruck

```
[x.lower() for x in ["A","B","C"]]
```

erzeugt die Liste ["a", "b", "c"]

- Der Ausdruck

```
[x+y for x in [10,30,50] for y in [20,40,60]]
```

erzeugt die Liste [30, 50, 70, 50, 70, 90, 70, 90, 110]

Dictionary Comprehensions

- liefern eine elegante Möglichkeit Wörterbücher (Dictionaries) zu erzeugen
- sie kombinieren Iteration, Erzeugung von Dictionaries und Kontrollfluß in einem
- Beispiel:

`{(i, j) : set(range(1, 10)) for i in range(9) for j in range(9)}`

geschweifte Klammern!

erzeugt ein Wörterbuch

- mit Schlüsselmenge = alle 2-Tupel von Zahlen von 0 bis 8
- das jedem Schlüssel (i, j) die Menge $\{1, 2, \dots, 9\}$ zuweist (für alle i und j von 0 bis 8)

Sudoku

- ist ein aus lateinischen Quadraten (typischerweise 9x9) bestehendes Logikrätsel folgender Form

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

- **Aufgabe:** Die leeren Kästchen so mit Ziffern (1 bis 9) füllen, so dass:
 - in jedem Kästchen genau eine Ziffer steht
 - in jeder **Zeile** jede Ziffer genau einmal vorkommt
 - in jeder **Spalte** jede Ziffer genau einmal vorkommt
 - in jedem der neun **3x3-Unterquadrate** jede Ziffer genau einmal vorkommt

Darstellung eines Spielstands

- wie stellen wir einen aktuellen „Spielstand“ dar?

```
stand = { (i, j) : set(range(1, 10)) for i in range(9) for j in range(9) }
```

im Folgenden auch
so bezeichnet!

- D.h. Paare (i, j) stellen **Kästchen** dar für alle i und j von 0 bis 8
- Jedem dieser Paare ordnet unser Wörterbuch `stand` eine **Teilmenge von $\{1, 2, \dots, 9\}$** zu (sogenannte **Kästchen-Menge**)
- Zu Beginn ist jede Kästchen-Menge auf **$\{1, 2, \dots, 9\}$ initialisiert**, mit der Idee, dass zu Beginn keine Einschränkung der Kästchen vorhanden ist (alle Zahlen in $\{1, 2, \dots, 9\}$ kommen als Kästcheneintrag in Frage)
- danach initialisieren wir `stand` per Hand, indem wir manche der Mengen einelementig machen (nächste Folie)
- wir meinen mit **Spielstand** im Folgenden immer ein Wörterbuch, das Kästchen auf Teilmengen von $\{1, 2, \dots, 9\}$ abbildet

Initialisierung des Spielstands

- The world's hardest Sudoku:

<https://www.telegraph.co.uk/news/science/science-news/9359579/Worlds-hardest-sudoku-can-you-crack-it.html>

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

```
def init1():  
    b[(0,0)]={8}  
    b[(1,2)]={3}  
    b[(1,3)]={6}  
    b[(2,1)]={7}  
    b[(2,4)]={9}  
    b[(2,6)]={2}  
    b[(3,1)]={5}  
    b[(3,5)]={7}  
    b[(4,4)]={4}  
    b[(4,5)]={5}  
    b[(4,6)]={7}  
    b[(5,3)]={1}  
    b[(5,7)]={3}  
    b[(6,2)]={1}  
    b[(6,7)]={6}  
    b[(6,8)]={8}  
    b[(7,2)]={8}  
    b[(7,3)]={5}  
    b[(7,7)]={1}  
    b[(8,1)]={9}  
    b[(8,6)]={4}
```

Darstellung des Spielstands

wollen wir so ausgeben

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

```
8 * * | * * * | * * *
* * 3 | 6 * * | * * *
* 7 * | * 9 * | 2 * *
-----
* 5 * | * * 7 | * * *
* * * | * 4 5 | 7 * *
* * * | 1 * * | * 3 *
-----
* * 1 | * * * | * 6 8
* * 8 | 5 * * | * 1 *
* 9 * | * * * | 4 * *
```


Ausgabe des Spielstands

```
def ausgabe(stand):  
    for i in range(9):  
        tmp=""
```

```
        if i==3 or i==6:
```

```
            print("-"*30)
```

```
        for j in range(9):
```

```
            if j==3 or j==6:
```

```
                tmp+=" | "
```

```
            d=stand[(i,j)]
```

```
            if len(d)==1:
```

```
                for e in d:
```

```
                    tmp+=" "+str(e)
```

```
            else:
```

```
                tmp+=" "+str(d)+" "
```

```
        print(tmp)
```

...ansonsten geben wir * aus

Grenze des 3x3 Unterquadrats bei Zeile
3 und 6 (der Zeilen 0 bis 8)

horizontale Linie, um Grenze des
3x3 Unterquadrats anzuzeigen

Grenze des 3x3 Unterquadrats
bei Spalte 3 und 6 (der Spalten 0 bis 8)

vertikale Linie, um Grenze des
3x3 Unterquadrats anzuzeigen

lese Menge des aktuellen
Kästchens aus

Kästcheninhalt wird nur ausgegeben
wenn Menge in Kästchen einelementig ist...

Ziel des Spiels als Mengen

- Nach der Initialisierung (wie mit dem oben abgebildeten “world’s hardest Sudoku”) sind die Mengen der Kästchen also entweder
 - nach wie vor **{1,2,...,9}** (wenn das Kästchen der Initialposition leer war)
 - einelementig, also von der Form **{k}** für ein k in $\{1,2,...,9\}$ (wenn das Kästchen gefüllt war)

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

hier bspw. {4}

- **grobe Idee des Sudoku-Lösers:** Entferne systematisch Kästchenmengen mit mindestens zwei Elementen so, dass am Ende nur noch einelementige Kästchenmengen vorhanden sind und die Sudoku-Bedingungen nicht verletzt werden

Kopie eines Spielstands

- da wir Sudoku mittels Backtracking lösen wollen, wird es notwendig sein vom aktuellen Spielstand eine Kopie anzufertigen

```
def kopie(stand):  
    return { (i,j): stand[(i,j)].copy() \  
            for i in range(9) for j in range(9) }
```

Zeilenumbruch

kopiert die Liste in `stand[(i,j)]`

„Bürsten“ von Spielständen

- Durch Anwendung der Sudoku-Bedingungen können wir u.U. einige der Kästchen-Mengen kleiner machen:

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

statt $\{1,2,3,4,5,6,7,8,9\}$ sollte diese Menge
weder eine 3 noch eine 6 noch eine 8
noch eine 2 enthalten!

- Wenn wir gewisse Mengen kleiner machen, können aus diesen Verkleinerungen weitere Verkleinerungen folgen!
- Wir sollten so lange verkleinern bis es nicht mehr möglich ist („bürsten“) und erst dann systematisch alle Möglichkeiten durchprobieren!

Beeinflussung von Kästchen

- wie können wir Abhängigkeit ausdrücken?

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

Kästchen (1,8) kann u.U. von folgenden Kästchen abhängig sein:

- in der gleichen Zeile:
 $\{(1,j) \mid j \in \{0,1,2,3,4,5,6,7\}\}$

- in der gleichen Spalte:
 $\{(i,8) \mid i \in \{0,2,3,4,5,6,7,8\}\}$

- im gleichen Unterquadrat:
 $\{(0,6),(0,7),(0,8),(1,6),(1,7),(2,6),(2,7),(2,8)\}$

- wie drückt man aus, ob eine Position (x, y) von Position (i, j) abhängig sein kann?

```
def abhaengig(i, j, x, y):  
    return (not((i==x) and (j==y))) and  
           ((i==x) or (j==y) or (i//3==x//3) and (j//3==y//3))
```

Umsetzung von Bürsten

```
def buersten(sudoku):  
    def abhaengig(i,j,x,y):  
        return (not((i==x) and (j==y))) and  
                ((i==x) or (j==y) or (i//3==x//3) and (j//3==y//3))  
    sudokuk=kopie(sudoku)  
    entfernt=False
```

Anlegen einer Kopie

noch kein Element einer Menge
eines Kästchens entfernt

```
effects=[(wert,x,y) for i in range(9)\  
            for j in range(9)\  
            for x in range(9)\  
            for y in range(9)\  
            for wert in sudokuk[(i,j)]\  
            if len(sudokuk[(i,j)])==1 and\  
            wert in sudokuk[(x,y)] and\  
            abhaengig(i,j,x,y)]
```

Zeilenumbruch

```
for (wert,x,y) in effects:  
    if wert in sudokuk[(x,y)]:  
        sudokuk[(x,y)].remove(wert)  
        entfernt=True  
if entfernt:  
    sudokuk=buersten(sudokuk)  
return sudokuk
```

Sammele Tripel (wert,x,y) ein,
wobei wert in Kästchen (x,y)
vorkommt und nachweislich gelöscht
werden muss.

Lösen durch Backtracking

1. **initialisiere** das zu lösende Sudoku durch unsere Darstellung von Spielständen (d.h. Kästchen-Mengen sind entweder **einelementige Kästchenmengen** oder **$\{1,2,\dots,9\}$**) in Spielstand `stand`
2. Falls eine Kästchenmenge in `stand` leer ist, gib **False** zurück
3. Falls alle Kästchen in `stand` einelementig sind, gib `stand` aus und gib dann **True** zurück
4. Lege eine **Liste** `nonsingletons` von Tripeln $(i, j, \text{stand}[i, j])$ an, wobei
 - (i, j) ein Kästchen ist
 - Kästchenmenge `stand[len(i, j)]` mindestens zwei Elemente hat
5. **Sortiere** `nonsingletons` **absteigend** nach Größe der Kästchenmengen
6. Entferne letztes Element $(i, j, \text{ziffern})$ von `nonsingletons` und kopiere `ziffern` in `ziffernkopie` (damit keine Nebeneffekte auftreten)
7. Für alle Ziffern `k` in `ziffernkopie`:
 - A. Lege Kopie von `stand` an (damit keine Nebeneffekte auftreten)
 - B. Setze `stand[(i, j)] = {k}`
 - C. Löse Problem rekursiv mit diesem neuen `stand` (d.h. gehe zu 2.)
 - D. Falls Rückgabewert
 - **True**, gib **True** zurück
 - **False**, setze `stand` auf Kopie zurück und versuche nächste Ziffer (d.h. gehe zu 7.)
8. gib **False** zurück

Backtracking-Schritt

Lösen durch Backtracking

```
def erfuellbar(stand):
    if len([stand[(i,j)] for i in range(9) for j in range(9) \
            if len(stand[(i,j)])==0])>=1: return False
    elif len([stand[(i,j)] for i in range(9) for j in range(9) \
            if len(stand[(i,j)])==1])==81:
        print("Wir haben eine Loesung gefunden")
        ausgabe(stand)
        return True
    else:
        nonsingletons=[(i,j,stand[(i,j)]) \
                        for i in range(9) \
                        for j in range(9) \
                        if len(stand[(i,j)])>1]
        cmp = lambda a,b: len(a[2])>len(b[2])
        nonsingletons = sortsuite.sortiere(cmp,nonsingletons)
        (i,j,ziffern)=nonsingletons.pop()
        ziffernkopie=ziffern.copy()
        old=stand.copy()
        for x in ziffernkopie:
            stand[(i,j)].clear()
            stand[(i,j)].add(x)
            stand=buersten(stand)
            if erfuellbar(stand):
                return True
            stand=old.copy()
        return False
```

Das Hauptprogramm

```
import sortsuite
```

```
def ausgabe...
```

```
def kopie...
```

```
def buersten...
```

```
def erfuellbar...
```

```
def init1...
```

```
stand={ (i,j):set(range(1,10)) for i in range(9) for j in range(9) }
```

```
init1()
```

```
print("Wir wollen folgendes Sudoku loesen")
```

```
ausgabe(stand)
```

Rekursion in der Informatik...

- ... ist eine Art und Weise Objekte/Probleme zu beschreiben
(zum Beispiel wohlgeklammerte Ausdrücke)
 - ... ist eine Art und Weise Lösungen zu entwerfen
(zum Beispiel für Fakultätsberechnung und Türme von Hanoi)
-
- ... ist genauso mächtig wie Iteration
 - ... erlaubt oftmals elegante Lösungen für Probleme
 - ... ist von zentraler Bedeutung in der Berechenbarkeitstheorie
(was kann von Computern berechnet werden)
 - ...wird von einigen Programmiersprachen verlangt wie
z.B. in Scheme, LISP, ML, Haskell (keine Schleifen möglich)

Die O-Notation



- 1894 eingeführt vom deutschen Zahlentheoretiker Paul Bachmann
- Die O-Notation (großes O) gibt eine obere Schranke für das asymptotische Wachstum einer Funktion
- **Gross O oder $O()$** wird im Rahmen einer Worst-Case-Analyse angewendet
 - Wenn der schlechteste Fall eintritt, ist dieser der Flaschenhals einer Programmberechnung
 - drückt das Wachstum relativ zur Eingabelänge aus
 - bewertet **weder** die Maschine **noch** die Implementierung

Die O-Notation, formal

Seien $f, g : \mathbb{N} \rightarrow \mathbb{N}$ jeweils Funktionen von $\mathbb{N}$ nach $\mathbb{N}$.

Wir schreiben $f \in O(g)$ falls Konstanten $c \in \mathbb{R}^+$ und $n_0 \in \mathbb{N}$ existieren so, dass für alle $n \in \mathbb{N}$ mit $n > n_0$ gilt:

$$f(n) \leq c \cdot g(n)$$

Wir sagen dann „ f ist/liegt in $O(g)$ “

Beispiele:

- Seien $f(n) = 2n$ und $g(n) = n$ für alle $n \in \mathbb{N}$.

Dann gilt $f \in O(g)$ denn wir wählen $c = 2$ und $n_0 = 0$.

- Seien $f(n) = 4n + 4$ und $g(n) = n^2$ für alle $n \in \mathbb{N}$.

Dann gilt $f \in O(g)$ denn wir wählen $c = 1$ und $n_0 = 5$

und es gilt $4n + 4 \leq 1 \cdot n^2$ für alle $n > 5 = n_0$.

Wir schreiben der Einfachheit halber
gleich $O(n)$

Genaueres Wachstum vs $O()$

```
def fakultaet_iterativ(n):  
    antwort = 1  
    while n >= 1:  
        antwort *= n  
        n -= 1  
    return antwort
```

*antwort = antwort * n*
temp = n-1
n=temp

- Berechnet die Fakultätsfunktion
- Anzahl der Operationen: $1+5n+1$
- Asymptotische Worst-Case Komplexität: $O(n)$
 - additive Konstanten werden vernachlässigt
 - multiplikative Konstanten werden vernachlässigt

Was ist Programmeffizienz?

- Computer werden immer schneller: Vielleicht ist Effizienzanalyse daher irgendwann unwichtig?
 - Datenmengen können jedoch sehr gross werden (Google hat im Jahre 2014 30 Billionen Webseiten verwaltet, was rund 100 Millionen Gigabyte entspricht)
 - Einfache algorithmische Lösungen **skalieren nicht** mit noch größeren Datenmengen
 - Wichtige Frage daher: Nach welchen Kriterien ist ein Programm effizient?
 - Wir trennen **Zeit-** und **Speichereffizienz** eines Programms
 - Kompromiss manchmal nötig:
 - Verwende Speicher um bereits Berechnetes zu sichern (siehe Fibonacci mit einem Wörterbuch)
 - wir fokussieren uns auf Zeiteffizienz
- Gewisse Zusammenhänge bestehen doch:
- Speicherbedarf $\leq$ Zeitbedarf
 - Zeitbedarf ist maximal exponentiell im Speicherbedarf

Was ist Programmeffizienz?

Es gibt gewisse Herausforderungen Effizienz eines algorithmischen Problems zu verstehen:

- Für einen Algorithmus gibt es **verschiedene Implementierungen**
- Für jedes Problem gibt es **verschiedene Algorithmen**, die man einsetzen könnte
- Wir wollen Implementierungen und abstrakte Algorithmen stets trennen

Messung von Programmeffizienz

Vorschläge:

- durch eine Stoppuhr
- die Anzahl der Operationen zählen
- abstraktes Größenwachstum

wir werden uns noch überzeugen, dass dies die angemessene Art und Weise ist

- zu messen welche Auswirkung die Wahl eines Algorithmus für ein gegebenes Problem hat und
- zu messen welche inhärente algorithmischen Schwierigkeiten einem gegebenen Problem zugrundeliegen.

Ausführungszeiten stoppen

- wir binden das `time`-Modul ein
- wir können die Methoden und Funktionen des Moduls `time` ins eigene Programm einbinden

- Starte die Stoppuhr

```
import time
```

- Rufe Funktion auf

```
def f(c):  
    return c*9/5+32
```

- Halte Stoppuhr an

```
t0 = time.process_time()  
  
f(10000)  
t1 = time.process_time() - t0  
print("Benötigte Zeit ",t1,"Sekunden")
```


Stoppuhrmessung ist inkonsistent

- Ziel: Bewertung verschiedener Algorithmen
- Laufzeit verschiedener Algorithmen **variiert** ✓
- Laufzeit verschiedener Implementierungen **variiert** ✗
- Laufzeit verschiedener Computer **variiert** ✗
- Laufzeit ist **nicht einmal vorhersagbar** bei kleinen Eingaben ✗
- Laufzeit **variiert** bei unterschiedlichen Eingaben **ohne** dass dadurch ein **Bezug** zwischen Eingabegröße und Laufzeit ableitbar ist ✗
- Schlimmer noch: Laufzeit bei fester Eingabe **nicht reproduzierbar** ✗

Zählen von Operationen

- wir nehmen der Einfachheit halber an, dass folgende Operationen konstante Zeit benötigen
 - mathematische Operationen
 - Vergleiche
 - Zuweisungen
 - Zugriff auf Objekte im Hauptspeicher
- Zähle die Anzahl der ausgeführten Operationen als Funktion der Eingabelänge

```
def f(c):  
    return c*9/5+32
```

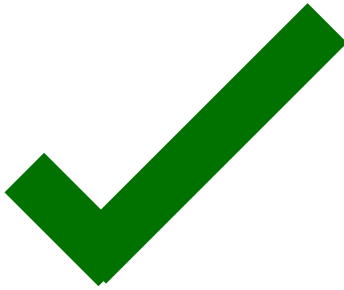
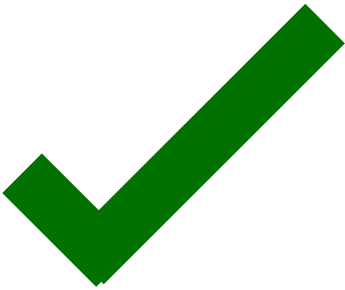
3 Operationen

```
def meinesumme(x):  
    total = 0  
    for i in range(x+1):  
        total += i  
    return total
```

1 Operation
x Durchläufe
1 Operation
2 Operationen

meinesumme hat $3x+1$ Operationen

Operationen zählen ist gut aber...

- Ziel: Bewertung verschiedener Algorithmen 
- Laufzeit verschiedener Algorithmen **variiert**
- Laufzeit verschiedener Implementierungen **variiert** 
- **unklar** welche Operationen gezählt werden sollen 
- Laufzeit **variiert** bei unterschiedlichen Eingaben **so, dass** dadurch jedoch ein **Bezug** zwischen Eingabegröße und Laufzeit ableitbar ist 
- Laufzeit bei fester Eingabe **reproduzierbar** 

Wir benötigen eine noch bessere Lösung

- Stoppuhrmessung und Zählen **bewertet Implementierungen**
- Stoppuhrmessung bewertet **Maschinen**
- Wir wollen **Algorithmen bewerten**
- Wir wollen **Skalierbarkeit bewerten**
- Wir wollen anhand der **Eingabelänge bewerten**

Wir benötigen eine noch bessere Lösung

- Wir fokussieren uns immer noch darauf Operationen zu zählen aber sorgen uns **nicht um Implementierungsdetails** (z.B. ob wir 3 oder 4 elementare Operationen innerhalb einer Schleife durchlaufen)
- Wir analysieren was passiert, wenn Größe des Problems **beliebig groß** wird
- Wir wollen die **Laufzeit** bis zum Ende der Berechnung messen und diese **in Abhängigkeit zur Eingabe- bzw. Problemgröße** darstellen.
- Wir müssen uns entscheiden was wir genau messen wollen, da uns (manchmal) gewisse Details nicht interessieren

Was wollen wir messen?

- Wir wollen die Effizienz in Abhängigkeit der Eingabelänge messen, also müssen wir festlegen was die Eingabelänge genau ist

- Kann eine natürliche Zahl sein

`meinesumme(x)`

- Kann die Länge einer Liste sein

`listensumme(liste)`

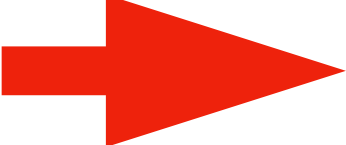
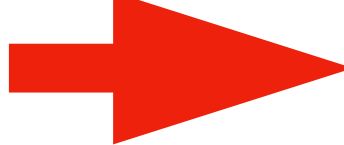
- Gewisse Entscheidungsoptionen möglich, bspw. bei mehreren Eingaben

`suche_nach_element(liste, element)`

Verhalten des Programms hängt von der Eingabe ab

- Sei folgende Funktion gegeben, die nach einem Element in einer Liste sucht

```
def suche_nach_element(liste, element)
    for i in liste:
        if i==element:
            return True
    return False
```

- Wenn `element` das erste Element von `liste` ist  **BEST CASE**
- Wenn `element` nicht in `liste` ist  **WORST CASE**
- Wenn man ungefähr die Hälfte aller Elemente von `liste` durchlaufen muss, um `element` zu finden  **AVERAGE CASE**
- Wir wollen eine allgemeine Laufzeitanalyse durchführen

Best, Average und Worst Case

- Nehmen wir eine Liste L der Länge $\text{len}(L)$ an
- Best Case: Die minimale Laufzeit über alle möglichen Eingaben einer gewissen Länge, nämlich $\text{len}(L)$:
 - Konstante Zeit für Funktion `suche_nach_element`
 - wird immer durch das erste Element der Liste erreicht
- Average Case: Durchschnittslaufzeit gemittelt über alle Eingaben einer gewissen Länge, nämlich $\text{len}(L)$ Unser Hauptaugenmerk wird hierauf gerichtet
 - kann recht aussagekräftiges und realistisches Maß sein
- Worst Case: Maximallaufzeit über alle Eingaben einer gewissen Länge, nämlich $\text{len}(L)$
 - linear in Listenlänge für Funktion `suche_nach_element`
 - die komplette Liste muss notfalls durchlaufen werden

Ordnungen des Größwachstums

- die Programmeffizienz soll bewertet werden, wenn die Eingabelänge **sehr groß** bzw. **beliebig groß** ist
- das Wachstum der Programmlaufzeit soll **in Abhängigkeit der Eingabelänge** gemessen werden
- es soll eine **obere Schranke** angegeben werden, die so klein wie möglich ist
- wir müssen nicht sonderlich genau sein, die **Größenordnung** reicht völlig aus, genaue Werte sind unwichtig
- wir sollen herausfinden können **welcher Teil** des Programms zur Maximallaufzeit beiträgt
- **In einem Satz: Wir wollen eine möglichst klein wachsende Größenordnung für das Worst-Case Verhalten eines Programms in Abhängigkeit der Eingabelänge ermitteln**

Die O-Notation



- 1894 eingeführt vom deutschen Zahlentheoretiker Paul Bachmann
- Die O-Notation (großes O) gibt eine obere Schranke für das asymptotische Wachstum einer Funktion
- **Gross O oder $O()$** wird im Rahmen einer Worst-Case-Analyse angewendet
 - Wenn der schlechteste Fall eintritt, ist dieser der Flaschenhals einer Programmberechnung
 - drückt das Wachstum relativ zur Eingabelänge aus
 - bewertet **weder** die Maschine **noch** die Implementierung

Die O-Notation, formal

Seien $f, g : \mathbb{N} \rightarrow \mathbb{N}$ jeweils Funktionen von $\mathbb{N}$ nach $\mathbb{N}$.

Wir schreiben $f \in O(g)$ falls Konstanten $c \in \mathbb{R}^+$ und $n_0 \in \mathbb{N}$ existieren so, dass für alle $n \in \mathbb{N}$ mit $n > n_0$ gilt:

$$f(n) \leq c \cdot g(n)$$

Wir sagen dann „ f ist/liegt in $O(g)$ “

Beispiele:

- Seien $f(n) = 2n$ und $g(n) = n$ für alle $n \in \mathbb{N}$.

Dann gilt $f \in O(g)$ denn wir wählen $c = 2$ und $n_0 = 0$.

- Seien $f(n) = 4n + 4$ und $g(n) = n^2$ für alle $n \in \mathbb{N}$.

Dann gilt $f \in O(g)$ denn wir wählen $c = 1$ und $n_0 = 5$

und es gilt $4n + 4 \leq 1 \cdot n^2$ für alle $n > n_0 = 5$.

Wir schreiben der Einfachheit halber
gleich $O(n)$

Genaueres Wachstum vs $O()$

```
def fakultaet_iterativ(n):  
    antwort = 1  
    while n >= 1:  
        antwort *= n  
        n -= 1  
    return antwort
```

- Berechnet die Fakultätsfunktion
- Anzahl der Operationen: $5n+2$
- Asymptotische Worst-Case Komplexität: $O(n)$
 - additive Konstanten werden vernachlässigt
 - multiplikative Konstanten werden vernachlässigt

Was genau misst $O(n)$?

- uns interessiert das asymptotische Wachstum der Laufzeit während die Eingabegröße anwächst
- falls wir also einen Ausdruck (wie $5n+2$) für die Anzahl der benötigten Operationen vor uns haben, wollen wir wissen wie dieser Ausdruck asymptotisch wächst, wenn die Eingabegröße (n) anwächst
- daher fokussieren wir uns auf jenen Teil des Terms, der asymptotisch am stärksten anwächst
- wir ignorieren multiplikative und additive Konstanten, da diese unabhängig von der Eingabegröße sind

Beispiele für Vereinfachungen

- entferne Konstanten und multiplikative Faktoren
- der Fokus liegt auf den asymptotisch am stärksten anwachsenden Termen

$$n^2 + 2n + 3$$

$$\in O(n^2)$$

$$n^2 + 1002n + 3^{700000}$$

$$\in O(n^2)$$

$$\log(n) + n + 4$$

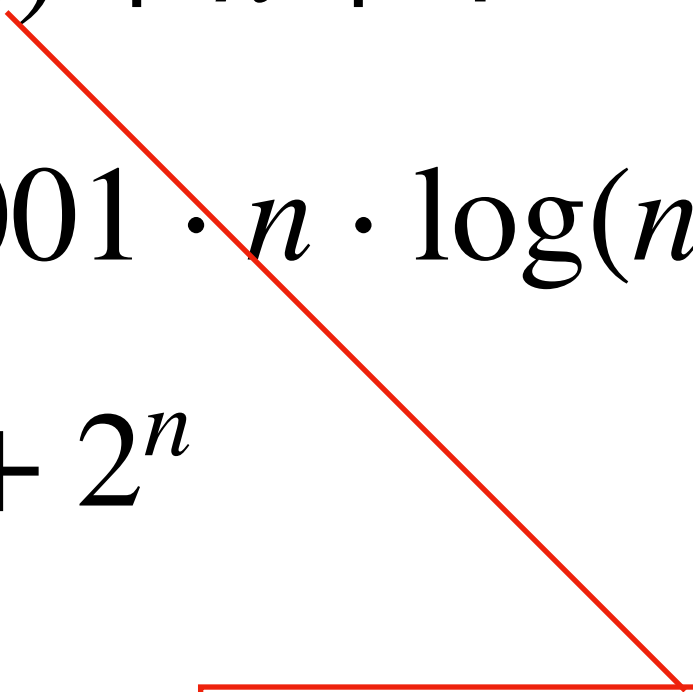
$$\in O(n)$$

$$0,00001 \cdot n \cdot \log(n) + 300n$$

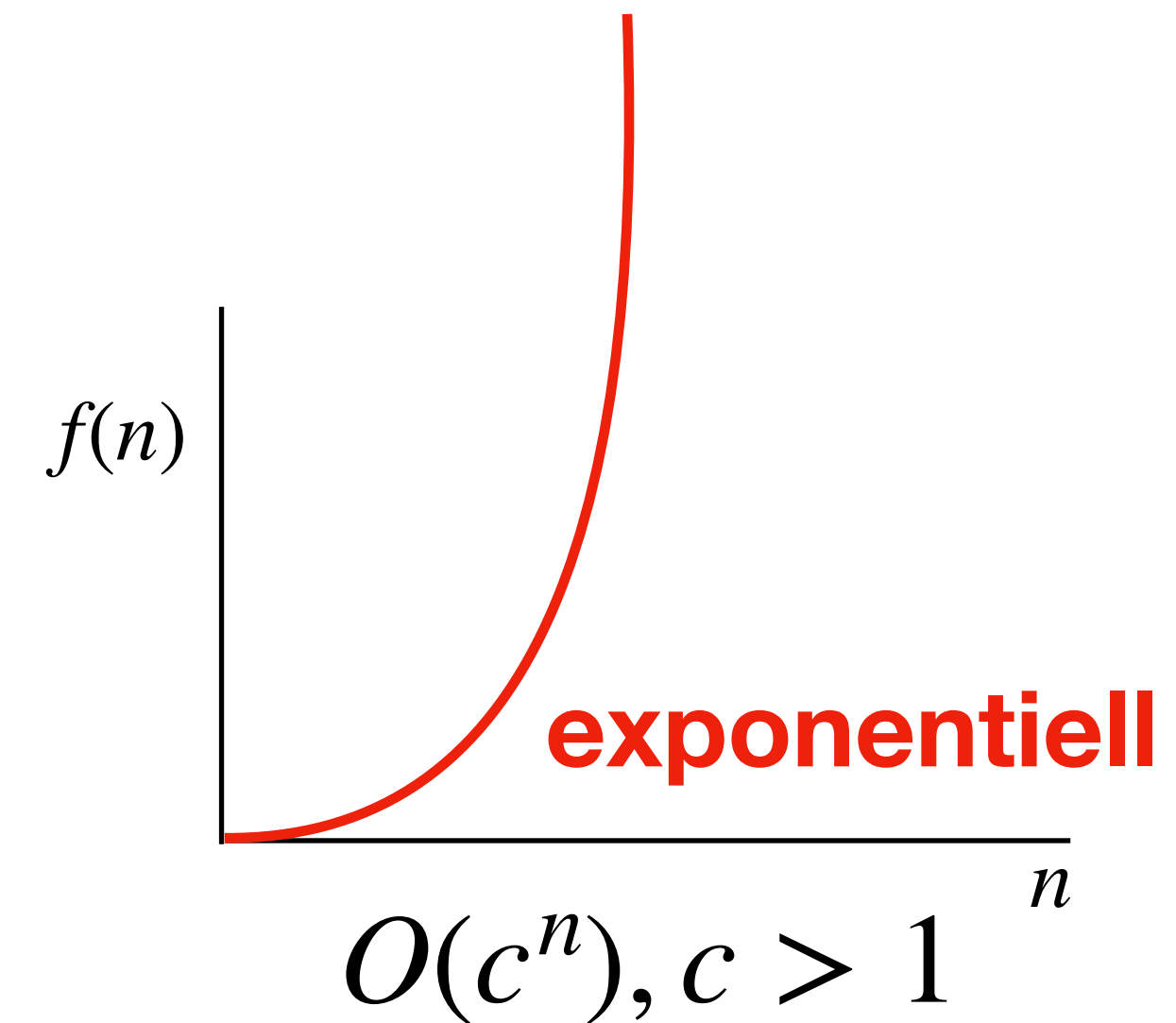
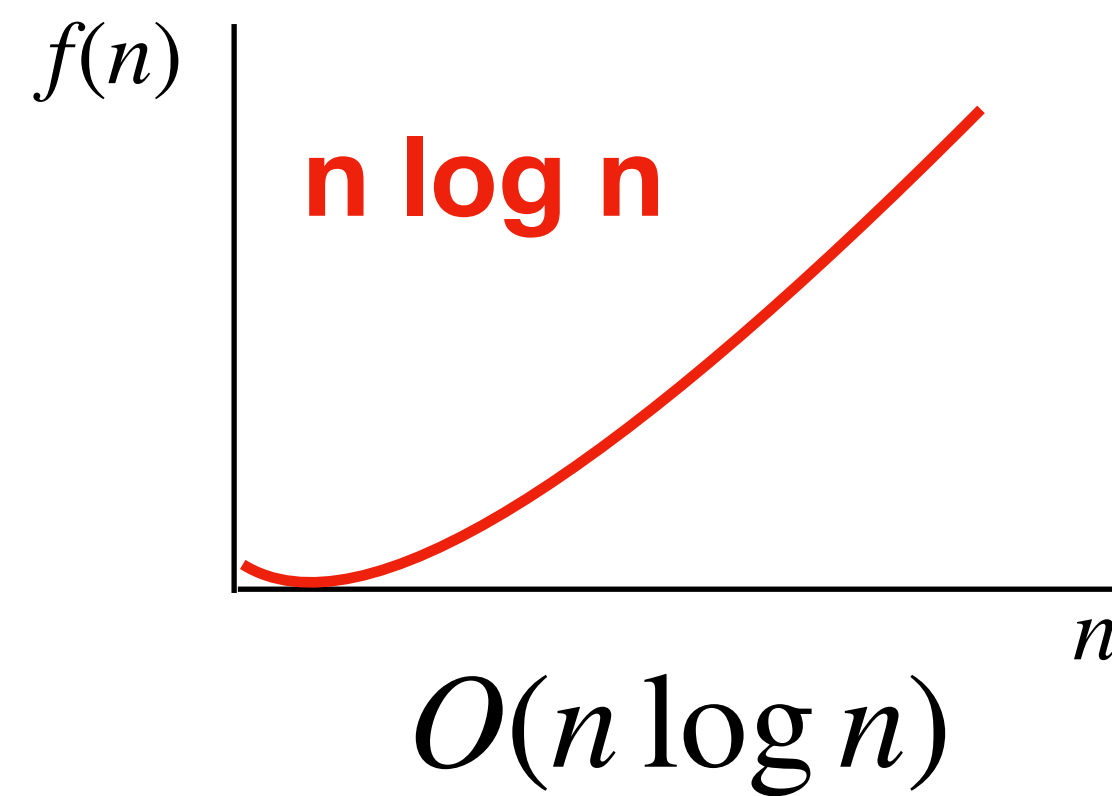
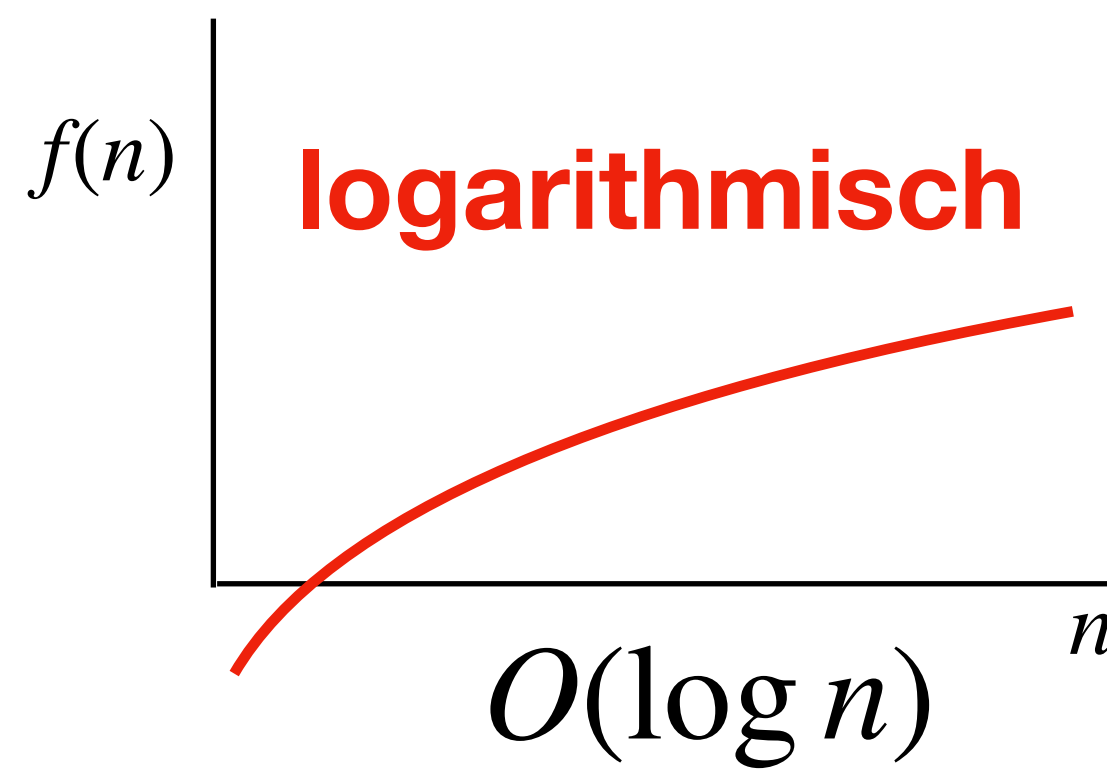
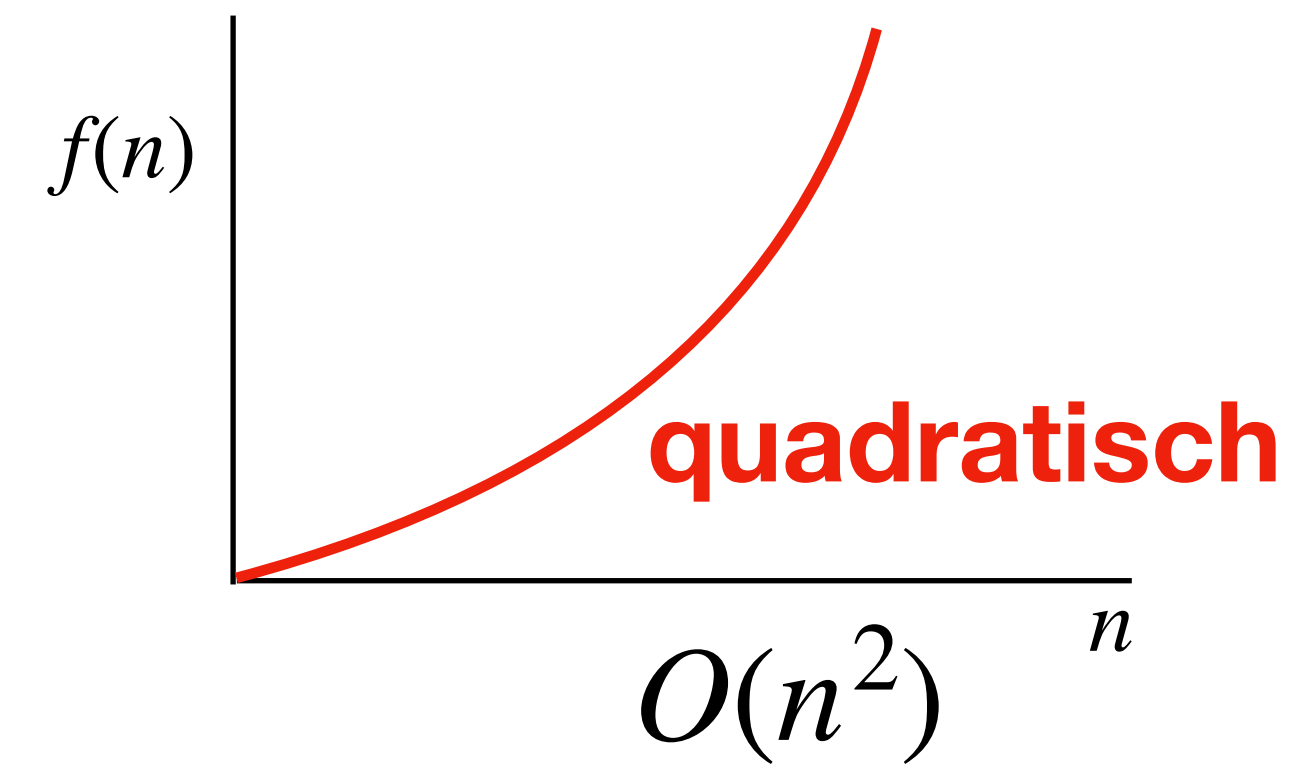
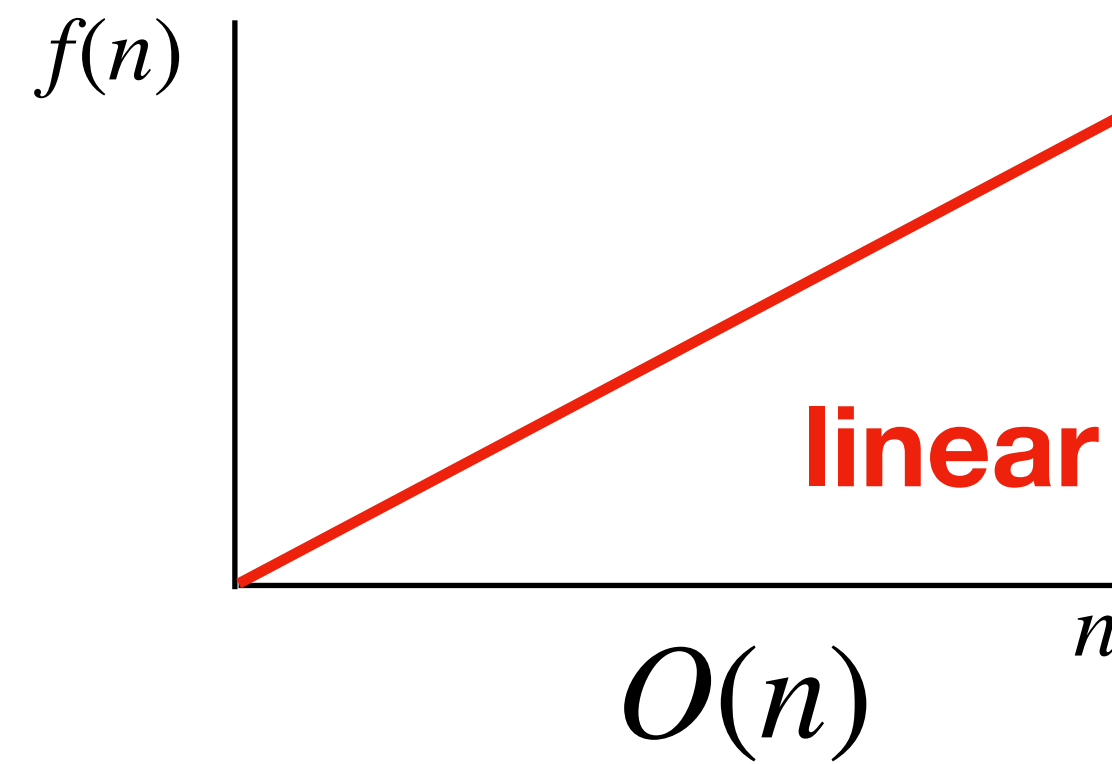
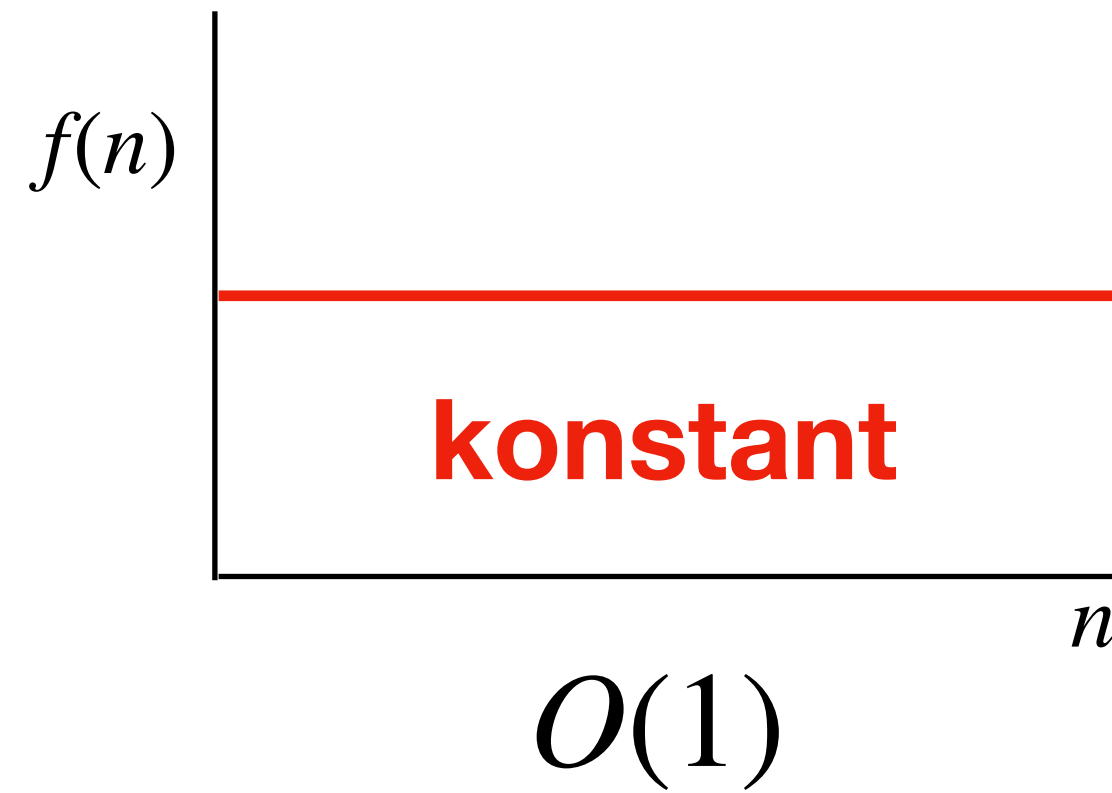
$$\in O(n \log n)$$

$$n^{300} + 2^n$$

$$\in O(2^n)$$


$$\log(n) = \min\{m \mid 2^m \geq n\}$$

Arten von Größenwachstum



Programme und ihre Komplexität analysieren

- **kombiniere** asymptotische Verhaltensweisen
 - Analysiere Verhaltensweisen innerhalb von Funktionen
 - wende gewisse Regeln an, Fokus auf dominanten Termen

Additionsgesetz für $O()$:

- wird oftmals für sequentiell Verhaltensweisen angewendet
- $O(f(n)) + O(g(n)) = O(f(n) + g(n))$
- Zum Beispiel:

```
for i in range(n) :  
    print("a")  
for j in range(n*n) :  
    print("b")
```

$O(n)$
 $O(n^2)$ } $O(n^2) + O(n) = O(n^2)$

ist $O(n^2) + O(n) = O(n^2)$ wegen des dominanten Terms

Programme und ihre Komplexität analysieren

- **kombiniere** asymptotische Verhaltensweisen
 - Analysiere Verhaltensweisen innerhalb von Funktionen
 - wende gewisse Regeln an, Fokus auf dominanten Termen

Additionsgesetz für $O()$:

- wird oftmals für sequentiell Verhaltensweisen angewendet
- $O(f(n)) + O(g(n)) = O(f(n) + g(n))$
- Zum Beispiel:

```
for i in range(n):  
    for j in range(n):  
        print("b")
```

**n Schleifendurchläufe
je $O(n)$**

$O(n)$

$\rightarrow O(n^2)$

ist **$O(n^2)$** da es **n** äußere Schleifendurchläufe gibt und jeder innere Schleifendurchlauf **n** Schritte benötigt.

Typische Laufzeitasymptotiken von Algorithmen

- $O(1)$ bezeichnet **konstantes Wachstum**
- $O(\log n)$ bezeichnet **logarithmisches Wachstum**
- $O(n)$ bezeichnet **lineares Wachstum**
- $O(n \log n)$ bezeichnet **log-lineares Wachstum**
- $O(n^2)$ bezeichnet **quadratisches Wachstum**
- $O(n^3)$ bezeichnet **kubisches Wachstum**
- **polynomielles Wachstum bedeutet** $O(n^c)$ für eine Konstante $c \geq 1$
- **Exponentielles Wachstum** $O(c^n)$ bedeutet für eine Konstante $c > 1$



aufsteigend sortiert

Asymptotik anschaulicher

Wachstum	n=10	n=100	n=1000	n=1000000
$O(1)$	1	1	1	1
$O(\log n)$	4	7	10	16
$O(n)$	10	100	1000	1000000
$O(n \log n)$	40	700	10000	160000000
$O(n^2)$	100	10000	10000000	1000000000000
$O(2^n)$	1024	1267650600 2282294014 9670320537 6	1071508607186267320948425 0490600018105614048117055 3360744375038837035105112 4936122493198378815695858 1275946729175531468251871 4528569231404359845775746 9857480393456777482423098 5421074605062371141877954 1821530464749835819412673 9876755916554394607706291 4571196477686542167660429 8316526243868372056680693 76	Viel Glück! (hat 301030 Stellen)

Lineare Laufzeit

```
def lineare_suche(L, e):  
    gefunden = False  
    for i in range(len(L)):  
        if e==L[i]:  
            gefunden = True  
    return gefunden
```

*hier könnten wir die Laufzeit verbessern, indem wir direkt **True** zurückgeben, aber dies würde den Worst Case nicht wirklich umgehen können*

- notfalls muss die komplette Liste durchlaufen werden, um festzustellen, dass ein Element nicht vorhanden ist
- $O(\text{len}(L))$ Schleifendurchläufe * $O(1)$ um $e==L[i]$ zu testen
 - $O(1+4n+1)=O(4n+2)=O(n)$
- Gesamtlaufzeit ist **$O(n)$** , wenn **n die Länge $\text{len}(L)$** bezeichnet

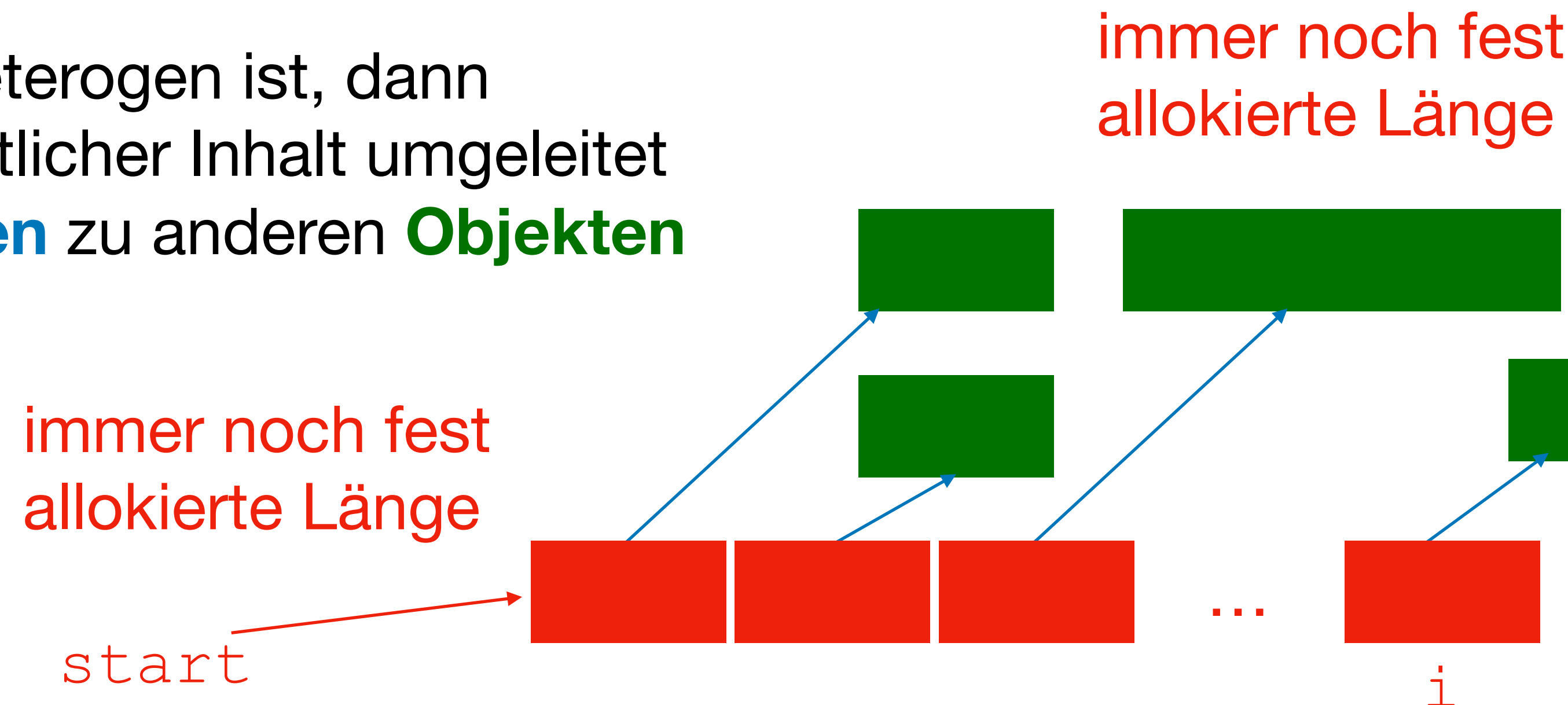
hier nehmen wir natürlich, an dass wir in konstanter Zeit auf ein Listenelement zugreifen können

Listenzugriff in $O(1)$

- wenn Liste **nur aus ints** besteht, dann ist das i -te Element an Position $\text{start} + 4 * i$



- falls Liste heterogen ist, dann
 - wird eigentlicher Inhalt umgeleitet
 - Referenzen** zu anderen **Objekten**



Lineare Laufzeit auf unsortierten Listen

```
def komische_suche(L, e):  
    for i in range(len(L)):  
        if e==L[i]:  
            return True  
        if L[i]>e:  
            return False  
    return False
```

im Worst Case muss die Liste immer noch komplett durchlaufen werden

- es muss lediglich gewartet werden, bis eine Zahl größer e erreicht wird (um abubrechen) oder davor abzulehnen
- $O(\text{len}(L))$ für die Schleife * $O(1)$ für den Test $e==L[i]$
- **Gesamtlaufzeit ist $O(n)$** wenn n die Länge $\text{len}(L)$ bezeichnet
- **Man beachte:** Die Größenordnung der Laufzeit dieses Algorithmus für den Worst Case ist dieselbe wie die des vorherigen Algorithmus

Lineare Laufzeit

wenn wir nichts weiteres über diese Liste wissen (z.B. dass sie sortiert vorliegt)

- ist im Allgemeinen notwendig, um ein Element in einer Liste zu finden
- Beispiel: Vertausche eine Zeichenkette

```
def spiegeln(s):  
    ergebnis=""  
    for c in s:  
        ergebnis = c + ergebnis  
    return ergebnis
```

- Laufzeit($O(\text{len}(s))$)

Lineare Laufzeit

- Komplexität hängt oftmals von der Anzahl der Iterationen ab

```
def fakultaet_iterativ(n):  
    prod = 1  
    for i in range(1, n+1):  
        prod *= i  
    return prod
```

- Anzahl der Schleifendurchläufe ist n
- Anzahl der Operationen innerhalb einer Schleife ist konstant (hier 4: inkrementiere i , weise Summe i zu, multipliziere, weise Produkt zu)
 - $O(1+4n+1)=O(4n+2)=O(n)$
- insgesamt nur **$O(n)$**

Verschachtelte Schleifen

aber nicht immer!

- **einfache Schleifen** bedeuten oftmals **lineare Laufzeit**
- wie sieht es mit **verschachtelten Schleifen** (d.h. Schleifen in Schleifen) aus?
- Generelle Antwort: Kann manchmal schwierig aber oftmals auch einfach zu analysieren sein

Quadratische Komplexität

*Listen sind keine Mengen sondern
Sequenzen!*

- **Beispiel:** Bestimme ob die erste Liste „in der zweiten enthalten ist“, genauer ob jedes Element der ersten Liste auch in der zweiten Liste irgendwo vorkommt (ohne Vielfachheiten zu zählen)

```
def istEnthalten(L1,L2):  
    for e1 in L1:  
        gefunden = False  
        for e2 in L2:  
            if e1 == e2:  
                gefunden = True  
                break  
        if not gefunden:  
            return False  
    return True
```

Quadratische Komplexität

```
def istEnthalten(L1,L2):  
    for e1 in L1:  
        gefunden = False  
        for e2 in L2:  
            if e1 == e2:  
                gefunden = True  
                break  
        if not gefunden:  
            return False  
    return True
```

- äußere Schleife wird **len (L1)** Mal durchlaufen
- jede Iteration (jeder Schleifendurchlauf) führt **len (L2)** innere Iterationen durch, die wiederum jeweils konstant viele Operationen durchführen

$O(\text{len}(L1) * \text{len}(L2))$

- Worst Case tritt bspw. ein, wenn alle Elemente in L1 gleich einem Element sind und dieses Element in L2 nur an letzter Position auftaucht

Quadratische Komplexität

*Listen sind keine Mengen sondern
Sequenzen!*

- **Beispiel:** Bestimme den „Durchschnitt“ zweier Listen, genauer eine Sequenz der Elemente, die sowohl mindestens einmal in beiden Listen vorkommen ohne Vielfachheiten (d.h. die Ergebnisliste darf jedes Element nur einmal enthalten)

```
def schneide(L1, L2):  
    tmp = []  
    for e1 in L1:  
        for e2 in L2:  
            if e1 == e2:  
                tmp.append(e1)  
                break  
    ergebnis = []  
    for e in tmp:  
        if not (e in ergebnis):  
            ergebnis.append(e)  
    return ergebnis
```


Quadratische Komplexität

```
def schneide(L1, L2):  
    tmp = []  
    for e1 in L1:  
        for e2 in L2:  
            if e1 == e2:  
                tmp.append(e1)  
                break  
    ergebnis = []  
    for e in tmp:  
        if not (e in ergebnis):  
            ergebnis.append(e)  
    return ergebnis
```

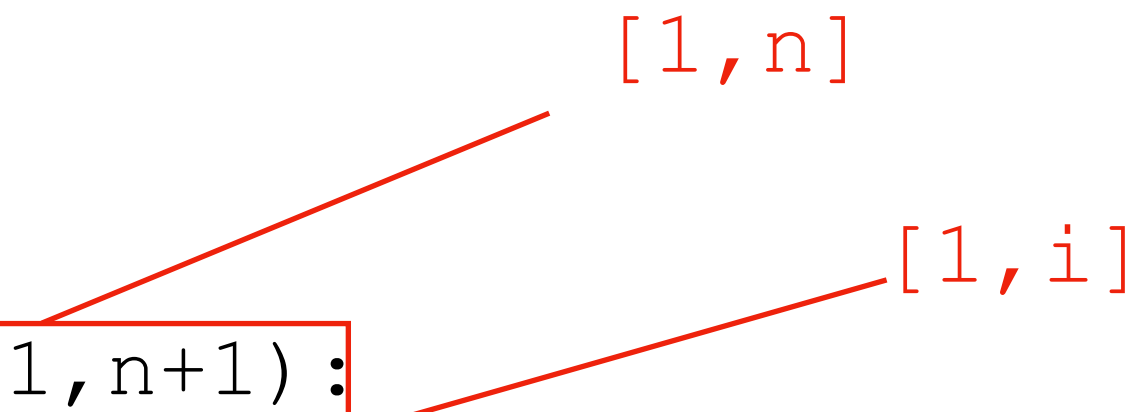
Ist $O(\text{len}(L1))$

wird $O(\text{len}(L1))$ mal durchlaufen

- erste (geschachtelte) Schleife hat eine Gesamtlaufzeit von $O(\text{len}(L1) * \text{len}(L2))$
 - zweite Schleife hat Laufzeit $O(\text{len}(L1)^2)$
- $O(\text{len}(L1) * \max(\text{len}(L1), \text{len}(L2)))$
- Worst Case tritt bspw. ein, wenn es ein Element e gibt so, dass $L1$ die Form $[e, \dots, e]$ hat und $L2$ die Form $[e]$ hat oder auch wenn $L1$ und $L2$ gleiche Länge haben und disjunkt sind.

Laufzeit geschachtelter Schleifen

```
def g(n):  
    x = 0  
    for i in range(1, n+1):  
        for j in range(1, i+1):  
            x += 1  
    return x
```

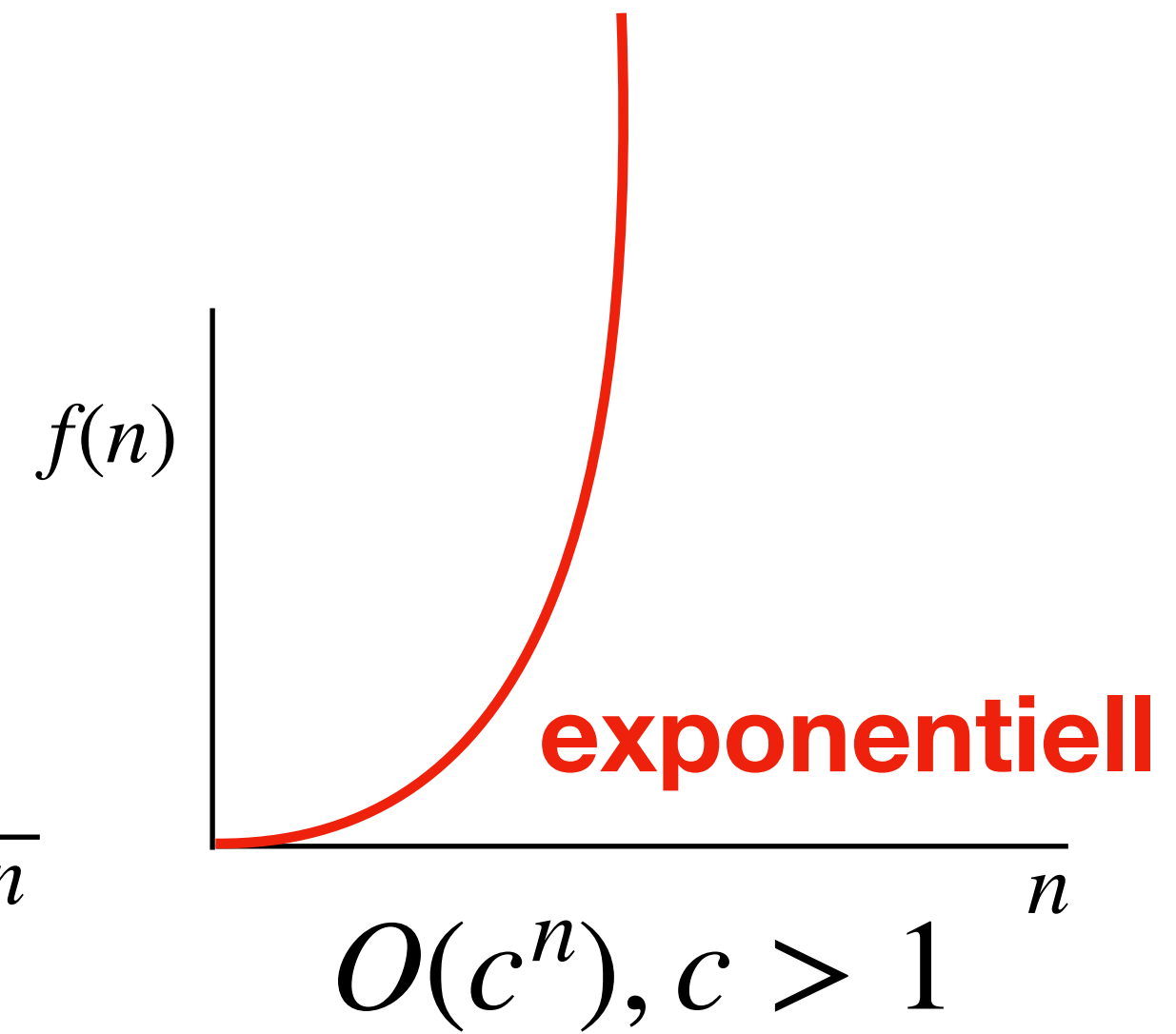
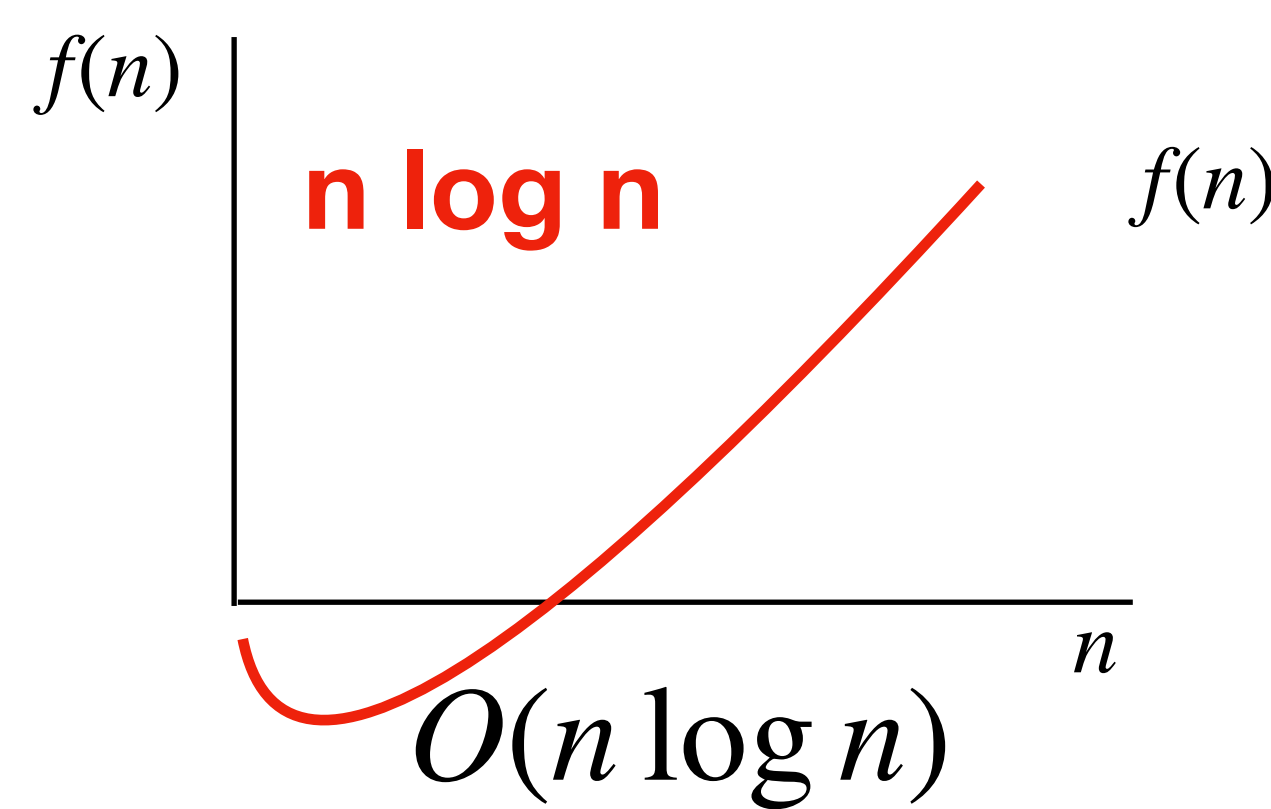
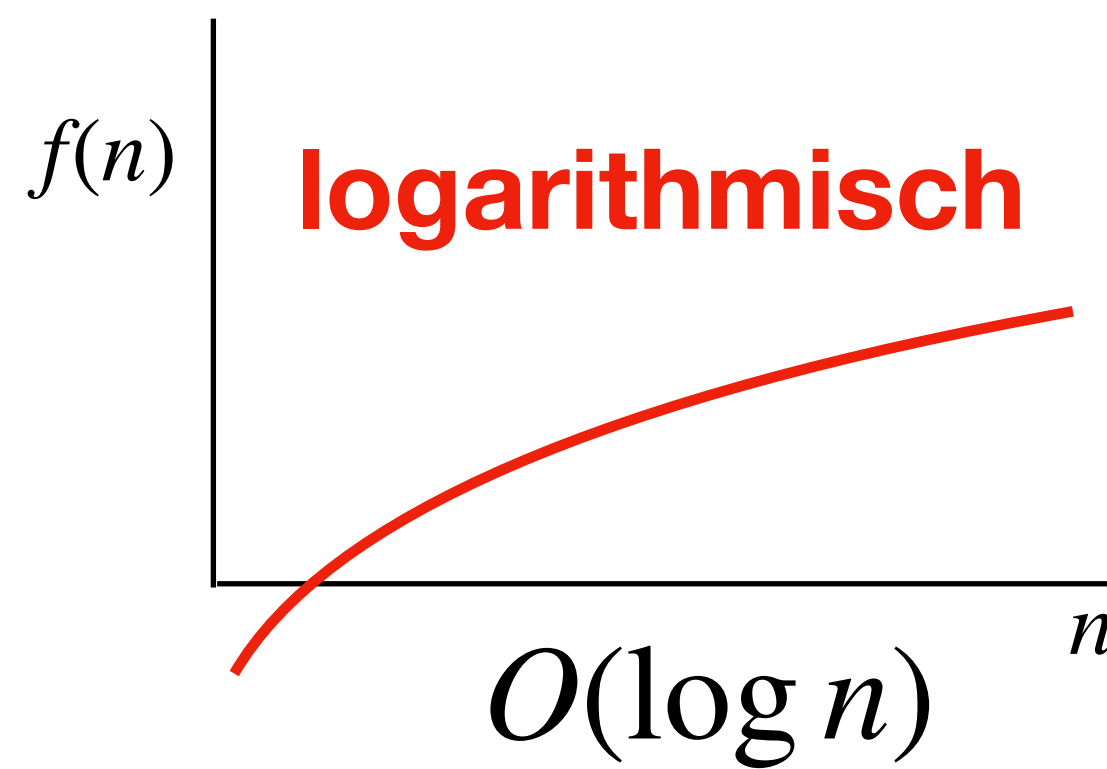
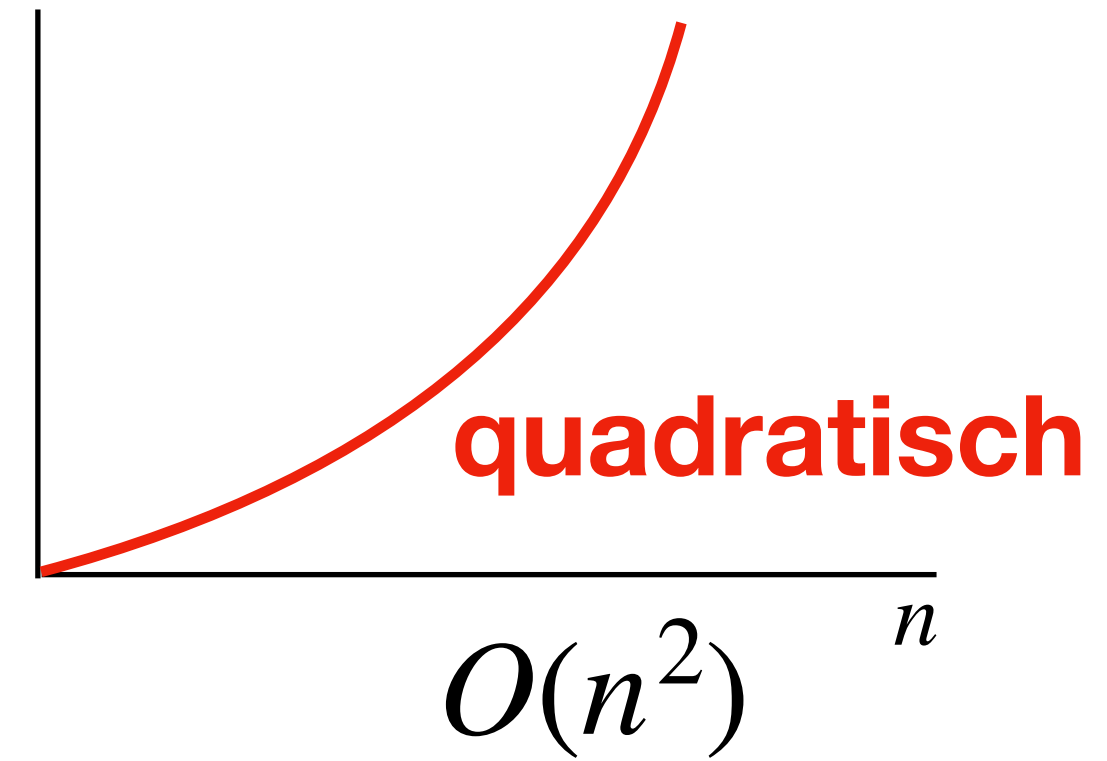
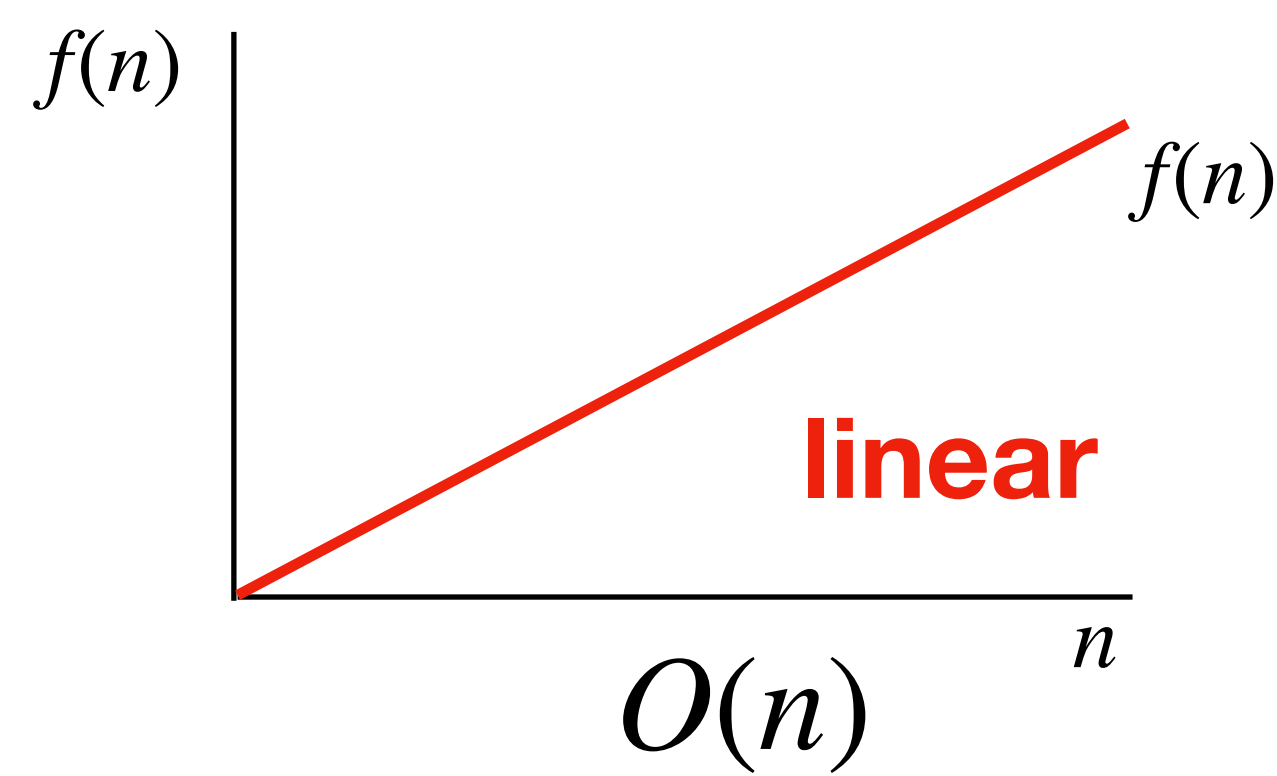
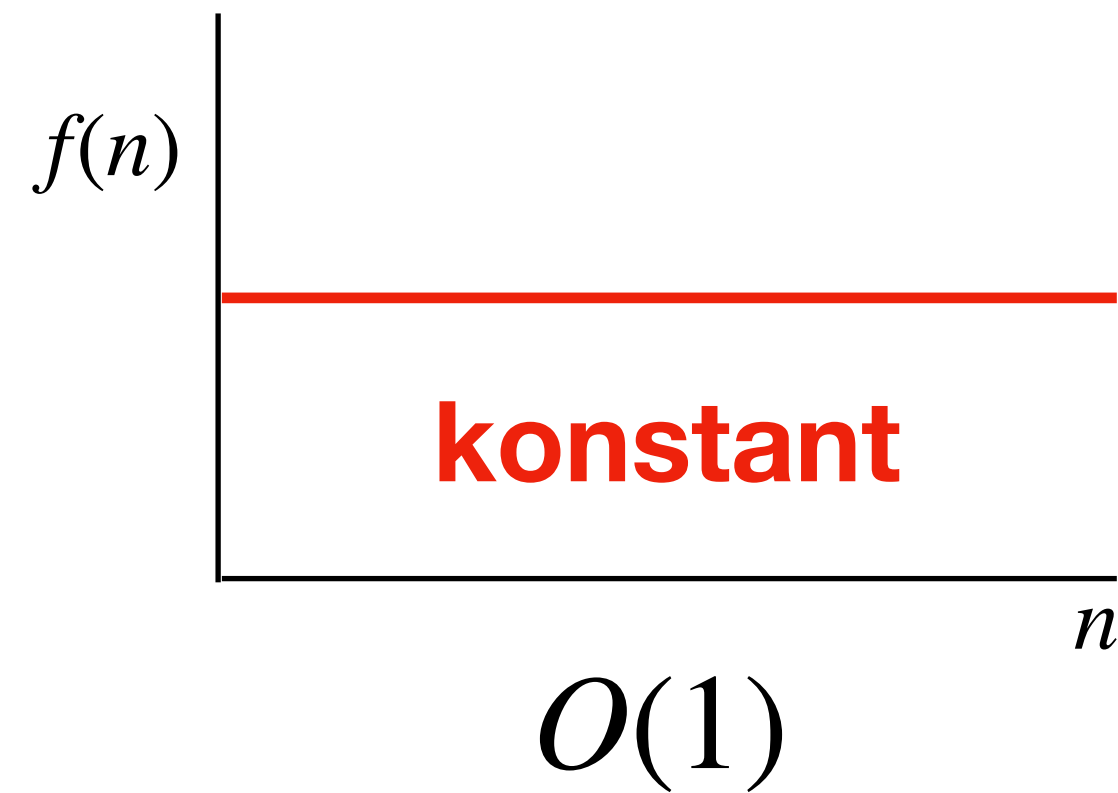


- was berechnet dieses Programm?
- es berechnet $1 + 2 + 3 + \dots + n$
- es gibt n äußere Schleifendurchläufe
- beim i -ten äußeren Schleifendurchlauf gibt es i innere Schleifendurchläufe
- Erinnerung:

$$\sum_{i=1}^n i = n \cdot (n + 1) / 2$$

- **Gesamtlaufzeit: $O(n^2)$**

Arten von Größenwachstum



Typische Laufzeitasymptotiken von Algorithmen

- $O(1)$ bezeichnet **konstantes Wachstum**
- $O(\log n)$ bezeichnet **logarithmisches Wachstum**
- $O(n)$ bezeichnet **lineares Wachstum**
- $O(n \log n)$ bezeichnet **log-lineares Wachstum**
- $O(n^2)$ bezeichnet **quadratisches Wachstum**
- $O(n^3)$ bezeichnet **kubisches Wachstum**
- **polynomielles Wachstum** bedeutet $O(n^c)$ für eine Konstante $c \geq 1$
- **Exponentielles Wachstum** $O(c^n)$ bedeutet für eine Konstante $c > 1$



aufsteigend sortiert

Konstante Laufzeit

- bedeutet, dass die Laufzeit unabhängig von der Eingabelänge ist
- insbesondere kann nicht die komplette Eingabe berücksichtigt werden
- Schleifen durchaus auch bei konstanter Laufzeit möglich, wenn die Anzahl der Durchläufe bzw. die Gesamtlaufzeit nicht von der Eingabe abhängt
- es gibt kaum interessante Algorithmen mit konstanter Laufzeit

Logarithmische Laufzeit

- auch bei solchen Algorithmen wird nicht die komplette Eingabe berücksichtigt
- Übungsaufgabe: $\forall c, \forall d \in \mathbb{R}, c > 1, d > 1, O(\log_c(n)) = O(\log_d(n))$
- **Binäre Suche hat logarithmische Laufzeit**, da in jeder Iteration das zu explorierende Intervall **halbiert** wird
- nächstes Beispiel hierfür: Bisection Search

Bisection Search

- nehmen wir an wir wollen ein bestimmtes Element e in einer gegebenen Liste $\mathbb{L}$ suchen
- wir haben bereits einen Algorithmus (Sequenzielle Suche) kennengelernt, der die komplette Liste durchläuft, dieser hat eine Laufzeit $O(n)$, wobei n die Länge der Liste beschreibt.
- nehmen wir nun an, dass die Liste aufsteigend sortiert ist
 - wir können nach wie vor sequentiell suchen, $O(n)$
 - können wir ein (laufzeit-)effizienteres Verfahren angeben?

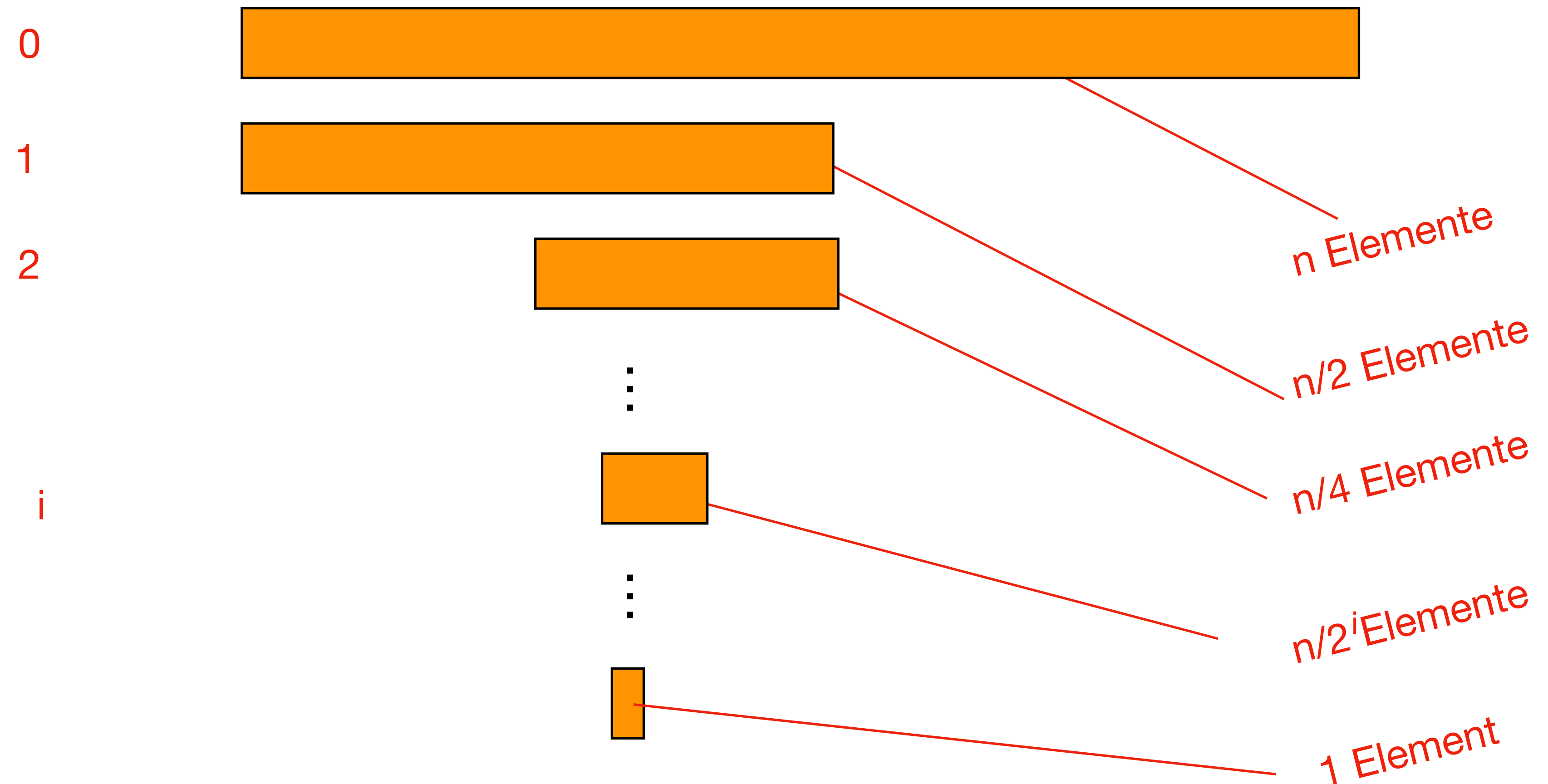
Bisection Search, Idee

1. wähle einen Index i , der die Liste in zwei Teile teilt
2. teste ob $L[i] == e$
3. falls ungleich, teste ob $L[i] < e$ oder ob $L[i] > e$
4. abhängig vom Ergebnis wird entweder in der linken oder der rechten Hälfte gesucht

Dies entspricht dem **Divide-and-Conquer Algorithmus**

- teile Problem in kleinere Teilprobleme (hier: kleinere Liste) plus ein paar elementare Operationen (hier: Vergleiche)
- Falls noch nicht fertig, fahre mit kleinerem Problem fort

Analyse von Bisection Search



- wir sind erst fertig, wenn das erste Mal gilt

$$1 \geq n/2^i \implies 2^i \geq n \implies i \geq \log(n)$$

- Gesamtlaufzeit **$O(\log(n))$**

Bisection Search, Algorithmus 1

```
def bisection1(L, e):
```

```
    if L == []:  
        return False
```

konstant
 $O(1)$

```
    elif len(L) == 1:  
        return L[0] == e
```

konstant
 $O(1)$

```
    else:
```

```
        haelfte = len(L) // 2
```

konstant
 $O(1)$

```
        if L[haelfte] > e:  
            return bisection1(L[:haelfte], e)
```

```
        else:  
            return bisection1(L[haelfte:], e)
```

Liste wird kopiert!

nicht konstant
 $O(\text{len}(L))$

nicht konstant
 $O(\text{len}(L))$

Laufzeit-Analyse Algorithmus 1

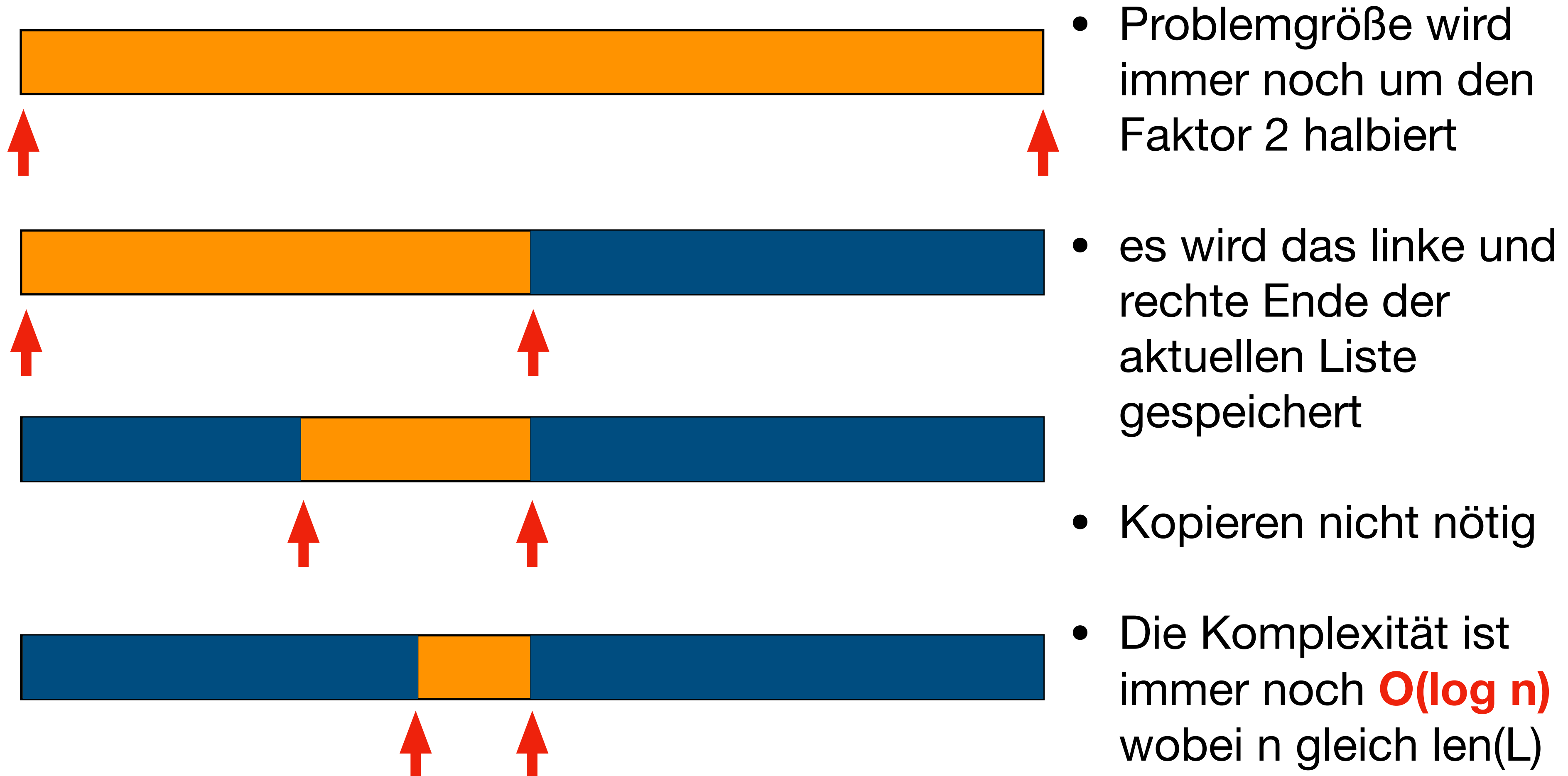
- $O(\log n)$ Rekursionsaufrufe
 - in jedem Rekursionsaufruf wird Listenlänge halbiert
 - wenn die ursprüngliche Liste Länge n hat, dann ist im Worst Case die Listenlänge irgendwann 1, also wenn $n/2^k \leq 1$
- $O(n)$ innerhalb des ersten Funktionsaufrufs, um die halbe Liste (die linke oder rechte Hälfte) zu kopieren
 - genauer $n/2$ für den ersten Aufruf, $n/4$ für den zweiten, ...
 $n/2^k$ für den k -ten
- Worst-Case Analyse: $O(\log n) \cdot O(n) = O(n \log n)$

- Genauere Analyse:

$$\sum_{k=1}^{\infty} n/2^k \leq n \quad \rightarrow \quad O(n)$$

also nicht wirklich eine Verbesserung zur sequentiellen Suche

Analyse von Bisection Search



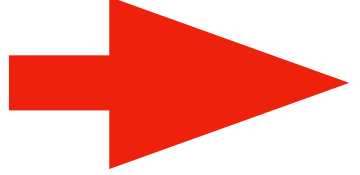
Bisection Search Algorithmus 2

```
def bisection2(L,e):  
    def bisection_hilfe(L, e, links, rechts):  
        if links >= rechts:  
            return links==rechts and L[links] == e  
        mitte = (links + rechts) // 2  
        if L[mitte] == e:  
            return True  
        elif L[mitte] > e:  
            return bisection_hilfe(L,e,links,mitte-1)  
        else:  
            return bisection_hilfe(L,e,mitte+1,rechts)  
    if len(L) == 0:  
        return False  
    else:  
        return bisection_hilfe(L,e,0,len(L)-1)
```

*konstante statt
lineare Laufzeit*

*konstante statt
lineare Laufzeit*

Laufzeit-Analyse Algorithmus 2

- $O(\log n)$ Rekursionsaufrufe
 - in jedem Rekursionsaufruf wird Listenlänge halbiert
 - wenn die ursprüngliche Liste Länge n hat, dann ist im Worst Case die Listenlänge irgendwann 1, also wenn $n/2^k \leq 1$
- linker und rechter Index (und ursprüngliche Liste und gesuchtes Element) werden übergeben
- Liste wird nicht kopiert, nur linker und rechter Index werden ggf. neu gesetzt
- innerhalb eines Funktionsaufrufs also Laufzeit $O(1)$
- Naive Analyse: $O(\log n) * O(1)$  **$O(\log n)$**

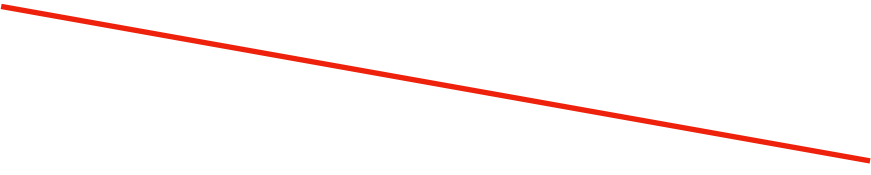
Rekursionstiefe

Logarithmische Laufzeit

```
def intnachstr(n):  
    ziffern = '0123456789'  
    if n==0:  
        return '0'  
    else:  
        return intnachstr(n//10)+ziffern[n%10]
```

Was macht folgendes Programm?

Es wandelt eine Zahl in einen String zur Basis 10 um.



inklusive führender
Nullen

Logarithmische Laufzeit

```
def intrnachstr(n):  
    ziffern = '0123456789'  
    if n==0:  
        return '0'  
    else:  
        return intrnachstr(n//10)+ziffern[n%10]
```

- Konstante Laufzeit innerhalb eines Funktionsaufrufs
- Bei rekursivem Aufruf wird die Eingabe verkleinert, nämlich durch 10 geteilt (Integer-Division)
- Rekursionstiefe entspricht die Anzahl der Male, die man n durch 10 teilen kann, um schließlich 0 zu werden.
- **$O(\log n)$**

Nochmal lineare Laufzeit

```
def fact_iter(n):  
    prod = 1  
    for i in range(1, n+1):  
        prod *= i  
    return prod
```

- insgesamt **$O(n)$** Schleifendurchläufe (genau n), konstante Zeit pro Schleifendurchlauf

Nochmal lineare Laufzeit

```
def fakultaet(n):  
    if n==1:  
        return 1  
    else:  
        return n*fakultaet(n-1)
```

- berechnet die Fakultätsfunktion rekursiv
- genau genommen (bei genauem Messen mittels Stoppuhr) kann man feststellen, dass die mittels Stoppuhr gemessene Zeit langsamer als die iterative Variante ist
- jedoch immer noch **$O(n)$** , denn die Rekursionstiefe ist linear in n , mit konstanter Laufzeit innerhalb eines Funktionsaufrufs
- Iterative und rekursive Implementierungen für die Fakultätsfunktion haben dieselbe asymptotische Laufzeit, nämlich $O(n)$

Log-lineare Laufzeit

- viele praktische Algorithmen haben log-lineare Laufzeit, also $O(n \log n)$
- häufig verwendete log-lineare Algorithmus ist Merge-Sort (später mehr)
- ist die inhärente^{???} Laufzeit für Sortieren (später mehr)

Polynomielle Laufzeit

- viele nicht-lineare Algorithmen haben quadratische Laufzeit
- entsteht häufig, wenn wir geschachtelte Schleifen haben
- haben wir **bisher** kennengelernt
- entspricht im Wesentlichen dem, was man als „effizient berechenbar“ bezeichnet (superpolynomielle bezeichnet man typischerweise als ineffizient)

Exponentielle Laufzeit

- Die meisten rekursiven Algorithmen besaßen nur einen rekursiven Aufruf. (keine streng mathematische Aussage, mit Vorsicht zu genießen!)
 - Bei den Türmen von Hanoi und der rekursiven Berechnung von `fib` (ohne Wörterbuch) war das anders: Zwei rekursive Aufrufe

```
def Tuerme(n, von, nach, zwischen):
```

```
    if n==1:
```

```
        print(von+ " -> " + nach)
```

```
    else:
```

```
        Tuerme(n-1, von, zwischen, nach)
```

```
        Tuerme(1, von, nach, zwischen)
```

```
        Tuerme(n-1, zwischen, nach, von)
```

```
def fib(n):
```

```
    if n==0 or n==1:
```

```
        return 1
```

```
    else:
```

```
        return fib(n-1) + fib(n-2)
```

(Genaue Laufzeit werden wir noch exakt bestimmen)

- Einige Probleme sind vermutlich inhärent exponentiell, bei den meisten von ihnen ist das aber tatsächlich noch nicht bekannt!!!
 - sehr schade, da man sehr lange auf Ergebnisse warten muss
 - approximative Lösungen liefern uns oftmals trotzdem zufriedenstellende (aber nicht exakte) Lösungen mit denen wir etwas anfangen können

Laufzeit von Türme von Hanoi

- Bezeichne t_n die Laufzeit, um Türme der Höhe n zu berechnen
- Es gilt folgende Rekurrenz $t_1 = 1$ und $t_n = 2t_{n-1} + 1$ für alle $n > 1$

$$\begin{aligned}\text{also } t_n &= 2t_{n-1} + 1 \\ &= 2(2t_{n-2} + 1) + 1 \\ &= 4t_{n-2} + 2 + 1 \\ &= 4(2t_{n-3} + 1) + 2 + 1 \\ &= 2^{n-1} + 2^{n-2} + \dots + 4 + 2 + 1 \\ &= 2^n - 1\end{aligned}$$

- die Gesamtlaufzeit ist also **$O(2^n)$**

Exponentielle Laufzeit

- gegeben sei eine Menge von Integers (d.h. ohne Wiederholungen) gegeben als Liste
- wir wollen die Menge aller Teilmengen (Untermengen) berechnen
- bspw. besitzt die Menge $\{1,2,3,4\}$ folgende Untermengen:
 - $\{\}, \{1\}, \{2\}, \{3\}, \{4\}, \{1,2\}, \{1,3\}, \{1,4\}, \{2,3\}, \{2,4\}, \{3,4\},$
 $\{1,2,3\}, \{1,2,4\}, \{1,3,4\}, \{2,3,4\}, \{1,2,3,4\}$
- man beachte, dass es nicht auf die Ordnung der Elemente ankommt also bspw. $\{1,2,3,4\}=\{3,2,1,4\}$ oder $\{1,2\}=\{2,1\}$

Potenzmengenkonstruktion, Idee

- wie wollen die Potenzmenge (Menge aller Untermengen) von $\{1, \dots, n\}$ generieren
- nehmen wir an wir könnten bereits die Potenzmenge von $\{1, \dots, n-1\}$ generieren
- dann ist die Potenzmenge von $\{1, \dots, n\}$ die Menge aller Mengen, die entsteht, wenn für jede Menge X in der Potenzmenge von $\{1, \dots, n-1\}$ sowohl X als auch $X \cup \{n\}$ aufnehmen
 - $\{\{\}, \{1\}, \{2\}, \{1,2\}, \{3\}, \{1,3\}, \{2,3\}, \{1,2,3\}, \{4\}, \{1,4\}, \{2,4\}, \{1,2,4\}, \{3,4\}, \{1,3,4\}, \{2,3,4\}, \{1,2,3,4\}\}$
- liefert ein einfaches rekursives Verfahren

Potenzmengenkonstruktion

Algorithmus

```
def generiereUntermenge(L):
```

```
    ergebnis = []
```

```
    if len(L) == 0:
```

```
        return [[]]
```

```
    kleiner = generiereUntermenge(L[:-1])
```

```
    extra = L[-1:]
```

```
    neu = []
```

```
    for klein in kleiner:
```

```
        neu.append(klein+extra)
```

```
    return kleiner + neu
```

Liste, die nur eine
leere Liste enthält

Eingabeliste ohne das letzte Element

Einelementige Liste, die nur das letzte Element enthält

Potenzmengenkonstruktion

Laufzeitanalyse

```
def generiereUntermenge(L):  
    ergebnis = []  
    if len(L) == 0:  
        return [[]]  
    kleiner = generiereUntermenge(L[:-1])  
    extra = L[-1:]  
    neu = []  
    for klein in kleiner:  
        neu.append(klein+extra)  
    return kleiner + neu
```

wir nehmen an append
hat Laufzeit $O(1)$

wesentliche Gesamtlaufzeit:

Laufzeit für das kleinere Problem plus die Laufzeit, um Lösung des kleineren Problems einmal zu duplizieren.

Potenzmengenkonstruktion

Laufzeitanalyse

- bezeichne t_n die Laufzeit für Eingabegröße n
- bezeichne s_n die Größe der Lösung für Eingabegröße n
- es gilt folgende Rekurrenz: $t_n \leq t_{n-1} + c \cdot (s_{n-1} + n)$
für eine Konstante c

- es gilt $t_n \leq t_{n-1} + c \cdot (s_{n-1} + n)$
 $= t_{n-1} + c \cdot (2^{n-1} + n)$

$$s_0 = 1 \text{ und } s_n = 2s_{n-1} \text{ für alle } n > 0$$

$$\text{also } s_n = 2^n \text{ für alle } n \geq 0$$

$$\leq t_{n-2} + c \cdot (s_{n-2} + n - 1) + c \cdot (2^{n-1} + n)$$

$$\leq t_{n-k} + c \cdot \left(2^{n-k} + \dots + 2^{n-1} + 2 \cdot \sum_{i=n-k+1}^n i \right)$$

$$= \boxed{t_0} + c \cdot (2^0 + \dots + 2^{n-1} + n(n+1))$$

2

$$= 2 + c \cdot (2^n - 1 + n(n+1))$$

liegt in $O(2^n)$

Typische Algorithmen für typische Laufzeiten

- $O(1)$ - Programm hängt nicht von der Größe des Problems ab
- $O(\log n)$ - Verkleinere Eingabe wiederholt durch einen konstanten Faktor größer 1
- $O(n)$ - Einfache iterative oder rekursive Programme
- $O(n \log n)$ - sehen wir gleich
- $O(n^c)$ - mehrere geschachtelte Schleifen oder rekursive Aufrufe
- $O(c^n)$ - mehrere Rekursionsaufrufe in jeder Stufe

Wieder keine streng mathematische Aussage, eher eine Tendenz!
Es gibt einige Gegenbeispiele!

Fibonaccizahlen iterativ

```
def fib_iter(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    elif n == 1:
```

```
        return 1
```

```
    else:
```

```
        fib_i = 1
```

```
        fib_ii = 1
```

```
        for i in range(n-1):
```

```
            tmp = fib_i
```

```
            fib_i = fib_ii
```

```
            fib_ii = tmp + fib_i
```

```
        return fib_ii
```

konstant
 $O(1)$

konstant
 $O(1)$

linear
 $O(n)$

konstant
 $O(1)$

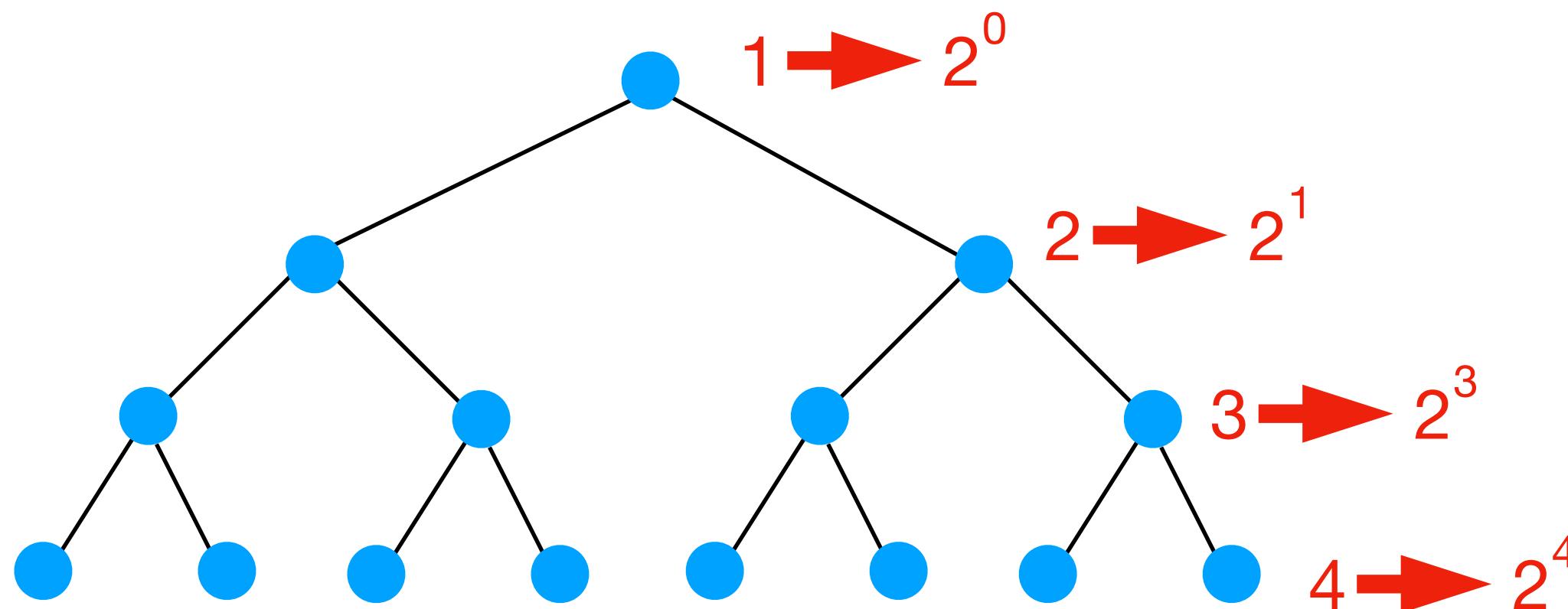
- Worst Case:
 $O(n)$

Laufzeit von Fibonacci rekursiv

(ohne Wörterbuch)

```
def fib(n):  
    if n==0 or n==1:  
        return 1  
    else:  
        return fib(n-1)+fib(n-2)
```

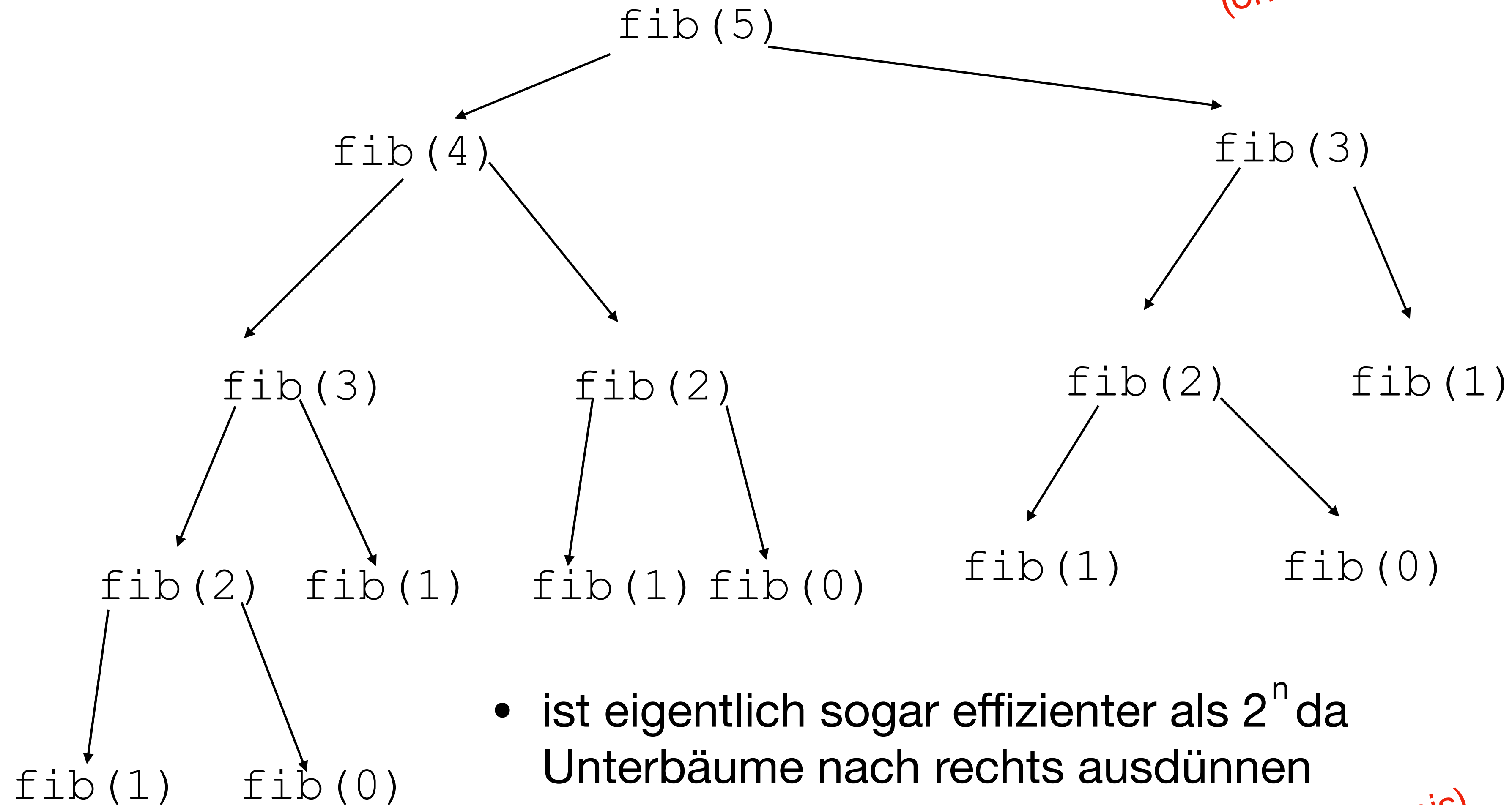
- Worst Case **$O(2^n)$** denn $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2) \leq 2 \cdot \text{fib}(n-1)$



entspräche $\text{fib}(n) = 2^n$

Laufzeit von Fibonacci rekursiv

(ohne Wörterbuch)



- ist eigentlich sogar effizienter als 2^n da Unterbäume nach rechts ausdünnen

„Goldener Schnitt“

- es gilt $\lim_{n \rightarrow \infty} \frac{\text{fib}(n+1)}{\text{fib}(n)} = \frac{1 + \sqrt{5}}{2} \approx 1,61803$ (ohne Beweis)

Zusammenfassung O-Notation

- dient die Effizienz eines Algorithmus zu beschreiben
 - ist eine Notation, die das Wachstum beschreibt
 - je kleiner die Größenordnung, desto effizienter
 - maschinen- und implementierungsunabhängig
- Verwenden Sie die O-Notation
 - um die Größenordnung der Laufzeit zu klassifizieren
 - als eine asymptotische Notation
 - als eine obere Schranke
 - bei Worst Case Analysen (meistens dort anzutreffen)

Laufzeit einiger Python-Funktionen

- **Listen** L , n sei $\text{len}(L)$

Worst Case

- | | |
|--|--------|
| • Zugriff ($L[i]$) | $O(1)$ |
| • Speichern (Zuweisung) | $O(1)$ |
| • Länge bestimmen (len) | $O(1)$ |
| • Am Ende anhängen (append) | $O(1)$ |
| • $==$ | $O(n)$ |
| • Element entfernen (remove) | $O(n)$ |
| • Liste kopieren (copy) | $O(n)$ |
| • Liste invertieren (reverse) | $O(n)$ |
| • Einmal traversieren | $O(n)$ |
| • Enthaltensein eines Elements testen | $O(n)$ |

- **Wörterbücher** w , n sei $\text{len}(L)$

Worst Case

- | | |
|---|--------|
| • Zugriff ($w[\text{element}]$) | $O(n)$ |
| • Speichern (Zuweisung) | $O(n)$ |
| • Länge bestimmen (len) | $O(n)$ |
| • Element entfernen (remove) | $O(n)$ |
| • Einmal traversieren | $O(n)$ |

- | | |
|-----------------------------------|--------|
| • Zugriff ($w[\text{element}]$) | $O(1)$ |
| • Speichern (Zuweisung) | $O(1)$ |
| • Element entfernen | $O(1)$ |
| • Einmal traversieren | $O(1)$ |

Average Case

Suchalgorithmen

- Suchalgorithmen sind Methoden, um ein Element in einer Menge von Elementen mit gewissen Eigenschaften zu finden
- zugrundeliegende Menge kann u.U. nur **implizit gegeben** sein
 - Beispiel: Berechnung der Quadratwurzel, mögliche Ansätze:
 - Brute-Force-mäßig alle möglichen Lösungen aufzählen und jeweils testen
 - Bisection Search
 - Newton-Verfahren
- zugrundeliegende Menge kann **explizit gegeben** sein
 - Beispiel: Ist eine gegebene Matrikelnummer in der Menge aller Erstsemester (bspw. als Wörterbuch gegeben) vorhanden?